COURSE SLIDES

Certified Kubernetes Application Developer (CKAD)

WSQ Course Code: TGS-2025053212

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version 1.0

COURSE ADMINISTRATION

Welcome & Housekeeping

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your phone camera and submit your attendance.
- A minimum of 75% attendance is required for assessment and funding.

Minimum 75% attendance required

YOUR TRAINER

About the Trainer

Mohan Pothula

Kubernetes & Cloud-Native Trainer

Role

Principal Kubernetes & DevOps trainer at Tertiary Infotech Academy.

Qualifications

Certified Kubernetes Application Developer (CKAD) · CKA · Docker Certified Associate · AWS Solutions Architect.

Expertise

Kubernetes · Docker · CI/CD · Cloud-native platform engineering.

Experience

20+ years across DBS Bank, SingTel, Mediacorp and SPH Media.

YOUR TRAINER

About the Trainer

[Trainer Name]

[Title / Role]

Qualifications: ______________________________

Expertise: ______________________________

Experience: ______________________________

Contact / Profile: ______________________________

ICE-BREAKER

Let's Know Each Other

Take a minute to introduce yourself to the class:

Your role

Your name and organisation / role.

Your experience

Any experience with Docker, Kubernetes or the cloud.

Your goal

Whether you are targeting the CKAD exam or production skills.

HOUSE RULES

Ground Rules

Phones on silent

Set your phone to silent mode.

Participate actively

No question is too small — ask freely.

Mutual respect

One conversation at a time.

Be punctual

Return from breaks on time.

Step out quietly

For calls or short breaks.

75% attendance

Required for funding eligibility.

SCHEDULE

Lesson Plan — 3½ Days, 8 hours/day

1 Day 1 · Domain 1

Application Design & Build

- Container images & multi-stage builds (Labs 1–2)
- Pods, Jobs & CronJobs (Labs 3–5)
- Multi-container Pods & Volumes (Labs 6–8)

2 Day 2 · Domain 2

Application Deployment

- Deployments, ReplicaSets & rollouts (Labs 9–10)
- Blue/green & canary (Labs 11–12)
- Helm, Kustomize, Daemon/StatefulSet (Labs 13–15)

3 Day 3 · Domains 3 & 4

Observability + Config & Security

- Probes, logging, metrics, debug (Labs 16–20)
- ConfigMaps, Secrets, SecurityContext (Labs 21–23)
- ServiceAccounts, RBAC, Quotas (Labs 24–26)

4 Day 4 (½) · Domain 5

Services & Networking + Assessment

- Services & Service DNS (Labs 27–28)
- Ingress with TLS & NetworkPolicy (Labs 29–30)
- 1:00 PM mock-exam practice, then final assessment from 2:00 PM

Daily timing: 9:00am – 6:00pm · 1-hour lunch · short tea breaks within each day

WHAT YOU'LL ACHIEVE

Learning Outcomes

1. **LO1** Build and optimise container images with single- and multi-stage Dockerfiles.
2. **LO2** Create and manage Pods imperatively and declaratively with kubectl and --dry-run.
3. **LO3** Run batch workloads with Jobs and scheduled workloads with CronJobs.
4. **LO4** Design multi-container Pods using the sidecar and init-container patterns.
5. **LO5** Persist and share Pod data with emptyDir, tmpfs and hostPath volumes.

LO6 Deploy, scale, observe, secure and network applications across the five CKAD domains.

FINAL ASSESSMENT

Assessment

Final Assessment

- Written / MCQ assessment on the LMS
- Mock-exam practice from 1:00 PM; final assessment from 2:00 PM
- Covers all five CKAD domains and the hands-on labs
- Open book — slides, Learner Guide & kubernetes.io
- Must be attempted for SSG funding

Funding & Competency

Minimum 75% attendance

Based on SSG digital attendance records.

Assessed as Competent

Pass the written / MCQ assessment.

ASSESSMENT RULES

Briefing for Assessment

Clear your space

Keep only approved materials on the table.

No recording

No photos of the assessment.

Work individually

No discussion during the assessment.

Use the LMS

The assessment is delivered online.

Open book

Slides, Learner Guide & kubernetes.io are allowed.

Time's up

Submit when time is up.

01

DAY 1

Application Design and Build

Images · Pods · Jobs · CronJobs · Multi-container · Volumes (CKAD Domain 1 — 20%)

TOPIC 1

Container Images

Dockerfiles, image layers & multi-stage builds

OVERVIEW

Why Container Images?

- An image is a read-only blueprint that packages your app with all its dependencies.
- A container is a running instance of an image — it starts in milliseconds.
- Kubernetes never builds images; it pulls them from a registry by name:tag and runs them.
- CKAD expects you to read, write and fix Dockerfiles under time pressure.

What is Kubernetes?

Kubernetes (k8s)

- Open-source container orchestration platform
- Originally designed by Google
- CNCF graduated project since 2016
- Automates deployment, scaling, and management of containerised workloads
- Declarative: describe desired state; K8s makes it so

Why Container Orchestration?

- Containers are ephemeral — orchestration ensures availability
- Scale to thousands of containers across many nodes
- Self-healing: restart failed containers automatically

Key Capabilities

Auto-scaling | **Self-healing** | **Load balancing** | **Rolling updates**

Virtual Machines vs Containers

Virtual Machines

- Include full OS (kernel + libraries + app)
- GBs in size; minutes to boot
- Strong isolation via hypervisor
- Each VM has its own kernel
- Good for: legacy apps, OS-level testing

vs

Containers

- Share host OS kernel; package only app + libs
- MBs in size; milliseconds to start
- Process-level isolation via namespaces + cgroups
- Very low overhead; high density per host
- Good for: microservices, CI/CD, cloud-native apps

Kubernetes High-Level Architecture

Control Plane (Master Node)

- **kube-apiserver** — Front-end; all comms go through it
- **etcd** — Distributed key-value store; source of truth
- **kube-scheduler** — Assigns Pods to nodes
- **controller-manager** — Runs Deployment, ReplicaSet, Node controllers…

Worker Node

- **kubelet** — Agent; ensures containers run in Pods
- **kube-proxy** — Network rules; enables Service networking
- **Container Runtime** — Runs containers (containerd, CRI-O…)
- **Pod (App)** — One or more containers; your workload

kubectl — The Kubernetes CLI

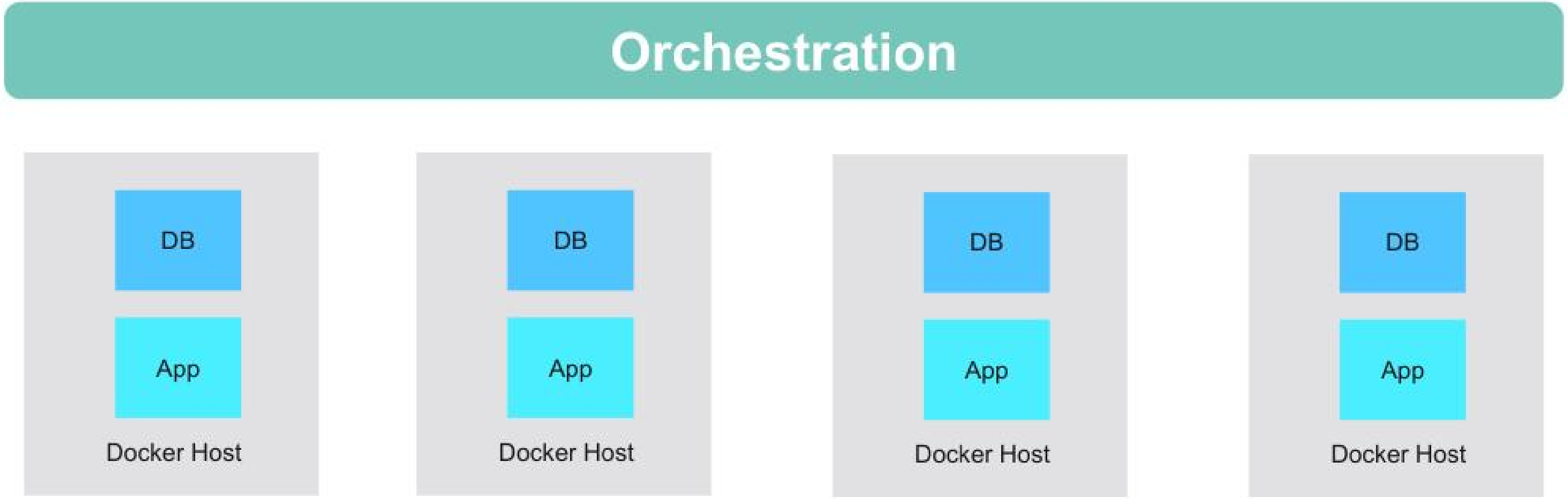
Structure: verb + resource + flags

```
kubectl <verb> <resource> [name] [flags]

kubectl get       # list resources
kubectl describe  # detailed info
kubectl apply     # create/update from YAML
kubectl delete    # remove a resource
kubectl exec      # run command in container
kubectl logs      # view container logs
```

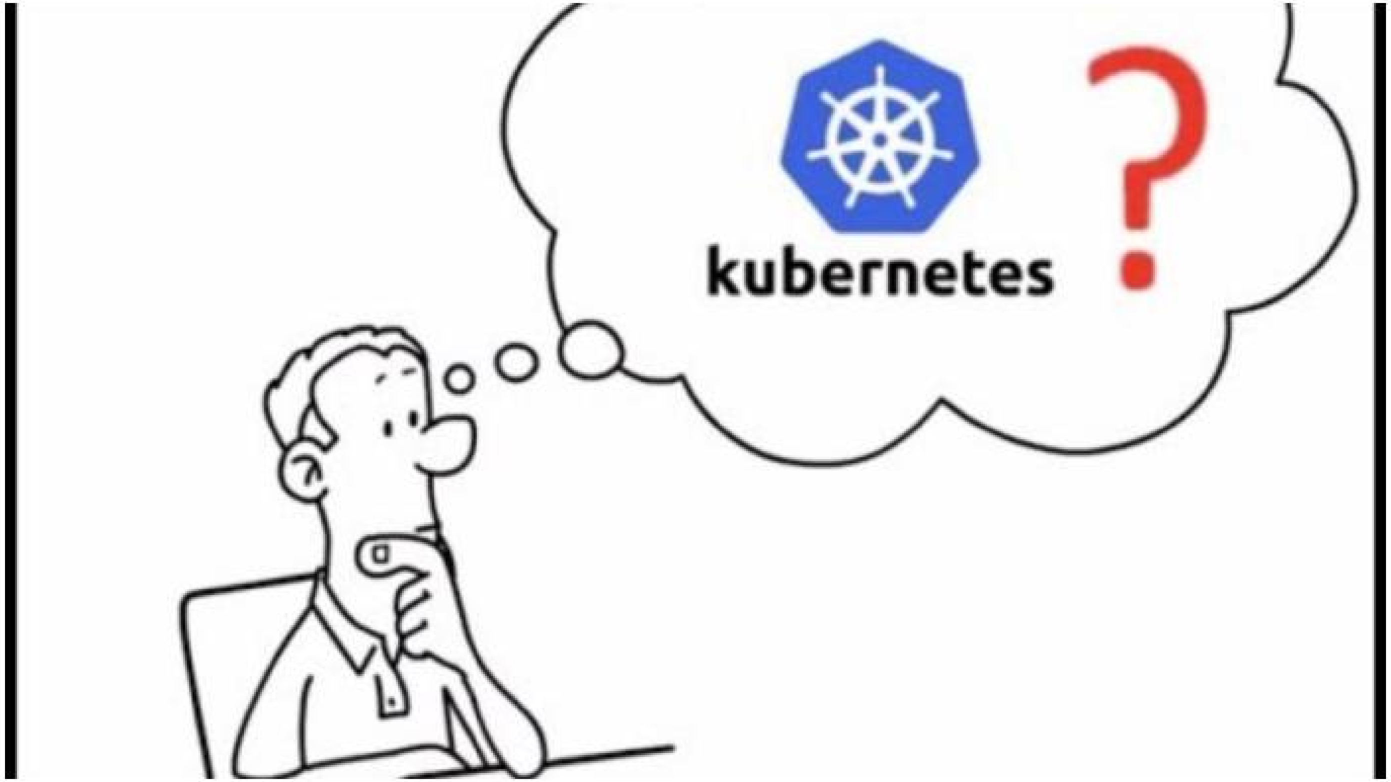
Useful flags for the exam

```
kubectl get pods -n <namespace>                         # specify namespace
kubectl get all --all-namespaces                        # across all ns
kubectl run nginx --image=nginx --dry-run=client -o yaml  # generate YAML
kubectl explain pod.spec.containers                     # inline API docs
```

Container Orchestration Tool



What is Kubernetes?



The founding of “κυβερνήτης”

History

- Greek for “Helmsman”
- Founded by Joe Beda, Brendan Burns and Craig McLuckie in Google
- First announced to public in mid-2014
- Version 1 released on July 2015
- We call it k8s for short

Orchestration tools

Kubernetes became the market leader after mass support from the community and support from key market players in the cloud space

WIN

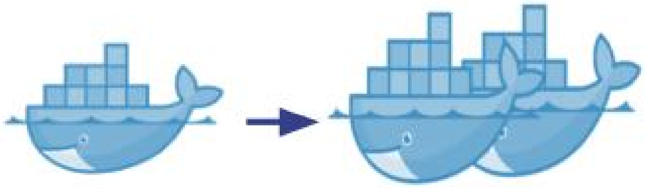
Docker Swarm

Kubernetes

Mesos

Kubernetes (k8s) - Key Benefits

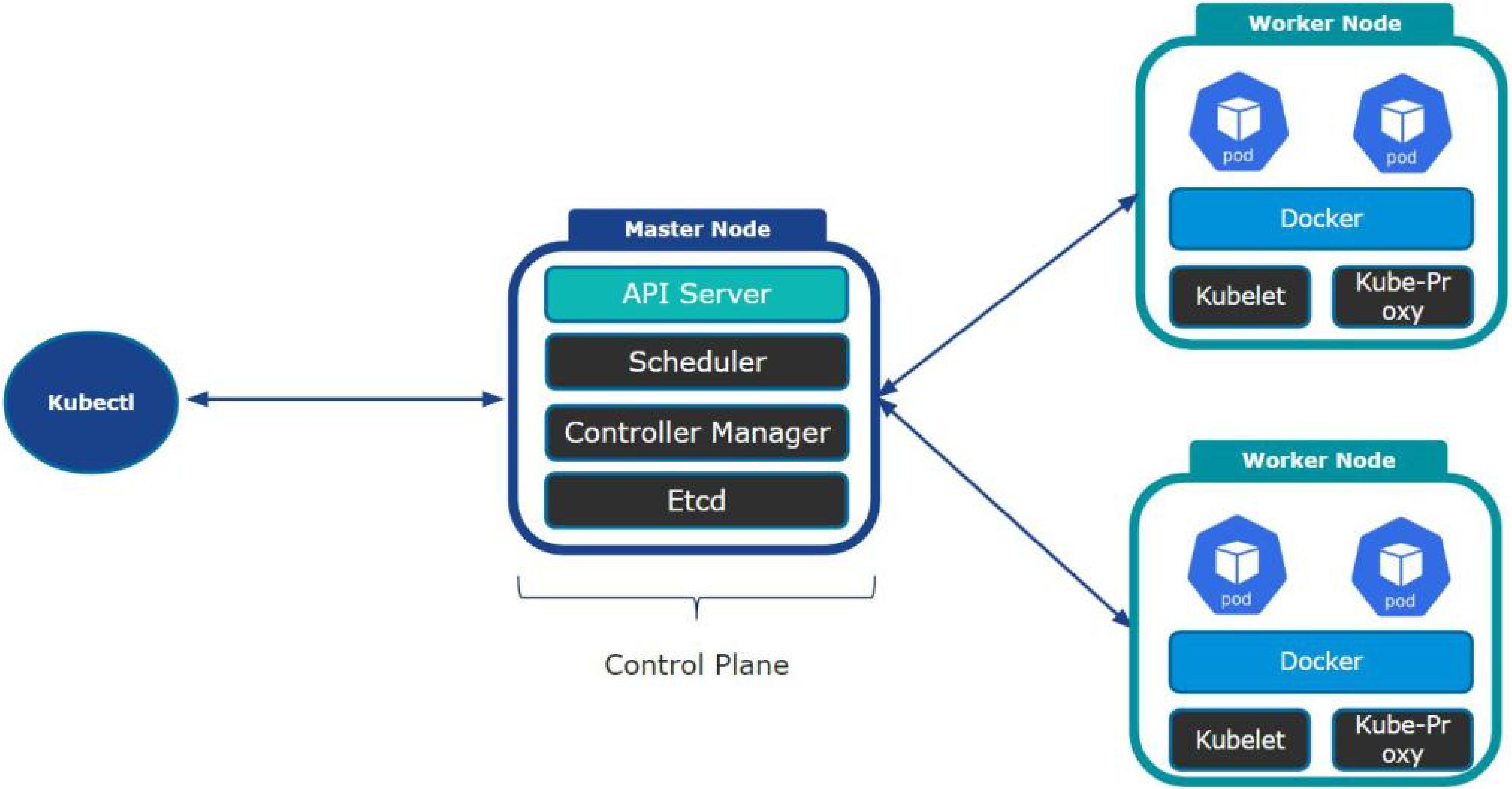
Portable



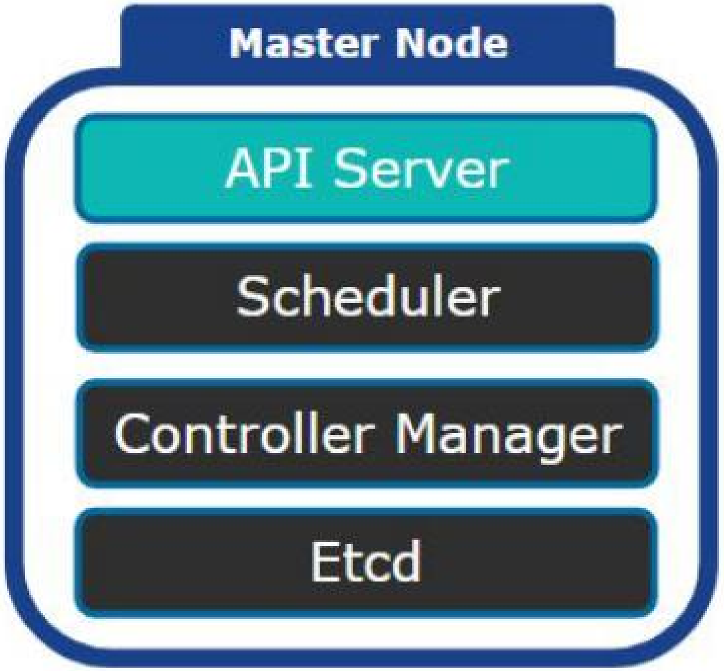
Scalable

Highly Available

Kubernetes High-Level Architecture

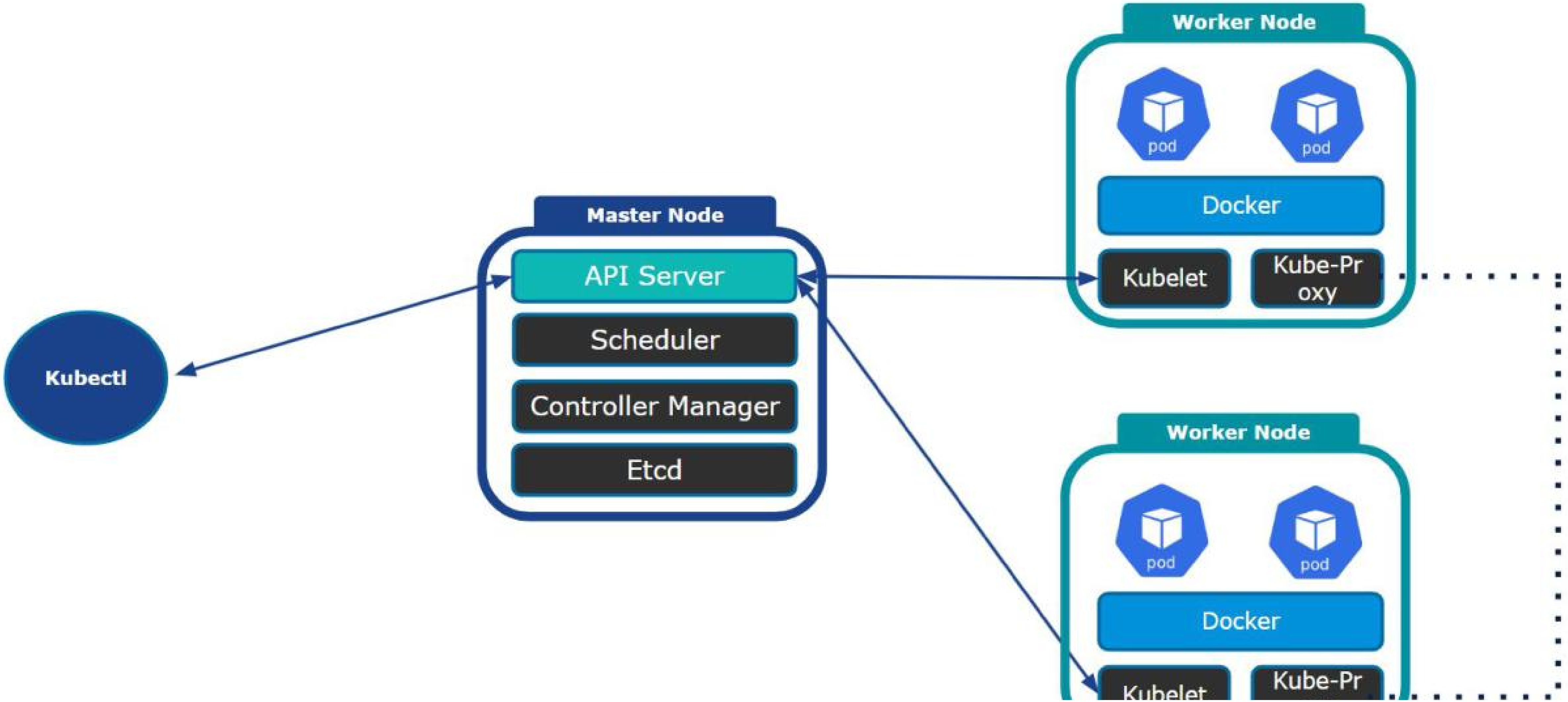


Master Node



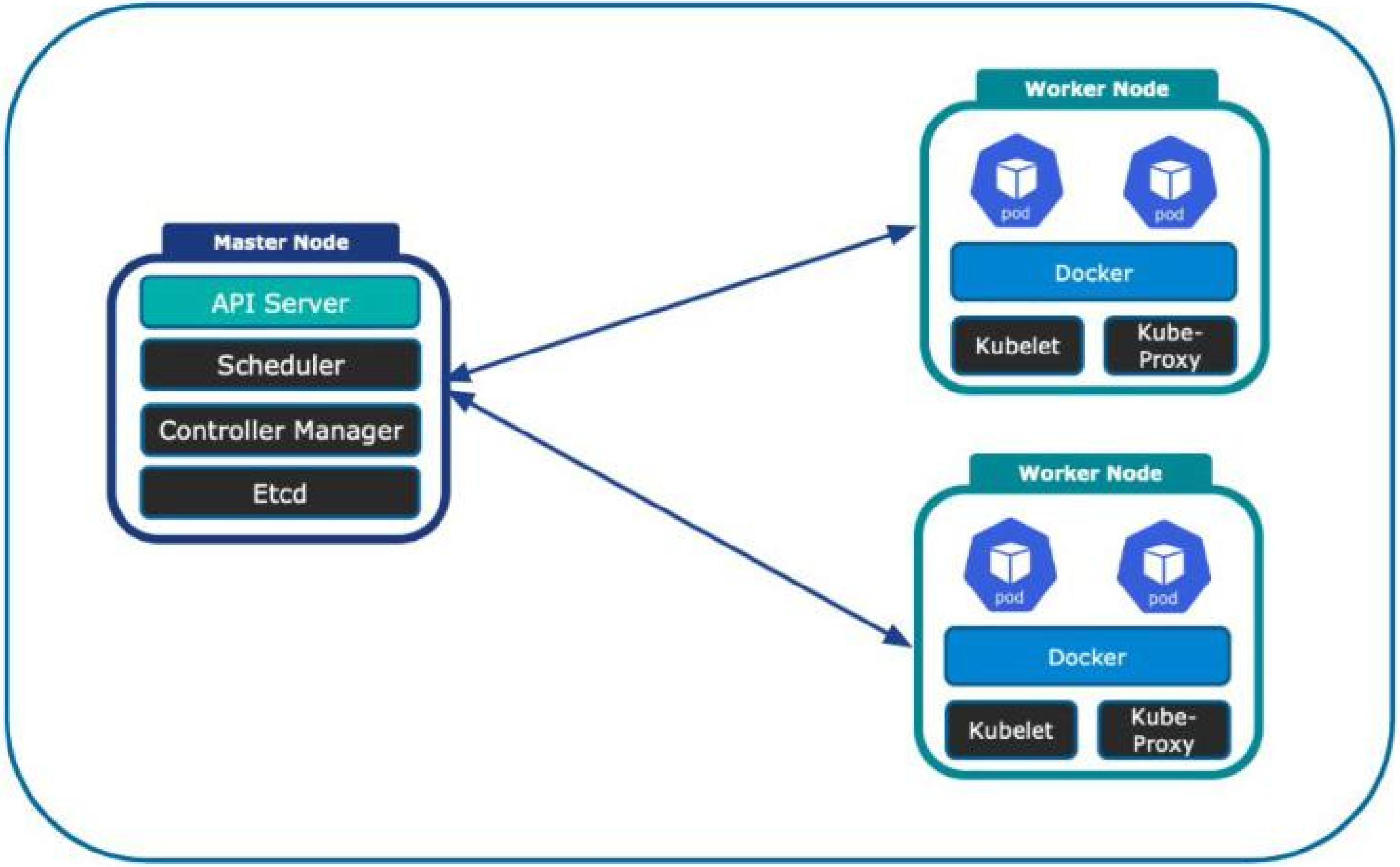
- **API Server (kube-apiserver):** Orchestrating all operations within the cluster
- **Scheduler (kube-scheduler):** Selects optimal node to run pods
- **Controller Manager (kube-controller-manager):** Different controllers with different functions
- **Etcd:** Stores data in key-value format

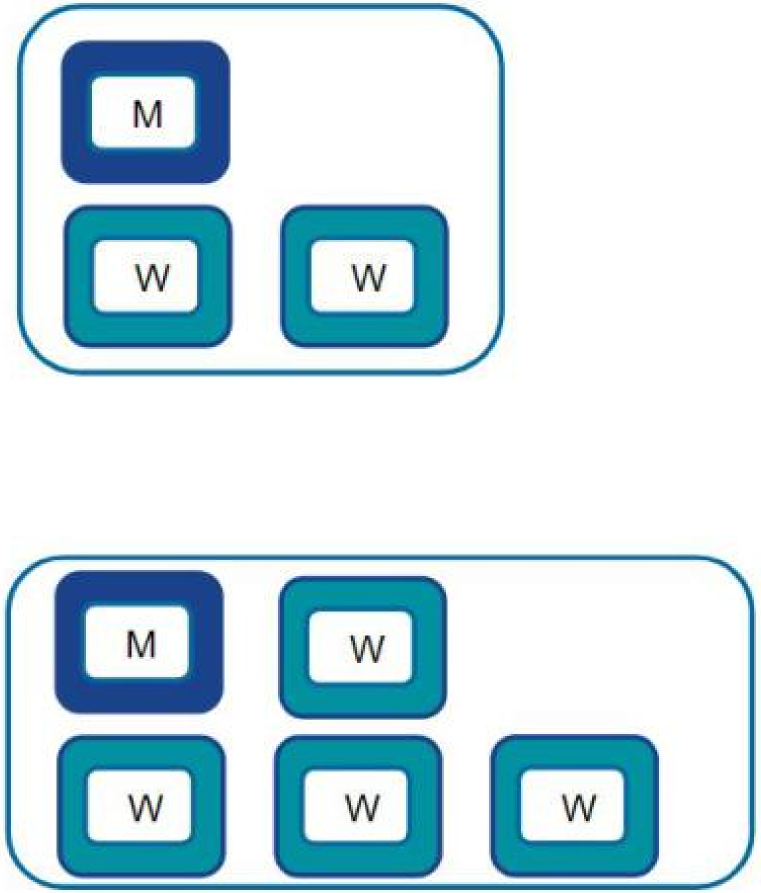
Kubernetes High-Level Architecture



Kubernetes Cluster

A set of nodes grouped together





Declarative Configurations

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: redis-pod1
spec:
  containers:
  - name: myapp-container
    image: redis
```

- YAML
  - Lines and white spaces delimiters

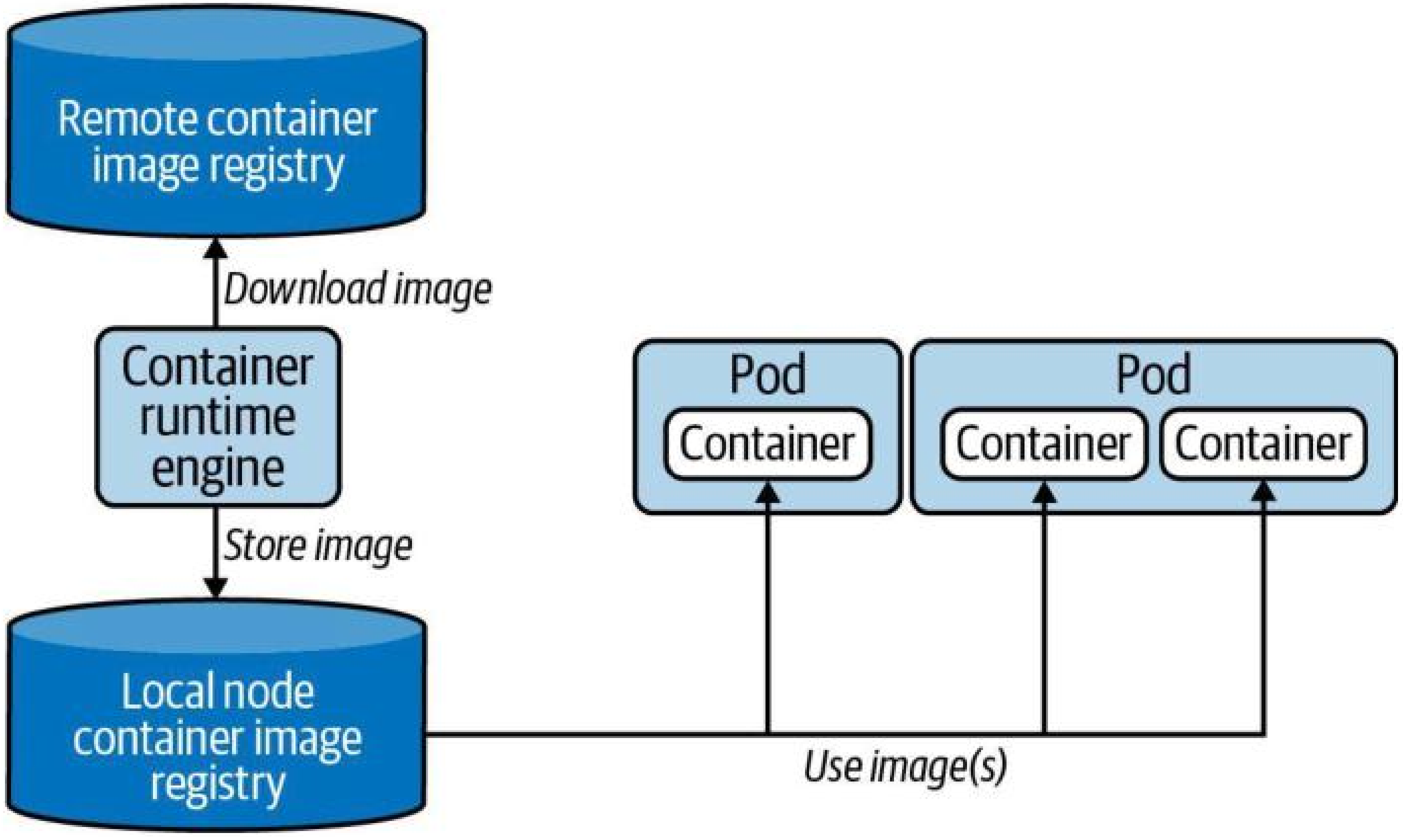
Imperative Commands

"Execute Hazelcast and configure it using parameters"

```
$ kubectl run hazelcast↵
  --image=hazelcast/hazelcast↵
  --restart=Never↵
  --port=5701↵
  --env="DNS_DOMAIN=cluster"↵
  --labels="app=hazelcast,env=prod"
pod/hazelcast created
```

Container Image Retrieval

Downloads container image from registry upon Pod creation



Creating a Pod with Imperative Command

- Get YAML output from Kubernetes

```
kubectl get pod webapp-pod-2 -o yaml > webapp-pod-2.yaml
```

Get pod command | Name of pod | Output type | Name of yaml file output

https://kubernetes.io/docs/reference/kubectl/cheatsheet/

Creating a Pod with Imperative Command

```yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: "2020-09-24T08:31:42Z"
  labels:
    run: webapp-pod-2
  name: webapp-pod-2
  namespace: default
  resourceVersion: "1931003"
  selfLink: /api/v1/namespaces/default/pods/webapp-pod-2
  uid: 18f28bba-83f1-471e-a40b-93112406f92a
spec:
  containers:
  - image: nginx
    imagePullPolicy: Always
    name: webapp-pod-2
    resources: {}
    terminationMessagePath: /dev/termination-log
    terminationMessagePolicy: File
    volumeMounts:
    - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
      name: default-token-dl9sb
      readOnly: true
  dnsPolicy: ClusterFirst
  enableServiceLinks: true
  nodeName: minikube
  priority: 0
  restartPolicy: Always
  schedulerName: default-scheduler
  securityContext: {}
  serviceAccount: default
  serviceAccountName: default
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: webapp-pod-2
spec:
  containers:
  - name: webapp-pod-2
    image: nginx
```

Creating a Namespace with Imperative Command

"Organize objects in namespace called `code-red`"

```
$ kubectl create namespace code-red
namespace/code-red created

$ kubectl run nginx --image=nginx --restart=Never -n code-red
pod/nginx created

$ kubectl get pods -n code-red
NAME    READY    STATUS   RESTARTS AGE
nginx   1/1  Running  0    13s
```

Creating a Namespace with Imperative Command

```
kubectl run sample-pod-3 --image=nginx --namespace=dev
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: sample-pod-3
  namespace: dev
spec:
  containers:
  - name: sample-pod-1
    image: nginx
```

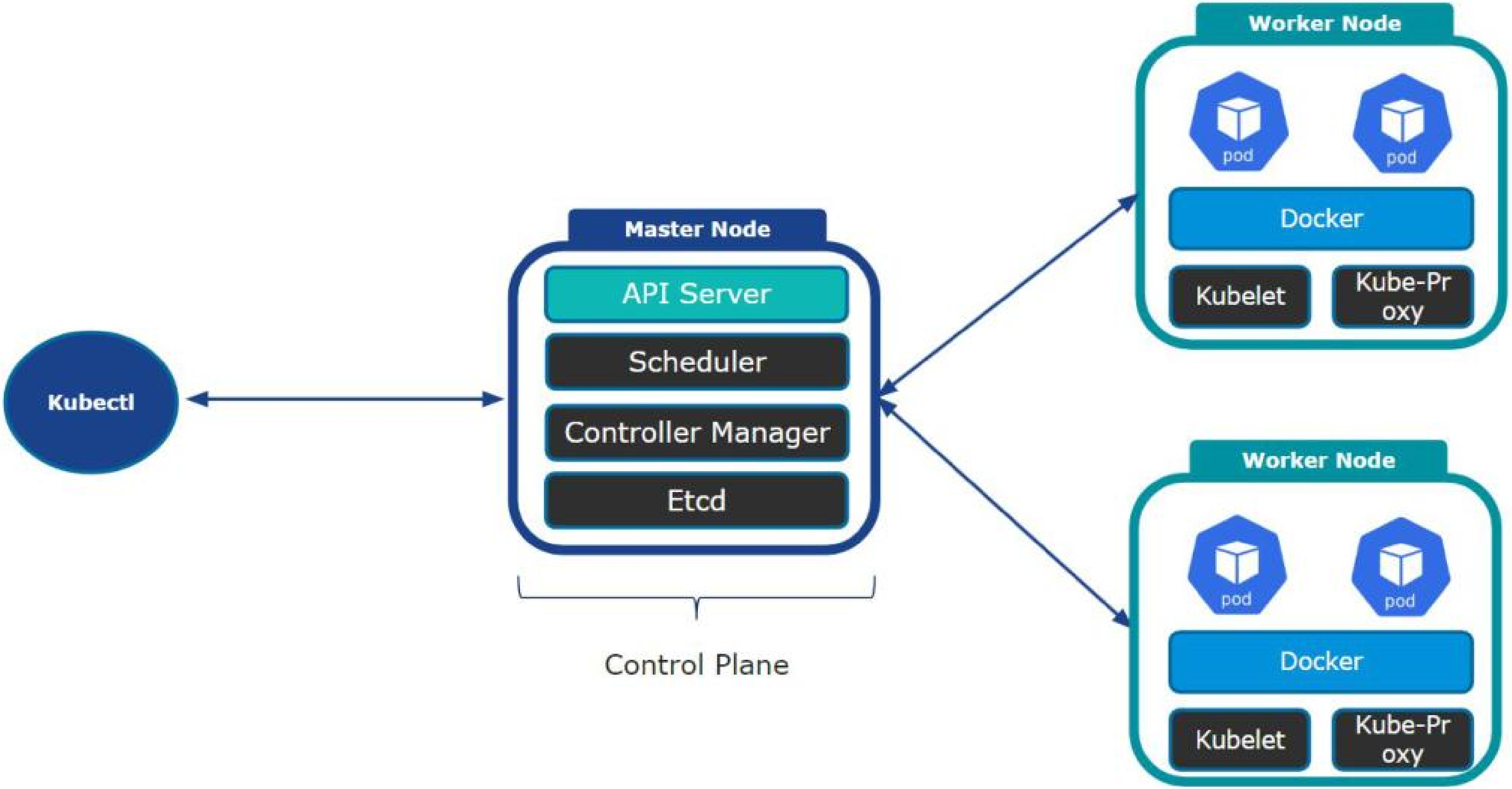
```
kubectl get pods --namespace=dev
```

Creating a CronJob with Imperative Command

"Render the current date on standard output every hour"

```
$ kubectl create
  cronjob↲ current-date↲
  --schedule="* * * * *"↲
  --image=nginx:1.24.0
  -- /bin/sh -c 'echo "Current date: $(date)"'
cronjob.batch/current-date created
```

Kubernetes High-Level Architecture



Declarative Configurations

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: redis-pod1
spec:
  containers:
  - name: myapp-container
    image: redis
```

- YAML
  - Lines and white spaces delimiters

Imperative Commands

"Execute Hazelcast and configure it using parameters"

```
$ kubectl run hazelcast↵
  --image=hazelcast/hazelcast↵
  --restart=Never↵
  --port=5701↵
  --env="DNS_DOMAIN=cluster"↵
  --labels="app=hazelcast,env=prod"
pod/hazelcast created
```

ONE INSTRUCTION = ONE LAYER

Anatomy of a Dockerfile

Base & setup

- FROM — pick a small, pinned base
- WORKDIR — set the working dir
- COPY / ADD — bring files in

Build steps

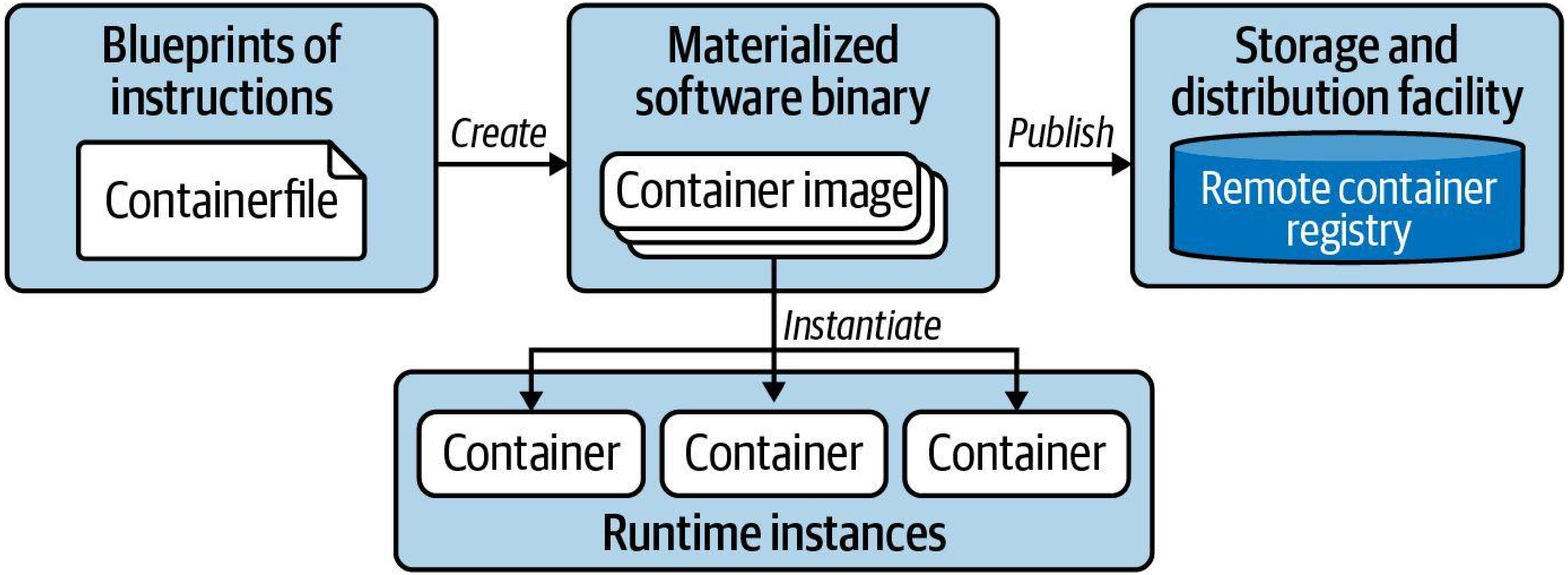
- RUN — install dependencies
- Each line = one cached layer
- Order steps for cache reuse

Runtime

- EXPOSE — document the port
- ENV — runtime configuration
- CMD / ENTRYPOINT — what runs

Containerization Process

Building and publishing an image, running image in container



SHRINK THE IMAGE

Single-Stage vs Multi-Stage Builds

Single-stage

- One FROM — build + runtime together
- Ships the whole toolchain
- Large image (~800 MB for Go)
- Bigger attack surface

Multi-stage

- Multiple FROM stages with AS <name>
- COPY --from=builder copies only the binary
- Tiny image (~8 MB, distroless)
- No shell, no package manager

Multi-Stage Build — Shrink the Image

Stage 1 — builder

FROM golang:1.22
WORKDIR /src
COPY . .
RUN go build -o /server

Ships compiler + source + modules

COPY --from=builder

only the binary

Stage 2 — runtime

FROM gcr.io/distroless
COPY --from=builder /server /
ENTRYPOINT ["/server"]

No shell · no package manager
Just your binary

~800 MB final image

~8 MB final image

Build a Container Image with Docker

LAB 1

Container images

Write a Dockerfile from scratch, build a tagged image, run it locally and inspect its layers. CKAD expects you to read and write Dockerfiles confidently — you may be asked to fix a broken one or produce one under time pressure.

You'll build

Write a Dockerfile, build and tag an image, run it, then inspect its layers with docker history.

Key commands: FROM python:3.12-slim · » `EXPOSE` · docker build · docker run · docker history

Build a Container Image with Docker

Step 1 — Write the Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
EXPOSE 8080
CMD ["python", "app.py"]
```

Note EXPOSE is documentation only — it does not publish the port. -p host:container is what actually opens it.

Step 2 — Build and tag the image

```
docker build -t ckad/hello:1.0 .
docker images ckad/hello
```

Build a Container Image with Docker (cont.)

Step 3 — Run and test the container

```
docker run -d --name hello -p 8080:8080 ckad/hello:1.0
curl http://localhost:8080
```

Step 4 — Inspect the image layers

```
docker history ckad/hello:1.0
docker rm -f hello && docker rmi ckad/hello:1.0
```

Build a Container Image with Docker

Test it

curl http://localhost:8080 returns hello from CKAD 2026 lab 1, and docker history shows a distinct layer for each Dockerfile instruction.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Multi-Stage Dockerfile

LAB 2 Container images

Separate the build stage from the runtime stage to produce images that are 10x smaller and contain no compiler toolchain. Multi-stage builds are a CKAD staple — you must be able to write one from scratch and explain why it shrinks the image.

You'll build

Build a single-stage baseline, then a multi-stage image with COPY --from and distroless, and compare sizes.

Key commands: FROM golang:1.22 · docker build · » Only

Multi-Stage Dockerfile

Step 1 — Single-stage baseline (large image)

```
FROM golang:1.22
WORKDIR /src
COPY main.go .
RUN go mod init demo && go build -o app main.go
CMD ["./app"]
```

Step 2 — Multi-stage Dockerfile (small image)

```
# Stage 1: compile
FROM golang:1.22 AS builder
WORKDIR /src
COPY main.go .
RUN go mod init demo && CGO_ENABLED=0 go build -o /out/app main.go

# Stage 2: runtime only
FROM gcr.io/distroless/static-debian12
COPY --from=builder /out/app /app
ENTRYPOINT ["/app"]
```

Multi-Stage Dockerfile (cont.)

Step 3 — Build both and compare sizes

```
docker build -f Dockerfile.single -t demo:single .
docker build -f Dockerfile.multi  -t demo:multi .
docker images | grep demo        # ~800MB  vs  ~8MB
```

Note Only the last FROM stage ends up in the final image. CGO_ENABLED=0 produces a static binary that runs in distroless/scratch — no shell, smaller attack surface.

Multi-Stage Dockerfile

Test it

docker images | grep demo shows demo:multi roughly 100x smaller than demo:single, and demo:multi still serves hello from multi-stage build.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Tea Break

15 minutes

TOPIC 2

Pods

The smallest deployable unit & the exam-speed kubectl workflow

FOUNDATIONS

What is a Pod?

- A Pod is the smallest deployable unit — one or more containers sharing network & storage.
- Containers in a Pod share an IP and localhost; they can share volumes.
- Create Pods imperatively (kubectl run) or declaratively (kubectl apply -f).
- Pods are ephemeral — in production you run them via higher-level controllers (Day 2).

GENERATE → EDIT → APPLY

The CKAD kubectl Workflow

Generate

- kubectl run / create
- with --dry-run=client -o yaml
- Scaffold a manifest fast

Edit

- Redirect to a .yaml file
- Tweak labels, image, command
- Version-controllable

Apply

- kubectl apply -f file.yaml
- Idempotent & repeatable
- kubectl describe to verify

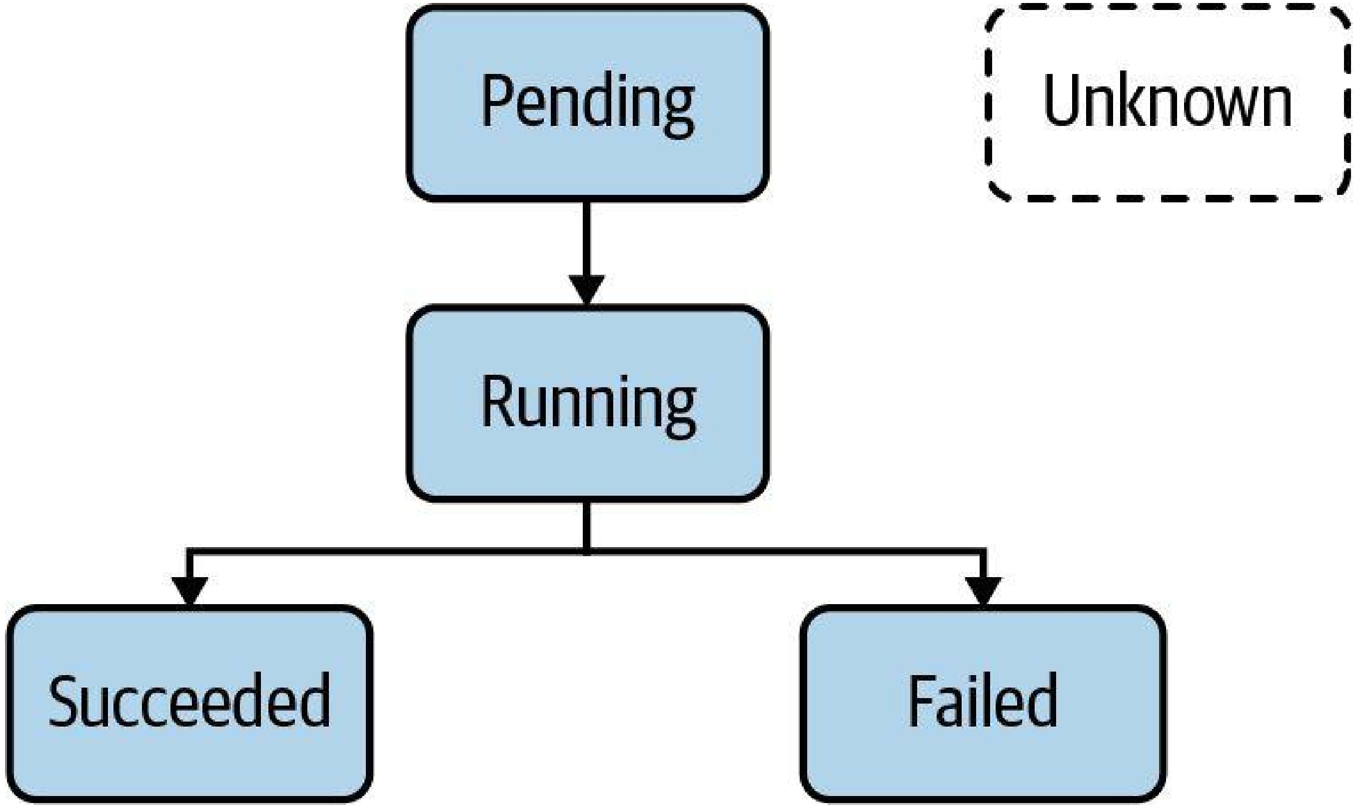
What is a Pod?

Runs a process or application in container(s)

- The smallest, most basic deployable objects in Kubernetes.
- It contains one or more containers, executed by the configured container runtime.
- Kubernetes enhances the basic functionality of a container with features such as security, storage, and more.

Pod Life Cycle Phases

Indicates operational status of Pod



SET THESE ON EXAM DAY

Exam-Speed kubectl (kubernetes.io quick reference)

- alias k=kubectl — set this first; saves keystrokes on every command.
- export do="--dry-run=client -o yaml" — scaffold any manifest without creating it.
- source <(kubectl completion bash); complete -o default -F __start_kubectl k — tab-completion.
- kubectl get -o wide / -o yaml / -o jsonpath · kubectl describe · kubectl explain — inspect & learn.

DAY 1 · PODS

Create and Manage Pods

LAB 3 Pods

The Pod is the smallest schedulable unit in Kubernetes. Create Pods imperatively, generate YAML with --dry-run, edit live manifests, and use the exam-critical $do alias that saves 30+ seconds per question.

You'll build

Set exam-speed aliases, run Pods imperatively, scaffold YAML with $do, then apply and inspect.

Key commands: alias k=kubectl · k run · k describe · » `--restart=Never`

Create and Manage Pods

Step 1 — Set exam-speed aliases (run these first on exam day)

```
alias k=kubectl
export do="--dry-run=client -o yaml"
```

Step 2 — Create a Pod imperatively

```
k run web --image=nginx:1.25 --port=80
k get pod web -o wide          # shows node and Pod IP
```

Step 3 — Generate a manifest without creating it

```
k run web2 --image=nginx:1.25 --port=80 $do > web2.yaml
k apply -f web2.yaml
k get pod web2 --show-labels
```

Create and Manage Pods (cont.)

Step 4 — Inspect a running Pod

```
k describe pod web
k get pod web -o jsonpath='{.status.podIP}'; echo
```

Step 5 — Override the container command

```
k run sleeper --image=busybox --restart=Never -- sh -c 'sleep 3600'
k delete pod web web2 sleeper --force --grace-period=0
```

Note --restart=Never creates a bare Pod (not a Deployment). Everything after -- becomes the container command. --force --grace-period=0 skips the 30s grace period.

Create and Manage Pods

Test it

k get pod web2 --show-labels shows the tier=frontend label, and the --dry-run=client -o yaml workflow produces a valid manifest you can apply.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Pod Lifecycle Phases

Phase	Description
Pending	Accepted by cluster; image pulling or scheduling
Running	At least one container is running
Succeeded	All containers exited with code 0
Failed	At least one container exited non-zero
Unknown	Pod state could not be obtained (node issue)

Check phase: kubectl get pod <name> -o jsonpath='{.status.phase}'

Common Pod Error Statuses

- CrashLoopBackOff — container keeps crashing; K8s backs off restart time
  - Causes: bad entrypoint, missing config, OOM kill, application crash
- ImagePullBackOff / ErrImagePull — cannot pull container image
  - Causes: wrong image name/tag, private registry, no imagePullSecret
- OOMKilled — container exceeded its memory limit; killed by kernel
- Pending (forever) — no node can satisfy scheduling requirements
  - Causes: insufficient CPU/memory, missing node label, taints without tolerations
- RunContainerError — container config error (bad entrypoint, missing volume mount)

Troubleshooting Pods & Containers

1 Check Pod status and events

```
kubectl get pod <name> -n <ns>
kubectl describe pod <name> -n <ns>    # shows Events section
```

2 View container logs

```
kubectl logs <pod-name>                # current logs
kubectl logs <pod-name> -p             # previous (crashed) container
kubectl logs <pod-name> -c <container> # specific container
```

3 Shell and ephemeral debug containers

```
kubectl exec -it <pod-name> -- /bin/sh
kubectl debug <pod-name> -it --image=busybox    # ephemeral container
  (use when main image has no shell, e.g. distroless images)
```

Environment Variables and Command/Args in Pods

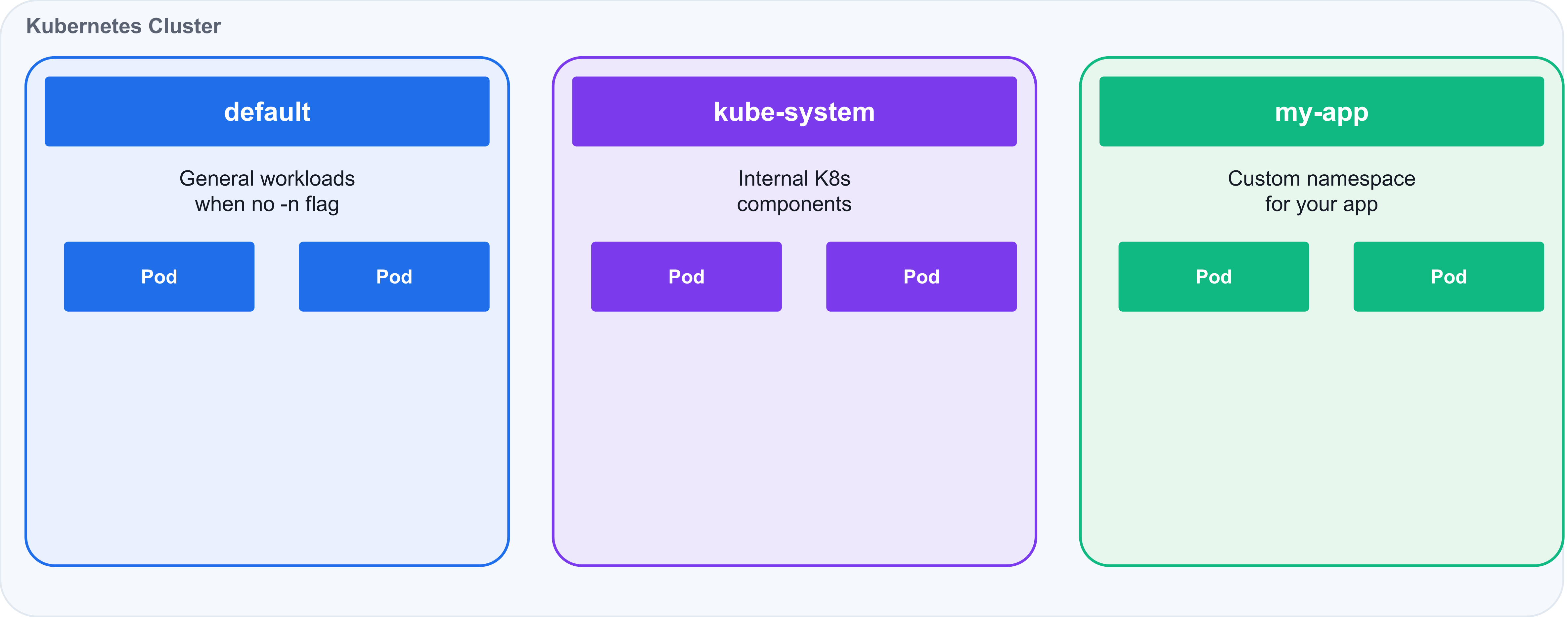
Declaring environment variables

```
spec:
  containers:
  - name: app
    image: myapp:1.0
    env:
    - name: APP_ENV
      value: production
    - name: DB_URL
      value: postgres://db:5432/mydb
```

Overriding container ENTRYPOINT and CMD

```
spec:
  containers:
  - name: app
    image: myapp:1.0
    command: ["/bin/sh"]       # overrides ENTRYPOINT
    args: ["-c", "echo hello"]  # overrides CMD
```

What is a Namespace?



Objects in different namespaces are isolated by default — cross-namespace access requires explicit Service DNS or RBAC policies

Namespace — Create, List, Delete

Create a namespace (imperative and declarative)

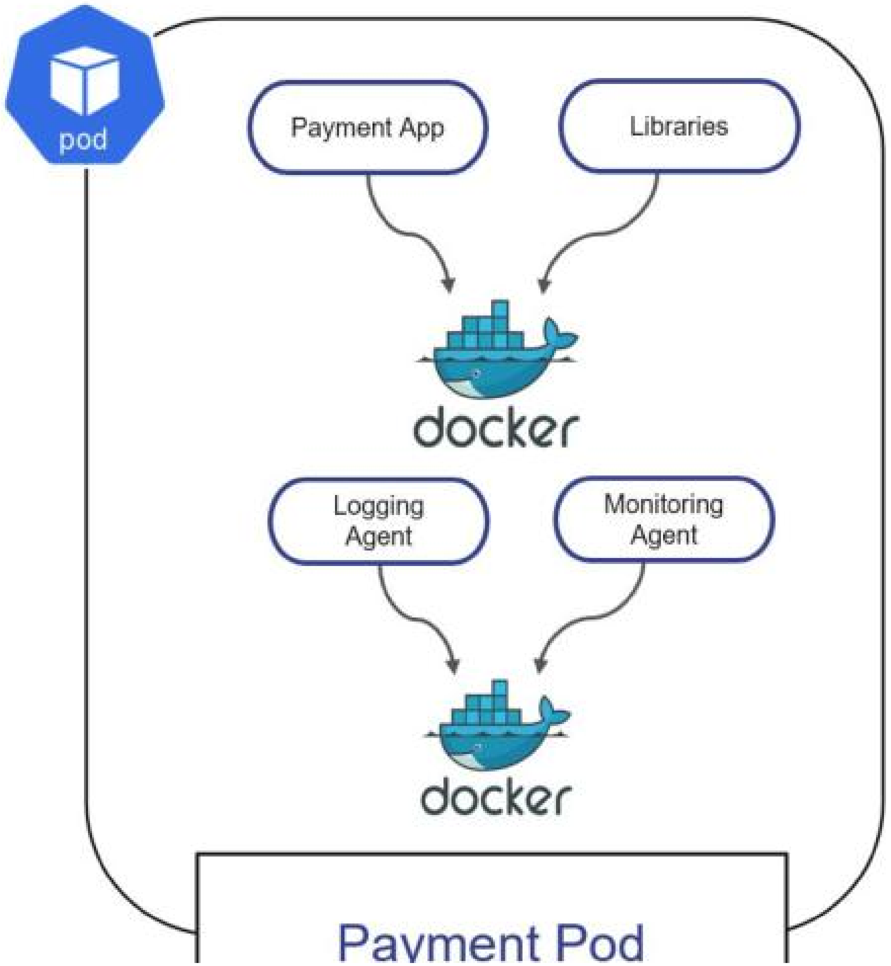
```
kubectl create namespace staging

apiVersion: v1
kind: Namespace
metadata:
  name: staging
```

List, inspect, delete, and set default namespace

```
kubectl get namespaces              # list all
kubectl describe namespace staging  # details
kubectl delete namespace staging    # cascade-deletes all objects inside!

kubectl run nginx --image=nginx -n staging
kubectl config set-context --current --namespace=staging
```

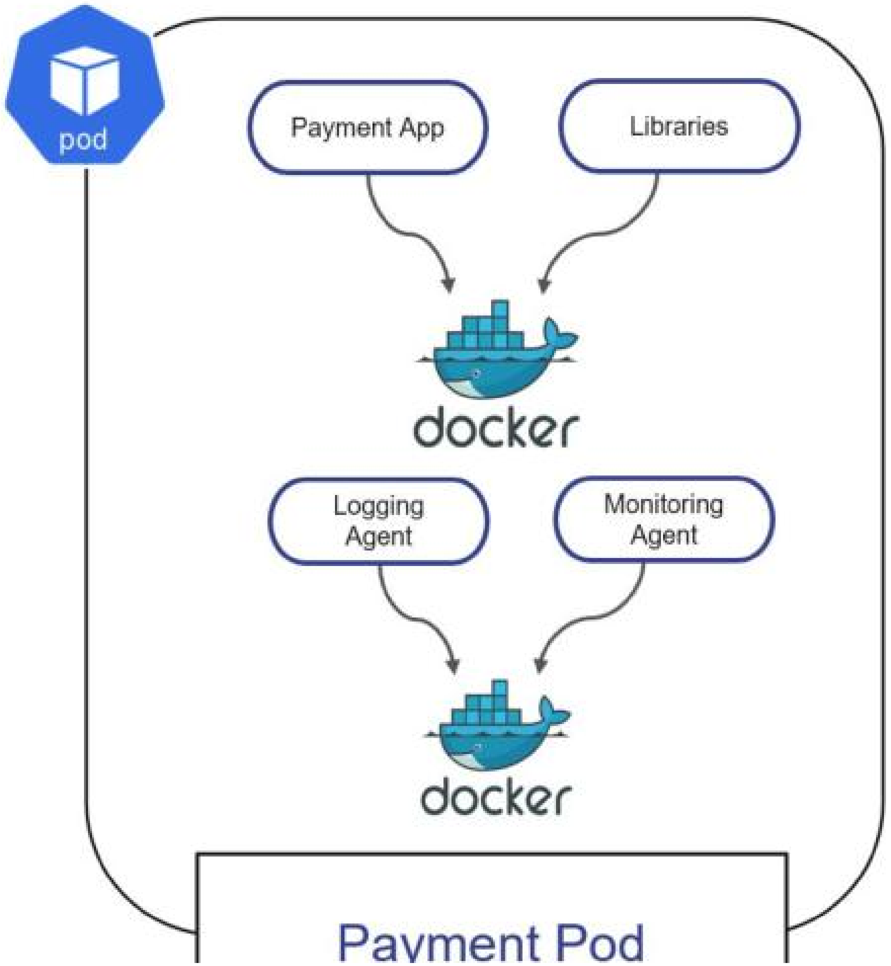
What is a Pod?

The smallest, most basic deployable objects in Kubernetes.



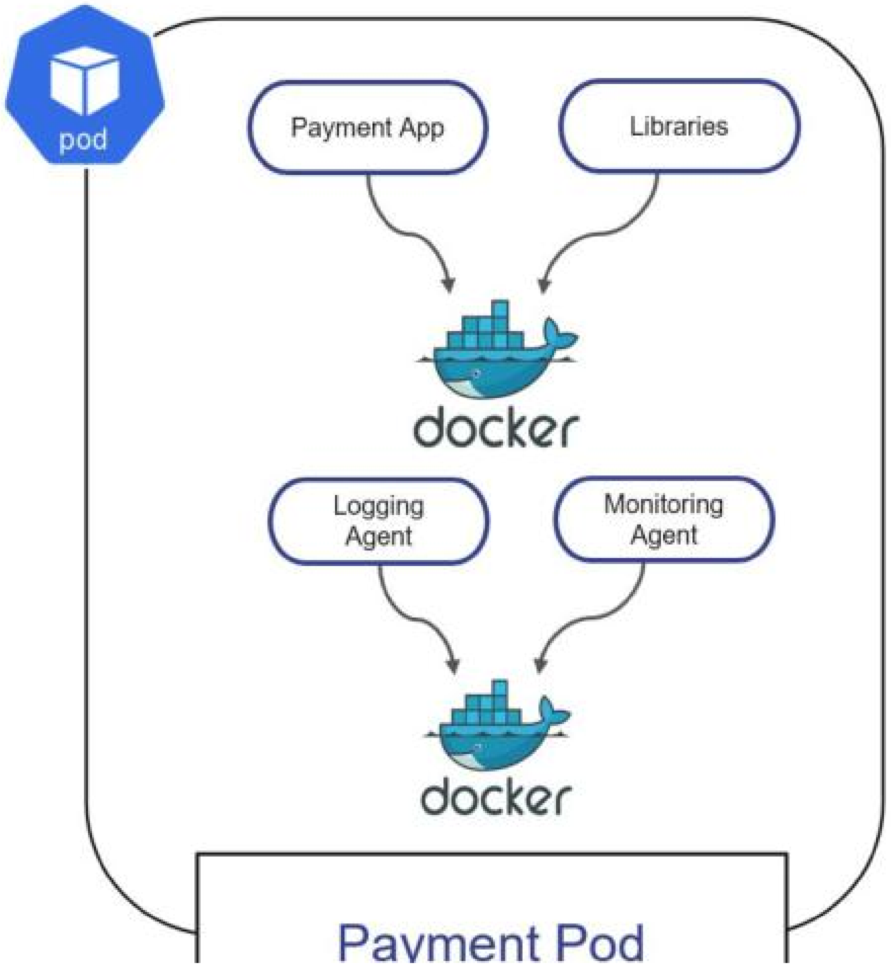
What is a Pod?

The smallest, most basic deployable objects in Kubernetes.



What is a Pod?

The smallest, most basic deployable objects in Kubernetes.



Retrieving High-Level Pod Information

Check the status of a Pod first

```
$ kubectl get pods -n t67
NAME READY     STATUS                         RESTARTS AGE
misbehaving-pod   0/1  ErrImagePull            0     2s
$ kubectl get pods -n k31
NAME READY     STATUS                   RESTARTS AGE
successful-pod    1/1  Running          0     5s
```

Failed to pull container image

Looks successful on the surface-level

Inspecting Events Across all Pods

Lists events for all Pods in the given namespace

```
$ kubectl get events
LAST SEEN TYPE REASON                     OBJECT                 MESSAGE
3m14s     Warning   BackOff               pod/custom-cmd         Back-off restarting
                                                                 ↵failed container
                                          cronjob/google-ping
2s   Warning   FailedNeedsStart                                  Cannot determine if
                                                                 ↵job needs to be ↵
                                                                 started: too many ↵
                                                                 missed start time ↵
                                                                 (> 100). Set or ↵
                                                                 decrease .spec. ↵
                                                                 startingDeadline ↵
                                                                 Seconds or check ↵
                                                                 clock skew
```

Opening an Interactive Shell to a Container

Allows for exploring container file system and processes

```
$ kubectl logs failing-pod
/bin/sh: can't create /root/tmp/curr-date.txt: nonexistent directory

$ kubectl exec failing-pod -it -- /bin/sh
# mkdir -p
~/tmp # cd
~/tmp
# ls
-l
total
4
-rw-r--r-- 1 root root 112 May 9 23:52
```

Copying a File from the Container

The `tar` binary needs to exist in container

```
$ kubectl exec -it nginx -n code-red -- /bin/sh
# tar cvf error-log.tar
/var/log/nginx/error.log # exit

$ kubectl cp code-red/nginx:error-log.tar error-log.tar

$ ls
error-log.tar
```

File to be copied

Namespace

Accessing Container Logs

Renders logs produced by application or process running in a container

```
$ kubectl logs hazelcast
...
May 25, 2020 3:36:26 PM com.hazelcast.core.LifecycleService
INFO: [10.1.0.46]:5701 [dev] [4.0.1] [10.1.0.46]:5701 is STARTED
```

- You can stream the logs with the command line option `-f`. This option is helpful if you want to see logs in real time.
- You can still get back to the logs of the previous container by adding the `-p` command line option in case the container crashes or is restarted.

Executing a Command in a Container

Interactive exploration of a container, or seeing the direct effect of a command

```
$ kubectl exec -it hazelcast -- /bin/sh
# ...

$ kubectl exec hazelcast -- env
...
DNS_DOMAIN=cluster
```

- The command line option `-it` opens an interactive shell. This is useful if you want to inspect the configuration of your application or debug the current state of your application.

- For direct execution of a command in the container simply append to `--`

Declaring Command and Args

Assign or overwrite container entrypoint and arguments

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: spring-boot-app
spec:
  containers:
  - image: bmuschko/spring-boot-app:1.5.3
    name: spring-boot-app
    command: ["/bin/sh"]
    args: ["-c", "while true; do date; sleep 10; done"]
```

Declaring Command and Args

Assign or overwrite container entrypoint and arguments

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: sample-pod-1
spec:
  containers:
  - name: sample-pod-1
    image: python:3.6-alpine
    command: ["sleep"]
    args: ["5"]
```

Namespace YAML Manifest

"Organize objects in namespace called `code-red`"

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: code-red
```

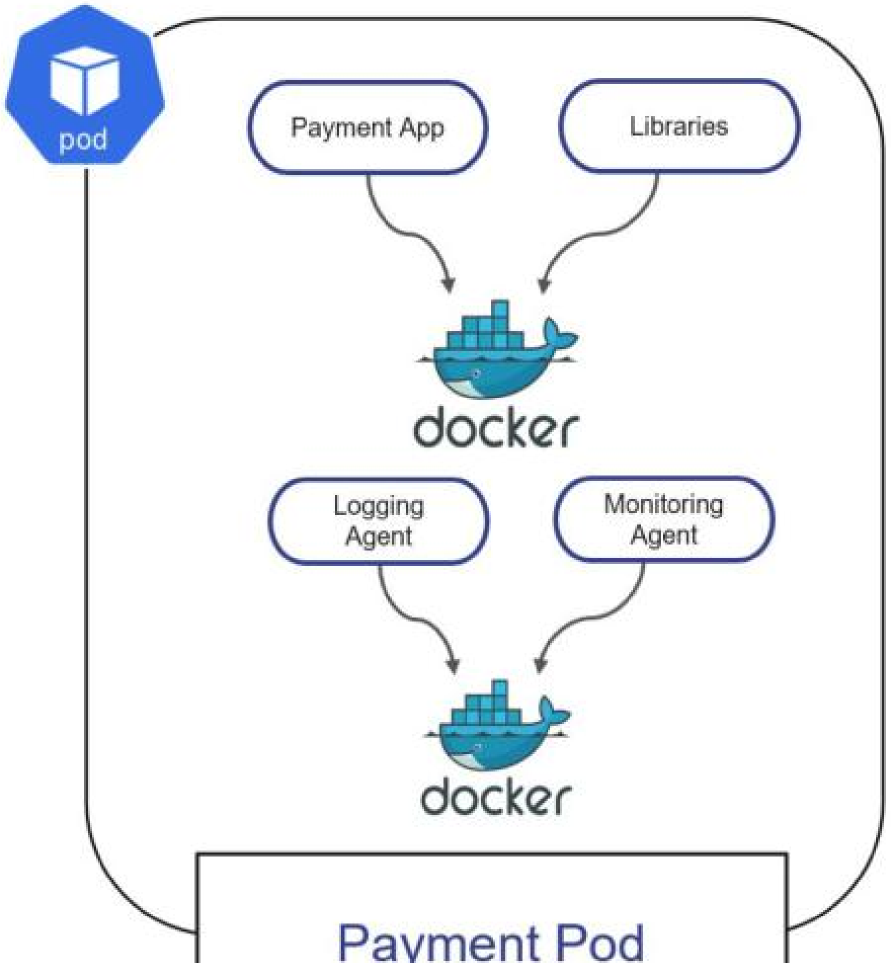
Deleting a Namespace

Cascade-deletes containing objects

```
$ kubectl delete namespace code-red
namespace "code-red" deleted

$ kubectl get pods -n code-red
No resources found in code-red namespace.
```

What is a Pod?

The smallest, most basic deployable objects in Kubernetes.



Lunch Break

1 hour

TOPIC 3

Jobs & CronJobs

Run-to-completion & scheduled batch workloads

RUN TO COMPLETION

Batch Workloads in Kubernetes

- A Job runs Pods until a set number of successful completions, then stops — it does not restart on success.
- A CronJob creates Jobs on a schedule using standard cron syntax (*/1 * * * * = every minute).
- Job Pod templates must use restartPolicy: Never or OnFailure — never Always.
- Trigger a CronJob now with: kubectl create job --from=cronjob/<name> <job-name>.

EXAM-TESTED EVERY SITTING

Key Fields to Memorise

Job

- completions — successes needed
- parallelism — Pods at once
- backoffLimit — failures allowed

Job (timing)

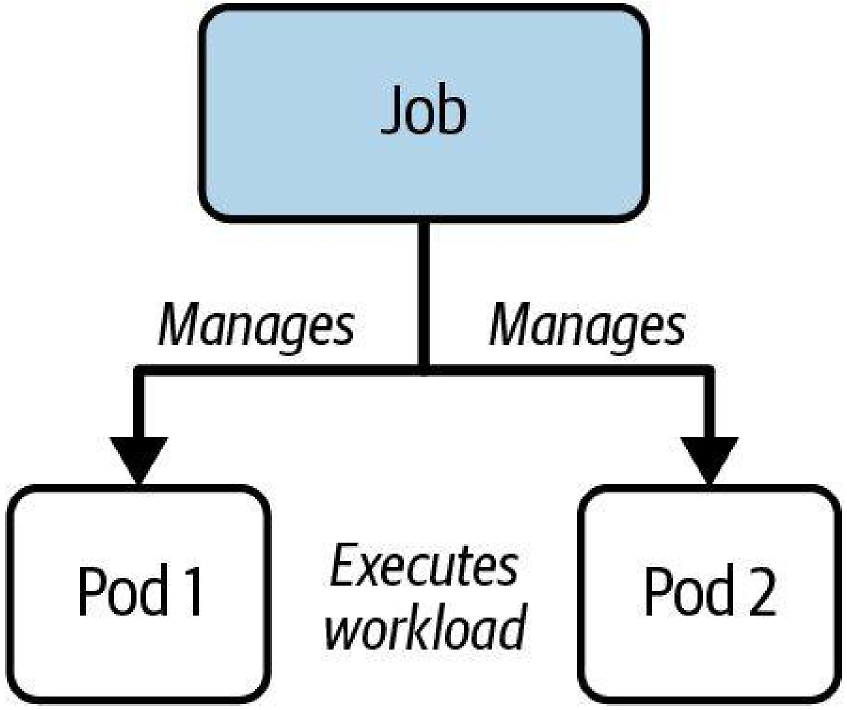
- activeDeadlineSeconds
- Hard wall-clock ceiling
- Overrides backoffLimit

CronJob

- schedule + timeZone
- concurrencyPolicy: Allow/Forbid/Replace
- successful/failedJobsHistoryLimit

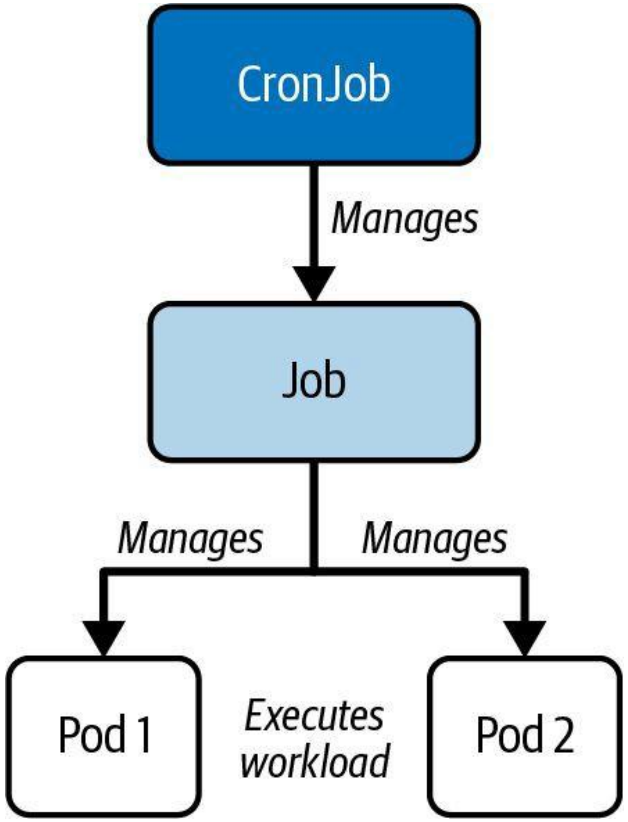
Job Object Hierarchy

A Job manages Pods to run the workload



CronJob Object Hierarchy

A CronJob manages Jobs which delegate the workload to Pods



Jobs — Run-to-Completion Workloads

LAB 4

Batch workloads

A Job runs Pods until a required number of successful completions is reached, then stops. CKAD tests completions, parallelism, backoffLimit and activeDeadlineSeconds in almost every sitting — you must write a Job manifest from memory.

You'll build

Create a one-shot Job, a parallel Job with completions, and explore backoffLimit and activeDeadlineSeconds.

Key commands: k create · apiVersion: batch/v1 · k apply · » `restartPolicy:

LAB 4 · COMMANDS

Jobs — Run-to-Completion Workloads

Step 1 — One-shot Job (imperative)

```
k create job hello --image=busybox -- echo "hello CKAD 2026"
k logs -l job-name=hello
```

Jobs — Run-to-Completion Workloads (cont.)

Step 2 — Parallel Job with multiple completions

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: pi-parallel
spec:
  completions: 5
  parallelism: 2
  backoffLimit: 4
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: pi
        image: perl:5.34
        command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(50)"]
```

Jobs — Run-to-Completion Workloads (cont.)

Step 3 — Apply and watch completions climb

```
k apply -f parallel.yaml
k get job pi-parallel -w        # COMPLETIONS ticks 0/5 → 5/5
```

Note restartPolicy: Never is mandatory in Job Pod templates. activeDeadlineSeconds is a hard wall-clock ceiling — it overrides backoffLimit.

Jobs — Run-to-Completion Workloads

Test it

k get job pi-parallel reaches 5/5 completions with no more than two Pods running simultaneously during the run.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

What is a Job?

Kubernetes primitive for executing a one-time operation

- It runs a specific task until a specified number of completions has been reached.
- The actual work managed by a Job is still running inside of a Pod. Therefore, you can think of a Job as a higher-level coordination instance for Pods executing the workload.
- Upon completion of a Job and its Pods, Kubernetes does not automatically delete the objects - they will stay until they're explicitly deleted. Keeping those objects helps with debugging the command run inside of the Pod and gives you a chance to inspect the logs.

Job Operation Types

Execution runtime behavior of workload can be fine-tuned

- The default behavior of a Job is to run the workload in a single Pod and expect one successful completion (non-parallel Job).
- The attribute `spec.completions` controls the number of required successful completion.
- The attribute `spec.parallelism` allows for executing the workload by multiple pods in parallel.

Job Operation Types

Pick combination of completions and parallelism based on workload use case

Type	Completions	Parallelism	Description
Non-parallel with one completion count	1	1	Completes as soon as its Pod terminates successfully.
Parallel with a fixed completion count	>= 1	>= 1	Completes when specified number of tasks finish successfully.
Parallel with worker queue	unset	>= 1	Completes when at least one Pod has terminated successfully and all Pods are terminated.

Restart Behavior

Re-running workload if execution is unsuccessful

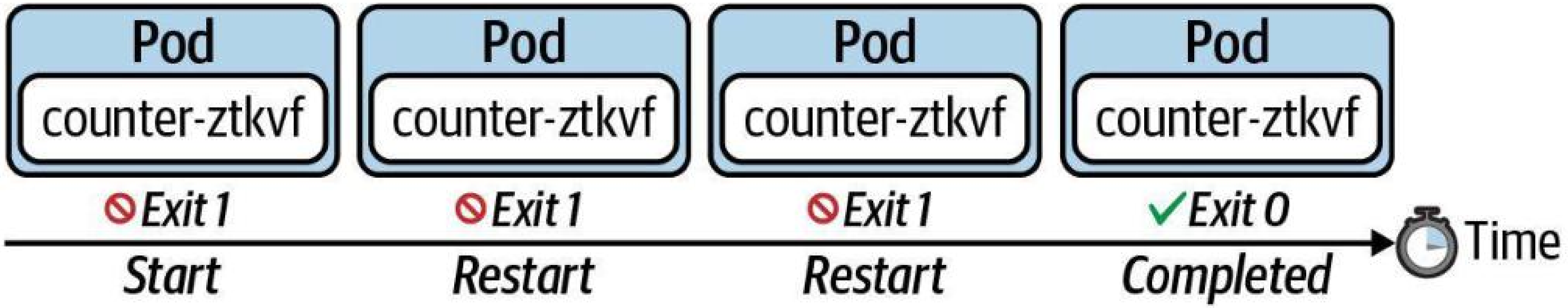
- The attribute `spec.backoffLimit` determines the number of retries a Job attempts to successfully complete the workload until the executed command finishes with an exit code 0. The default is 6, which means it will execute the workload 6 times before the Job is considered unsuccessful.

- The attribute `spec.template.spec.restartPolicy` needs to be declared explicitly. The restart policy of a Job can only be `OnFailure` or `Never`.

Restarting a Container on Failure

Restart container if it exits with a non-zero exit code

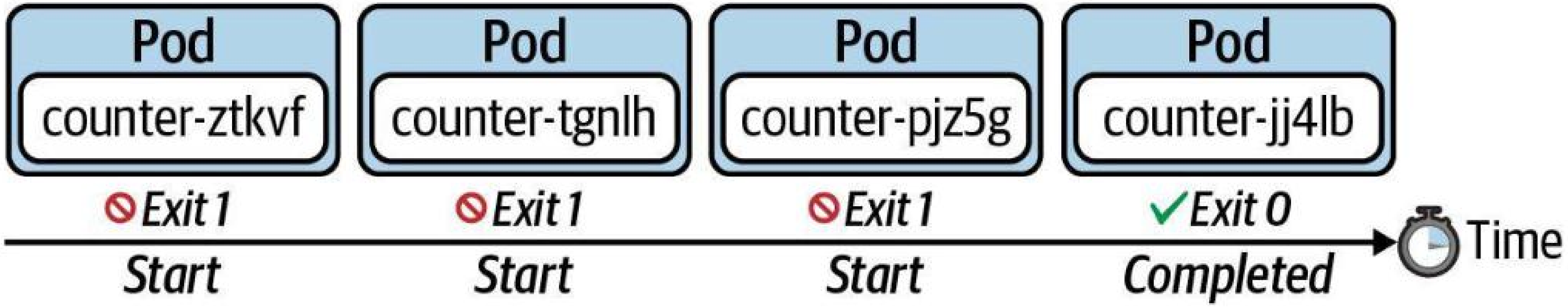
`spec.template.spec.restartPolicy: OnFailure`



Starting a New Pod on Failure

The policy does not restart the container upon a failure, it starts a new Pod.

`spec.template.spec.restartPolicy: Never`



CronJob YAML Manifest

"Increment a counter and render its value on the terminal"

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: counter
spec:
  schedule: "*/1 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: Never
          containers:
          - args:
            - /bin/sh
            - -c
            - ...
            image: nginx:1.24.0
```

The crontab expression used to run CronJob periodically

Run in a new Pod

CronJob YAML Manifest

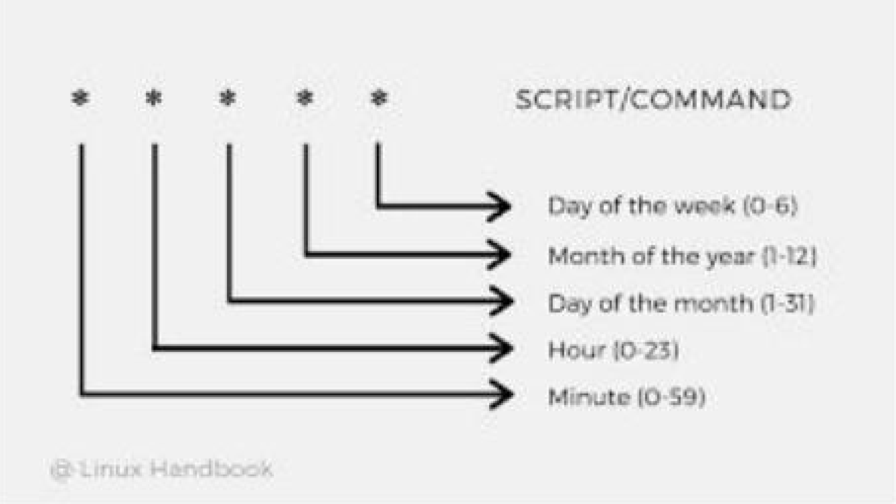
job-definition.yaml

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: calculate-pod-job
spec:
  completions: 3
  parallelism: 3
  template:
   spec:
     containers:
     - image: ubuntu
       name: calculate-pod
       command: [‘expr’,’2’, ‘+’,’1’]
  restartPolicy: Never
```

cron-job-definition.yaml

```yaml
apiVersion: batch/v1beta1
kind: CronJob
metadata:
  name: calculate-pod-cron-job
spec:
  schedule: “*/1 * * * *”
  jobTemplate:
   spec:
    spec:
  completions: 3
  parallelism: 3
  template:
   spec:
     containers:
     - image: ubuntu
       name: calculate-pod
       command: [‘expr’,’2’, ‘+’,’1’]
  restartPolicy: Never
```

CronJob YAML Manifest



cron-job-definition.yaml

```
apiVersion: batch/v1beta1
kind: CronJob
metadata:
 name: calculate-pod-cron-job
spec:
 schedule: “*/1 * * * *”
 jobTemplate:
  spec:
   spec:
 completions: 3
 parallelism: 3
 template:
  spec:
   containers:
   - image: ubuntu
    name: calculate-pod
    command: [‘expr’,’2’, ‘+’,’1’]
 restartPolicy: Never
```

What is a Job?

Kubernetes primitive for executing a one-time operation

- It runs a specific task until a specified number of completions has been reached.
- The actual work managed by a Job is still running inside of a Pod. Therefore, you can think of a Job as a higher-level coordination instance for Pods executing the workload.
- Upon completion of a Job and its Pods, Kubernetes does not automatically delete the objects - they will stay until they're explicitly deleted. Keeping those objects helps with debugging the command run inside of the Pod and gives you a chance to inspect the logs.

CronJobs — Scheduled Workloads

LAB 5 Batch workloads

A CronJob creates Jobs on a time-based schedule using standard cron syntax. CKAD tests concurrencyPolicy, timeZone, startingDeadlineSeconds, history limits, manual triggers and suspend/resume — these fields appear verbatim in exam questions.

You'll build

Create a CronJob imperatively and declaratively, suspend/resume it, then trigger it manually.

Key commands: k create · apiVersion: batch/v1 · k patch · » `concurrencyPolicy`:

LAB 5 · COMMANDS

CronJobs — Scheduled Workloads

Step 1 — Create a CronJob imperatively

```
k create cronjob date-printer --image=busybox \
  --schedule="*/1 * * * *" -- sh -c "date; echo from cronjob"
k get cronjob date-printer
```

CronJobs — Scheduled Workloads (cont.)

Step 2 — Declarative CronJob with all key fields

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: report
spec:
  schedule: "*/2 * * * *"
  timeZone: "Asia/Singapore"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 2
  startingDeadlineSeconds: 30
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - name: report
            image: busybox
            command: ["sh", "-c", "echo report at $(date)"]
```

LAB 5 · COMMANDS

CronJobs — Scheduled Workloads (cont.)

Step 3 — Suspend, resume and trigger manually

```
k patch cronjob report -p '{"spec":{"suspend":true}}'    # pause
k patch cronjob report -p '{"spec":{"suspend":false}}'   # resume
k create job --from=cronjob/report report-manual           # run now
```

Note concurrencyPolicy: Allow (default) / Forbid / Replace. timeZone is a newer field — always set it. create job --from=cronjob/<name> runs the workload immediately.

CronJobs — Scheduled Workloads

Test it

k get cronjob report shows SUSPEND True after the patch, and k create job --from=cronjob/report report-manual produces an immediate one-off run.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Configuring Retained History

Being able to know what happened during workload execution retroactively

- Even after completing a task in a Pod controlled by a CronJob, it will not be deleted automatically. Keeping a historical record of Pods can be tremendously helpful for troubleshooting failed workloads or inspecting the logs.
- The attribute `spec.successfulJobsHistoryLimit` configures the number of successful executions. Defaults to 3 successful Jobs.
- The attribute `spec.failedJobsHistoryLimit` configures the number of failed executions. Defaults to 1 failed Job.

Tea Break

15 minutes

TOPIC 4

Multi-Container Pods

Sidecar & init-container design patterns

POD DESIGN PATTERNS

Why More Than One Container?

- Containers in a Pod share the network namespace and can share volumes (emptyDir is the usual glue).
- Sidecar — a helper alongside the main app (log shipper, proxy) sharing a volume.
- Init container — runs to completion before main containers (seed a volume, wait for a service).
- Always target a specific container with kubectl logs -c <name> and kubectl exec -c <name>.

ONE POD, MANY ROLES

Container Patterns

Sidecar

- Runs beside the main app
- Shares an emptyDir volume
- Log shipping / proxy / sync

Init container

- Runs first, to completion
- Pod shows Init:N/M
- Seed volume · wait for DNS

Native sidecar

- initContainer + restartPolicy: Always
- Kubernetes 1.29+
- Runs for the Pod's lifetime

Design Patterns

Emerged from Pod requirements

Init Container

- Provide initialization logic concerns to be run before the main application even starts.
- Example: Downloading a configuration files required by application.

Sidecar

- The sidecars are not part of the main traffic or API of the primary application and operate asynchronously.
- Example: Watcher capabilities.

Adapter

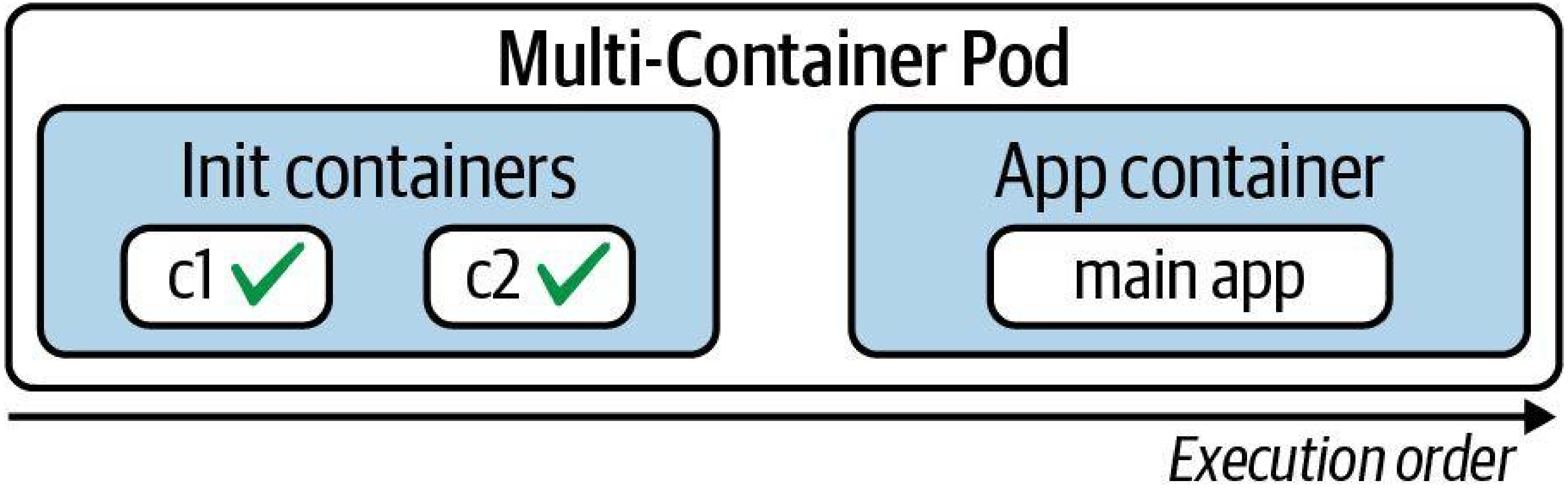
- Transforms the output produced by the application to make it consumable in the format needed by another part of the system.
- Example: Massaging log data.

Ambassador

- Provides a proxy for communicating with external services to hide and/or abstract the complexity.
- Example: Rate-limiting functionality for HTTP(S) calls to an external service.

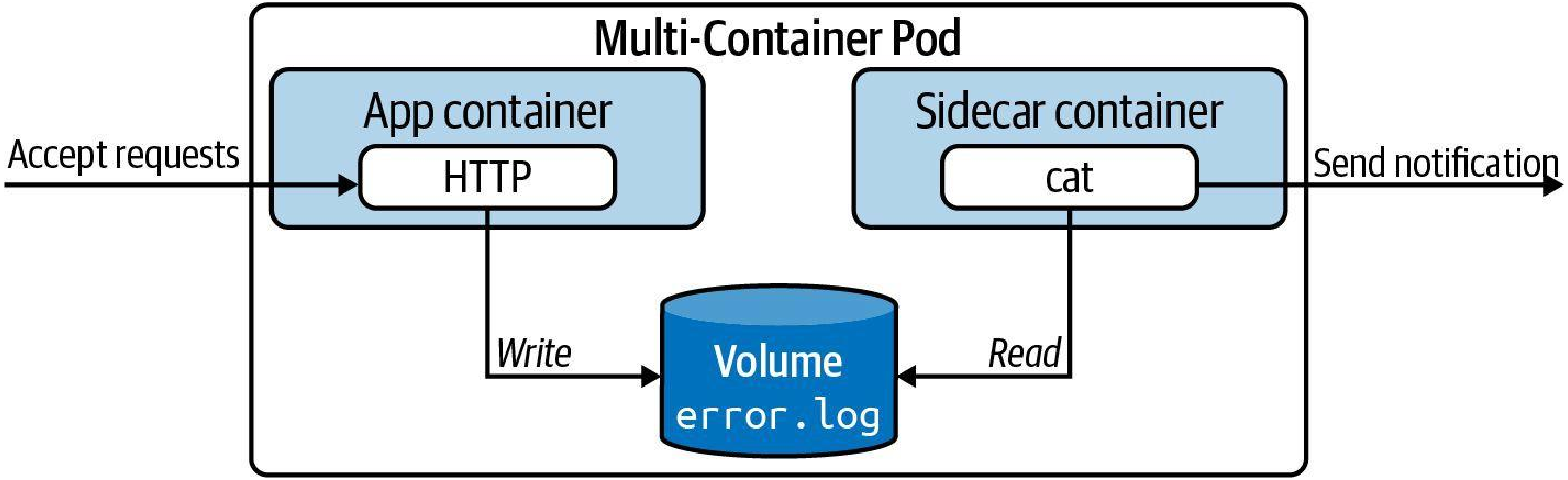
Init Container Life Cycle

Initialization logic before main application containers



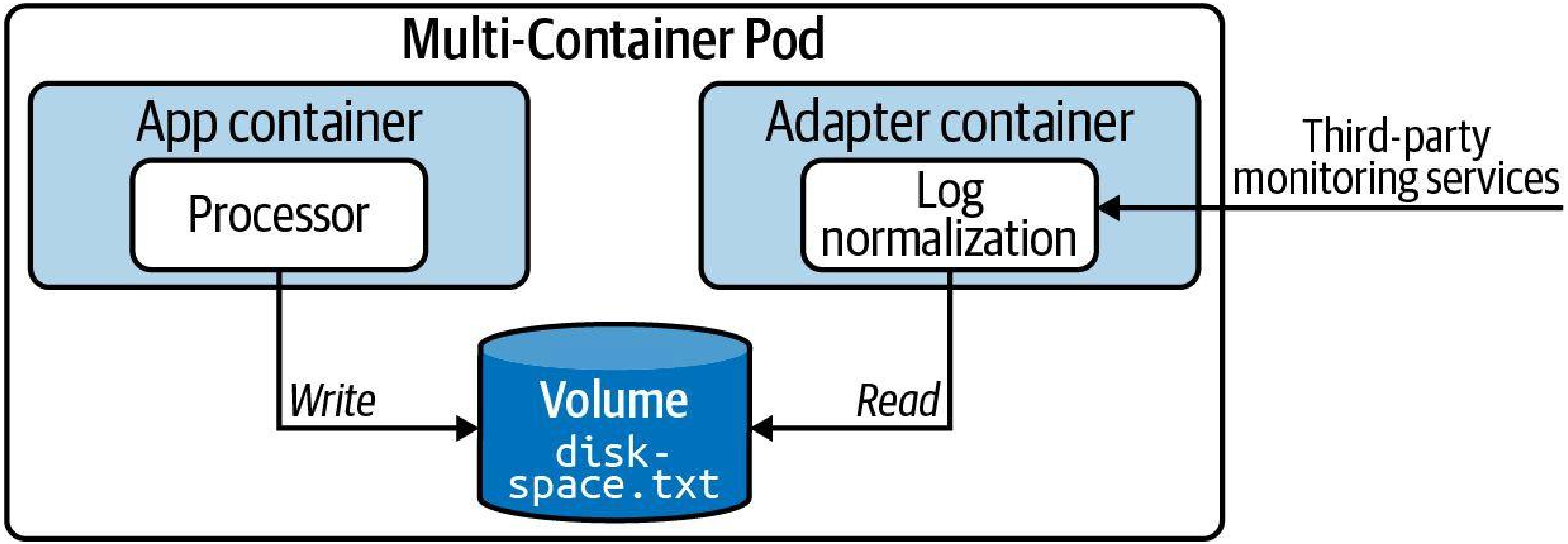
Sidecar Pattern

"Send a notification of error log contains entries"



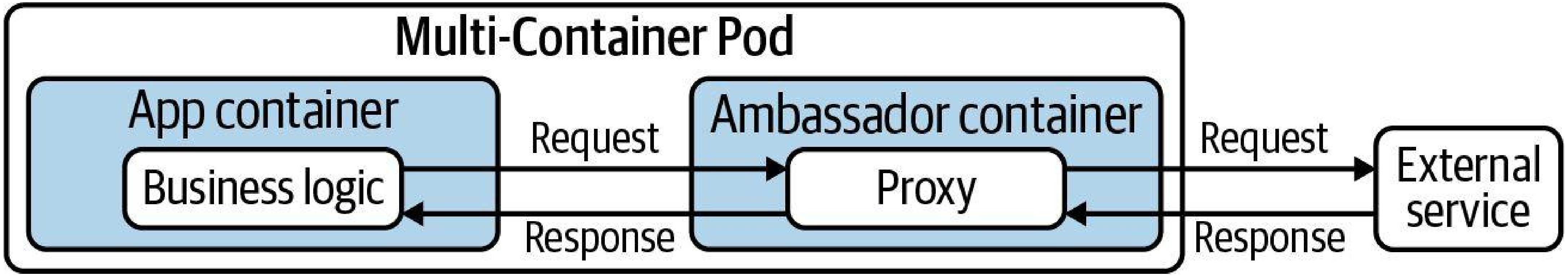
Adapter Pattern

"Transform log data to JSON to make it consumable by external system"



Ambassador Pattern

"OAuth authentication handled by network communication proxy"



Multi-Container Pods — Sidecar Pattern

LAB 6 Multi-container Pods

Containers in the same Pod share a network namespace and can share volumes. CKAD tests the sidecar pattern (a helper reads/writes a shared volume) and the native sidecar feature (Kubernetes 1.29+). You must exec into and read logs from each container.

You'll build

Run a sidecar that tails a shared emptyDir log, read each container's logs, then build a native sidecar.

Key commands: apiVersion: v1 · k logs · initContainers: · » With

Multi-Container Pods — Sidecar Pattern

Step 1 — Sidecar sharing a log file via emptyDir

```
apiVersion: v1
kind: Pod
metadata:
  name: app-with-sidecar
spec:
  volumes:
  - name: logs
    emptyDir: {}
  containers:
  - name: app
    image: busybox
    command: ["sh","-c","while true; do date >> /var/log/app.log; sleep 2; done"]
    volumeMounts:
    - {name: logs, mountPath: /var/log}
  - name: log-shipper
    image: busybox
    command: ["sh","-c","tail -F /var/log/app.log"]
    volumeMounts:
    - {name: logs, mountPath: /var/log}
```

Multi-Container Pods — Sidecar Pattern (cont.)

Step 2 — Read logs from each container separately

```
k logs app-with-sidecar -c app | head -5
k logs app-with-sidecar -c log-shipper | head -5
k exec -it app-with-sidecar -c app -- sh -c 'wc -l /var/log/app.log'
```

Step 3 — Native sidecar container (Kubernetes 1.29+)

```
initContainers:
- name: log-collector
  image: busybox
  restartPolicy: Always
  command: ["sh","-c","while true; do echo alive; sleep 5; done"]
```

Note With multiple containers in a Pod, always specify -c <name> for logs and exec. emptyDir is the standard glue volume between sidecar and main containers.

Multi-Container Pods — Sidecar Pattern

Test it

k logs app-with-sidecar -c log-shipper shows the same lines app is writing, proving both containers share the emptyDir volume.

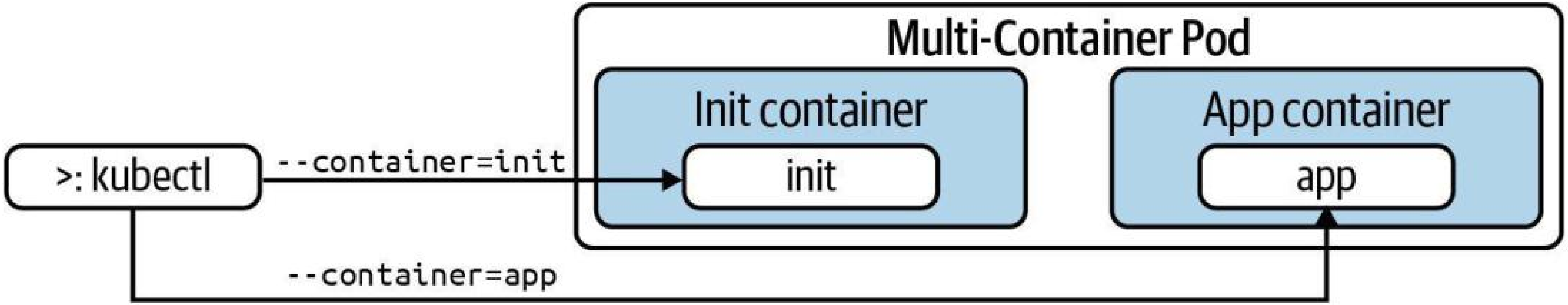
Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Multi-Container Pods

Defining and interacting with multiple containers, design patterns

Interacting with a Container

Run kubectl with `--container` or `-c` CLI option



Shelling into a Container

Defaults to main application container if not explicit

```
$ kubectl exec -it -c web-server business-app -- /bin/sh
# exit

$ kubectl exec -it business-app -- /bin/sh
Defaulted container "web-server" out of: web-server, ↵
app-initializer
(init) # exit
```

Init Containers

LAB 7

Multi-container Pods

Init containers run in order, to completion, before any main container starts. Use them to seed shared volumes, wait for upstream services, or do one-time setup. CKAD asks you to write init containers and read Init:N/M Pod status correctly.

You'll build

Seed an nginx web root with an init container, then add an init container that waits for a Service via DNS.

Key commands: apiVersion: v1 · k exec · initContainers: · » The

Init Containers

Step 1 — Init container that seeds the web root

```
apiVersion: v1
kind: Pod
metadata:
  name: web-init
spec:
  volumes:
  - name: html
    emptyDir: {}
  initContainers:
  - name: seed
    image: busybox
    command: ["sh","-c","echo '<h1>Seeded by init</h1>' > /work/index.html"]
    volumeMounts:
    - {name: html, mountPath: /work}
  containers:
  - name: web
    image: nginx:1.25
    volumeMounts:
    - {name: html, mountPath: /usr/share/nginx/html}
```

Init Containers (cont.)

Step 2 — Verify the seeded content

```
k exec web-init -- cat /usr/share/nginx/html/index.html
```

Step 3 — Init container that waits for a Service

```
initContainers:
- name: wait-for-db
  image: busybox
  command: ["sh","-c","until nslookup db.default.svc.cluster.local; do sleep 2; done"]
```

Note The Pod stays in Init:0/1 until DNS resolves. Creating the db Service unblocks it. If an init container fails it is restarted per the Pod's restartPolicy.

Init Containers

Test it

k exec web-init -- cat /usr/share/nginx/html/index.html returns the seeded heading, proving the init container populated the volume before nginx started.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

TOPIC 5

Volumes

Persist & share Pod data: emptyDir, tmpfs & hostPath

EPHEMERAL VS SHARED

Pod Storage Basics

- Anything written to the container filesystem is lost when the container restarts.
- Volumes are declared under spec.volumes and mounted via spec.containers[].volumeMounts.
- A volume can be shared by every container in the Pod — that is how sidecars exchange data.
- Pod-scoped volumes live and die with the Pod; durable storage (PV/PVC) comes later in the course.

POD-SCOPED STORAGE

Volume Types in Domain 1

emptyDir: {}

- Created with the Pod
- Shared by all containers
- Deleted with the Pod

emptyDir: Memory

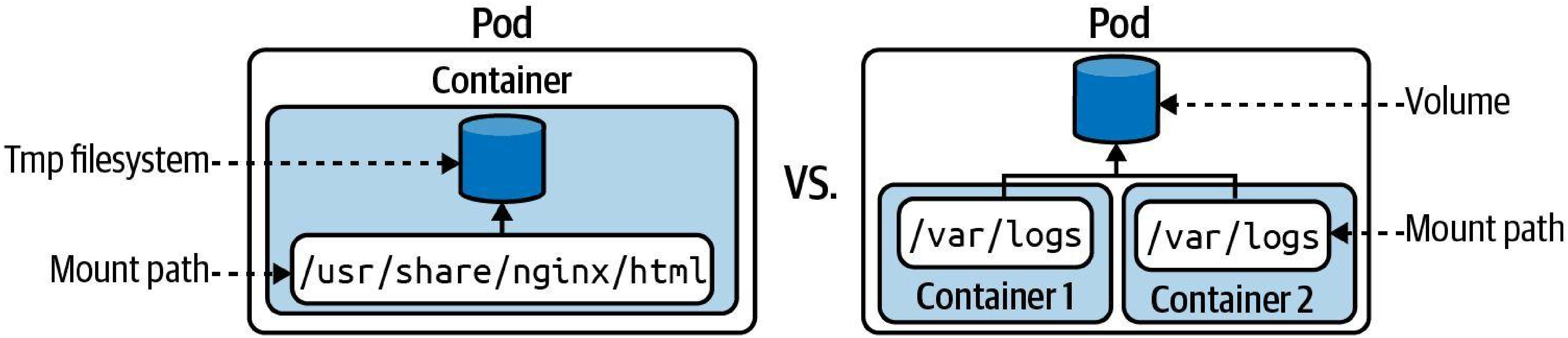
- tmpfs — backed by RAM
- Fast scratch space
- Counts to memory limit

hostPath

- Mounts a node directory
- DaemonSets / node tools only
- Security risk in multi-tenant

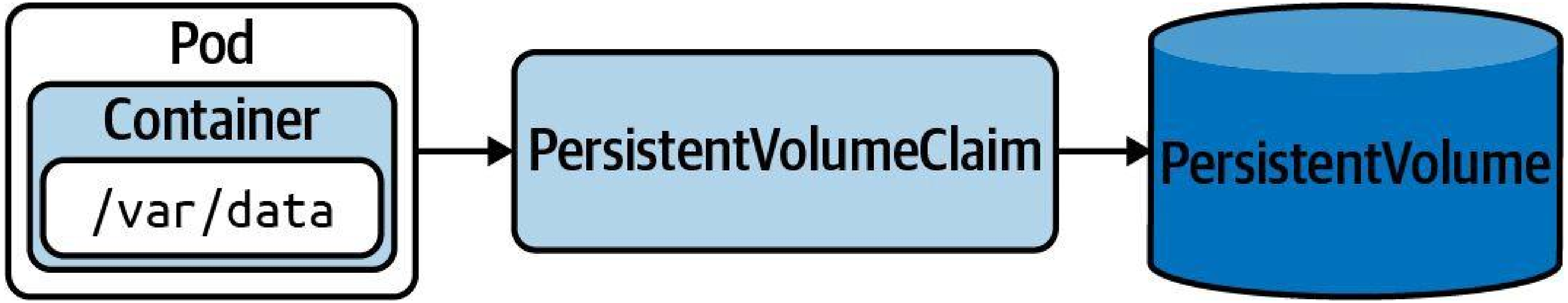
Ephemeral Volume

Useful for sharing data between containers, or for caching data



Persistent Volume

Persist data that outlives a Pod, node, or cluster restart



Volumes — emptyDir and hostPath

LAB 8 Volumes

Containers are ephemeral — data written to the container filesystem is lost on restart. Volumes survive restarts and can be shared between containers. CKAD tests emptyDir, emptyDir.medium: Memory and hostPath, plus mounting strategies.

You'll build

Share an emptyDir between two containers, mount a RAM-backed tmpfs volume, then mount a node directory with hostPath.

Key commands: volumes: · » Volumes

Volumes — emptyDir and hostPath

Step 1 — emptyDir shared between two containers

```
volumes:
- name: shared
  emptyDir: {}
containers:
- name: writer
  image: busybox
  command: ["sh","-c","echo hello-shared > /data/msg.txt; sleep 3600"]
  volumeMounts: [{name: shared, mountPath: /data}]
- name: reader
  image: busybox
  command: ["sh","-c","sleep 5; cat /data/msg.txt; sleep 3600"]
  volumeMounts: [{name: shared, mountPath: /data}]
```

Volumes — emptyDir and hostPath (cont.)

Step 2 — emptyDir backed by RAM (tmpfs)

```
volumes:
- name: fast
  emptyDir:
    medium: Memory
    sizeLimit: 64Mi
```

Step 3 — hostPath: access files on the node

```
volumes:
- name: etc
  hostPath:
    path: /etc
    type: Directory
```

Note Volumes are declared under spec.volumes and consumed via spec.containers[].volumeMounts. Avoid hostPath in multi-tenant clusters — it is a security risk; use it only for DaemonSets and node-level tools.

Volumes — emptyDir and hostPath

Test it

k logs scratch -c reader prints hello-shared, proving the writer and reader containers share the same emptyDir volume.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Volumes

Ephemeral and persistent storage for Pods

Pods are ephemeral

Can be stopped or destroyed

ephemeral

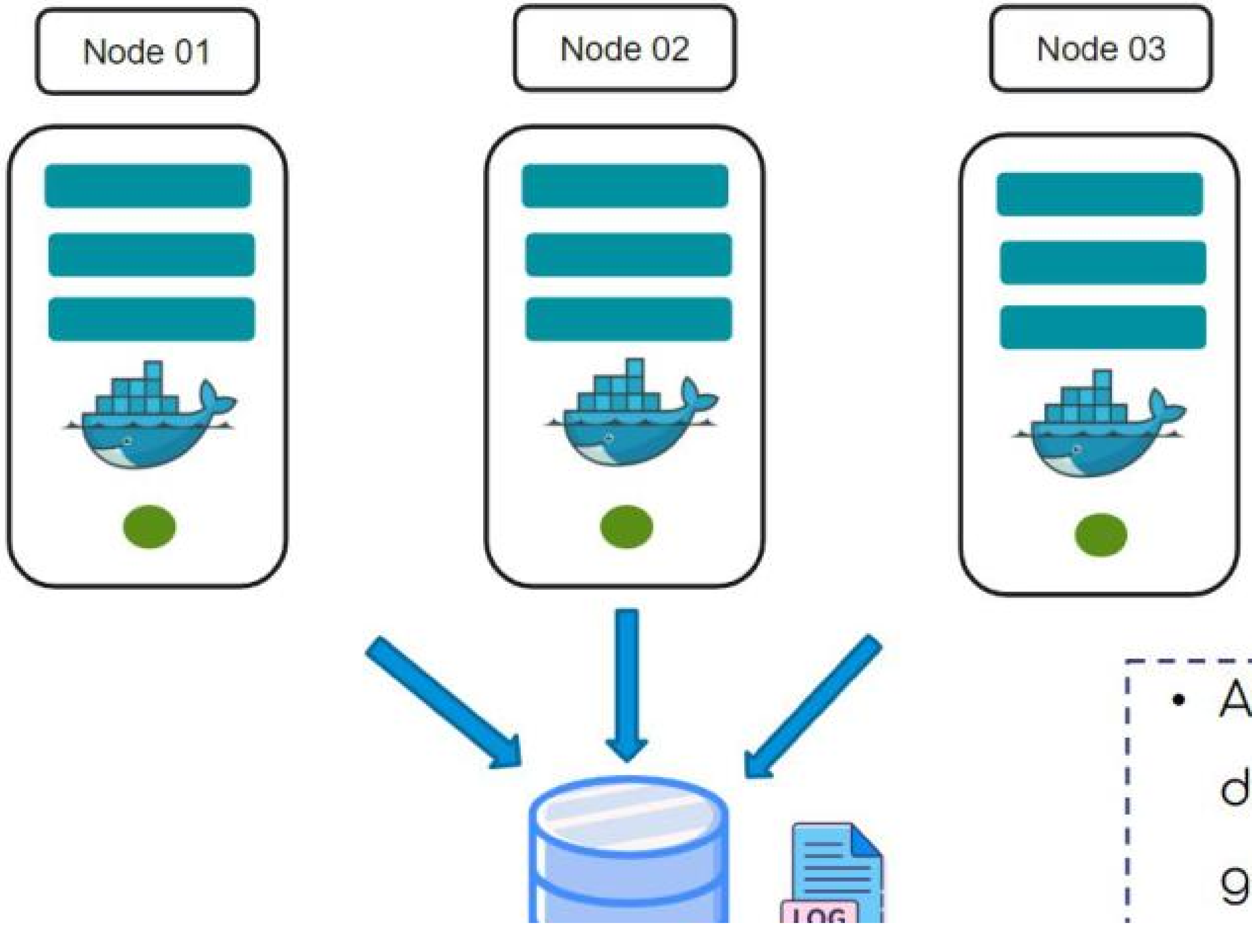
/ɪˈfɛm(ə)r(ə)l,ɪˈfiːm(ə)r(ə)l/

adjective

lasting for a very short time.
"fashions are ephemeral: new ones regularly drive out the old"

Similar: transitory | transient | fleeting | passing | short-lived | momentary

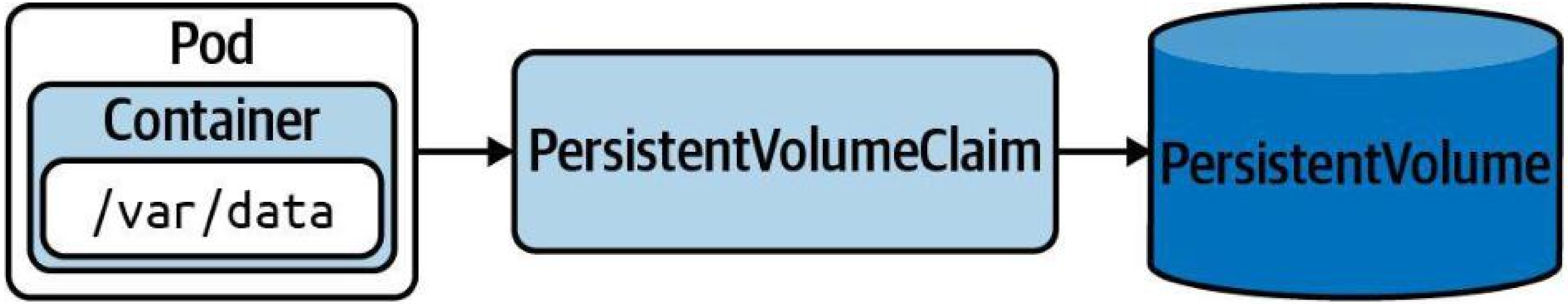
How do you store the data?



- Allows you to store data even if the pod gets destroyed

Persistent Volume

Persist data that outlives a Pod, node, or cluster restart



Static vs. Dynamic Provisioning

PersistentVolume object is created manually or automatically

- *Static Provisioning:* Storage device needs to be created first. The PersistentVolume object references the storage device and needs to be created manually.

- *Dynamic Provisioning:* The PersistentVolumeClaim object references a storage class. The PersistentVolume object is created automatically.

- *Storage Class:* Object that knows how to provision a storage device with specific performance requirements.

Defining a PersistentVolume

Define storage capacity, access mode, and host path

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0001
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /tmp/data
```

apiVersion, kind of object

Defining a PersistentVolume

Define storage capacity, access mode, and host path

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0001
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /tmp/data
```

Name of persistent volume

Defining a PersistentVolume

Define storage capacity, access mode, and host path

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0001
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /tmp/data
```

Capacity of the storage

Defining a PersistentVolume

Define storage capacity, access mode, and host path

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0001
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /tmp/data
```

- ReadWriteOnce – mounted as read write by a single node
- ReadWriteMany – mounted as read write by multiple nodes
- ReadOnlyMany – mounted as read only by many nodes

Defining a PersistentVolume

Define storage capacity, access mode, and host path

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0001
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /tmp/data
```

Volume plugin for the type of storage

Access Mode

Defines read/write capabilities of Volume

Type	Description
`ReadWriteOnce`	Read-write access by a single node.
`ReadOnlyMany`	Read-only access by many nodes.
`ReadWriteMany`	Read-write access by many nodes.
`ReadWriteOncePod`	Read/write access mounted by a single Pod.

Creating a PersistentVolume

Summarized most important configuration and status

```
$ kubectl create -f db-pv.yaml
persistentvolume/db-pv
created
$ kubectl get pv db-pv
NAME     CAPACITY   ACCESS MODES    RECLAIM POLICY   STATUS      ...
db-pv    1Gi        RWO             Retain           Available   ...
```

The object hasn't been consumed by a claim yet

Reclaim Policy

What should happen to PersistentVolume when Claim is deleted?

Type	Description
`Retain`	Default. When PVC is deleted, PV is “released” and can be reclaimed.
`Delete`	Deletion removes PV and associated storage.
`Recycle`	*Deprecated*. Use dynamic binding instead.

Mounting a PersistentVolumeClaim in a Pod

Use Volume type `persistentVolumeClaim`

```
apiVersion: v1
kind: Pod
metadata:
  name: app-consuming-pvc
spec:
  volumes:
    - name: app-storage
      persistentVolumeClaim:
        claimName: db-pvc
  containers:
  - image: alpine
    ...
    volumeMounts:
      - mountPath:
        "/mnt/data" name:
        app-storage
```

Reference the Volume by claim name

Assigning a StorageClass for Dynamic Provisioning

Creates PersistentVolume object automatically via storage class

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: aws-ebs
provisioner: kubernetes.io/aws-ebs
```

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 256Mi
  storageClassName: aws-ebs
```

Helm

Deploying an application defined by a set of YAML manifests

Bundling the Template Files into a Chart Archive File

Archive file is usually published to a chart repository for consumption

```
$ helm package .
Successfully packaged chart and saved it to:
/Users/bmuschko/helm/web-app-2.5.4.tgz

$ ls
web-app-2.5.4.tgz
```

Kustomize

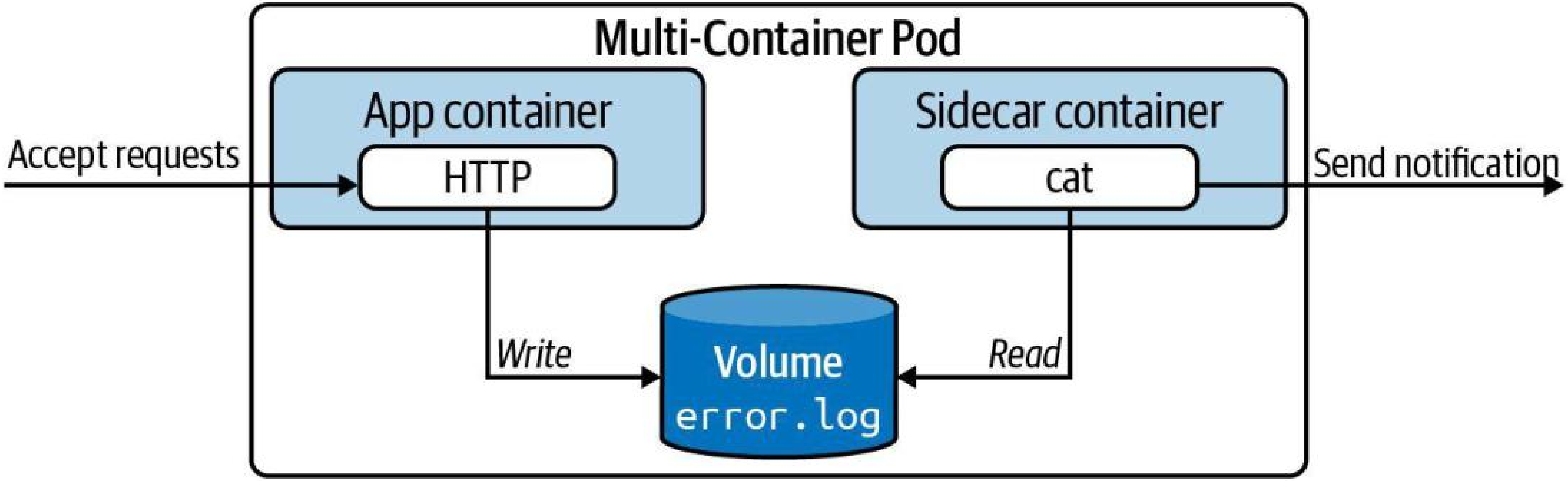
Template-free customization of application configuration

Probes

Readiness, liveness, and startup probes

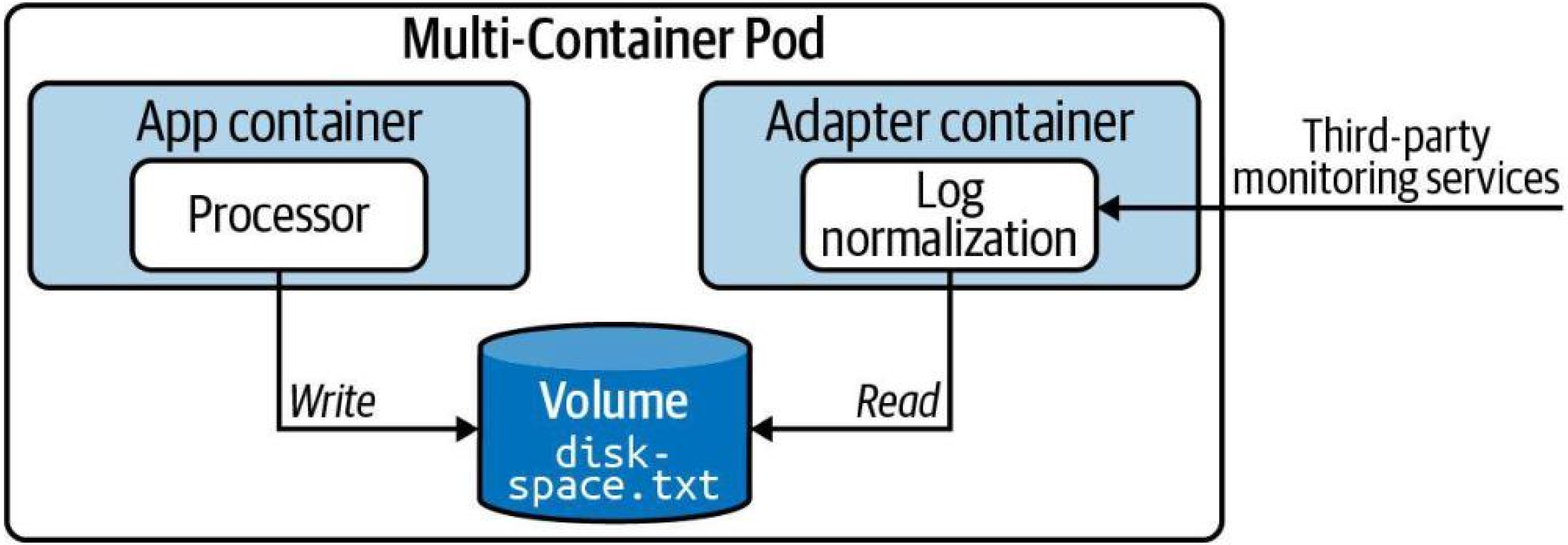
Sidecar Pattern

"Send a notification of error log contains entries"



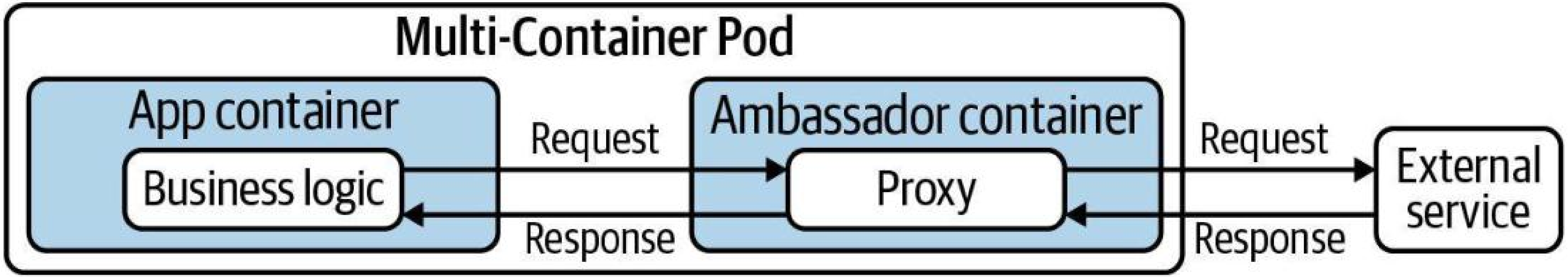
Adapter Pattern

"Transform log data to JSON to make it consumable by external system"



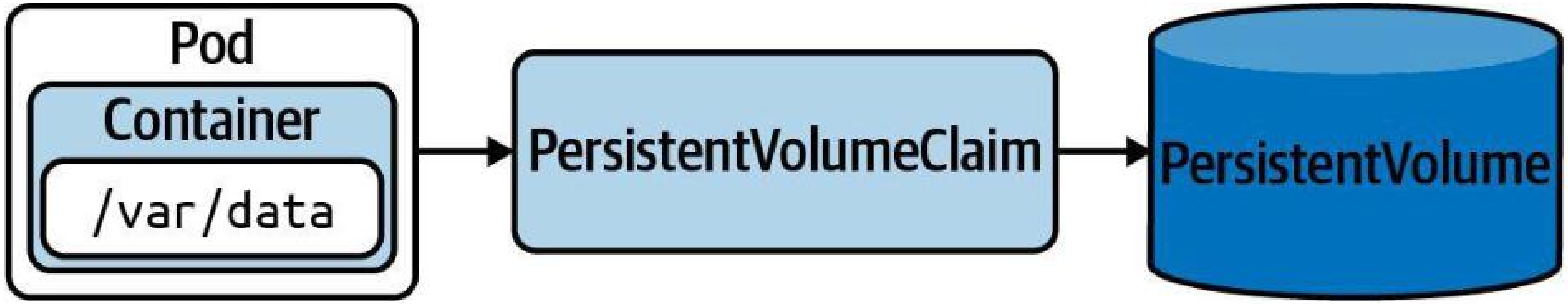
Ambassador Pattern

"OAuth authentication handled by network communication proxy"



Persistent Volume

Persist data that outlives a Pod, node, or cluster restart



WRAP-UP

Day 1 done — you can design and build applications.

Tomorrow: Application Deployment — Deployments, rollouts, Helm & Kustomize.

COURSE SLIDES

Certified Kubernetes Application Developer (CKAD)

WSQ Course Code: TGS-2025053212

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

02

DAY 2

Application Deployment

Deployments · Rollouts · Blue/Green & Canary · Helm · Kustomize · Daemon/StatefulSet (CKAD Domain 2 — 20%)

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your phone camera and submit your attendance.
- A minimum of 75% attendance is required for assessment and funding.

Minimum 75% attendance required

DAY 2 OBJECTIVES

Recap & Today

- Yesterday (Domain 1): images, Pods, Jobs/CronJobs, multi-container Pods and volumes.
- Today (Domain 2 — 20% of the exam): how applications are deployed, scaled and upgraded.
- From a single Pod to controllers that keep replicas running and roll out new versions safely.
- Packaging with Helm and Kustomize, plus DaemonSets and StatefulSets for specialised workloads.

KUBERNETES PRIMITIVES

Labels and Annotations

Label assignment and selection, providing meta information to objects

METADATA · KEY-VALUE PAIRS

Labels and Annotations

Labels

- Key-value pairs assigned to an object
- Must follow naming conventions
- Used to query and filter objects from the CLI or from other objects

Annotations

- Represents human-readable metadata for an object
- Not meant for querying purposes
- Reserved annotations can add runtime behaviour to objects

What is a Label?

- Key-value pairs assigned to Kubernetes objects with the imperative label command or using the metadata.labels[] attribute.
- Not meant for elaborate, multi-word descriptions of its functionality.
- Kubernetes limits the length of a label to a maximum of 63 characters and a range of allowed alphanumeric and separator characters.

Example Labels

- Three different Pods, each with multiple label key-value pairs:
  - Pod 1: tier=backend, env=prod, app=miracle
  - Pod 2: tier=frontend, env=dev, app=crawler
  - Pod 3: tier=frontend, env=staging, version=v2.1
- Labels are set at creation time or added later with kubectl label.
- Multiple labels on the same object create a unique identity for filtering.

Assigning Labels with Imperative Approach

Assign multiple labels to a Pod

```
kubectl label pod nginx tier=backend env=prod app=miracle
pod/nginx labeled
```

List Pods with their labels

```
kubectl get pods --show-labels
NAME    ...LABELS
nginx   ...tier=backend,env=prod,app=miracle
```

Remove a label by appending '-' to its key

```
kubectl label pod nginx tier-
pod/nginx labeled
```

LABELS · DECLARATIVE

Assigning Labels in a YAML Manifest

Pod manifest with three label key-value pairs

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod1
  labels:
    tier: backend
    env: prod
    app: miracle
spec:
  ...
```

LABELS · NAMING CONVENTIONS (app.kubernetes.io/*)

Recommended Labels

Well-known label keys with the app.kubernetes.io prefix

```yaml
metadata:
  labels:
    app.kubernetes.io/name: wordpress
    app.kubernetes.io/instance: wordpress-abcxzy
    app.kubernetes.io/version: "4.9.4"
    app.kubernetes.io/managed-by: helm
    app.kubernetes.io/component: server
    app.kubernetes.io/part-of: wordpress
```

Selecting Labels

- A label selector uses a set of criteria to query for Kubernetes objects.
- Equality-based requirements — match objects with a specific key or key=value:
  - tier=frontend (equal) • tier!=frontend (not equal)
- Set-based requirements — match objects where a key's value is in a set:
  - 'tier in (frontend, backend)' • 'version notin (v1)'
- Multiple selectors are ANDed together.

Label Selection from the CLI

Equality-based: Pods with tier=frontend AND env=dev

```
kubectl get pods -l tier=frontend,env=dev --show-labels
NAME    ...LABELS
pod2    ...app=crawler,env=dev,tier=frontend
```

Key-only: Pods that have a 'version' label (any value)

```
kubectl get pods -l version --show-labels
NAME    ...LABELS
pod3    ...app=crawler,env=staging,version=v2.1
```

Set-based with Boolean OR

```
kubectl get pods -l 'tier in (frontend,backend),env=dev' --show-labels
```

Label Selection in YAML

A NetworkPolicy selects Pods via podSelector.matchLabels

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: frontend-network-policy
spec:
  podSelector:
    matchLabels:
      tier: frontend
  ...
```

What is an Annotation?

- Key-value pairs for providing descriptive, human-readable metadata to Kubernetes objects via the metadata.annotations[] attribute.
- Cannot be used for querying or selecting objects.
- Make sure to put the value of an annotation into single- or double-quotes if it contains special characters or spaces.

Example Annotations

- A Pod can carry any number of annotation key-value pairs:
  - commit: 866a8dc
  - author: 'Benjamin Muschko'
  - branch: 'bm/bugfix'
- Unlike labels, annotation values are unrestricted in length.
- Annotations are visible via kubectl describe and kubectl get -o yaml.

Assigning Annotations with Imperative Approach

Assign annotations to a Pod

```
kubectl annotate pod nginx commit='866a8dc' branch='bm/bugfix'
pod/nginx annotated
```

Inspect annotations via describe

```
kubectl describe pods nginx
Annotations:  branch: bm/bugfix
              commit: 866a8dc
```

Remove an annotation by appending '-' to its key

```
kubectl annotate pod nginx commit-
pod/nginx annotated
```

ANNOTATIONS · DECLARATIVE

Assigning Annotations in a YAML Manifest

Pod manifest with three annotation key-value pairs

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod1
  annotations:
    commit: 866a8dc
    author: 'Benjamin Muschko'
    branch: 'bm/bugfix'
spec:
  ...
```

ANNOTATIONS · RESERVED KEYS

Well-Known Annotations

Reserved annotations may influence runtime behaviour (e.g. Pod Security Standards)

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: secured
  annotations:
    kubernetes.io/description: "Manages objects for business app."
    pod-security.kubernetes.io/warn: "baseline"
```

EXERCISE

Using Labels and Annotations

Exam Essentials

- Labels are key-value pairs assigned to Kubernetes objects. Values must follow naming and length restrictions (max 63 chars, alphanumeric + separators).
- Primitives such as Deployments, Services, and NetworkPolicies use label selection for filtering, querying, and routing.
- Use annotations to provide human-readable metadata that cannot be queried. Some reserved annotations influence runtime behaviour (e.g. Pod Security Standards at namespace level).

Well-Known Annotations

Reserved annotations that may influence runtime treatment

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: secured
  annotations:
    kubernetes.io/description: "Manages objects for business app."
                                    "baseline"
```

Enforces Pod
Security Standards
at the namespace

TOPIC 1

Deployments & ReplicaSets

The controller that keeps your Pods running

DESIRED STATE

Why Deployments?

- A bare Pod is not rescheduled if it dies — you need a controller to keep the desired count.
- A Deployment manages a ReplicaSet, which manages the Pods — a three-tier ownership chain.
- kubectl scale changes the replica count; the ReplicaSet controller reconciles immediately.
- Old ReplicaSets are kept for rollback (revisionHistoryLimit, default 10).

DEPLOYMENT → REPLICASET → POD

The Ownership Chain

Deployment

- You declare desired state
- replicas + Pod template
- Owns one active ReplicaSet

ReplicaSet

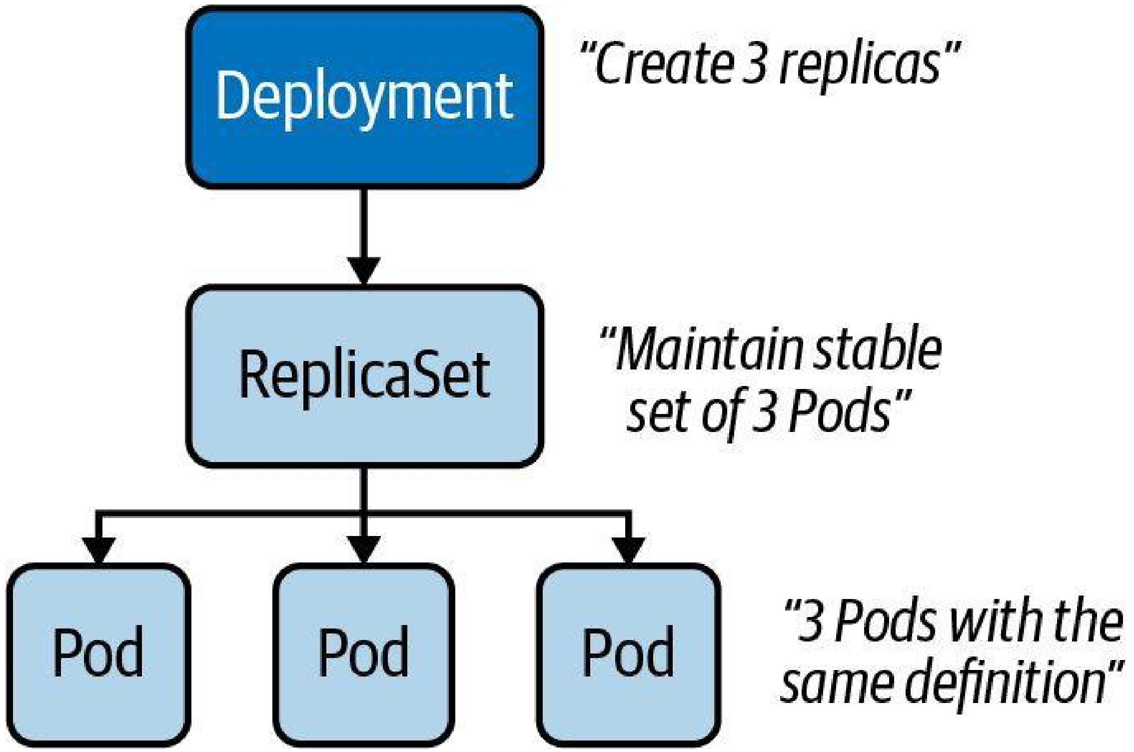
- Keeps N Pods running
- Hash-suffixed name
- Self-heals deleted Pods

Pod

- The running workload
- Recreated if it dies
- Never edited directly

Example Deployment

A Deployment that controls three replicas



Deployments and ReplicaSets

LAB 9 Deployments & ReplicaSets

A Deployment manages a ReplicaSet, which manages Pods. CKAD tests the full ownership chain, scaling, environment-variable injection and reading ReplicaSet history — required to debug failed rollouts.

You'll build

Create a Deployment imperatively, scale it, generate YAML, inject env vars, then read its ReplicaSet history.

Key commands: k create · k scale · k set · » `kubectl

Deployments and ReplicaSets

Step 1 — Create a Deployment imperatively

```
k create deployment web --image=nginx:1.25 --replicas=3
k get deploy,rs,pod -l app=web
```

Step 2 — Scale up and down

```
k scale deployment web --replicas=5
k scale deployment web --replicas=2
k get pods -l app=web
```

Step 3 — Generate Deployment YAML and apply

```
k create deployment api --image=httpd:2.4 --replicas=2 $do > api.yaml
k apply -f api.yaml
```

LAB 9 · COMMANDS

Deployments and ReplicaSets (cont.)

Step 4 — Inject an env var and observe ReplicaSet history

```
k set env deployment/api APP_COLOR=blue
k get rs -l app=api    # old RS 0/0/0, new RS 2/2/2
```

Note kubectl set env / set image mutate the Pod template and trigger a new ReplicaSet (a rolling update). Old ReplicaSets are kept for rollback — revisionHistoryLimit (default 10) controls how many.

Deployments and ReplicaSets

Test it

k get deploy,rs,pod -l app=web shows one Deployment owning one ReplicaSet owning the Pods, and k get rs -l app=api shows two ReplicaSets after the env change.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Creating a Deployment with Imperative Approach

Creates objects for the Deployment, ReplicaSet and Pod(s)

```
kubectl create deployment deployment-1 --image=nginx:1.18.0
```

Create command

Deployment name

Image

Deployment Template Attributes

Replica configuration is defined under `spec.template`

Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: my-deploy
  name: my-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      tier: backend
  template:
    metadata:
      labels:
        tier: backend
    spec:
      containers:
      - image: nginx:1.14.2
        name: nginx
```

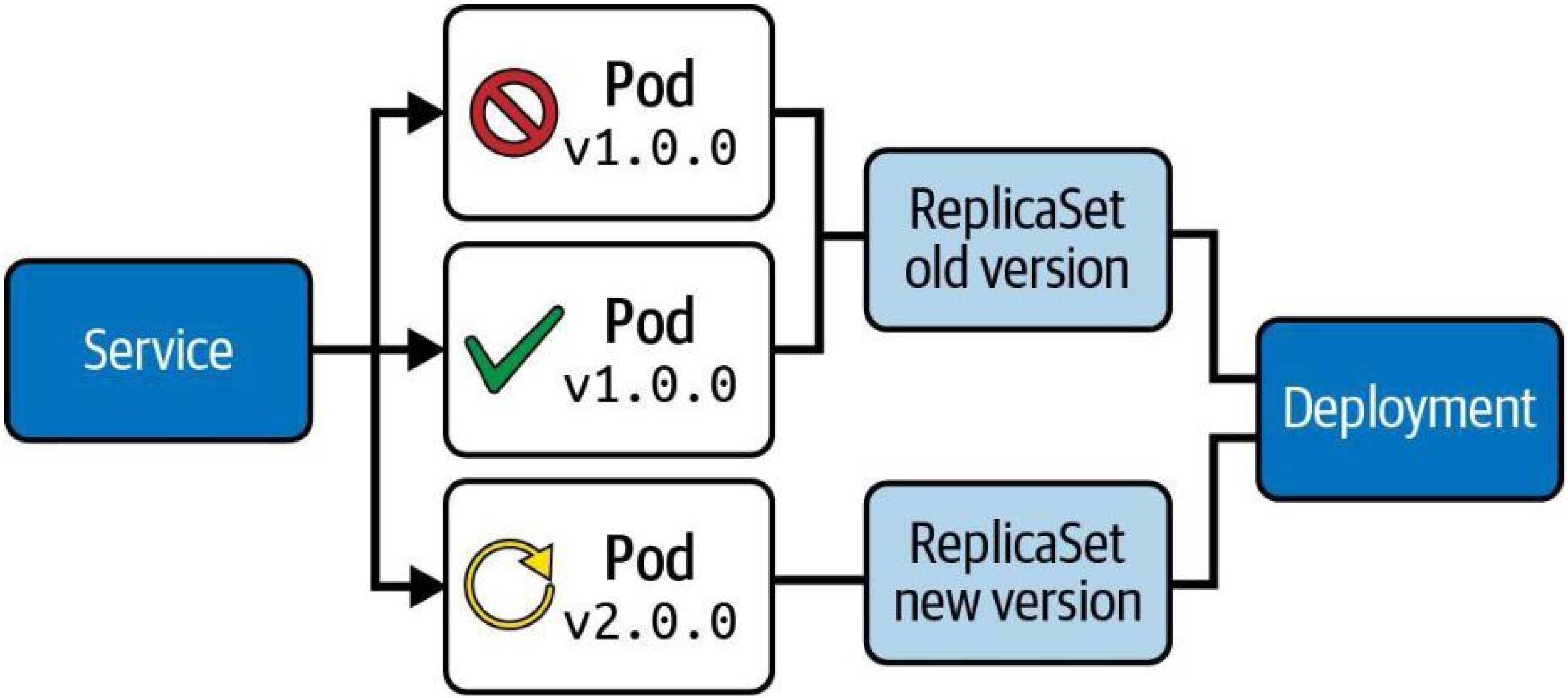
Template spec attributes are the same as for a Pod spec

Pod

```yaml
apiVersion:v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - image: nginx:1.14.2
    name: nginx
```

Rolling Update Deployment Strategy

Secondary ReplicaSet is created with new version of application



Rolling Update Deployment Strategy

Runtime behavior can be fine-tuned

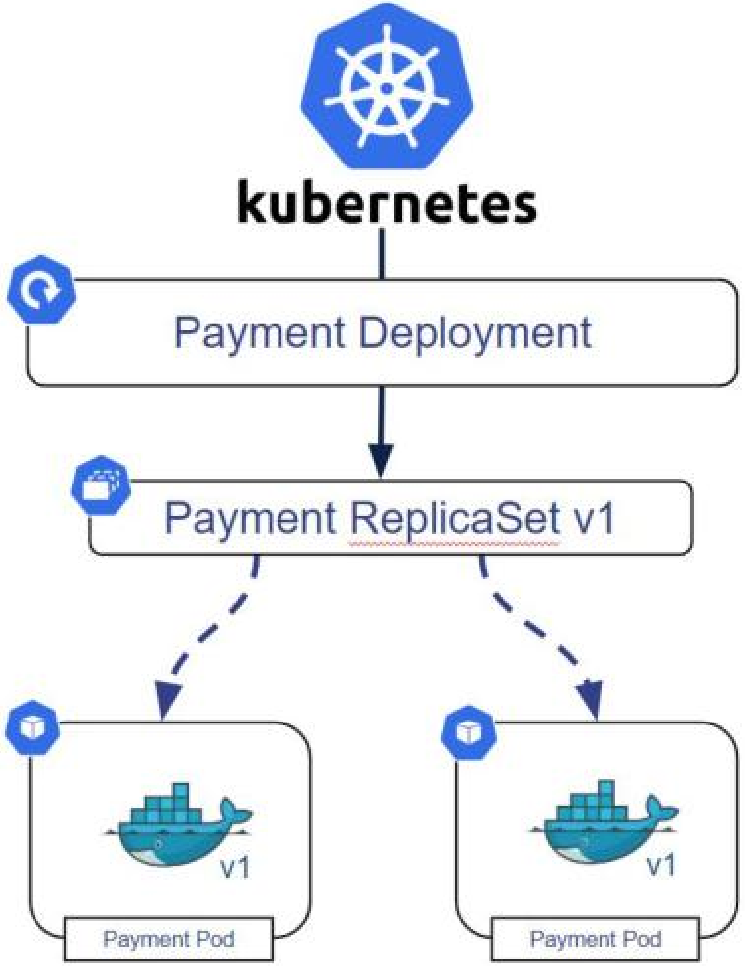
```yaml
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 2
      maxUnavailable: 0
```

Determines how update should be performed

- *Pros:* New version is slowly distributed over time, no downtime
- *Cons:* Breaking API changes can make consumers incompatible

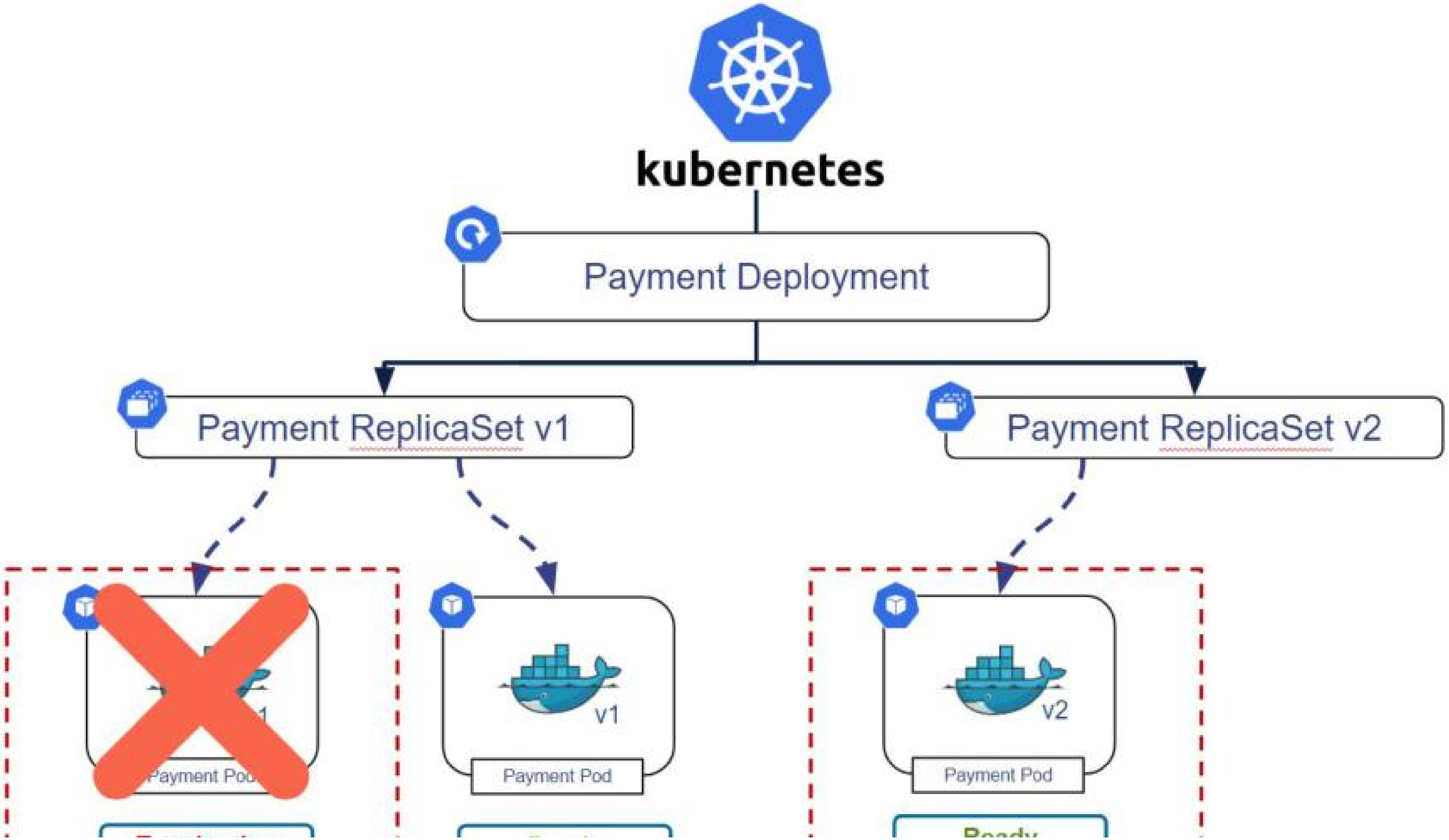
Rolling Update Deployment Strategy

Secondary ReplicaSet is created with new version of application



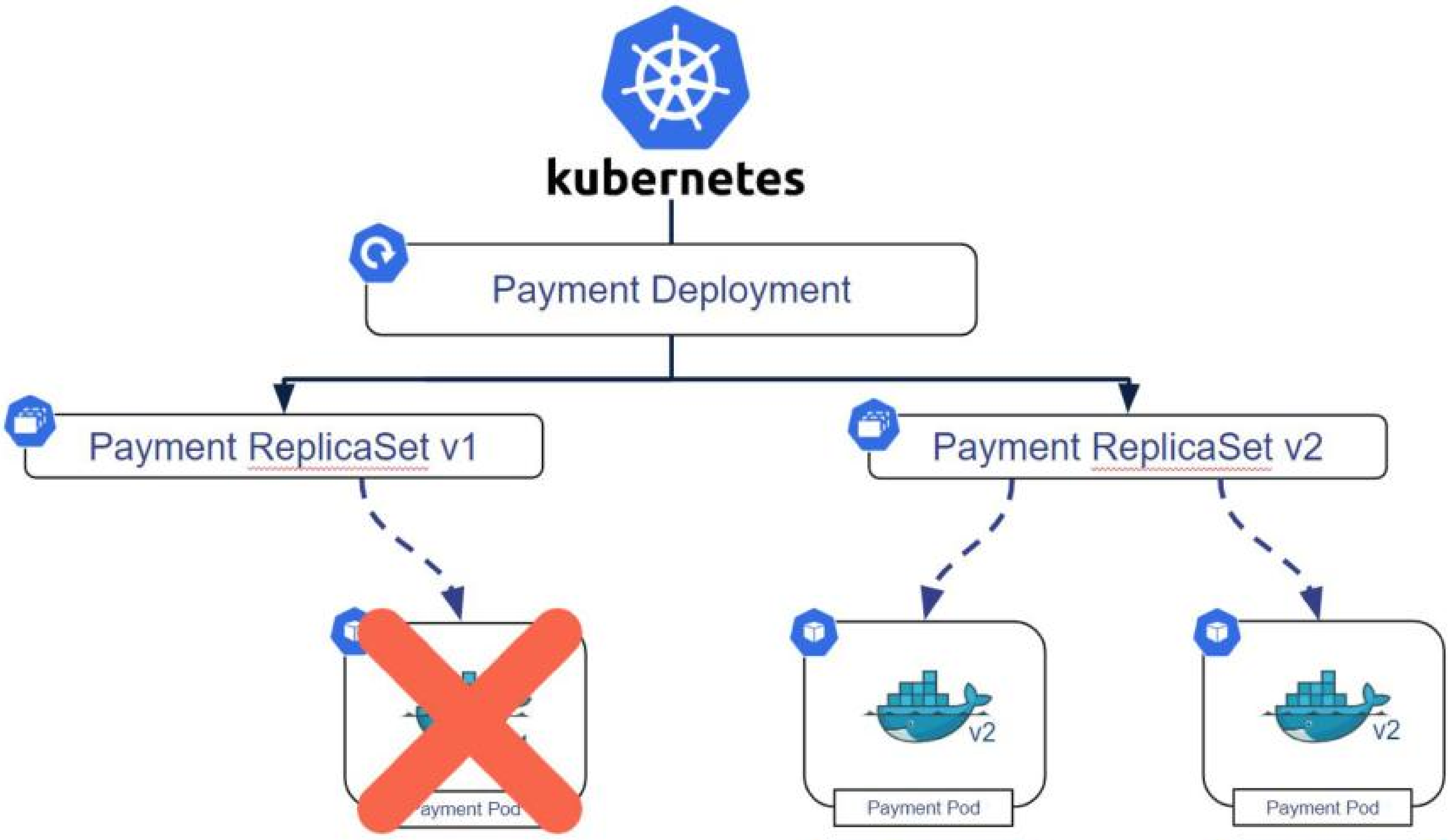
Rolling Update Deployment Strategy

Secondary ReplicaSet is created with new version of application



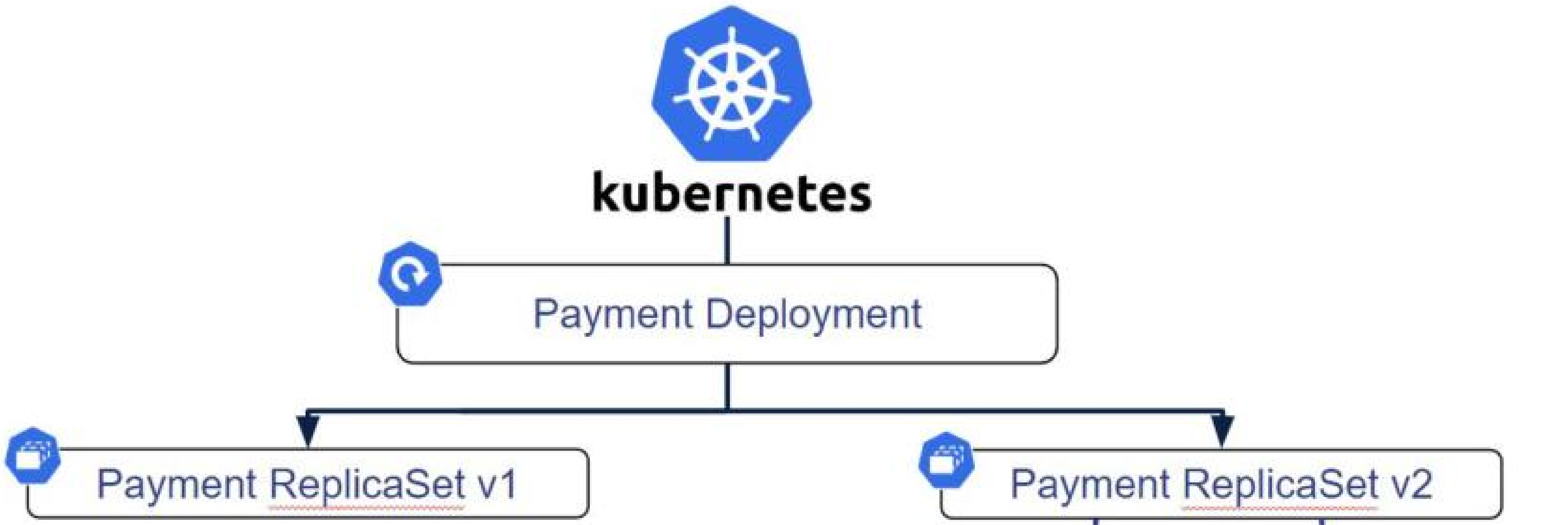
Rolling Update Deployment Strategy

Secondary ReplicaSet is created with new version of application



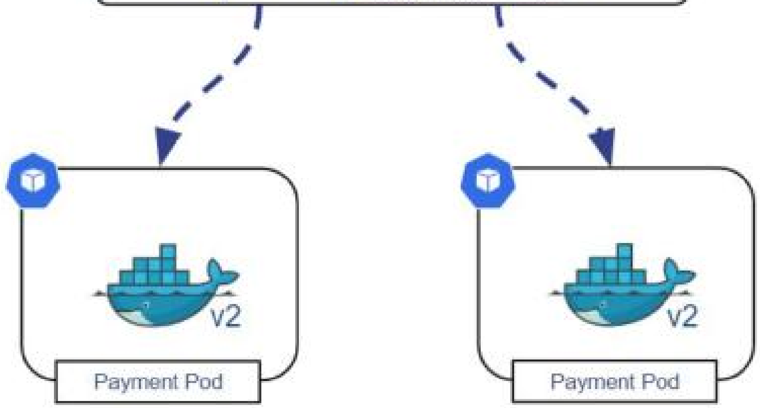
Rolling Update Deployment Strategy

Secondary ReplicaSet is created with new version of application




```
chtans-MacBook-Pro:~ chtan$ kubectl get replicaset
NAME                        DESIRED   CURRENT   READY   AGE
payment-deployment-89f98cf9   2         2         2       45s
```

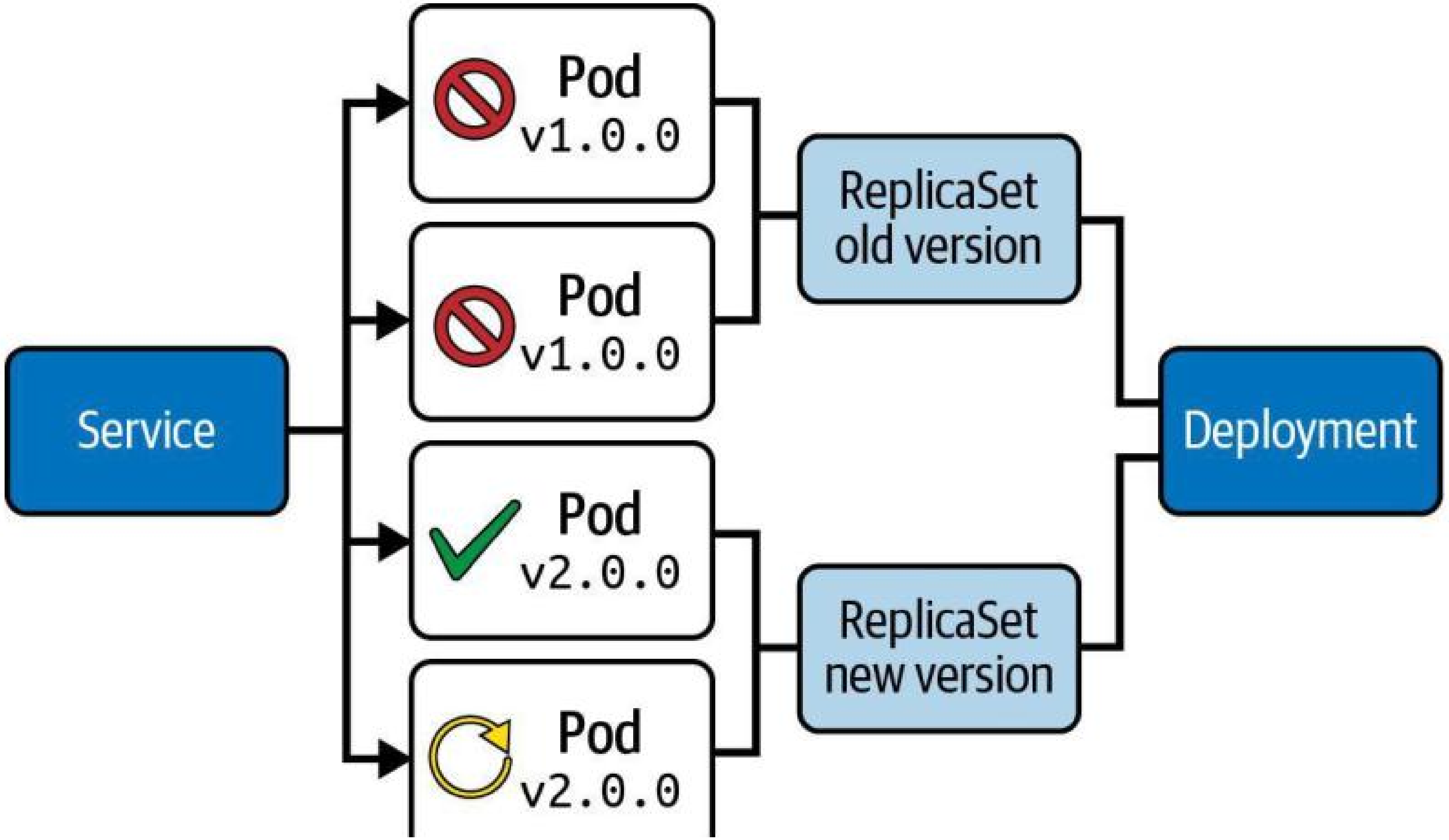
```
chtans-MacBook-Pro:~ chtan$ kubectl get replicaset
NAME                        DESIRED   CURRENT   READY   AGE
```



Deployment Strategy

Recreate Deployment Strategy

Terminates all the running instances then recreate them



Recreate Deployment Strategy

No configuration options available

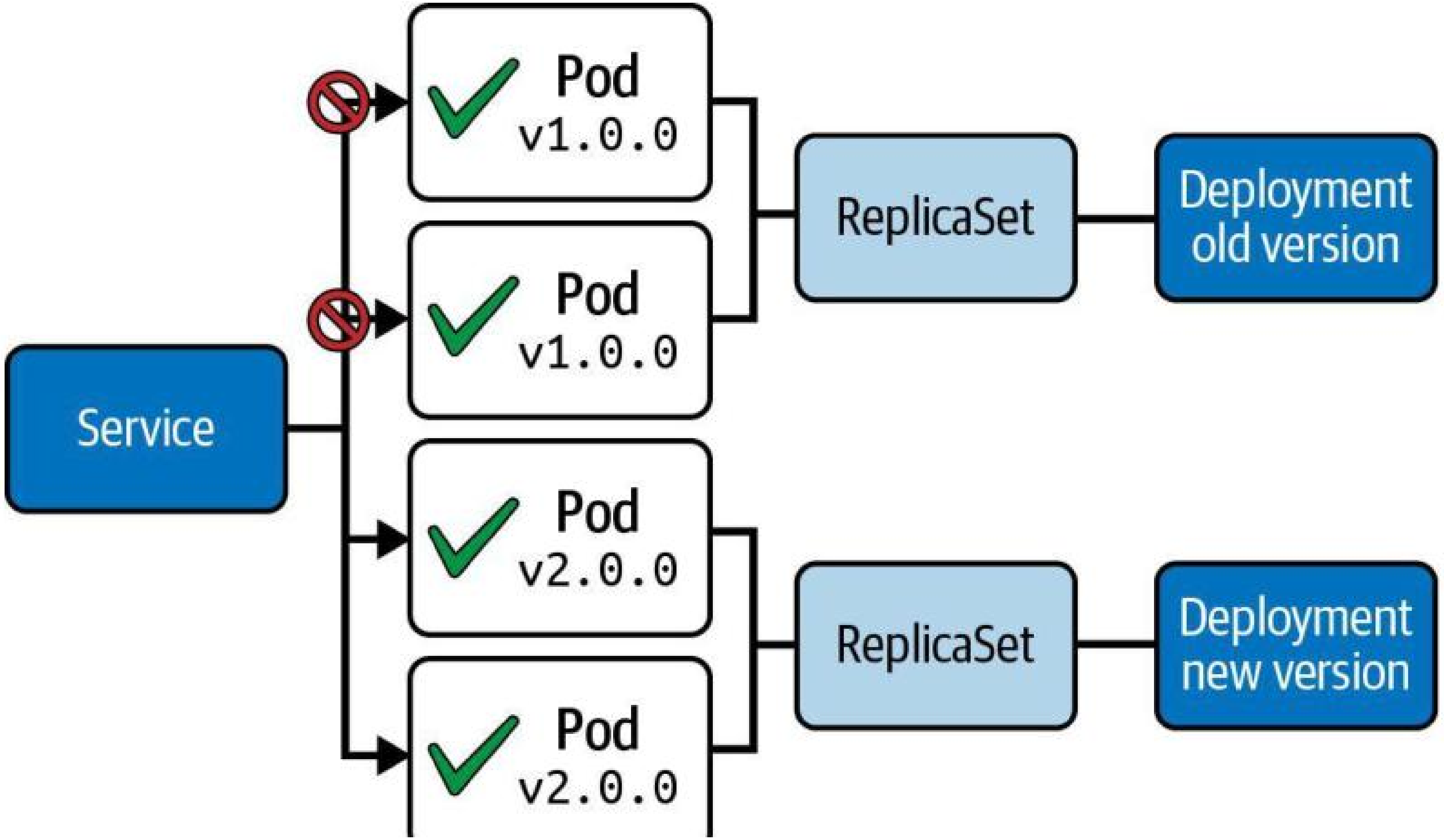
```yaml
spec:
  replicas: 3
  strategy:
    type: Recreate
```

Set the strategy explicitly

- *Pros:* Good option for development environments as everything is renewed at once
- *Cons:* Can cause downtime while applications are starting up

Blue/Green Deployment Strategy

Deployment object with new version is created alongside



Blue/Green Deployment Strategy

Requires a duplicate set of objects

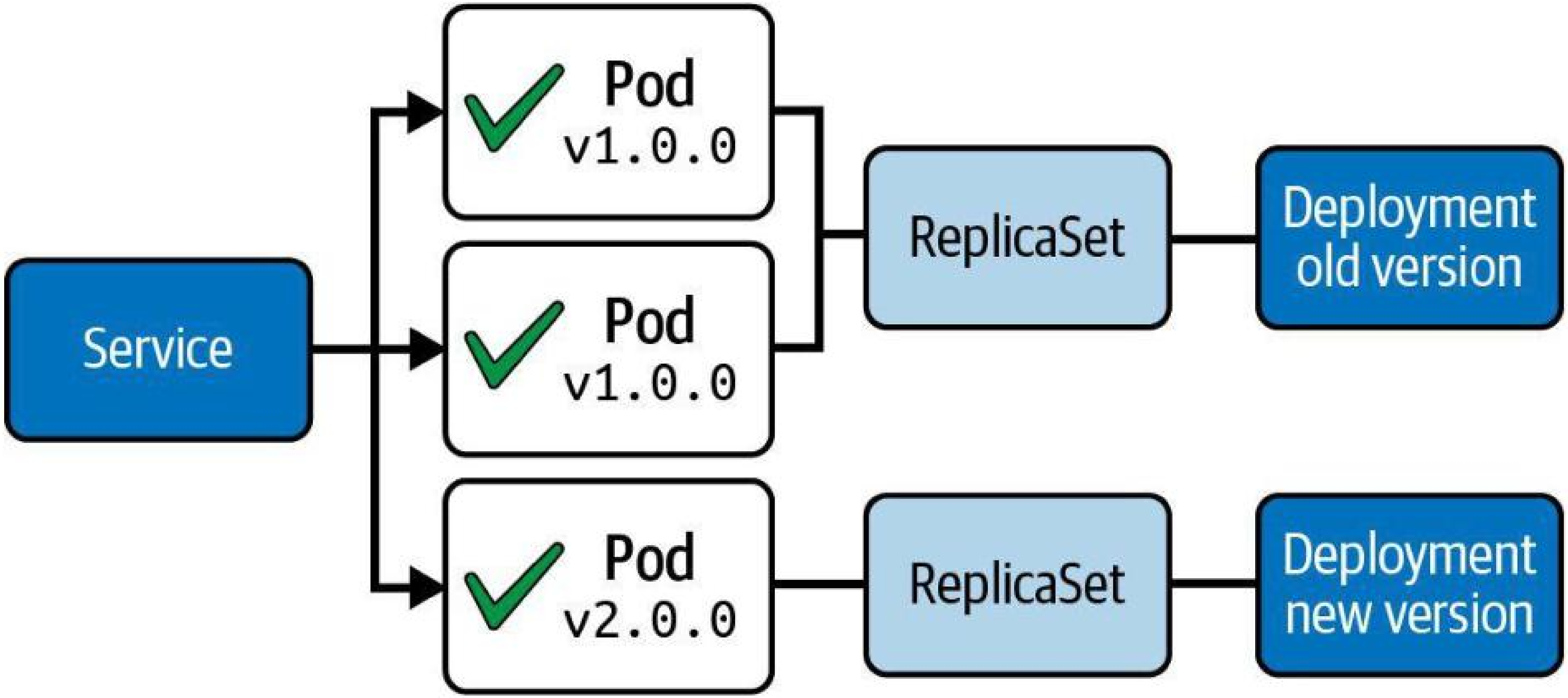
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: blue
spec:
  selector:
    matchLabels:
      app: my-deploy
      version: 1.0.0
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: green
spec:
  selector:
    matchLabels:
      app: my-deploy
      version: 2.0.0
```

- *Pros:* No downtime, traffic can be routed when ready
- *Cons:* Resource duplication, configuration of network routing

Canary Deployment Strategy

Two Deployment objects with different traffic distribution



Canary Deployment Strategy

The new Deployment initially starts with a smaller number of replicas

```
spec:
  replicas: 3
```

```
spec:
  replicas: 1
```

- *Pros:* New version released to subset of users, A/B testing of features and performance

- *Cons:* May require a load balancer for fine-grained distribution

Tea Break

15 minutes

TOPIC 2

Rolling Updates & Rollback

Zero-downtime upgrades, one ReplicaSet at a time

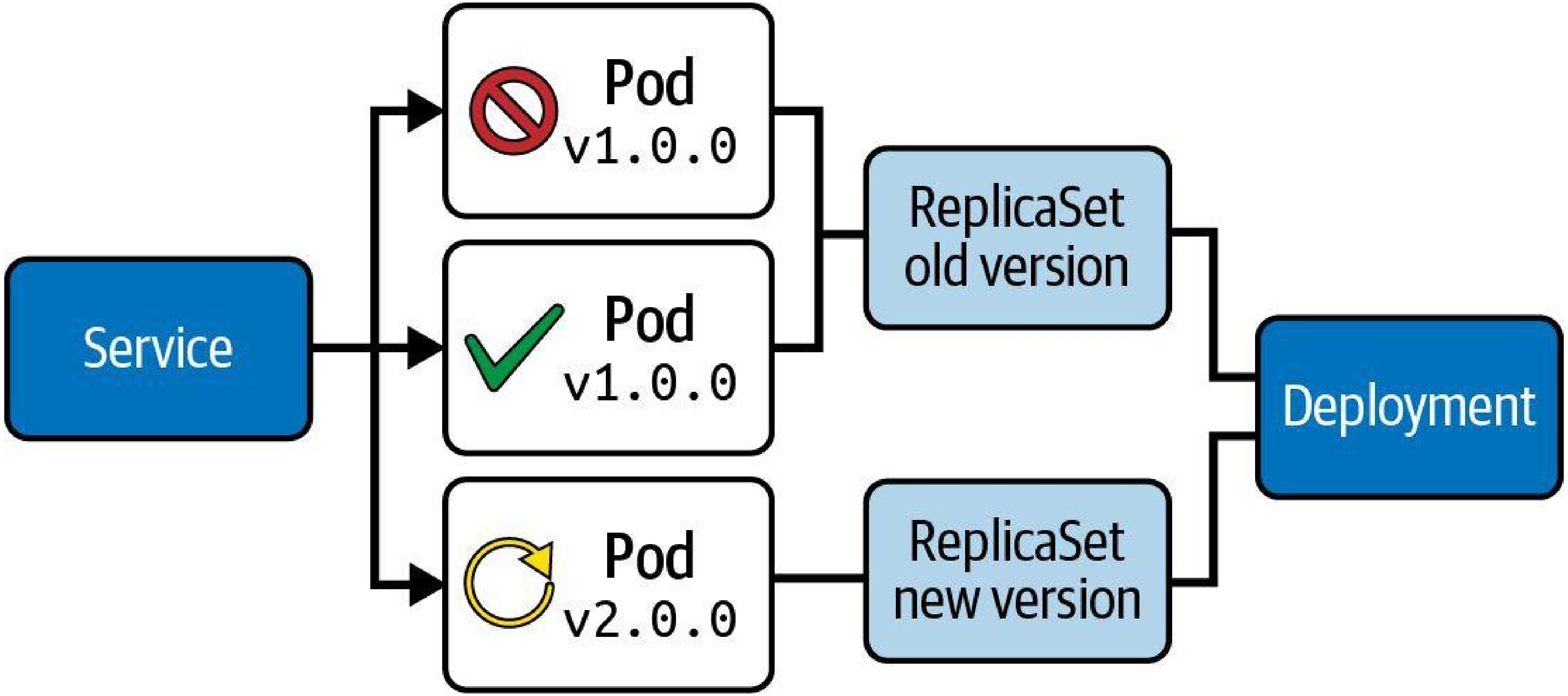
ZERO DOWNTIME

How a Rolling Update Works

- kubectl set image mutates the Pod template — this creates a new ReplicaSet and starts a rollout.
- maxSurge = extra Pods allowed above desired; maxUnavailable = Pods allowed down — together they tune speed vs availability.
- kubectl rollout status watches progress; rollout history lists every revision.
- rollout pause batches several changes into one ReplicaSet; rollout undo reverts instantly.

Rolling Update Deployment Strategy

Secondary ReplicaSet is created with new version of application



Rolling Update Deployment Strategy

Runtime behavior can be fine-tuned

```yaml
spec:
  replicas: 3
  strategy:
    type: RollingUpdate

     rollingUpdate:
       maxSurge: 2
       maxUnavailable: 0
```

Determines how update should be performed

- *Pros:* New version is slowly distributed over time, no downtime
- *Cons:* Breaking API changes can make consumers incompatible

Rolling Updates and Rollback

LAB 10

Rolling Updates & Rollback

A Deployment performs zero-downtime upgrades by gradually replacing Pods one ReplicaSet at a time. CKAD tests maxSurge, maxUnavailable, rollout pause/resume, history inspection and rollback — often as a timed multi-step question.

You'll build

Tune the rolling-update strategy, trigger an image update, inspect history, pause/resume, then roll back.

Key commands: k create · k patch · k set · k rollout · » Pausing

LAB 10 · COMMANDS

Rolling Updates and Rollback

Step 1 — Create the initial Deployment

```
k create deployment web --image=nginx:1.24 --replicas=4
k rollout status deployment/web
```

Step 2 — Tune the rolling-update strategy

```
k patch deployment web -p \
  '{"spec":{"strategy":{"rollingUpdate":{"maxSurge":1,"maxUnavailable":1}}}}'
```

Step 3 — Trigger an image update and watch the rollout

```
k set image deployment/web nginx=nginx:1.25
k rollout status deployment/web
k get rs -l app=web        # old RS drains to 0, new RS ramps to 4
```

Rolling Updates and Rollback (cont.)

Step 4 — View history, pause, batch changes, resume

```
k rollout history deployment/web
k rollout pause deployment/web
k set image deployment/web nginx=nginx:1.26
k set env deployment/web APP_ENV=production
k rollout resume deployment/web
```

Step 5 — Roll back

```
k rollout undo deployment/web
k rollout undo deployment/web --to-revision=1
```

Note Pausing batches multiple mutations into a single new ReplicaSet — one rollout, not two. rollout undo reverts to the previous (or a specified) revision instantly.

Rolling Updates and Rollback

Test it

k rollout history deployment/web lists each revision, and k rollout undo returns the Deployment to the prior image with zero downtime.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Listing Deployment with Imperative Approach

Listing objects for the Deployment, ReplicaSet and Pod(s)

```
Kubectl get deployments
```

Get all Deployments

```
Kubectl describe deployment deployment-1
```

Setting a Change Cause for a Revision

Tracked by annotation with key `kubernetes.io/change-cause`

```
$ kubectl rollout history deployment my-deploy
deployment.apps/my-deploy
REVISION CHANGE-CAUSE
2          <none>
3          <none>

$ kubectl annotate deployment
my-deploy kubernetes.io/change-cause="image updated to 1.16.1"
deployment.apps/my-deploy annotated

$ kubectl rollout history deployment my-deploy
deployment.apps/my-deploy
REVISION CHANGE-CAUSE
2          <none>
3          image updated to 1.16.1
```

What changed
with this revision?

Rolling Update Declaratively

```
kubectl edit deployment deployment-1
```

Edit deployment command

Deployment name

```
spec:
  containers:
  - image: nginx:1.18.0
```

Rolling Update Declaratively

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deployment-1
spec:
  replicas: 3
  selector:
    matchLabels:
      env: dev
  template:
    metadata:
      labels:
        env: dev
    spec:
      containers:
      - name: nginx-deployment
        image: nginx:1.18.0
```

NAME	READY	STATUS	RESTARTS	AGE
deployment-1-5c8bbc875-8tsc7	0/1	ContainerCreating	0	12s
deployment-1-b9d67f686-rp6kd	1/1	Running	0	3m7s
deployment-1-b9d67f686-w56vw	1/1	Running	0	3m7s
deployment-1-b9d67f686-wz5tg	1/1	Running	0	3m7s

NAME	READY	STATUS	RESTARTS	AGE
deployment-1-5c8bbc875-8tsc7	1/1	Running	0	26s
deployment-1-5c8bbc875-8tvdv	1/1	Running	0	12s
deployment-1-5c8bbc875-gp5cc	1/1	Running	0	14s

Rolling Update Declaratively

```
kubectl set image deployment deployment-1 nginx=nginx:1.19.1
```

$\underbrace{\text{kubectl set image deployment}}_{\text{Edit deployment image}}$ $\underbrace{\text{deployment-1}}_{\text{Deployment name}}$ $\underbrace{\text{nginx=nginx:1.19.1}}_{\text{Container name=version}}$

Rolling Update Declaratively

```
NAME                             READY   STATUS
deployment-1-68ff495bcc-4nvgg    1/1     Running
deployment-1-68ff495bcc-vd8lx    0/1     ContainerCreating
deployment-1-75fb796bf8-h542s    1/1     Running
deployment-1-75fb796bf8-j7747    1/1     Terminating
```

```
  Containers:
   deployment-1:
    Image:          nginx:1.19.1
```

Rollout Status

```
kubectl rollout status deployment payment-deployment
```

Rollout command

Deployment name

What happens if there is an error?

Lunch Break

1 hour

TOPIC 3

Blue/Green & Canary

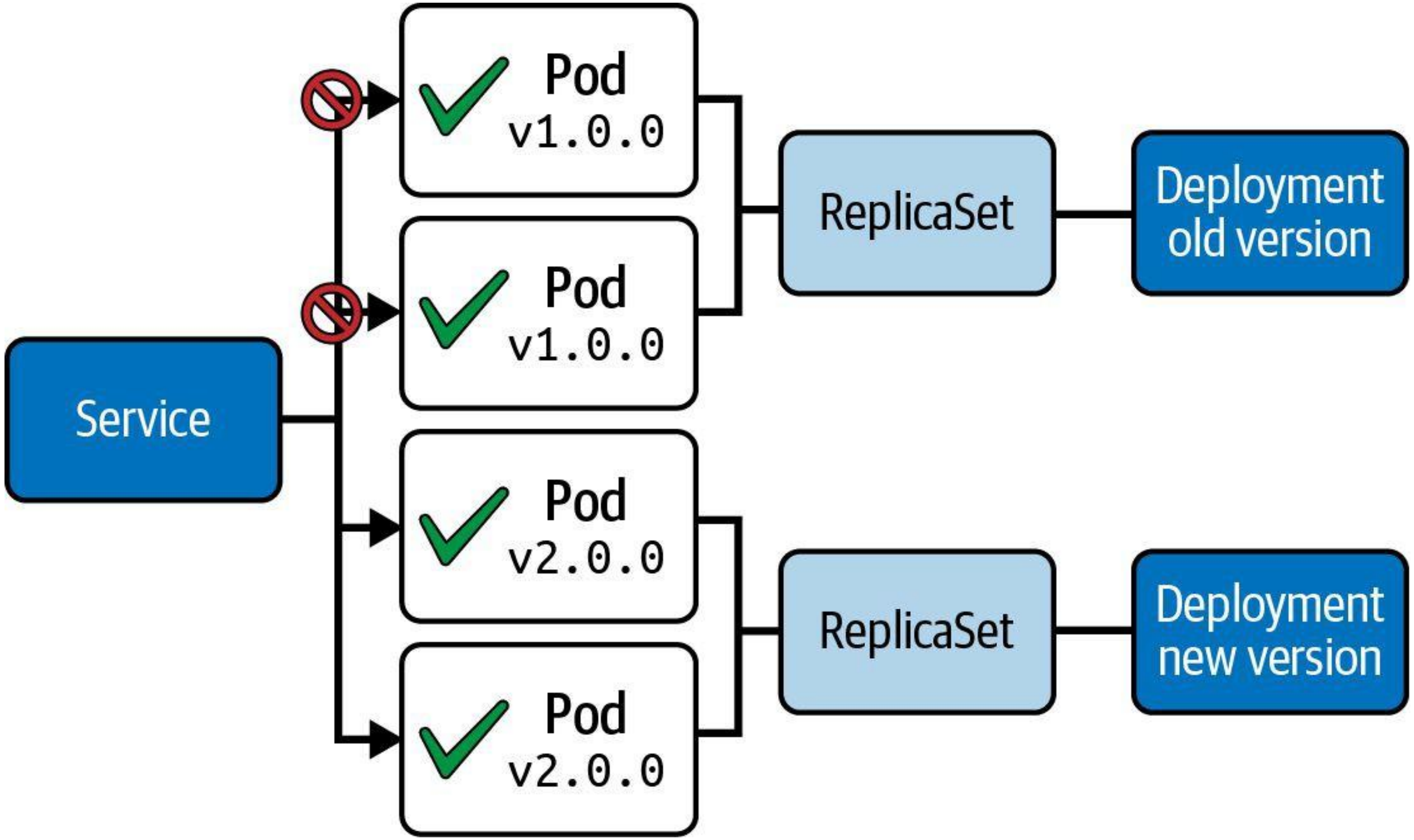
Release strategies built from Deployments + Services

Beyond the Default Rollout

- Blue/Green runs two complete versions; a Service selector switch flips 100% of traffic atomically.
- Canary sends a small slice of traffic to the new version by running it at a low replica ratio.
- Both are built from plain Deployments and Services using labels — no extra tooling for CKAD.
- Rollback is instant: re-patch the selector (blue/green) or scale the canary back down.

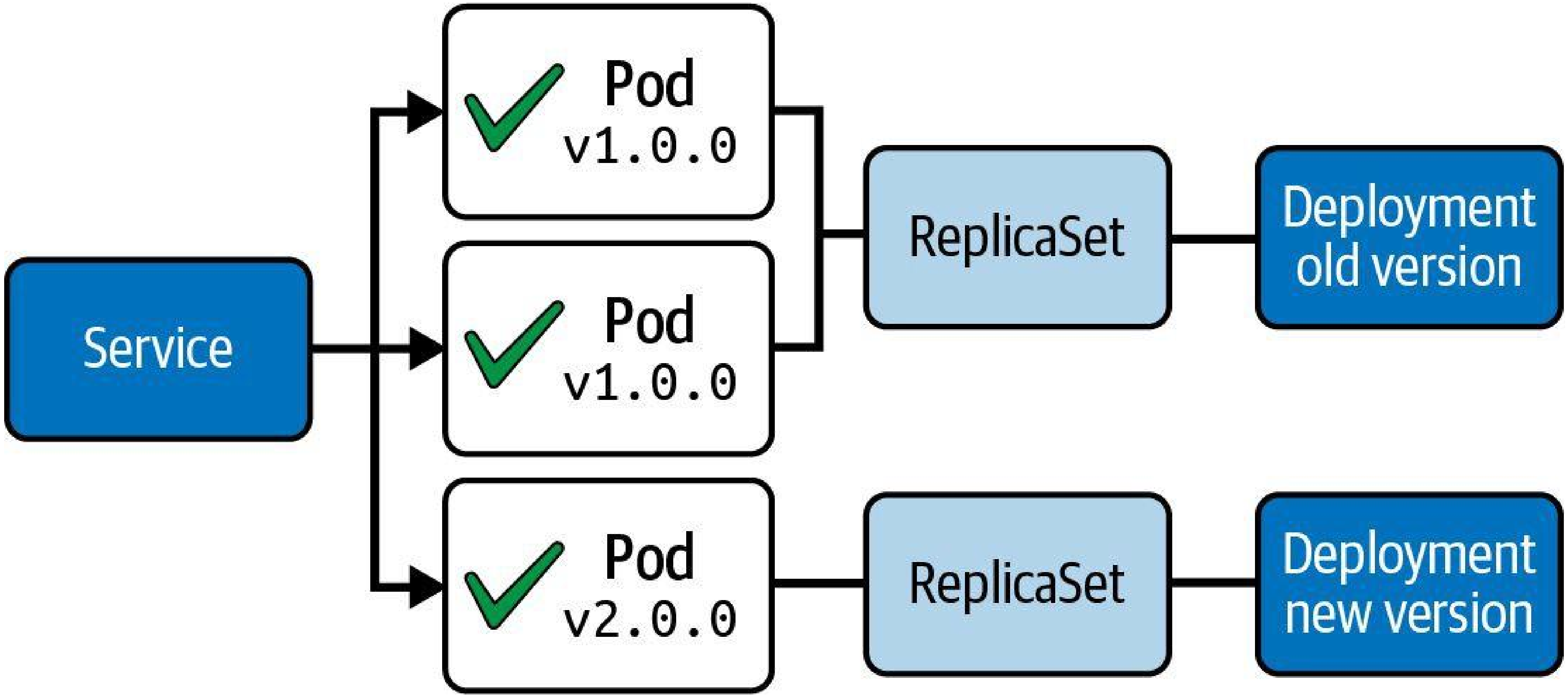
Blue/Green Deployment Strategy

Deployment object with new version is created alongside



Canary Deployment Strategy

Two Deployment objects with different traffic distribution



Blue/Green Deployment

LAB 11

Blue/Green Deployment

Blue/Green keeps two complete copies of the app alive at once. A Service selector switch flips 100% of traffic from blue to green in one atomic operation — with instant rollback if anything breaks.

You'll build

Deploy blue (live) and green (idle), smoke-test green, flip the Service selector, then roll back with one patch.

Key commands: k apply · GREEN_POD=$(k get · k patch · » Cutover

Blue/Green Deployment

Step 1 — Deploy blue (current production) behind a Service

```
# Deployment web-blue (labels app=web, version=blue) + Service web
# selector: {app: web, version: blue}
k apply -f blue.yaml
k get pods -l app=web --show-labels
```

Step 2 — Deploy green (next version, no live traffic yet)

```
# Deployment web-green (labels app=web, version=green), nginx:1.25
k apply -f green.yaml
k get pods -l app=web --show-labels
```

Step 3 — Smoke-test green directly before cutover

```
GREEN_POD=$(k get pod -l version=green -o jsonpath='{.items[0].metadata.name}')
k exec $GREEN_POD -- curl -sI localhost:80 | head -2
```

Blue/Green Deployment (cont.)

Step 4 — Flip the Service to green (atomic cutover)

```
k patch service web -p '{"spec":{"selector":{"app":"web","version":"green"}}}'
k describe svc web | grep Selector
```

Step 5 — Roll back instantly if needed

```
k patch service web -p '{"spec":{"selector":{"app":"web","version":"blue"}}}'
```

Note Cutover and rollback are both a single kubectl patch service selector update — atomic, milliseconds, no Pod churn. Blue stays running as an instant fallback.

Blue/Green Deployment

Test it

After the patch, k describe svc web | grep Selector shows version=green, and patching it back to version=blue restores the old version instantly.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Canary Deployment

LAB 12 Canary Deployment

A canary release sends a small fraction of live traffic to a new version while the bulk stays on stable. In Kubernetes the split is approximated by replica ratios behind one broad Service selector — no special tooling required.

You'll build

Run a 9-replica stable and 1-replica canary behind one Service, verify the ~10% split, then promote.

Key commands: k apply · k run · k scale · » Traffic

LAB 12 · COMMANDS

Canary Deployment

Step 1 — Stable Deployment (90% of traffic)

```
# Deployment web-stable: replicas 9, labels app=web,track=stable
# Service web: selector {app: web}, port 5678
k apply -f stable.yaml
```

Step 2 — Canary Deployment (10% of traffic)

```
# Deployment web-canary: replicas 1, labels app=web,track=canary
k apply -f canary.yaml
k get pods -l app=web --show-labels
```

Step 3 — Verify the traffic split

```
k run probe --image=busybox --restart=Never -it --rm -- sh -c \
  'for i in $(seq 1 30); do wget -qO- web:5678; done | sort | uniq -c'
# ~27 stable, ~3 canary
```

Canary Deployment (cont.)

Step 4 — Promote: scale up canary, retire stable

```
k scale deployment web-canary --replicas=9
k scale deployment web-stable --replicas=0
```

Note Traffic split is approximated by replica ratio (9:1 ≈ 90/10). Promotion is just kubectl scale on both Deployments — no Service change. For precise percentage splits use a service mesh (Istio/Linkerd) — beyond CKAD scope.

Canary Deployment

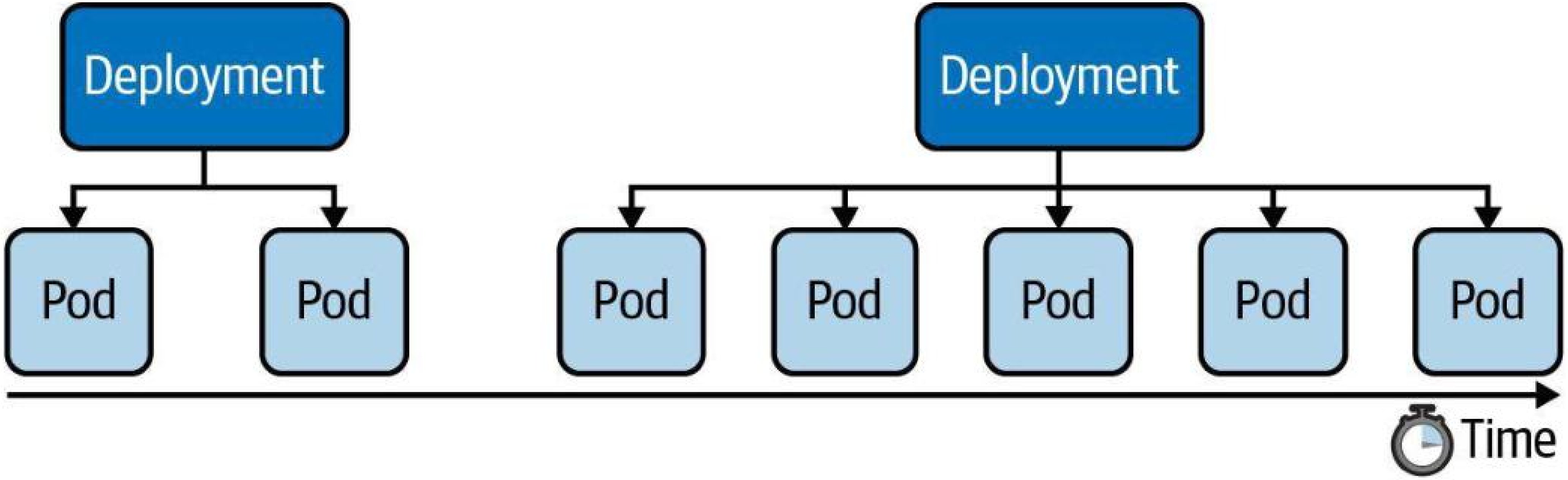
Test it

The probe loop returns roughly 90% stable and 10% canary, and after scaling, all traffic shifts to the canary version with no Service edit.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Manually Scaling a Deployment

"We measured performance under load. Scale up to exactly *x* number of Pods."



Common Deployment Strategies

Choosing the right deployment procedure depends on the needs

- *Rolling Update:* Release a new version on a rolling update fashion, one after the other.
- *Recreate:* Terminate the old version and release the new one.
- *Blue/green:* Release a new version alongside the old version then switch traffic.
- *Canary:* Release a new version to a subset of users, then proceed to a full rollout.

Common Deployment Strategies

Choosing the right deployment procedure depends on the needs

- *Rolling Update:* Release a new version on a rolling update fashion, one after the other.
- *Recreate:* Terminate the old version and release the new one.
- *Blue/green:* Release a new version alongside the old version then switch traffic.
- *Canary:* Release a new version to a subset of users, then proceed to a full rollout.

Types of Autoscalers

HPA — Horizontal

- Horizontal Pod Autoscaler (HPA)
- Scales the number of Pod replicas based on CPU/memory thresholds
- Standard Kubernetes feature
- Uses the metrics-server component

VPA — Vertical

- Vertical Pod Autoscaler (VPA)
- Scales CPU/memory allocation for existing Pods from historic metrics
- Provided by cloud providers as an add-on or installed manually
- Both autoscalers require metrics-server and resource requests/limits on Pods

Horizontal Pod Autoscaler (HPA)

- The HPA is a Kubernetes primitive that queries the metrics server at regular intervals.
- It compares current resource utilisation against the defined threshold.
- If utilisation exceeds the threshold → HPA increases the replica count (scale out).
- If utilisation drops below the threshold → HPA decreases the replica count (scale in).
- The HPA targets a Deployment (or ReplicaSet) via scaleTargetRef.

HPA · CPU SCALING

Autoscaling a Deployment by CPU

- The HPA monitors average CPU utilisation across all running replicas.
- Example: threshold = 80% CPU, current = 15% → no scale-out needed.
- Example: threshold = 80% CPU, current = 95% → HPA adds replicas.
- Scaling decisions respect minReplicas and maxReplicas bounds.
- kubectl autoscale is the imperative shortcut to create an HPA.

Horizontal Pod Autoscaler YAML Manifest

HPA targeting a Deployment, scaling on CPU utilisation

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: app-cache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: app-cache
  minReplicas: 3
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 80
```

Inspecting the Horizontal Pod Autoscaler

List HPAs — shows current utilisation vs threshold

```
kubectl get hpa
NAME        REFERENCE               TARGETS    MINPODS   MAXPODS   REPLICAS   AGE
app-cache   Deployment/app-cache    15%/80%    3         5         3          2m14s
```

Describe for detailed events and scaling history

```
kubectl describe hpa app-cache
```

EXERCISE

Creating a Horizontal Pod Autoscaler for a Deployment

Tea Break

15 minutes

TOPIC 4

Helm

The Kubernetes package manager

Charts, Releases & Revisions

- A chart is a bundle of templated YAML; a release is one deployed instance of a chart.
- Workflow: repo add → install → upgrade → rollback → uninstall.
- --set overrides values at install/upgrade; --reuse-values keeps prior overrides.
- helm history tracks every revision; helm get manifest shows what was actually applied.

What is Helm?

Package manager and templating engine for a set of manifests

- The artifact produced by the Helm executable is a so-called *chart file* bundling the manifests that comprise the API resources of an application.
- Chart files can be distributed to a chart repository e.g. central chart repository and consumed from there (e.g., for running Grafana or PostgreSQL).
- At runtime, it replaces placeholders in YAML template files with actual, end-user defined values.

Standard Chart Structure

Predefined directory structure with mandatory `Chart.yaml` and `values.yaml`

```
$ tree
.
├──
├── templates
│   ├── web-app-pod-template.yaml
│   └── web-app-service-template.yaml
└──

     values.yaml
```

Helm — Install, Upgrade, Rollback

LAB 13 Helm

Helm is the Kubernetes package manager. A chart is a bundle of templated YAML; a release is a deployed instance. CKAD tests install, upgrade, rollback, list and value overrides — all under exam time pressure.

You'll build

Add a chart repo, install a release with overrides, inspect it, upgrade, roll back, then uninstall.

Key commands: helm repo · helm install · helm get · helm upgrade · » `--reuse-values`

Helm — Install, Upgrade, Rollback

Step 1 — Add a chart repository

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo bitnami/nginx | head -5
```

Step 2 — Install a release with value overrides

```
helm install web bitnami/nginx \
  --set service.type=ClusterIP --set replicaCount=2
helm list
```

Step 3 — Inspect the generated manifests and values

```
helm get manifest web | head -40
helm get values web
```

Helm — Install, Upgrade, Rollback (cont.)

Step 4 — Upgrade, then roll back

```
helm upgrade web bitnami/nginx --set replicaCount=4 --reuse-values
helm history web
helm rollback web 1
helm uninstall web
```

Note --reuse-values preserves prior overrides while changing only the new one. helm history <release> tracks every revision; rollback creates a new revision rather than deleting history.

Helm — Install, Upgrade, Rollback

Test it

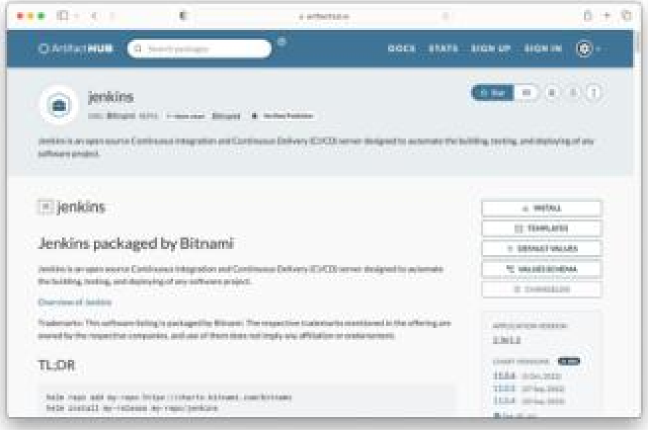
helm history web lists each revision with its status, and helm rollback web 1 returns the release to its first revision as a new history entry.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Finding a Chart

Searches the Artifact Hub for a chart with a matching name

```
$ helm search hub jenkins
URL                                                 CHART    VERSION APP
https://artifacthub.io/packages/helm/bitnami/jenkins 11.0.6  2.361.2
...
```



Installing a Chart

You may have to add an external repository that hosts chart file

```
$ helm repo add bitnami https://charts.bitnami.com/bitnami
"bitnami" has been added to your repositories

$ helm install jenkins bitnami/jenkins
...
CHART NAME: jenkins
CHART VERSION: 11.0.6
APP VERSION: 2.361.2

** Please be patient while the chart is being deployed **
```

Listing Installed Charts

Information about chart details

```
$ helm list
NAME NAMESPACE          REVISION ...    STATUS     CHART            APP VERSION
jenkins   default        1              deployed   jenkins-11.0.6   2.361.2

$ kubectl get pods
NAME                                    READY   STATUS     RESTARTS   AGE
pod/bitnami-jenkins-764b44cc99-twf6f    1/1     Running    0          7m34s
...
```

Uninstalling a Chart

Deletes Kubernetes objects controlled by chart

```
$ helm uninstall jenkins
release "jenkins" uninstalled

$ kubectl get pods
No resources found in default namespace.
```

Values File

Key-value pairs used at runtime to replace placeholders in the YAML manifests

values.yaml

```yaml
db_host: mysql-service
db_user: root
db_password: password
service_port: 3000
```

Additional built-in objects are available for use.

Previewing Final Templates

Replaces placeholders with actual values from `values.yaml`

```
$ helm template .
---
# Source: Web Application/templates/web-app-pod-template.yaml
apiVersion: v1
kind: Pod
metadata:
  name: web-app
spec:
  containers:
  - image: bmuschko/web-app:1.0.1
    name: web-app
    env:





 ...
```

Installing a Chart from Local Files

Helpful for trying out the deployment before publishing the chart file

```
$ helm install web-app .
NAME: web-app
LAST DEPLOYED: Wed Apr 19 11:58:34
2023
NAMESPACE:
default STATUS:
deployed
REVISION: 1
TEST SUITE: None
$ kubectl get pods,services
NAME            READY               RESTARTS   AGE
                STATUS
pod/web-app     1/1   Running       0          3m24s
NAME                          TYPE       CLUSTER-IP      EXTERNAL-IP    PORT(S)          AGE
service/web-app-service       NodePort   10.109.44.239   <none>         3000:30456/TCP   3m24s
```

Overriding Values

End user can tweak runtime behavior to personal requirements

```
$ helm install --namespace custom --create-namespace --set service_port=9090 web-app .
NAME: web-app
LAST DEPLOYED: Wed Apr 19 11:58:34
2023
NAMESPACE:
custom STATUS:
deployed
REVISION: 1
TEST SUITE: None
```

Providing a custom namespace (defaults to `default` namespace)

Override values in `values.yaml`

TOPIC 5

Kustomize

Environment overlays without templating

REUSE WITHOUT TEMPLATES

Base + Overlays

- Kustomize reuses one base set of manifests and layers environment-specific changes on top.
- namePrefix and commonLabels are applied to every resource in the overlay.
- The images block overrides image tags without editing the base files; patches do strategic merges.
- Built into kubectl: kubectl apply -k and kubectl kustomize — no extra binary required.

What is Kustomize?

Customizing application manifests without templates

- Writing manifests for application stacks is straightforward, however, deploying them to different Kubernetes runtime environments with varying configuration gets tricky.
- Instead of copying and modifying manifests per environment, you can use kustomize to merge configuration change. Kustomize does not require using a template engine or changing the original manifests.
- Kustomize can be used as a subcommand/flag of `kubectl` or as a separate executable.

Environment Kustomize Setup

Subdirectories contain environment-specific configuration

```
.
├── base
│   ├── deployment.yaml
│   ├── kustomization.yaml
│   └── service.yaml
├── dev

│   ├── config.properties
│   └── kustomization.yaml
└── prod
    ├── config.properties
    └── kustomization.yaml
```

Create a dedicated directory + configuration per environment

Kustomize Overlays

LAB 14 Kustomize

Kustomize reuses a single base set of manifests and applies environment-specific overlays without templating. It is built into kubectl (-k). CKAD tests namePrefix, commonLabels, images and strategic-merge patches.

You'll build

Build a base, add dev and prod overlays (prefix, labels, image tag, replica patch), preview, then apply.

Key commands: resources: · kubectl kustomize · » `namePrefix`

Kustomize Overlays

Step 1 — Base kustomization (shared by all environments)

```
# base/kustomization.yaml
resources:
- deployment.yaml
- service.yaml
```

Step 2 — Dev overlay (name prefix + common label)

```
# overlays/dev/kustomization.yaml
resources:
- ../../base
namePrefix: dev-
commonLabels:
  env: dev
```

Kustomize Overlays (cont.)

Step 3 — Prod overlay (image bump + replica patch)

```
# overlays/prod/kustomization.yaml
resources:
- ../../base
namePrefix: prod-
commonLabels: {env: prod}
images:
- name: nginx
  newTag: "1.26"
patches:
- path: replica-patch.yaml
```

Step 4 — Preview, then apply

```
kubectl kustomize overlays/prod | grep -E "name:|replicas:|image:"
kubectl apply -k overlays/dev
kubectl apply -k overlays/prod
```

Kustomize Overlays (cont.)

Note namePrefix and commonLabels apply to every resource in the overlay. The images block overrides tags without editing base files. apply -k / kubectl kustomize are built into kubectl — no extra binary.

Kustomize Overlays

Test it

kubectl kustomize overlays/prod shows name prod-web, 5 replicas and nginx:1.26, while the dev overlay stays at 1 replica and nginx:1.25.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Kustomize Use Cases

Convenient manifest management

- Generating manifests from other sources. For example, creating a ConfigMap and populating its key-value pairs from a properties file.

- Adding common configuration across multiple manifests. For example, adding a namespace and a set of labels for a Deployment and a Service.

- Composing and customizing a collection of manifests. For example, setting resource boundaries for multiple Deployments.

Basic Kustomize Setup

Regular YAML manifests + a `kustomization.yaml` file

```
.
└── base
    ├── deployment.yaml
    ├── kustomization.yaml
    └── service.yaml
```

```yaml
apiVersion: apps/v1
kind: Deployment
...
```

```yaml
apiVersion: v1
kind: Service
...
```

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
...
```

Kustomization File Structure

Generate or transform other Kubernetes Resource Model (KRM) objects

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- {pathOrUrl}
- ...

generators:
- {pathOrUrl}
- ...

transformers:
- {pathOrUrl}
- ...

validators:
- {pathOrUrl}
- ...
```

What *existing* resources are to be customized

What *new* resources should be created

What to *do* to the aforementioned resources

A transformer that doesn't transform, but can (just like a

Example Kustomization File

Merges labels into a set of resources

kustomization.yaml

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

labels:
  layer: backend

resources:
  - deployment.yaml
  - service.yaml
```

Defines labels for all processed resources

Declares resources to be modified

Generating a Kustomize File

Imperative command to create `kustomization.yaml` file

```
$ cd base
$ ls .
deployment.yaml service.yaml

$ kustomize create --resources=deployment.yaml,service.yaml ↵
  --labels=layer:backend

$ ls .
deployment.yaml kustomization.yaml service.yaml
```

Building and Previewing a Set of Resources

Applies the rules in `kustomization.yaml` and renders the result to the output

```
$ kustomize build base
apiVersion: v1
kind: Service
metadata:
  labels:
    layer: backend
...
---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    layer: backend
...
```

Alternatively, you can also use `kubectl kustomize base`

Applied labels from rules

Example Production Environment

Base configuration is merged with the configuration of an environment

prod/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- ../base/ namespace:
  prod namePrefix:
  prod-
  configMapGenerator:
- name: backend-configmap
  env: config.properties
replicas:
- name: backend-deployment
  count: 10
```

Merging Environment Configuration

Base configuration + environment configuration

```
$ kustomize build prod
apiVersion: v1
data:
  DB_USERNAME: prod-user
kind: ConfigMap
metadata:
  name: prod-backend-configmap-7gbb88gbhc
  namespace: prod
---
apiVersion: v1
kind: Service
metadata:
  labels:
    layer: backend
  name: prod-backend-service
  namespace: prod
...
```

Kustomize Benefits

A good option for operators managing their own manifests

- *Native to `kubectl`:* Kustomize does not necessarily require you to install additional tooling.
- *DRY approach:* Define a base configuration just once with the ability to reuse it. Environment-specific configuration will be added as an overlay.
- *Easy to learn:* Does not require learning another template engine. You'll use standard YAML manifests to define resources.
- *Scalable variants:* New configuration e.g. environments can be easily added.

Tea Break

15 minutes

TOPIC 6

DaemonSets & StatefulSets

Node-wide agents & stable-identity workloads

Specialised Controllers

- A DaemonSet runs exactly one Pod per matching node — log collectors, CNI agents, node exporters.
- A StatefulSet gives Pods stable names (db-0, db-1) and ordered, graceful start/stop.
- A StatefulSet needs a headless Service (clusterIP: None) for per-Pod DNS.
- Scale-down always removes the highest ordinal first; scale-up adds the next in order.

THREE WORKLOAD SHAPES

Pick the Right Controller

Deployment

- Stateless, interchangeable
- Any number of replicas
- Rolling updates

DaemonSet

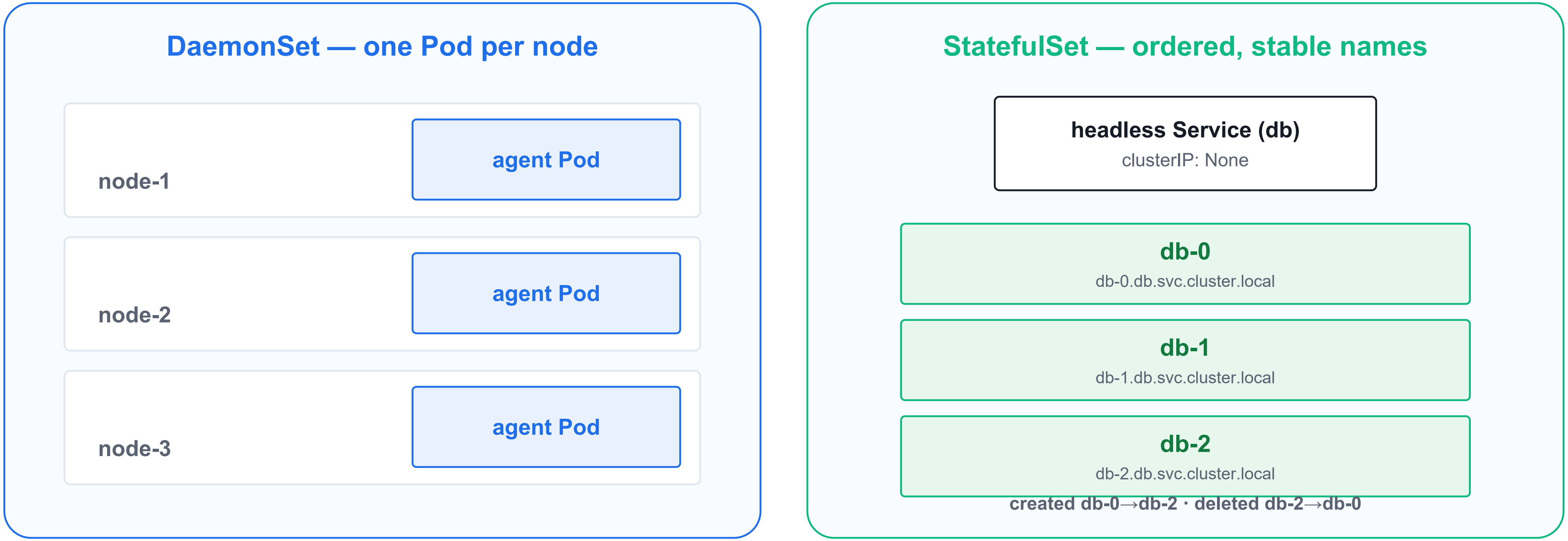
- One Pod per node
- Node-level agents
- Scales with the cluster

StatefulSet

- Stable identity + storage
- Ordered start/stop
- Databases, queues

WORKLOAD CONTROLLERS · NODE-WIDE vs STABLE IDENTITY

DaemonSet vs StatefulSet



DaemonSet = **one Pod per matching node**; StatefulSet = **stable identity + ordered start/stop**.

DaemonSets and StatefulSets

LAB 15

DaemonSets & StatefulSets

A DaemonSet runs exactly one Pod per matching node (log collectors, CNI agents). A StatefulSet gives Pods stable identities and ordered start/stop (databases). CKAD expects you to know when to use each and write the YAML.

You'll build

Run a DaemonSet on every node, then a StatefulSet with a headless Service, stable DNS and ordered scaling.

Key commands: k apply · nslookup db-0.db.default.svc.cluster.local · » DaemonSet

DaemonSets and StatefulSets

Step 1 — DaemonSet on every node

```
# DaemonSet node-agent, image busybox
k apply -f ds.yaml
k get ds,pods -l app=node-agent -o wide   # DESIRED = node count
```

Step 2 — Headless Service for the StatefulSet

```
# Service db: clusterIP: None, selector app=db, port 5432
k apply -f headless.yaml
```

Step 3 — StatefulSet with stable identities

```
# StatefulSet db: serviceName db, replicas 3
k apply -f sts.yaml
k rollout status sts/db
k get pods -l app=db        # db-0 → db-1 → db-2 in order
```

DaemonSets and StatefulSets (cont.)

Step 4 — Verify stable per-Pod DNS and scale

```
nslookup db-0.db.default.svc.cluster.local
k scale sts db --replicas=4    # adds db-3
k scale sts db --replicas=2    # removes db-3 then db-2
```

Note DaemonSet = one Pod per matching node. StatefulSet = stable names (db-0, db-1) and ordered start/stop; a headless Service (clusterIP: None) gives each Pod its own DNS. Scale-down always removes the highest ordinal first.

DaemonSets and StatefulSets

Test it

k get pods -l app=db shows db-0, db-1, db-2 created in order, and nslookup db-0.db.default.svc.cluster.local resolves to a single Pod IP.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Taints & Tolerations

Taint — mark a Node to repel Pods that do not tolerate it.

Toleration — added to a Pod to allow it to schedule onto tainted Nodes.

Node (tainted)

Taint: gpu=true:NoSchedule

BLOCKED

Pod (no toleration)

? schedule here

ALLOWED

Pod (with toleration)

gpu=true:NoSchedule

Exam Tip

- Taints repel Pods from Nodes
- Tolerations let a Pod tolerate a taint
- A Pod with a toleration MAY go to a tainted node — it's not forced

Taint Effects

NoSchedule

New Pods without matching toleration are NOT scheduled here.
Already-running Pods are unaffected.

PreferNoSchedule

Scheduler TRIES to avoid placing Pods here, but will if no other option exists.
Soft constraint.

NoExecute

New Pods are not scheduled AND existing Pods without toleration are EVICTED.
Use tolerationSeconds for grace period.

kubectl taint nodes node1 gpu=true:NoSchedule | Remove: kubectl taint nodes node1 gpu=true:NoSchedule-

Tolerations in a Pod Spec

Adding a taint to a Node

```
kubectl taint nodes node1 gpu=true:NoSchedule
kubectl taint nodes node1 gpu=true:NoSchedule-   # remove
```

Toleration in the Pod spec

```yaml
spec:
  tolerations:
  - key: "gpu"
    operator: "Equal"
    value: "true"
    effect: "NoSchedule"
  containers:
  - name: gpu-job
    image: cuda:11.0
```

Node Affinity

Node Affinity constrains which Nodes a Pod can be scheduled on, based on Node labels.

More expressive than nodeSelector — supports operators: In, NotIn, Exists, DoesNotExist, Gt, Lt

nodeSelector (simple)

```
spec:
  nodeSelector:
    disktype: ssd
```

- Key=value match only
- No operator support
- Cannot express 'not in' or 'exists'

Node Affinity (expressive)

```
spec:
  affinity:
    nodeAffinity:
      required...:
        nodeSelectorTerms:
        - matchExpressions:
          - key: disktype
            operator: In
            values: [ssd, nvme]
```

Node Affinity Types

- requiredDuringSchedulingIgnoredDuringExecution
  - Hard constraint — Pod WILL NOT schedule if no matching node
  - Use when workload requires specific hardware (GPU, SSD)
- preferredDuringSchedulingIgnoredDuringExecution
  - Soft constraint — scheduler PREFERS matching nodes but places elsewhere if needed
  - Use when you want to spread workloads across zones if possible
- IgnoredDuringExecution
  - Both types ignore affinity rules for already-running Pods
- Combine multiple matchExpressions for fine-grained control
  - Expressions within a nodeSelectorTerm are ANDed; multiple terms are ORed

What is a DaemonSet?

- Resource that ensures exactly one (or more) copy of a Pod runs on every (or every matching) Node in the cluster
- Contrast with a Deployment, which runs N replicas spread across nodes as the scheduler sees fit

Yaml Manifest

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd
  namespace: logging
spec:
  selector:
    matchLabels:
      app: fluentd
  template:
    metadata:
      labels:
        app: fluentd
    spec:
      containers:
      - name: fluentd
        image: fluent/fluentd:latest
        volumeMounts:
        - name: varlog
          mountPath: /var/log
      volumes:
      - name: varlog
        hostPath:
```

selector: must match the Pod template's labels

template: same as for Deployments—a Pod spec.

hostPath (common): mount

Deploy & Verify

- kubectl apply -f daemonset.yaml
- kubectl get daemonset -n logging
- kubectl get pods -n logging -o wide
- kubectl rollout status daemonset/fluentd -n logging

If you add a new node, the DaemonSet controller automatically launches a new Pod on it; when you remove a node, its Pod is cleaned up.

What is a StatefulSet?

- Set of pods with stable unique identities and optional stable storage
- Pods are created in ordinal order, from 0 up to N-1
- Controller waits for Pod 0 to be running and ready before creating pod 1 etc.
- Like Deployments, each Pod is scheduled by the kube-scheduler based on capacity, taints, selectors, affinity, etc.

Yaml Manifest

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
  namespace: default
spec:
  serviceName: web
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: nginx
        image: nginx:1.25.5
        ports:
        - containerPort: 80
        volumeMounts:
        - name: www-storage
          mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
  - metadata:
      name: www-storage         # will yield PVCs: www-storage-web-0, -1, -2
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
```

must match headless service

template: same as for Deployments—a Pod spec.

will yield PVCs:
www-storage-web-0
www-storage-web-1

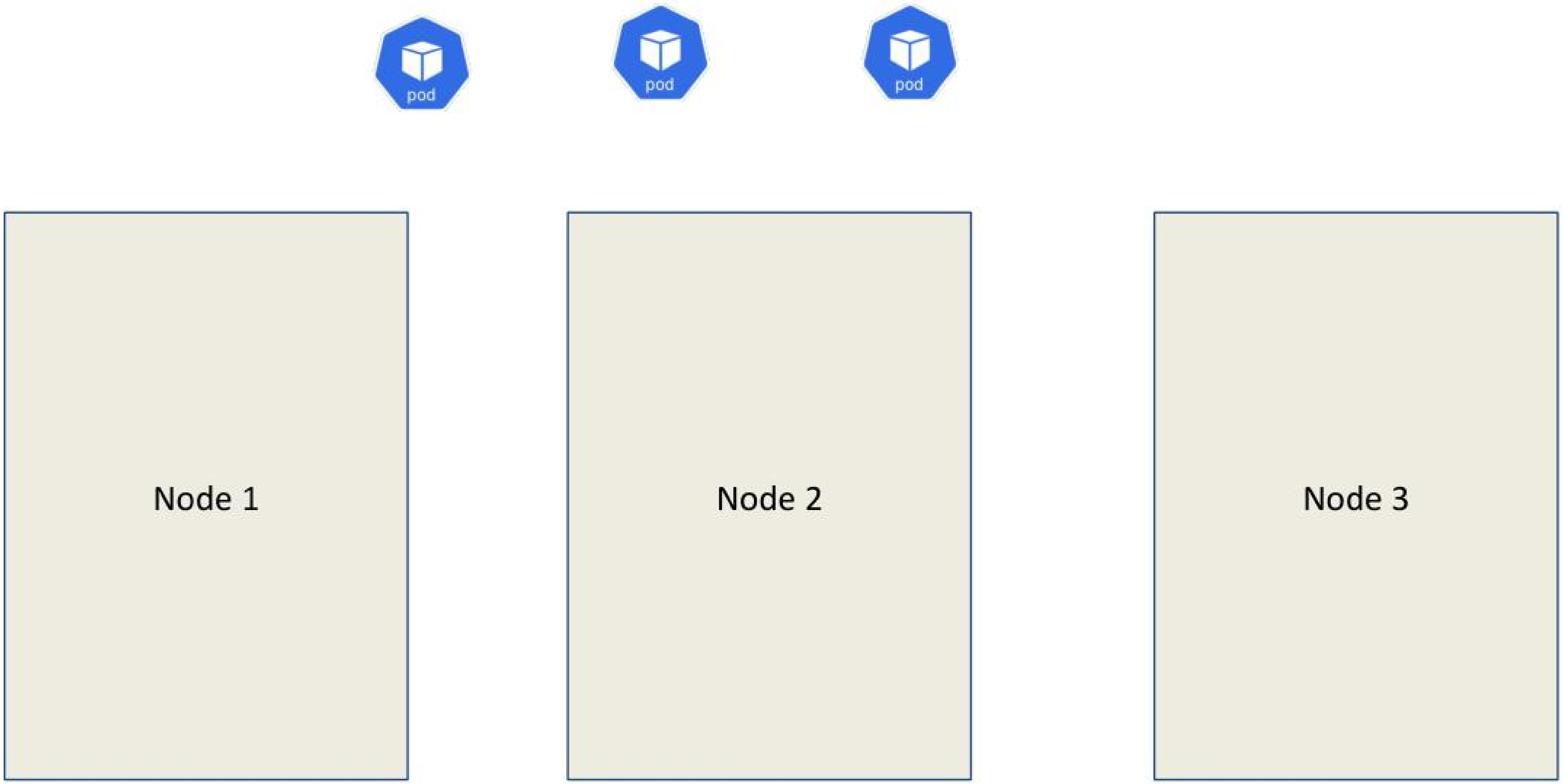
Deploy & Verify

- kubectl apply -f statefulset.yaml
- kubectl get statefulset

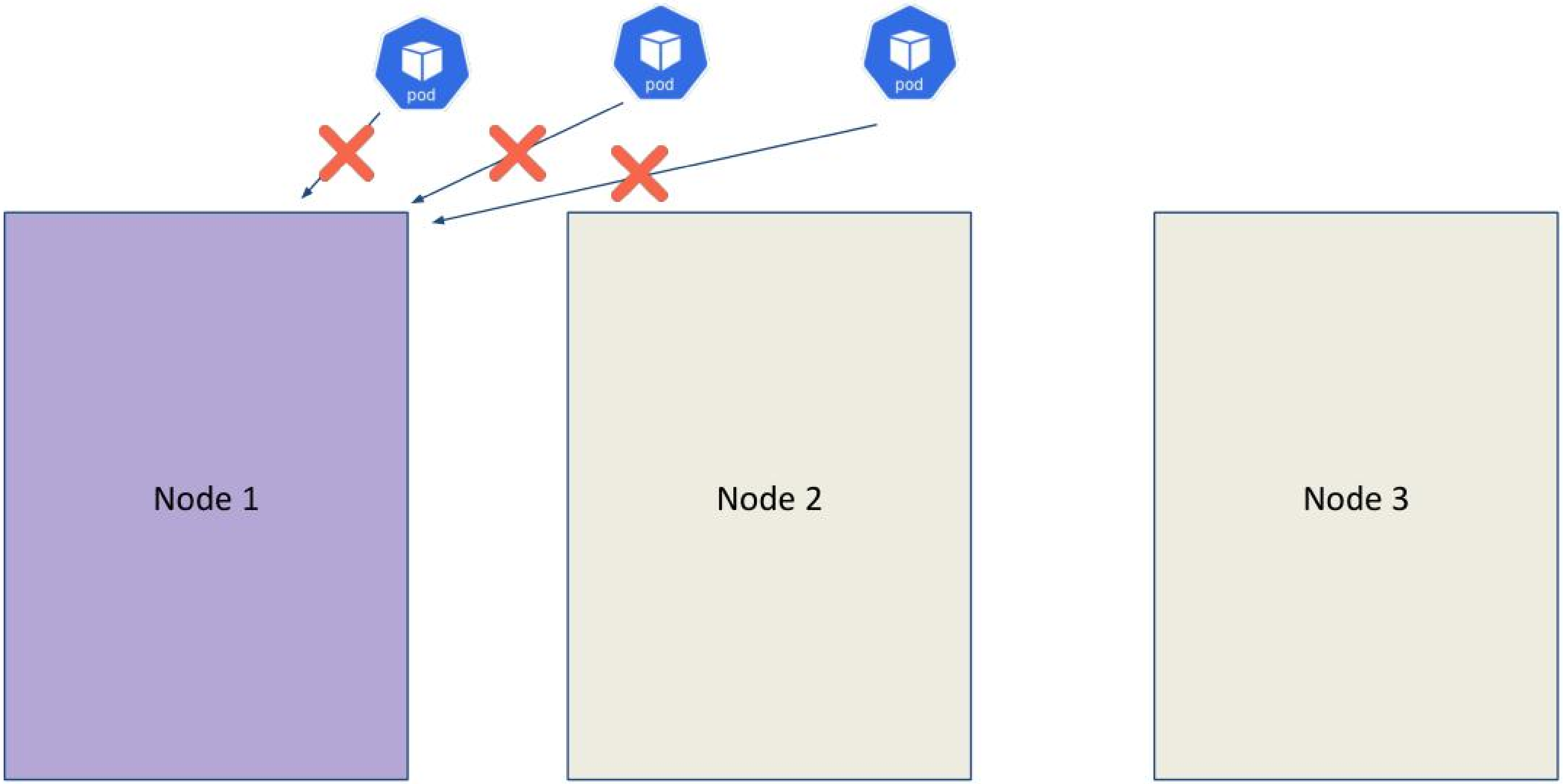
StatefulSet creates the amount of replica pods you specified, and **names them in order.**

What happens if we want to stop a pod from getting deployed on certain nodes?

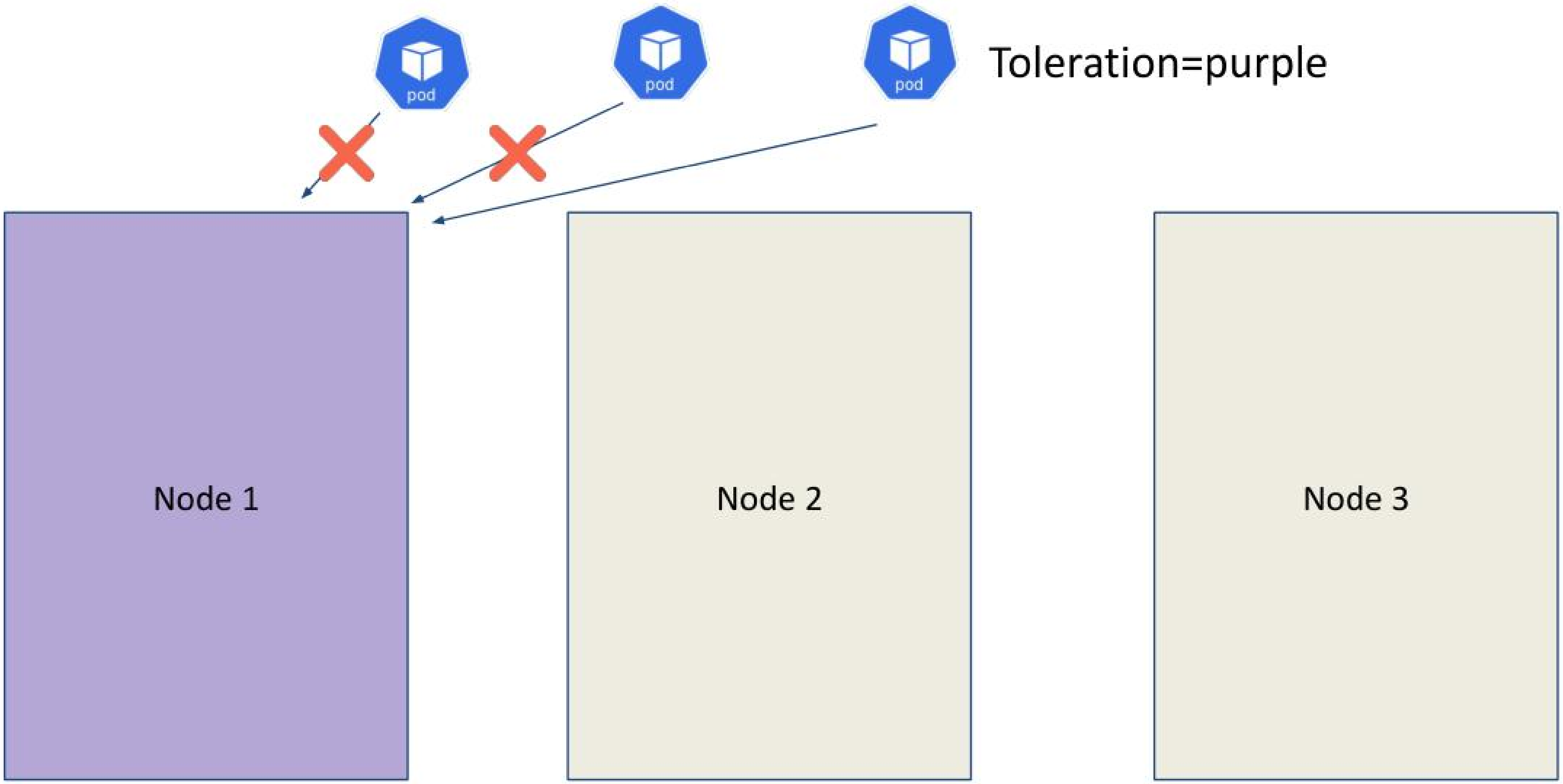
Taints & Toleration



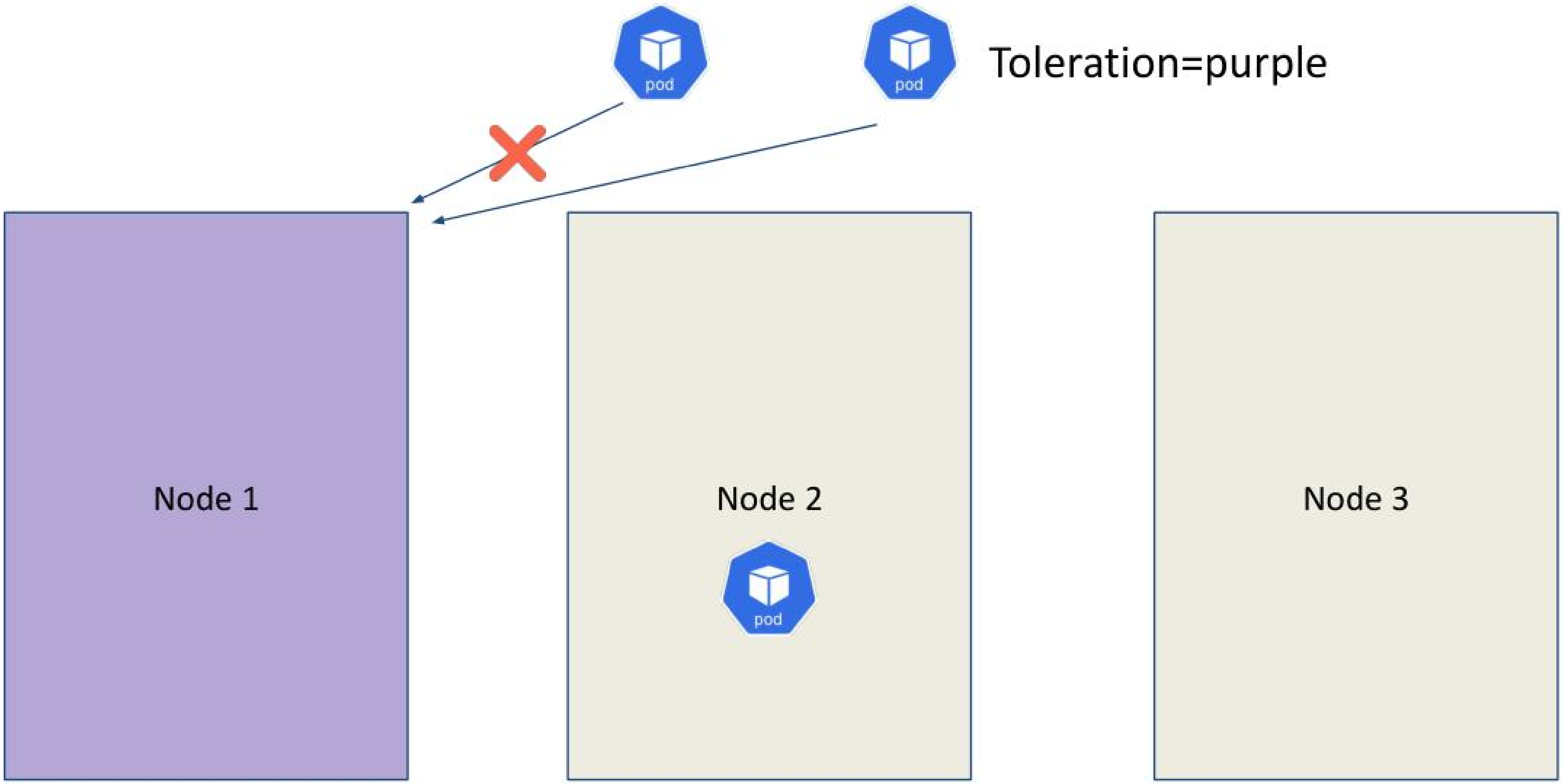
Taints & Toleration



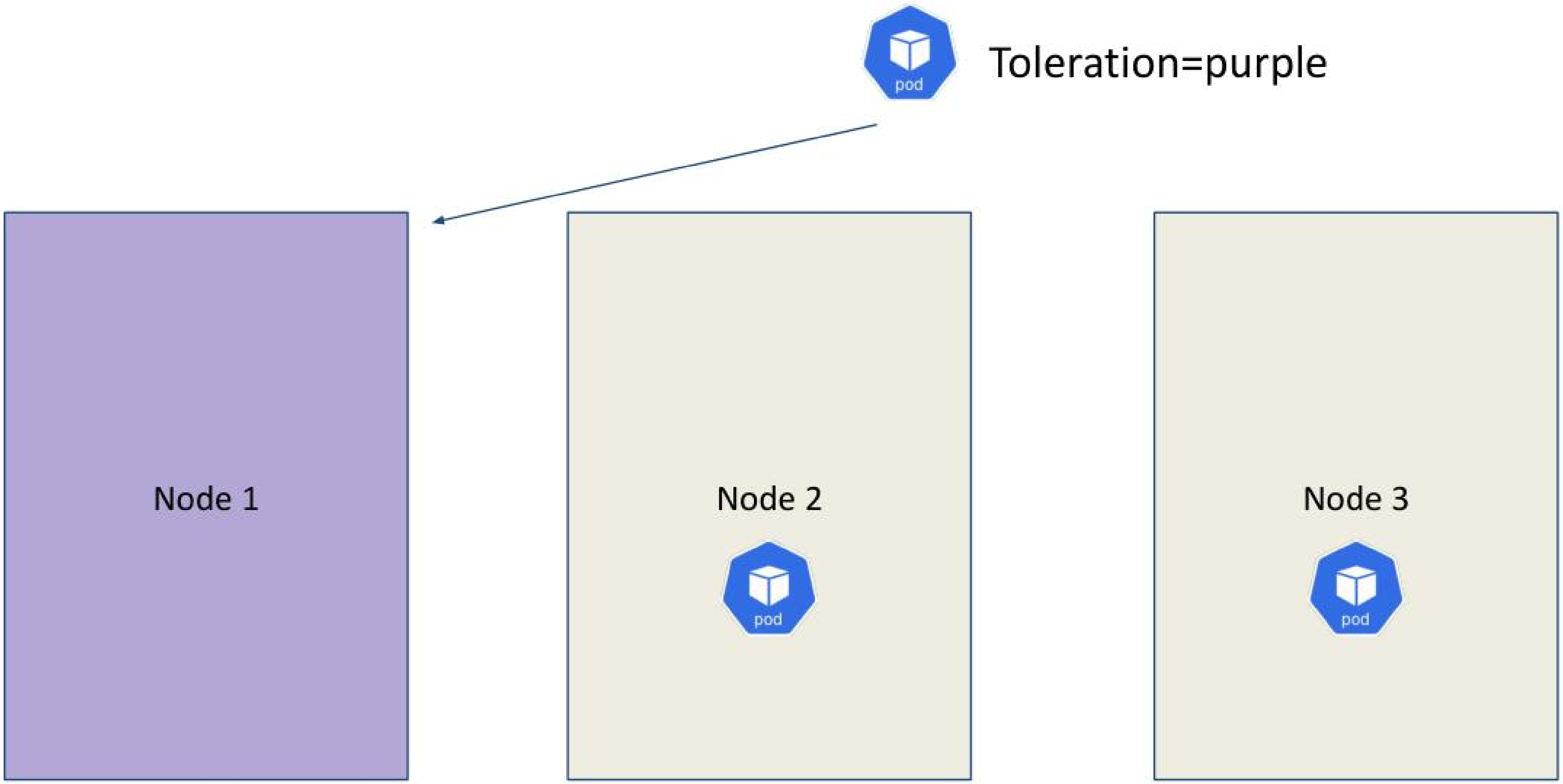
Taints & Toleration



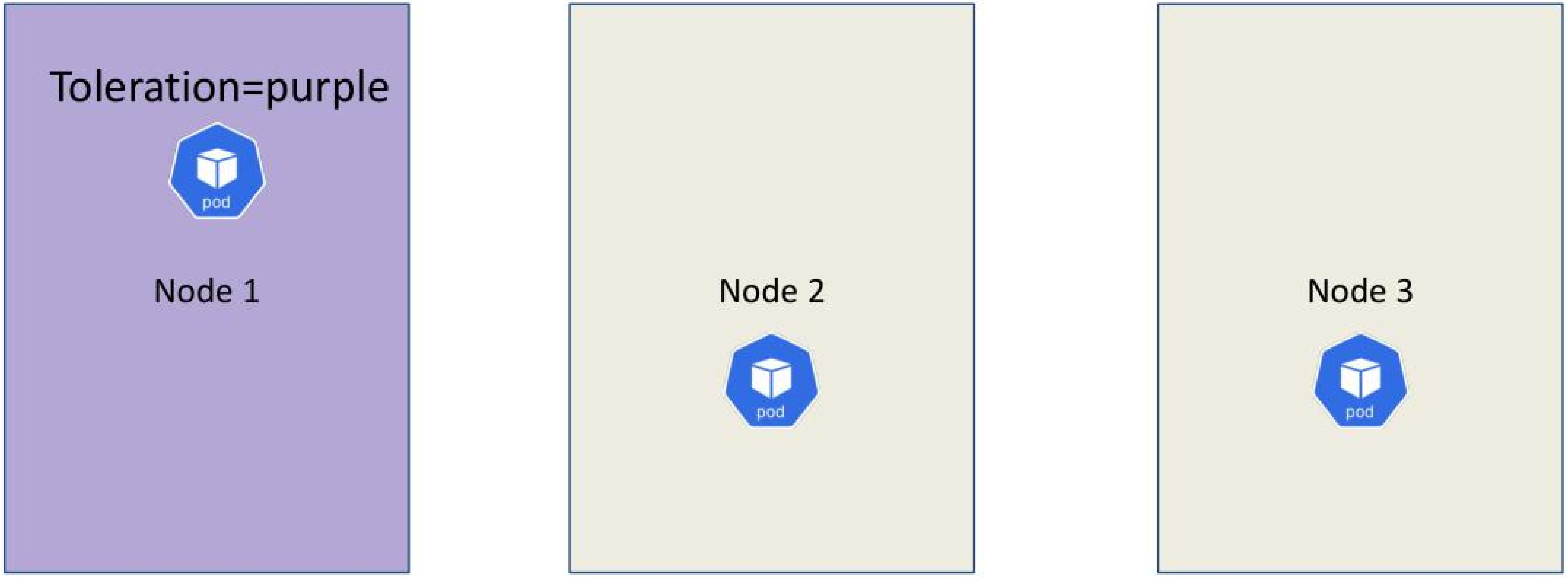
Taints & Toleration



Taints & Toleration



Taints & Toleration



What happens if we want a pod to only go to a specific node?

WRAP-UP

Day 2 done — you can deploy, scale and upgrade applications.

Tomorrow: Observability and Configuration & Security — probes, logging, ConfigMaps, Secrets, RBAC.

COURSE SLIDES

Certified Kubernetes Application Developer (CKAD)

WSQ Course Code: TGS-2025053212

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

DAY 3

Observability + Config & Security

Probes · Logging · Metrics · Debug · ConfigMaps · Secrets · SecurityContext · RBAC (CKAD Domains 3 & 4 — 40%)

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your phone camera and submit your attendance.
- A minimum of 75% attendance is required for assessment and funding.

Minimum 75% attendance required

DAY 3 OBJECTIVES

Recap & Today

- Yesterday (Domain 2): Deployments, rollouts, release strategies, Helm, Kustomize, Daemon/StatefulSet.
- Morning — Domain 3 Observability (15%): probes, logging, metrics, debugging, API deprecations.
- Afternoon — Domain 4 Configuration & Security (25%): ConfigMaps, Secrets, securityContext, RBAC, quotas.
- Together these two domains are 40% of the CKAD exam — the largest single day.

DOMAIN 3

Observability & Maintenance

Probes · logging · metrics · debugging · API versions (15%)

TOPIC 1

Health Probes

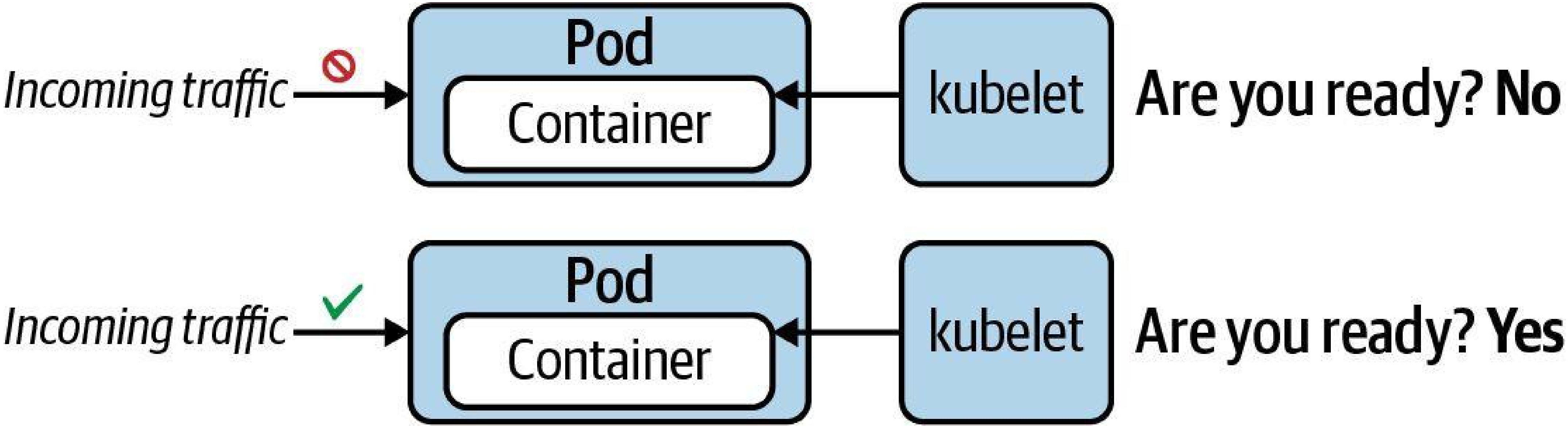
liveness · readiness · startup

Three Probes, Three Jobs

- livenessProbe — restarts the container when it has hung or deadlocked.
- readinessProbe — removes the Pod from Service endpoints until it can serve traffic (no restart).
- startupProbe — gives slow apps time to boot; it blocks the other two until it succeeds.
- Each probe uses one handler: httpGet, tcpSocket or exec — pick by what the container exposes.

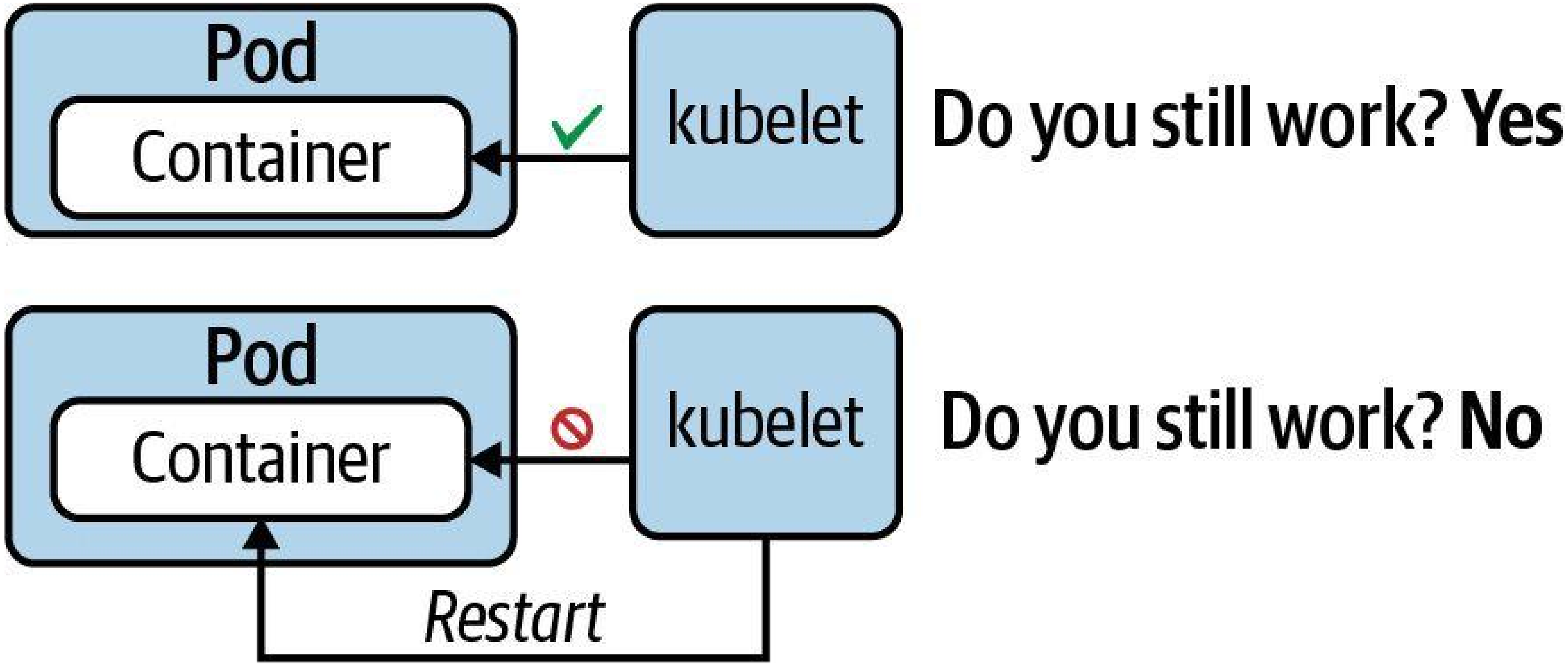
Readiness Probe

"Is application ready to handle requests?"



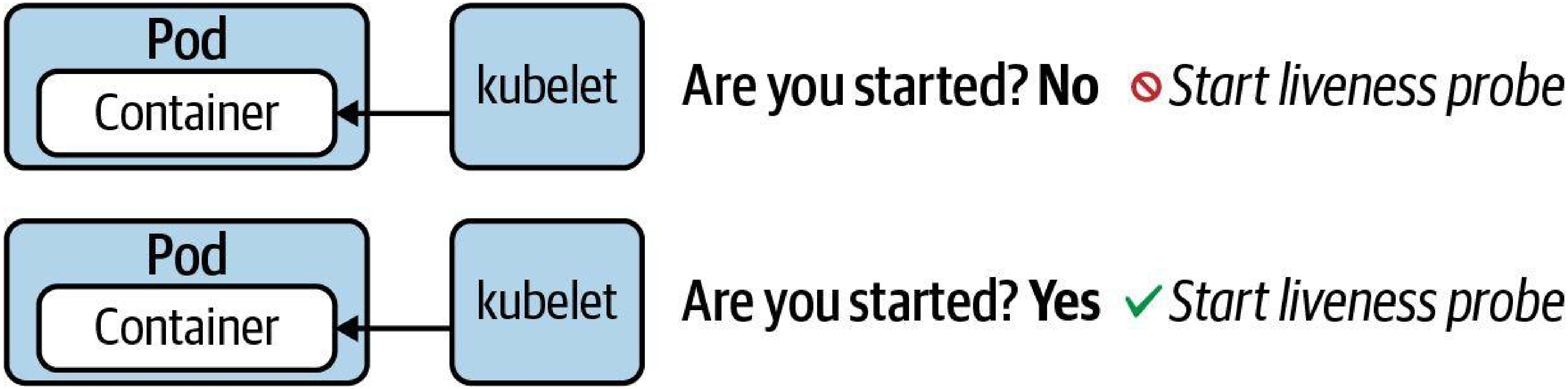
Liveness Probe

"Does the application still function without errors?"



Startup Probe

"Legacy application may need longer to start. Hold off on liveness probing."



Liveness, Readiness & Startup Probes

LAB 16 Probes

Kubernetes uses three probes to manage health: livenessProbe restarts a failed container, readinessProbe removes it from Service endpoints, startupProbe gives slow apps time to boot. CKAD tests all three handlers (httpGet, tcpSocket, exec) and timing.

You'll build

Add HTTP readiness+liveness probes, trigger a liveness restart, add a TCP probe, then a startupProbe with exec.

Key commands: readinessProbe: · k exec · startupProbe: · » Three

Liveness, Readiness & Startup Probes

Step 1 — HTTP liveness + readiness probe

```
readinessProbe:
  httpGet: {path: /, port: 80}
  initialDelaySeconds: 2
  periodSeconds: 5
livenessProbe:
  httpGet: {path: /, port: 80}
  initialDelaySeconds: 10
  periodSeconds: 10
  failureThreshold: 3
```

Step 2 — Trigger a liveness failure (watch RESTARTS climb)

```
k exec web-probes -- rm /usr/share/nginx/html/index.html
sleep 35
k get pod web-probes
```

Liveness, Readiness & Startup Probes (cont.)

Step 3 — TCP socket probe

```
readinessProbe:
  tcpSocket: {port: 6379}
  periodSeconds: 5
```

Step 4 — startupProbe for slow-starting apps

```
startupProbe:
  exec: {command: ["cat", "/tmp/ready"]}
  failureThreshold: 30
  periodSeconds: 5
```

Note Three handlers — httpGet, tcpSocket, exec. readinessProbe failure removes the Pod from Service endpoints without restarting it; livenessProbe failure restarts the container; startupProbe blocks the other two until the app is initialised.

Liveness, Readiness & Startup Probes

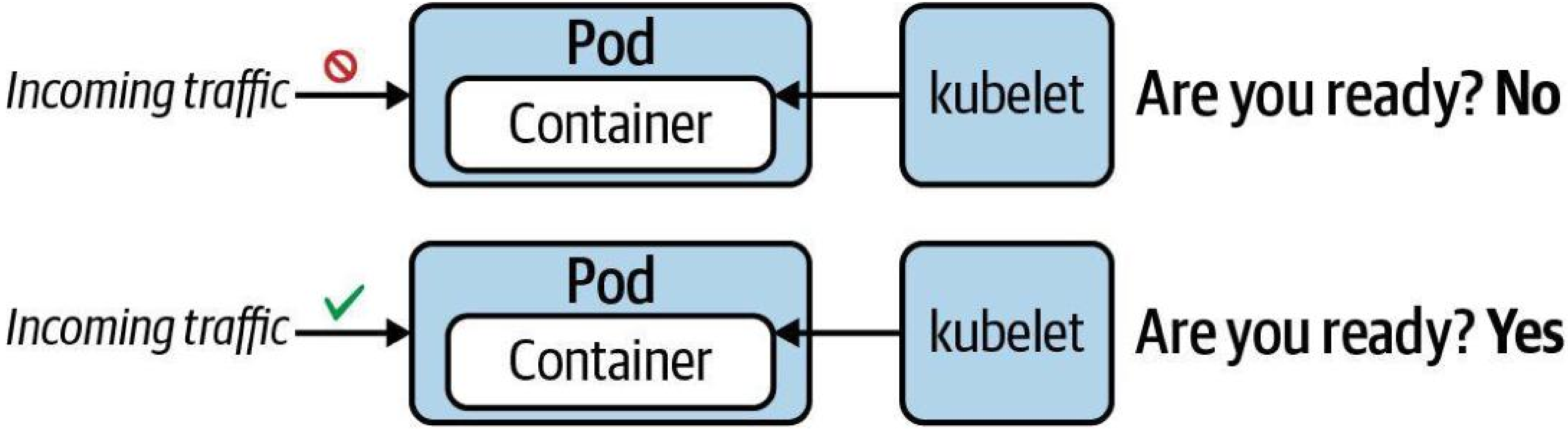
Test it

Deleting the index page makes the liveness probe fail three times and the container restarts (RESTARTS increments), while the TCP and startup Pods reach READY 1/1.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

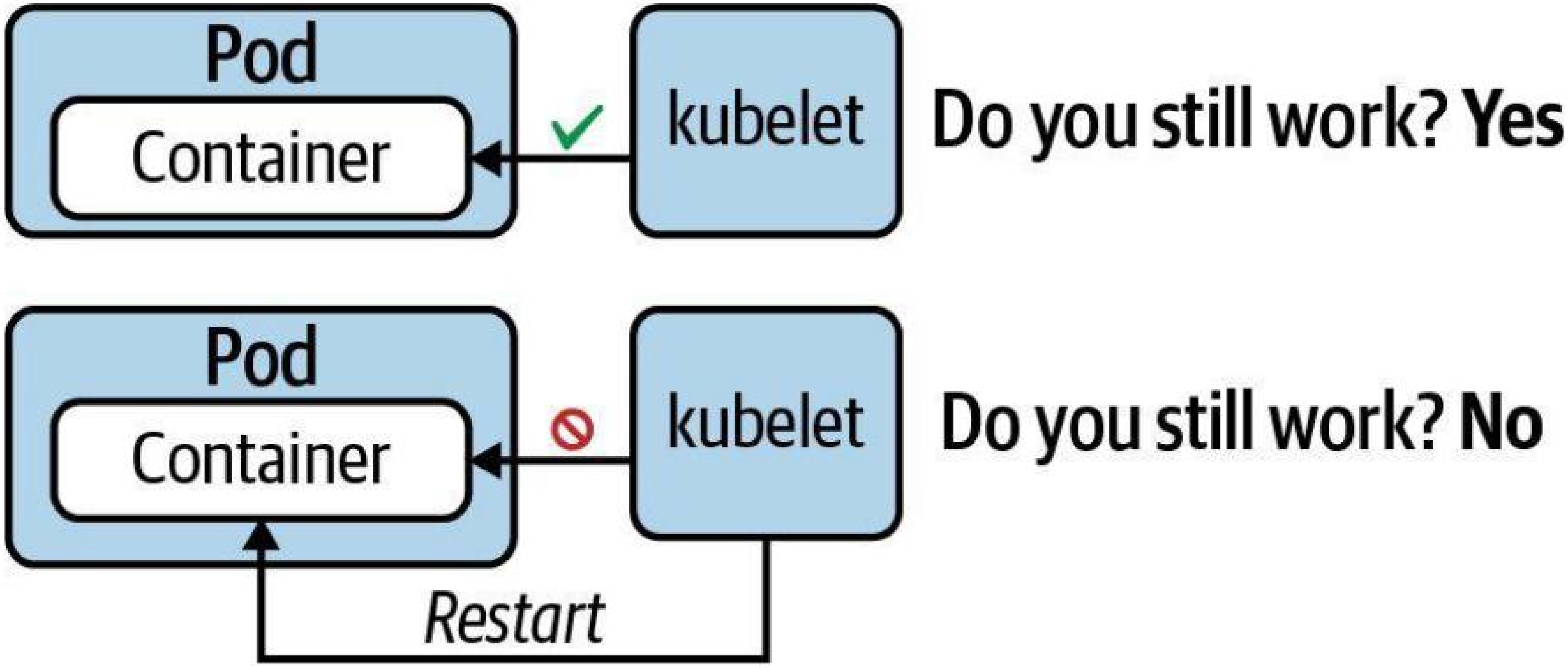
Readiness Probe

"Is application ready to handle requests?"



Liveness Probe

"Does the application still function without errors?"



Health Verification Methods

Any verification method can be used by any of the probe types

Method	Option	Description
Custom Command	`exec.command`	Executes a command inside of the container e.g. a cat command and checks its exit code. Kubernetes considers an zero exit code to be successful. A non-zero exit code indicates an error.
HTTP GET Request	`httpGet`	Sends an HTTP GET request to an endpoint exposed by the application. A HTTP response code in the range of 200 and 399 indicates success. Any other response code is regarded as an error.
TCP socket connection	`tcpSocket`	Tries to open a TCP socket connection to a port. If the connection could be established, the probing attempt was successful. The inability to connect is accounted for as an error.

Health Check Attributes

Any verification method can be used by any of the probe types

Attribute	Default Value	Description
`initialDelaySeconds`	0	Delay in seconds until first check is executed.
`periodSeconds`	10	Interval for executing a check (e.g., every 20 seconds).
`timeoutSeconds`	1	Maximum number of seconds until the check operation times out.
`successThreshold`	1	Number of successful check attempts until probe is considered successful after a failure.
`failureThreshold`	3	Number of failures for check attempts before probe is marked failed and takes action.
`terminationGracePeriodSeconds`	30	Grace period before forcing a container to stop upon failure.

Using a Named Port

Referencing a port by assigned name in multiple probes

```yaml
ports:
- name: http-port
  containerPort: 8080

readinessProbe:
  httpGet:
    path: /healthz
    port: http-port

livenessProbe:
  httpGet:
    path: /healthz
    port: http-port
```

TOPIC 2

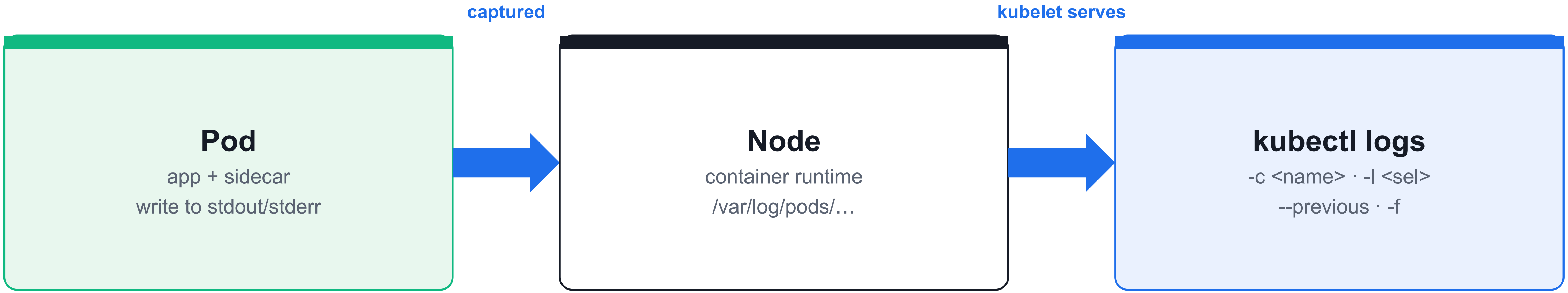
Logging

kubectl logs — your first debugging tool

Reading Container Logs

- Logs are whatever the container writes to stdout/stderr; the kubelet captures and serves them.
- --tail=N for recent lines; -f to stream live (like tail -f).
- --previous reads the prior terminated container — the key to diagnosing CrashLoopBackOff.
- -c <container> selects one container; -l <selector> aggregates logs across matching Pods.

Where Logs Come From



Logs are whatever the container prints to **stdout/stderr**. Use **--previous** for a crashed container.

Container Logging

LAB 17

Logging

kubectl logs is the first debugging tool on the exam. Read logs from single and multi-container Pods, follow a live stream, retrieve logs from a crashed container, and aggregate logs across a Deployment.

You'll build

Read and tail logs, stream with -f, recover crash logs with --previous, then aggregate by container and label.

Key commands: k logs · » `--tail=N`

LAB 17 · COMMANDS

Container Logging

Step 1 — Read recent lines and follow a live stream

```
k logs noisy --tail=10
k logs -f noisy &        # -f streams like tail -f
kill %1
```

Step 2 — Retrieve logs from a crashed container

```
k logs crasher --previous | head -5
```

Step 3 — Multi-container Pod logs

```
k logs multi -c app | head -3
k logs multi -c sidecar | head -3
k logs multi --all-containers=true --prefix=true | head -8
```

LAB 17 · COMMANDS

Container Logging (cont.)

Step 4 — Aggregate logs across a Deployment

```
k logs deployment/fleet --tail=3
k logs -l app=fleet --prefix=true --tail=2
```

Note --tail=N for recent lines, -f to stream, --previous for crash loops, -c <name> for a specific container, -l <selector> to aggregate matching Pods. --prefix=true labels each line with [pod/container].

Container Logging

Test it

k logs crasher --previous shows the message printed before the crash, and k logs -l app=fleet --prefix=true interleaves lines from all three Pods.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Tea Break

15 minutes

TOPIC 3

Metrics

kubectl top & the metrics-server add-on

MEASURE IT

Resource Usage at a Glance

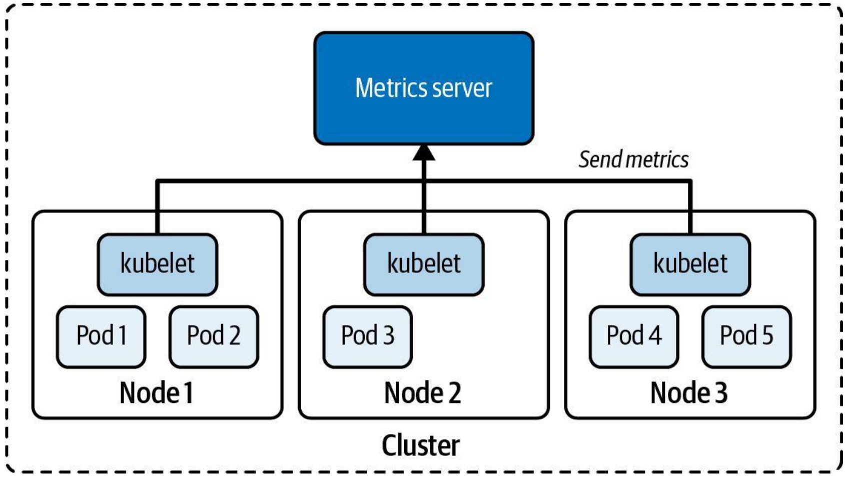
- kubectl top shows live CPU and memory for nodes and Pods — it requires metrics-server.
- metrics-server scrapes each kubelet's cAdvisor and serves the aggregated Metrics API.
- Sort with --sort-by=cpu / --sort-by=memory; span all namespaces with -A.
- On Killercoda's self-signed certs, patch the Deployment with --kubelet-insecure-tls.

What is the Metrics Server?

Cluster-wide resource metrics aggregator

- Gathers resource consumption metrics on nodes, e.g. CPU and memory. The kubelet sends those metrics to the Kubernetes API server and exposes them through the Metrics API.

- The metrics server stores data in memory and does not persist data over time.



kubectl top and Metrics Server

LAB 18 Metrics

kubectl top shows real-time CPU and memory for nodes and Pods. It requires the metrics-server add-on. CKAD tests installing metrics-server, reading node/Pod metrics, and sorting by resource usage.

You'll build

Install metrics-server, read node metrics, generate CPU load, then sort Pod metrics by CPU and memory.

Key commands: kubectl apply · k top · k run · » `kubectl

kubectl top and Metrics Server

Step 1 — Install metrics-server (and trust kubelet certs on Killercoda)

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubectl patch -n kube-system deployment metrics-server --type=json -p='
[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls"}]'
until kubectl top node 2>/dev/null; do sleep 5; done
```

Step 2 — Top nodes

```
k top node        # CPU cores/% and memory bytes/% per node
```

Step 3 — Generate load, then top Pods sorted

```
k run cpu-burner --image=busybox -- sh -c 'while true; do :; done'
sleep 30
k top pod --sort-by=cpu
k top pod --sort-by=memory
k top pod -A --sort-by=cpu | head -10
```

kubectl top and Metrics Server (cont.)

Note kubectl top needs metrics-server running. --kubelet-insecure-tls is required on Killercoda's self-signed certs. cpu-burner should top the CPU sort; -A spans all namespaces.

kubectl top and Metrics Server

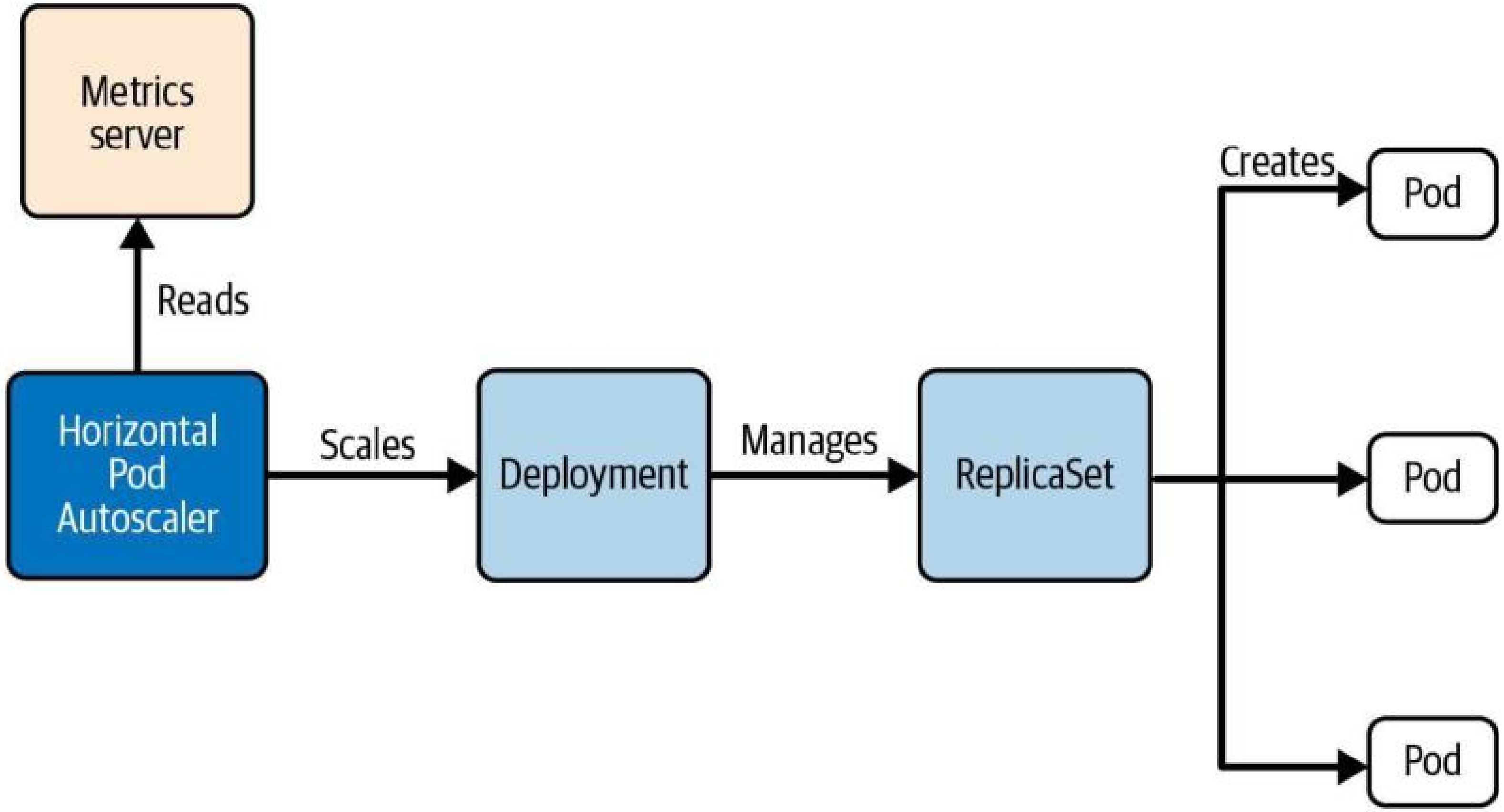
Test it

After ~30s, k top node reports CPU/memory and k top pod --sort-by=cpu lists cpu-burner at the top of the usage ranking.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Horizontal Pod Autoscaler (HPA)

Kubernetes primitive that reaches into metrics server



Metrics Server

Retrieving, inspecting, and using resource metrics

Querying for Metrics

Metrics for nodes and Pods can be rendered with the `top` command

```
$ kubectl top nodes
NAME            CPU(cores)   CPU%   MEMORY(bytes)
MEMORY%
controlplane    193m         9%     1239Mi          32%
node01          119m         5%     889Mi           23%

$ kubectl top pods
NAME       CPU(cores    MEMORY(bytes
fronten    ) 38m        ) 85Mi
d          29m          59Mi
backend
```

Kubernetes Features that Tap into Metrics

Where else does the metrics server come into play? (docs)

- A Horizontal Pod Autoscaler (HPA) can only determine if Deployment replicas need to be scaled up or down based on available resource consumption determined by metrics server. If the current resource consumption cannot be determined, it is indicated by `<unknown>` in the `TARGETS` column of an HPA.

- Pod scheduling on cluster nodes is based on the definition of resource requests. The scheduler looks at the resources capacity available on a cluster node and decides if the workload can be scheduled for the Pod's resource request.

TOPIC 4

Debugging

Diagnose and fix broken workloads under time pressure

TROUBLESHOOTING

The Three Failures You Will See

- ImagePullBackOff — wrong image name/tag; confirm in describe → Events, then fix and recreate.
- CrashLoopBackOff — container exits non-zero; read kubectl logs --previous.
- OOMKilled — exceeded the memory limit; raise resources.limits.memory.
- kubectl get events --sort-by=.lastTimestamp and kubectl debug (ephemeral container) are your fast tools.

Common Pod Error Statuses

Poorly-documented in Kubernetes docs, encountered often in practice

Status	Root Cause	Potential Fix
`ImagePullBackOff` or `ErrImagePull`	Image could not be pulled from registry.	Check correct image name, check that image name exists in registry, verify network access from node to registry, ensure proper authentication.
`CrashLoopBackOff`	Application or command run in container crashes.	Check command executed in container, ensure that image can properly execute (e.g., by creating a container with Docker Engine).
`CreateContainerConfigError`	ConfigMap or Secret referenced by container cannot be found.	Check correct name of the configuration object, verify the existence of the configuration object in the namespace.

Debugging Pods and Events

LAB 19 Debugging

CKAD regularly includes broken workloads you must diagnose and fix under time pressure. Identify and resolve ImagePullBackOff, CrashLoopBackOff and OOMKilled, and use kubectl debug for live triage.

You'll build

Diagnose three classic failures with describe/logs/events, then inject an ephemeral debug container.

Key commands: k describe · k get · » ImagePullBackOff

LAB 19 · COMMANDS

Debugging Pods and Events

Step 1 — ImagePullBackOff (wrong image name)

```
k describe pod typo | tail -15      # 'Failed to pull image'
k delete pod typo --force --grace-period=0
k run typo --image=nginx:1.25       # corrected
```

Step 2 — CrashLoopBackOff (container exits non-zero)

```
k describe pod crash | grep -A3 "Last State"
k logs crash --previous
```

Step 3 — OOMKilled (exceeds memory limit)

```
k describe pod hungry | grep -A4 "Last State"   # Reason: OOMKilled
# fix: raise resources.limits.memory
```

Debugging Pods and Events (cont.)

Step 4 — Cluster-wide events + ephemeral debug container

```
k get events -A --sort-by=.lastTimestamp | tail -20
k get events --field-selector type=Warning
k debug crash -it --image=busybox --target=crash -- sh
```

Note ImagePullBackOff → wrong image/tag. CrashLoopBackOff → use logs --previous. OOMKilled → raise the memory limit. kubectl debug injects an ephemeral container sharing the target's namespaces — vital for distroless images with no shell.

Debugging Pods and Events

Test it

k describe reveals the correct failure reason for each Pod (ImagePullBackOff, CrashLoopBackOff, OOMKilled), and k debug opens a shell inside the running Pod.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

TOPIC 5

API Deprecations

Find the right apiVersion with kubectl

STAY CURRENT

APIs Change — Know How to Look Them Up

- Old API versions are deprecated and eventually removed (e.g. extensions/v1beta1 Ingress, gone in 1.22).
- kubectl api-resources shows the correct apiVersion for every resource in its APIVERSION column.
- kubectl api-versions lists every version the cluster serves.
- kubectl explain <resource>.<field> --recursive documents fields without leaving the terminal.

Listing Available API version

`kubectl` can discover API version useable in manifests

```
$ kubectl api-versions
admissionregistration.k8s.io/v1
apiextensions.k8s.io/v1
apiregistration.k8s.io/v1 apps/v1
authentication.k8s.io/v1
authorization.k8s.io/v1 autoscaling/v1
autoscaling/v2
...
```

API Deprecations & kubectl explain

LAB 20 API Deprecations

The Kubernetes API evolves — old versions are deprecated and removed. CKAD tests kubectl explain, api-resources and api-versions. You must find the correct apiVersion for any resource on exam day using only kubectl.

You'll build

List resources and versions, explore schemas with explain, detect a removed API, and look up the current version.

Key commands: k api-resources · k explain · Pod/Service/ConfigMap/Secret/SA/Quota/LimitRange -> · » `kubectl

LAB 20 · COMMANDS

API Deprecations & kubectl explain

Step 1 — List resources, groups and served versions

```
k api-resources --api-group=apps
k api-resources --api-group=batch
k api-versions | sort
```

Step 2 — Explore a schema with explain

```
k explain deployment.spec.strategy.rollingUpdate
k explain pod.spec.containers.resources --recursive | head -30
```

Step 3 — Detect a removed API and find the replacement

```
# extensions/v1beta1 Ingress was removed in 1.22
k explain ingress --api-version=networking.k8s.io/v1 | head -10
k api-resources | grep -i ingress
```

API Deprecations & kubectl explain (cont.)

Step 4 — Stable apiVersions to memorise (v1.35)

```
Pod/Service/ConfigMap/Secret/SA/Quota/LimitRange  -> v1
Deployment/StatefulSet/DaemonSet/ReplicaSet        -> apps/v1
Job/CronJob                                        -> batch/v1
Ingress/NetworkPolicy                              -> networking.k8s.io/v1
Role/RoleBinding/ClusterRole(+Binding)             -> rbac.authorization.k8s.io/v1
HorizontalPodAutoscaler                            -> autoscaling/v2
```

Note kubectl api-resources shows the correct apiVersion in its APIVERSION column — the exam-legal way to look it up. kubectl explain <res>.<field> documents fields without leaving the terminal.

API Deprecations & kubectl explain

Test it

k api-resources | grep -i ingress shows networking.k8s.io/v1, and k explain deployment.spec.strategy.rollingUpdate prints the maxSurge/maxUnavailable fields.

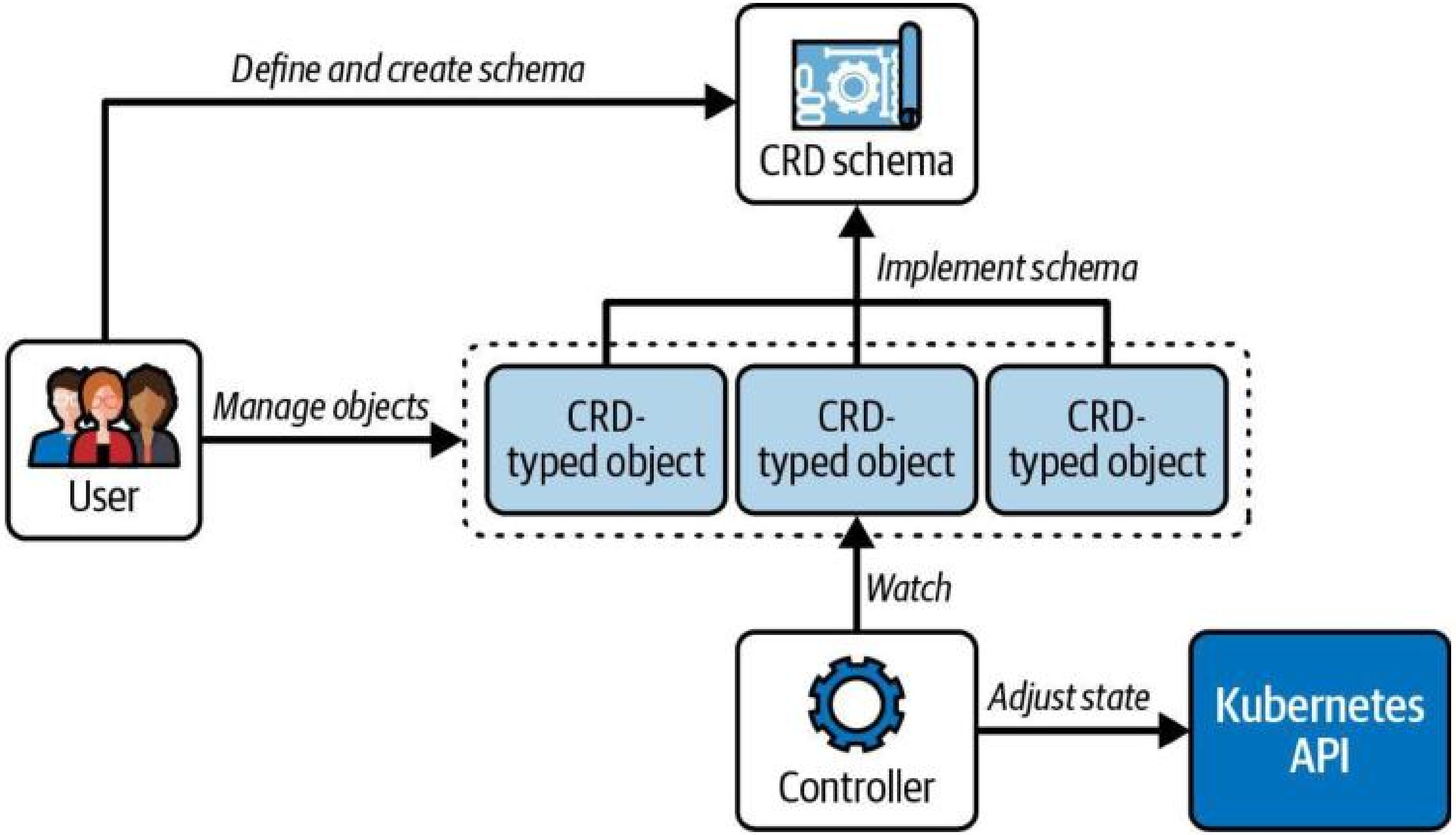
Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Custom Resource Definitions (CRDs)

Extending the Kubernetes API by custom primitives

The Operator Pattern

Consists of CRDs and Controllers



API Deprecations

Kubernetes release cycle, API deprecations and removals

Deprecation Policy

Some APIs will be deprecated and eventually removed

- The Kubernetes project releases 3 versions per year (see release cadence).
- With each release, an API can become deprecated which means that is scheduled for removal or replacement. You can find more information about the details in the Kubernetes Deprecation Policy.
- The use of a deprecated API renders a warning message when creating the object that uses it.
- The Deprecated API Migration Guide shows a list of deprecated APIs and the scheduled releases that will remove support for the API.

Deprecated HPA API Warning Message

`kubectl` renders a warning message, but creates object

```
$ kubectl create -f hpa.yaml
Warning: autoscaling/v2beta2 HorizontalPodAutoscaler is deprecated
↵in v1.23+, unavailable in v1.26+; use autoscaling/v2     ↵
HorizontalPodAutoscaler
```

Removed ClusterRole API

Uses removed API with Kubernetes version 1.22

```yaml
apiVersion:
kind: ClusterRole
metadata:
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```

The **rbac.authorization.k8s.io/v1beta1** API version of ClusterRole, ClusterRoleBinding, Role, and RoleBinding is no longer served as of v1.22.

Removed ClusterRole API Error Message

`kubectl` renders an error message, as API does not exist anymore

```
$ kubectl create -f clusterrole.yaml
error: resource mapping not found for name: "pod-reader" namespace:
↵"" from "clusterrole.yaml": no matches for kind "ClusterRole" in
  ↵version "rbac.authorization.k8s.io/v1beta1"
```

Using the ClusterRole API Replacement

Usage of replacement API

```
apiVersion:
kind: ClusterRole
metadata:
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```

- Migrate manifests and API clients to use the **rbac.authorization.k8s.io/v1** API version, available since v1.8.
- All existing persisted objects are accessible via the new APIs
- No notable changes

Lunch Break

1 hour

DOMAIN 4

Configuration & Security

ConfigMaps · Secrets · securityContext · ServiceAccounts · RBAC · quotas (25%)

TOPIC 6

ConfigMaps

Inject non-secret configuration into Pods

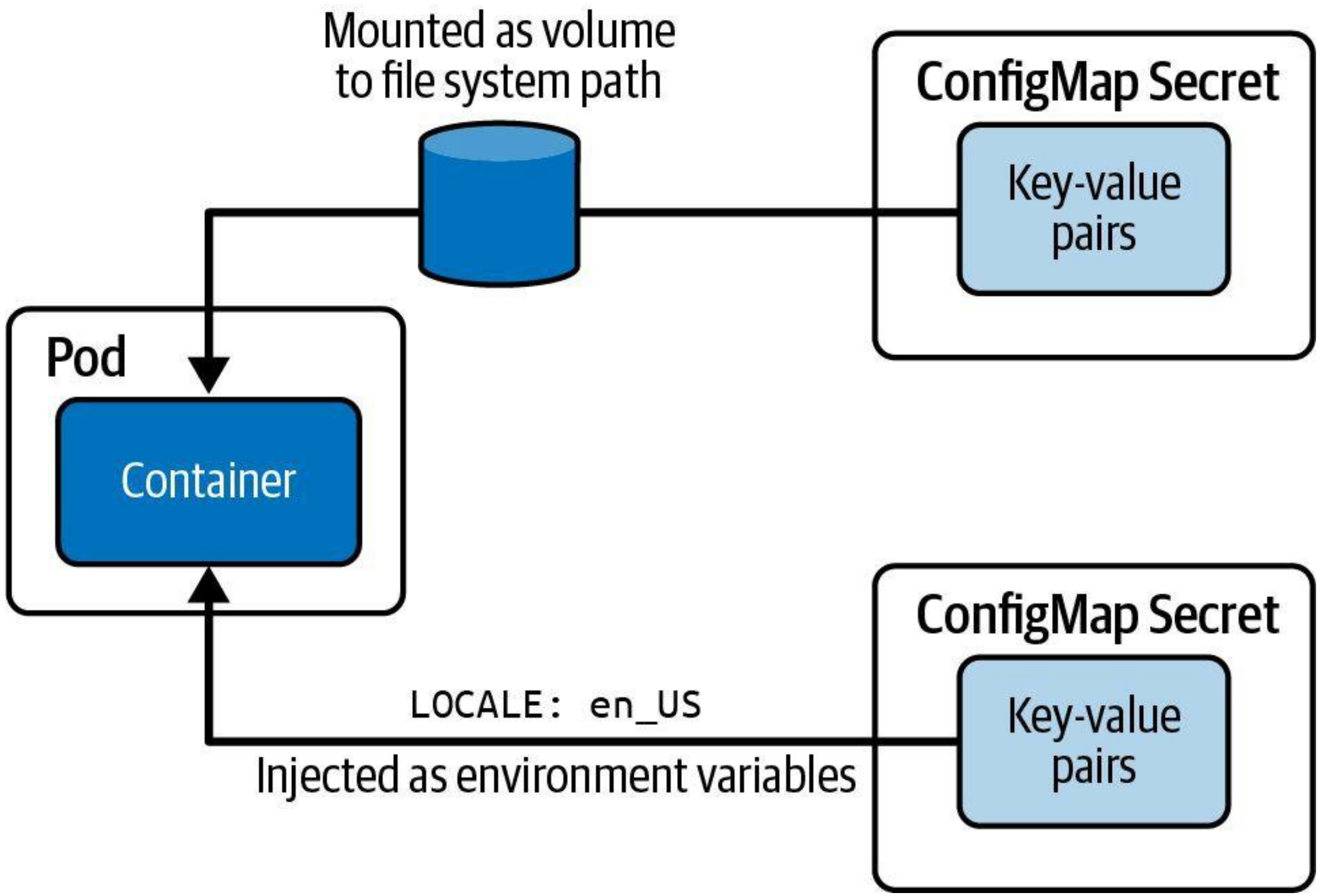
EXTERNAL CONFIG

Decouple Config from Image

- A ConfigMap holds non-secret key/value data; create it with --from-literal, --from-file or --from-env-file.
- Inject one key with configMapKeyRef, all keys with envFrom, or mount it as a file volume.
- Volume-mounted ConfigMaps update live (~60s); env-var injections are fixed at Pod startup.
- Keep config out of the image so the same image runs in every environment.

Consuming Configuration Data

Applicable to ConfigMap and Secret



Consuming a ConfigMap as Environment Variables

Controlling runtime behavior with the help of simple values

```yaml
apiVersion: v1 kind: Pod
metadata:
  name: backend
spec:
  containers:
  - image: bmuschko/web-app:1.0.1
    name: backend
    envFrom:
    - configMapRef: name: db-
        config
```

```
$ kubectl exec -it backend -- env
DB_HOST=mysql-service
DB_USER=backend
```

ConfigMaps — Env & Volume Injection

LAB 21 ConfigMaps

ConfigMaps inject non-secret configuration into Pods. CKAD tests all three injection styles: a single env var (configMapKeyRef), bulk env vars (envFrom), and file-based volume mounts — and that only file mounts update live.

You'll build

Create ConfigMaps three ways, inject one key, inject all keys with envFrom, mount as files, then update live.

Key commands: k create · env: · envFrom: · volumeMounts: · » Creation:

ConfigMaps — Env & Volume Injection

Step 1 — Create ConfigMaps three ways

```
k create configmap app-cfg --from-literal=COLOR=blue --from-literal=GREETING=hello
k create configmap app-conf --from-file=app.conf
k create configmap app-env  --from-env-file=env.list
```

Step 2 — Inject a single key as an env var

```
env:
- name: COLOR
  valueFrom:
    configMapKeyRef: {name: app-cfg, key: COLOR}
```

Step 3 — Inject all keys with envFrom

```
envFrom:
- configMapRef: {name: app-cfg}
- configMapRef: {name: app-env}
```

ConfigMaps — Env & Volume Injection (cont.)

Step 4 — Mount a ConfigMap as a file volume

```
volumeMounts:
- {name: cfg, mountPath: /etc/app}
volumes:
- name: cfg
  configMap: {name: app-conf}
```

Note Creation: --from-literal, --from-file, --from-env-file. Consumption: configMapKeyRef (one key), envFrom (all keys), volume mount (file per key). Volume-mounted ConfigMaps update live (~60s); env-var injections are fixed at Pod startup and need a restart.

ConfigMaps — Env & Volume Injection

Test it

The single-key Pod logs COLOR=blue, the envFrom Pod shows all four variables, and the mounted ConfigMap appears as files under /etc/app.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Creating a ConfigMap with Imperative Approach

Key-value pairs can be parsed from different sources

```
$ kubectl create configmap db-config --from-literal=db=staging
configmap/db-config created

$ kubectl create configmap db-config --from-env-file=config.env
configmap/db-config created

$ kubectl create configmap db-config --from-file=config.txt
configmap/db-config created

$ kubectl create configmap db-config --from-file=app-config
configmap/db-config created
```

Source Options for Data Parsed by a ConfigMap

Configuration options and example values

Option	Description	Example Value
`--from-literal`	Literal values, which are key-value pairs as plain text.	`--from-literal=locale=en_US`
`--from-env-file`	A file that contains key-value pairs and expects them to be environment variables.	`--from-env-file=config.env`
`--from-file`	Huge page resource type.	`--from-file=app-config.json`
`--from-file`	A directory with one or many files.	`--from-file=config-dir`

ConfigMap - Imperative Approach

Configuration options and example values

```
kubectl create configmap color-config-map --from-literal=APP_COLOR="black"
```

Create configmap command

Name of configmap

Key-value pair of configmap

ConfigMap YAML Manifest

Key-value pair definition under `data` attribute

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: hello-configmap
data:
  APP_COLOR: black
  demo_greeting: Welcome to Kubernetes 101
```

Data of configmap

ConfigMap YAML Manifest

Key-value pair definition under `data` attribute

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: hello-configmap
data:
  APP_COLOR: black
  demo_greeting: Welcome to Kubernetes 101
```

```
kubectl create –f configmap.yaml
```

ENV Variables - Define all key-value pairs in configmap

Key-value pair definition under `data` attribute

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: sample-pod-1
spec:
  containers:
  - name: sample-pod-1
    image: python:3.6-alpine
    envFrom:
    - configMapRef:
        name: color-configmap
```

```yaml
envFrom:
- configMapRef:
    name: color-configmap
```

```
Environment Variables from:
  color-configmap  ConfigMap
```

ENV Variables - Define via Multiple ConfigMaps

Key-value pair definition under `data` attribute

```
env:
 - name: color
   valueFrom:
    configMapKeyRef:
      name: color-configmap
      key: APP_COLOR
 - name: demo_greeting
   valueFrom:
     configMapKeyRef:
       name: hello-configmap-file
       key: demo_greeting
```

From color-configmap

From hello-configmap-file

```
Environment:
  color:          <set to the key 'APP_COLOR' of config map 'color-configmap'>
  demo_greeting:  <set to the key 'demo_greeting' of config map 'hello-configmap-file'>
```

Consuming Changed ConfigMap Data

Containers will not refresh consumed data upon a change

- The key-value pairs of a ConfigMap can be changed for a live object.
- Changed ConfigMap key-value pairs consumed by containers will not be reloaded automatically for an already-running Pod.
- The application code needs to implement logic to either reload the configuration data periodically or on-demand.

TOPIC 7

Secrets

Sensitive configuration, guarded by RBAC

SENSITIVE DATA

Secrets vs ConfigMaps

- Secrets work like ConfigMaps but values are base64-encoded and access is more tightly controlled.
- Types: generic, tls (kubernetes.io/tls) and docker-registry (for imagePullSecrets).
- Inject with envFrom: secretRef (all keys) or secretKeyRef (one key); mount with defaultMode: 0400.
- Base64 is not encryption — restrict access with RBAC and enable encryption at rest.

Consuming a Secret as Environment Variables

Accessed values are base64-decoded

```yaml
apiVersion: v1 kind: Pod
metadata:
  name: backend
spec:
  containers:
  - image: bmuschko/web-app:1.0.1
    name: backend



    envFrom:
    - secretRef:
        name: mysecret
```

```
$ kubectl exec -it backend -- env
pwd=s3cre!
```

Consuming a Secret as a Volume

A good choice for processing a machine-readable configuration file

- apiVersion: `v1` kind: `Pod` metadata:
- name: `backend`
- spec:
- containers:

  –name: `backend`
  - image:
- `bmuschko/web-app:1.0.1`
  - volumeMounts:
    - – name: `ssh-volume` mountPath: `/var/app` readOnly: `true`
- volumes:

```
$ kubectl exec -it backend -- /bin/sh
# ls -l /var/app ssh-privatekey
```

```
# cat /var/app/ssh-privatekey
-----BEGIN RSA PRIVATE KEY----- Proc-Type:
4,ENCRYPTED
DEK-Info:
AES-128-CBC,8734C9153079F2E8497C8075289E BBF1
...
-----END RSA PRIVATE KEY-----
```

Secrets

LAB 22 Secrets

Secrets are like ConfigMaps but base64-encoded with stricter access. CKAD tests generic, tls and docker-registry types, env-var injection, file mounts with defaultMode, and decoding values. Secrets are not encrypted — protect them with RBAC.

You'll build

Create a generic Secret, decode it, inject as env vars, mount with 0400 mode, then create tls and docker-registry Secrets.

Key commands: k create · envFrom: · volumes: · » Types:

Secrets

Step 1 — Create a generic Secret and decode a value

```
k create secret generic db-cred \
  --from-literal=DB_USER=admin --from-literal=DB_PASS=s3cr3t
k get secret db-cred -o jsonpath='{.data.DB_PASS}' | base64 -d; echo
```

Step 2 — Inject Secret keys as env vars

```
envFrom:
- secretRef: {name: db-cred}
```

Step 3 — Mount a Secret as files with restrictive mode

```
volumes:
- name: s
  secret:
    secretName: db-cred
    defaultMode: 0400
```

Secrets (cont.)

Step 4 — TLS and docker-registry Secrets

```
k create secret tls demo-tls --cert=tls.crt --key=tls.key
k create secret docker-registry myreg \
  --docker-server=registry.example.com --docker-username=u \
  --docker-password=p --docker-email=u@example.com
```

Note Types: generic, tls (kubernetes.io/tls), docker-registry. envFrom: secretRef injects all keys; secretKeyRef injects one. Reference a registry Secret via spec.imagePullSecrets. Base64 ≠ encryption — restrict with RBAC.

Secrets

Test it

base64 -d on .data.DB_PASS returns s3cr3t, the mounted Secret files are mode 0400, and the tls Secret reports type kubernetes.io/tls.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Creating a Secret with Imperative Approach

Parsed values are base64-encoded automatically

```
kubectl create secret generic system-secret --from-literal=system_admin=p@ssword999
```

Create secret command

Name of secret

Value of secret

```
apiVersion: v1
data:
  system_admin: cEBzc3dvcmQ5OTk=
kind: Secret
metadata:
  creationTimestamp: "2020-08-21T05:50:27Z"
  name: system-secret
  namespace: default
  resourceVersion: "1042130"
  selfLink: /api/v1/namespaces/default/secrets/system-secret
  uid: 50f8462b-1125-4f5b-ba82-4eeabf11c274
```

Source Options for Data Parsed by a Generic Secret

Configuration options and example values

Option	Description	Example Value
`--from-literal`	Literal values, which are key-value pairs as plain text.	`--from-literal=password=secret`
`--from-env-file`	A file that contains key-value pairs and expects them to be environment variables.	`--from-env-file=config.env`
`--from-file`	Huge page resource type.	`--from-file=id_rsa=~/.ssh/id_rsa`
`--from-file`	A directory with one or many files.	`--from-file=config-dir`

Create Secret from File

Configuration options and example values

secrets.properties

```
DB_Admin: admin
DB_Password: p@ssword
```

```
kubectl create secret generic credentials-secret-file --from-file=credentials_secret.properties
```

Create secret command

Name of secret

Filepath of the secret

Declarative Approach

Configuration options and example values

- Declarative way: create yaml file

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: credentials-secret
data:
  DB_Admin: admin
  DB_User: root
  DB_Password: p@ssword
```

```
Error from server (BadRequest): error when creating "credentials-secret.yaml": Secret in version "v1" cannot be handled
 as a Secret: v1.Secret.Data: decode base64: illegal base64 data at input byte 4, error found in #10 byte of ...|n":"ad
min","DB_Passw|..., bigger context ...|{"apiVersion":"v1","data":{"DB_Admin":"admin","DB_Password":"p@ssword","DB_User"
:"root"},"kind"|...
```

Declarative Approach

Encoding Secrets

```
DB_Admin: admin
DB_User: root
DB_Password: p@ssword
```

→

```
DB_Admin: YWRtaW4=
DB_User: cm9vdA==
DB_Password: cEBzc3dvcmQ=
```

echo –n ‘admin’ | base64 → YWRtaW4=

echo –n ‘root’ | base64 → cm9vdA==

echo –n ‘p@ssword’ | base64 → cEBzc3dvcmQ=

Declarative Approach

Decoding Secrets

```
DB_Admin: YWRtaW4=
DB_User: cm9vdA==
DB_Password: cEBzc3dvcmQ=
```

```
DB_Admin: admin
DB_User: root
DB_Password: p@ssword
```

echo ‘YMRtaW4==’ | base64 --decode	admin
echo ‘cm9vdA==’ | base64 --decode	root
echo ‘cEBzc3dvcmQ=’ | base64 --decode	p@ssword

Describing Secrets

```
kubectl describe secret credentials-secret
```

```
Name:          credentials-secret
Namespace:     default
Labels:        <none>
Annotations:   <none>

Type:  Opaque

Data
====
DB_Admin:      5 bytes
DB_Password:   8 bytes
DB_User:       4 bytes
```

Viewing Secrets

```
kubectl get secret credentials-secret –o yaml
```


```yaml
apiVersion: v1
data:
  DB_Admin: YWRtaW4=
  DB_Password: cEBzc3dvcmQ=
  DB_User: cm9vdA==
kind: Secret
metadata:
  creationTimestamp: "2020-06-19T06:31:51Z"
  name: credentials-secret
  namespace: default
  resourceVersion: "29616"
  selfLink: /api/v1/namespaces/default/secrets/credentials-secret
  uid: 276b2049-8588-418f-87a4-91a1851a9c88
type: Opaque
```

Secrets Are Not Really Secret

Techniques that help with making Secrets more secure

- Secret objects are stored in etcd in unencrypted form by default. Encryption of data in etcd is configurable.

- Define RBAC permissions to only allow developers to create, view, and modify Secret objects in dedicated namespaces. An even stricter policy could only allow administrators to manage Secrets.

- Use Kubernetes-external, third-party Secrets services for managing sensitive data, e.g. AWS Secrets Manager or HashiCorp Vault. You can consume the data with the help of the External Secrets Operator.

Source Options for Data Parsed by a Generic Secret

Configuration options and example values

Option	Description	Example Value
`--from-literal`	Literal values, which are key-value pairs as plain text.	`--from-literal=password=secret`
`--from-env-file`	A file that contains key-value pairs and expects them to be environment variables.	`--from-env-file=config.env`
`--from-file`	Huge page resource type.	`--from-file=id_rsa=~/.ssh/id_rsa`
`--from-file`	A directory with one or many files.	`--from-file=config-dir`

Tea Break

15 minutes

TOPIC 8

SecurityContext

Control container identity & privileges

LEAST PRIVILEGE

Run Containers Safely

- securityContext sets the user, group and privileges a container runs with.
- Pod-level applies to all containers; container-level overrides one container.
- runAsNonRoot: true blocks any image whose default user is root.
- Hardened pattern: readOnlyRootFilesystem + capabilities.drop:[ALL] + allowPrivilegeEscalation:false.

Defining a Security Context

Pod- and/or container-level definition

```yaml
apiVersion: v1 kind:
Pod metadata:
  name: secured-pod
spec:



   securityContext:
     fsGroup: 3000

   volumes:
   - name: data-vol
     emptyDir: {}
   containers:


   - image: nginx:1.18.0 name:
     secured-container volumeMounts:
     - name: data-vol
       mountPath: /data/app
```

Defined on the Pod-level (applies to all containers)

SecurityContext

LAB 23 SecurityContext

securityContext controls the identity and privileges of containers. CKAD asks you to enforce non-root execution, a read-only root filesystem and dropped Linux capabilities — at Pod level (all containers) or container level (one override).

You'll build

Run as a set user/group, enforce runAsNonRoot, lock the root filesystem, then drop all capabilities.

Key commands: securityContext: · » Pod-level

SecurityContext

Step 1 — Run as a specific user and group (Pod level)

```
securityContext:
  runAsUser: 1000
  runAsGroup: 3000
  fsGroup: 2000     # applies to volume mounts
```

Step 2 — Enforce runAsNonRoot (blocks root images)

```
securityContext:
  runAsNonRoot: true
# nginx runs as root -> Pod refuses to start
```

Step 3 — Read-only root filesystem (container level)

```
securityContext:
  readOnlyRootFilesystem: true
# mount an emptyDir for any writable path (e.g. /tmp)
```

SecurityContext (cont.)

Step 4 — Drop Linux capabilities (hardened container)

```
securityContext:
  capabilities:
    drop: ["ALL"]
    add: ["NET_BIND_SERVICE"]
  allowPrivilegeEscalation: false
```

Note Pod-level securityContext applies to all containers; container-level overrides one. The CKAD hardened pattern = runAsNonRoot: true + readOnlyRootFilesystem: true + capabilities.drop: [ALL] + allowPrivilegeEscalation: false.

SecurityContext

Test it

The non-root Pod logs uid=1000 gid=3000, the runAsNonRoot nginx Pod refuses to start, and writes to the read-only root filesystem are rejected.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Security Context

Privilege and access control for a Pod

Overriding a Value on the Container-Level

Container-level definition wins

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: non-root-success
spec:
  securityContext:
    runAsNonRoot: false
  containers:
  - image: bitnami/nginx:1.23.1
    name: nginx
    securityContext:
      runAsNonRoot: true
```

Takes precedence

TOPIC 9

ServiceAccounts

The identity a Pod presents to the API

WHO AM I

Pod Identity

- Every Pod runs as a ServiceAccount — the default one has minimal permissions.
- spec.serviceAccountName attaches a dedicated ServiceAccount to a Pod.
- A token, CA cert and namespace are projected into /var/run/secrets/kubernetes.io/serviceaccount/.
- automountServiceAccountToken: false enforces least privilege; kubectl create token mints short-lived tokens.

Service Account as Subject

Pod assign the Service Account by name

ServiceAccounts

LAB 24

ServiceAccounts

Every Pod runs as a ServiceAccount; the default one has minimal RBAC. CKAD tests creating dedicated ServiceAccounts, attaching them to Pods, disabling the auto-mounted token, and requesting short-lived tokens with kubectl create token.

You'll build

Create a ServiceAccount, attach it to a Pod, inspect the projected token, disable auto-mount, then mint a token.

Key commands: k create · k exec · k patch · TOKEN=$(k create · » `spec.serviceAccountName`

ServiceAccounts

Step 1 — Create a ServiceAccount and attach it to a Pod

```
k create serviceaccount app-sa
# Pod spec:
spec:
  serviceAccountName: app-sa
k get pod app -o jsonpath='{.spec.serviceAccountName}'; echo
```

Step 2 — Inspect the auto-mounted token inside the Pod

```
k exec app -- ls /var/run/secrets/kubernetes.io/serviceaccount/
```

Step 3 — Disable token auto-mount (least privilege)

```
k patch sa app-sa -p '{"automountServiceAccountToken": false}'
```

ServiceAccounts (cont.)

Step 4 — Request a short-lived ad-hoc token

```
TOKEN=$(k create token app-sa --duration=1h)
echo $TOKEN | cut -c1-60
```

Note spec.serviceAccountName attaches a custom SA. Token, CA and namespace are projected under /var/run/secrets/kubernetes.io/serviceaccount/. automountServiceAccountToken: false enforces least privilege; kubectl create token replaces the old Secret-backed tokens (removed in 1.24+).

ServiceAccounts

Test it

k get pod app -o jsonpath='{.spec.serviceAccountName}' returns app-sa, and kubectl create token app-sa prints a short-lived JWT.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

TOPIC 10

RBAC

Who can do what, on which resources, where

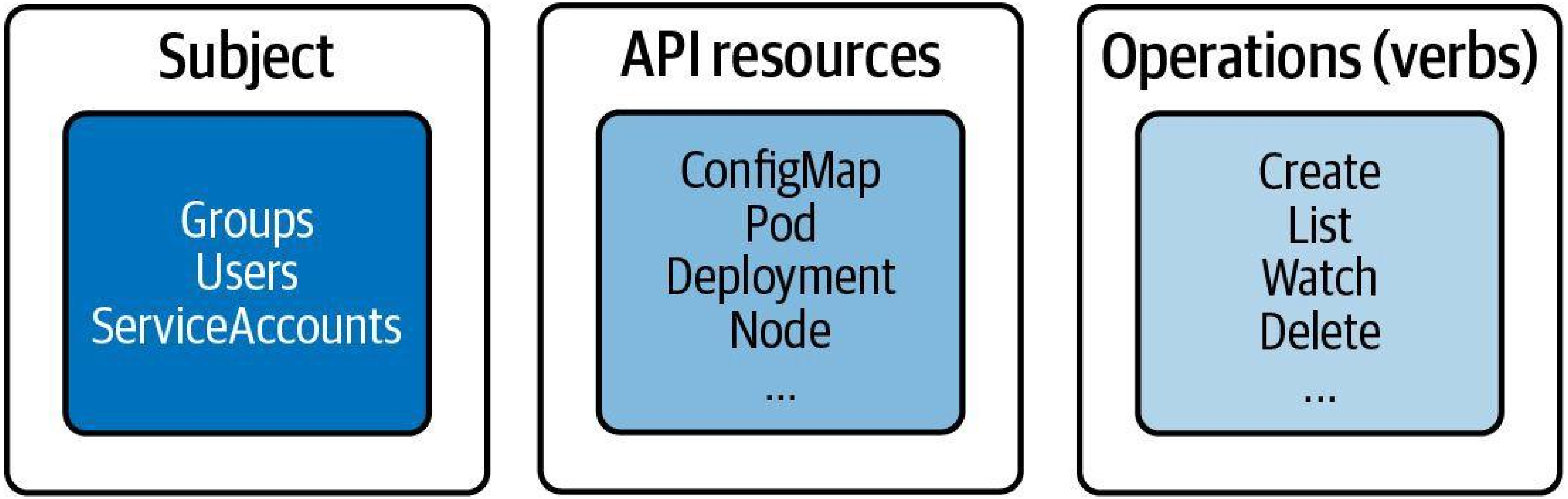
AUTHORIZATION

Role-Based Access Control

- A Role/ClusterRole lists allowed verbs (get, list, create, delete…) on resources.
- A RoleBinding/ClusterRoleBinding ties that Role to a subject (ServiceAccount, user or group).
- Role + RoleBinding = one namespace; ClusterRole + ClusterRoleBinding = cluster-wide.
- ClusterRole + RoleBinding limits a cluster role's verbs to a single namespace.
- Validate without logging in: kubectl auth can-i <verb> <resource> --as=<identity>.

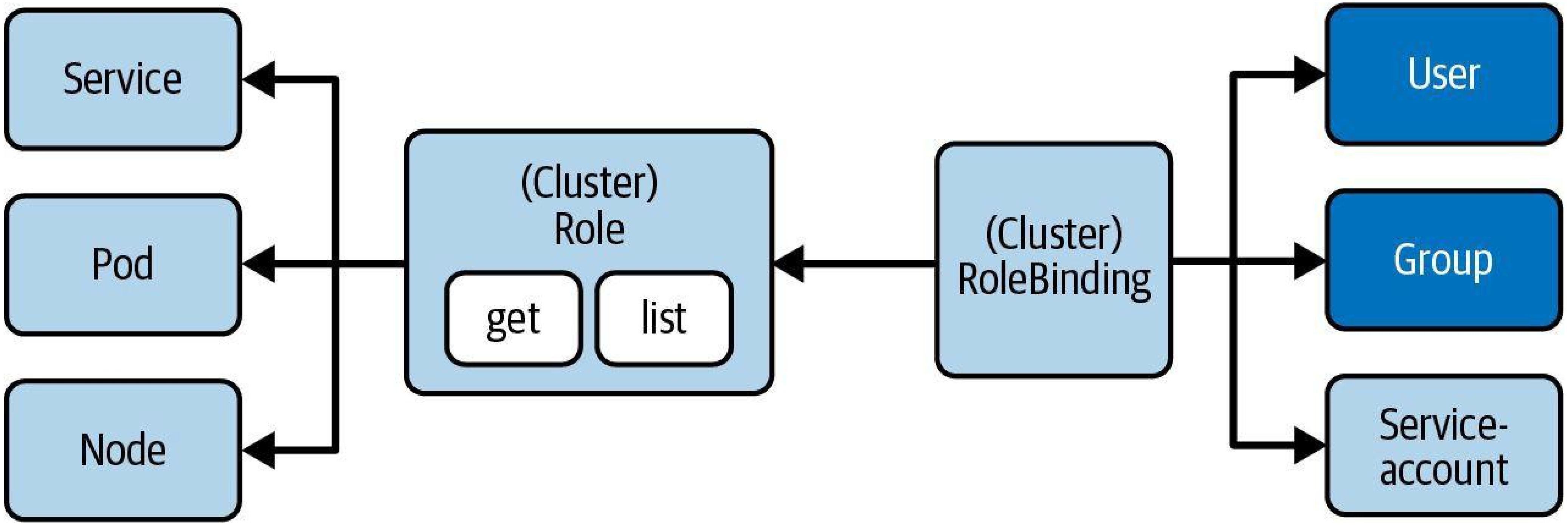
RBAC High-Level Overview

Three key elements for understanding concept



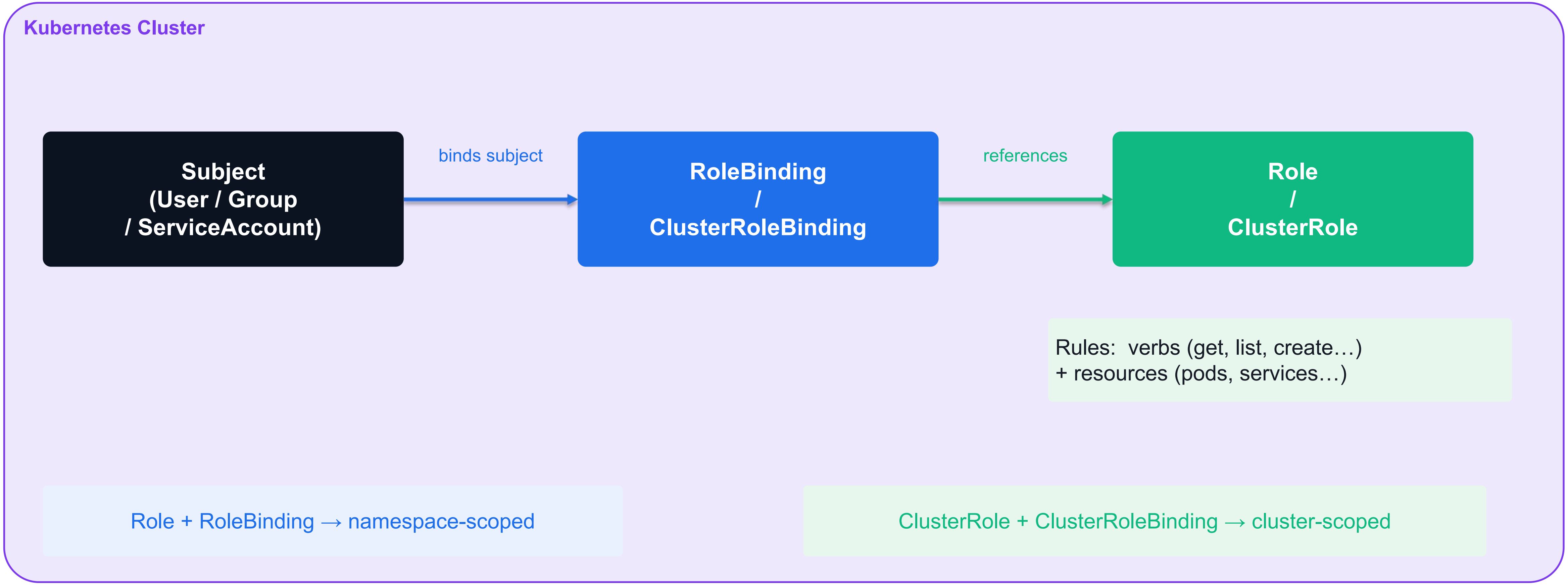
Involved RBAC Primitives

Restrict access to certain operations of API resources for subjects



RBAC · ARCHITECTURE

RBAC — Role-Based Access Control Model



Role vs ClusterRole

Role (Namespace-scoped)

- Scoped to a single namespace
- kubectl get roles -n <ns>
- Common uses: app-specific read/write access
  - Example: allow listing pods in 'default' namespace only
- Paired with RoleBinding in the same namespace

ClusterRole (Cluster-scoped)

- Applies across all namespaces (or non-namespaced resources)
- kubectl get clusterroles
- Common uses: admin, monitoring, ingress controller
  - Example: allow listing nodes, PVs, namespaces cluster-wide
- Paired with ClusterRoleBinding or namespaced RoleBinding

Defining a Role and RoleBinding

Role: grant read access to pods in the 'dev' namespace

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: dev
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

RoleBinding: bind the Role to a ServiceAccount

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: dev
subjects:
- kind: ServiceAccount
  name: my-app-sa
  namespace: dev
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

ClusterRole and ClusterRoleBinding

ClusterRole: read access to nodes (non-namespaced resource)

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "list"]
```

ClusterRoleBinding: bind to a user

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: read-nodes-global
subjects:
- kind: User
  name: jane
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: node-reader
  apiGroup: rbac.authorization.k8s.io
```

RBAC — Essential kubectl Commands

Create a Role imperatively (quick in exam)

```
kubectl create role pod-reader \
  --verb=get,list,watch \
  --resource=pods \
  --namespace=dev
```

Bind the Role to a ServiceAccount

```
kubectl create rolebinding read-pods \
  --role=pod-reader \
  --serviceaccount=dev:my-app-sa \
  --namespace=dev
```

Check permissions for a ServiceAccount (auth can-i)

```
kubectl auth can-i list pods \
  --as=system:serviceaccount:dev:my-app-sa \
  --namespace=dev
# yes
```

RBAC — Roles & RoleBindings

LAB 25 RBAC

RBAC controls who can do what on which resources. CKAD tests creating Roles, ClusterRoles, RoleBindings and ClusterRoleBindings imperatively, and validating with kubectl auth can-i --as. Know namespace- vs cluster-scope.

You'll build

Create a Role + RoleBinding for a ServiceAccount, validate with can-i, then a ClusterRole bound two ways.

Key commands: k create · k auth · » Role

RBAC — Roles & RoleBindings

Step 1 — Role: namespace-scoped permissions

```
k create role pod-reader \
  --verb=get,list,watch --resource=pods -n dev
```

Step 2 — RoleBinding: link Role to a ServiceAccount

```
k create rolebinding viewer-binding \
  --role=pod-reader --serviceaccount=dev:viewer -n dev
```

Step 3 — Validate with kubectl auth can-i

```
k auth can-i list pods   -n dev     --as=system:serviceaccount:dev:viewer  # yes
k auth can-i delete pods -n dev     --as=system:serviceaccount:dev:viewer  # no
k auth can-i list pods   -n default --as=system:serviceaccount:dev:viewer  # no
```

RBAC — Roles & RoleBindings (cont.)

Step 4 — ClusterRole bound cluster-wide vs to one namespace

```
k create clusterrole node-reader --verb=get,list --resource=nodes
k create clusterrolebinding nodes-binding \
  --clusterrole=node-reader --serviceaccount=dev:nodes-sa   # all namespaces
k create rolebinding dev-node-reader \
  --clusterrole=node-reader --serviceaccount=dev:viewer -n dev  # only dev
```

Note Role + RoleBinding = namespace-scoped. ClusterRole + ClusterRoleBinding = cluster-wide. ClusterRole + RoleBinding = the cluster role's verbs limited to one namespace. kubectl auth can-i <verb> <res> --as=<identity> validates without logging in.

RBAC — Roles & RoleBindings

Test it

k auth can-i list pods -n dev --as=system:serviceaccount:dev:viewer returns yes while delete pods and cross-namespace list return no.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

6

SECURITY

Authentication and Authorization

Access control for the Kubernetes API

Processing a Request to the API Server

- Every client request to the API server passes four sequential phases:
  - Authentication — who is the caller?
  - Authorization — is the caller allowed to perform this action?
  - Admission Control — is the request well-formed and valid?
  - Validation — schema validation of the requested resource
- All four phases must pass for the request to be processed.

API Request Processing Phases

- Authentication: Validates the identity of the caller using a supported strategy, e.g. client certificates or bearer tokens.
- Authorization: Determines if the authenticated identity may perform the requested verb + HTTP path. Typically implemented with RBAC.
- Admission Controller: Verifies the request is well-formed and may mutate or reject it before processing. Ensures the resource definition is valid.

Authentication via Credentials in Kubeconfig

The kubeconfig file at $HOME/.kube/config is used by kubectl

```yaml
apiVersion: v1
kind: Config
clusters:
- cluster:
    certificate-authority: /home/user/.minikube/ca.crt
    server: https://127.0.0.1:58936
  name: minikube
contexts:
- context:
    cluster: minikube
    user: bmuschko
  name: bmuschko
current-context: bmuschko
users:
- name: bmuschko
  user:
    client-certificate-data: REDACTED
    client-key-data: REDACTED
```

Managing Kubeconfig and Current Context

View the full kubeconfig

```
kubectl config view
apiVersion: v1
kind: Config
current-context: bmuschko
...
```

Show and switch the active context

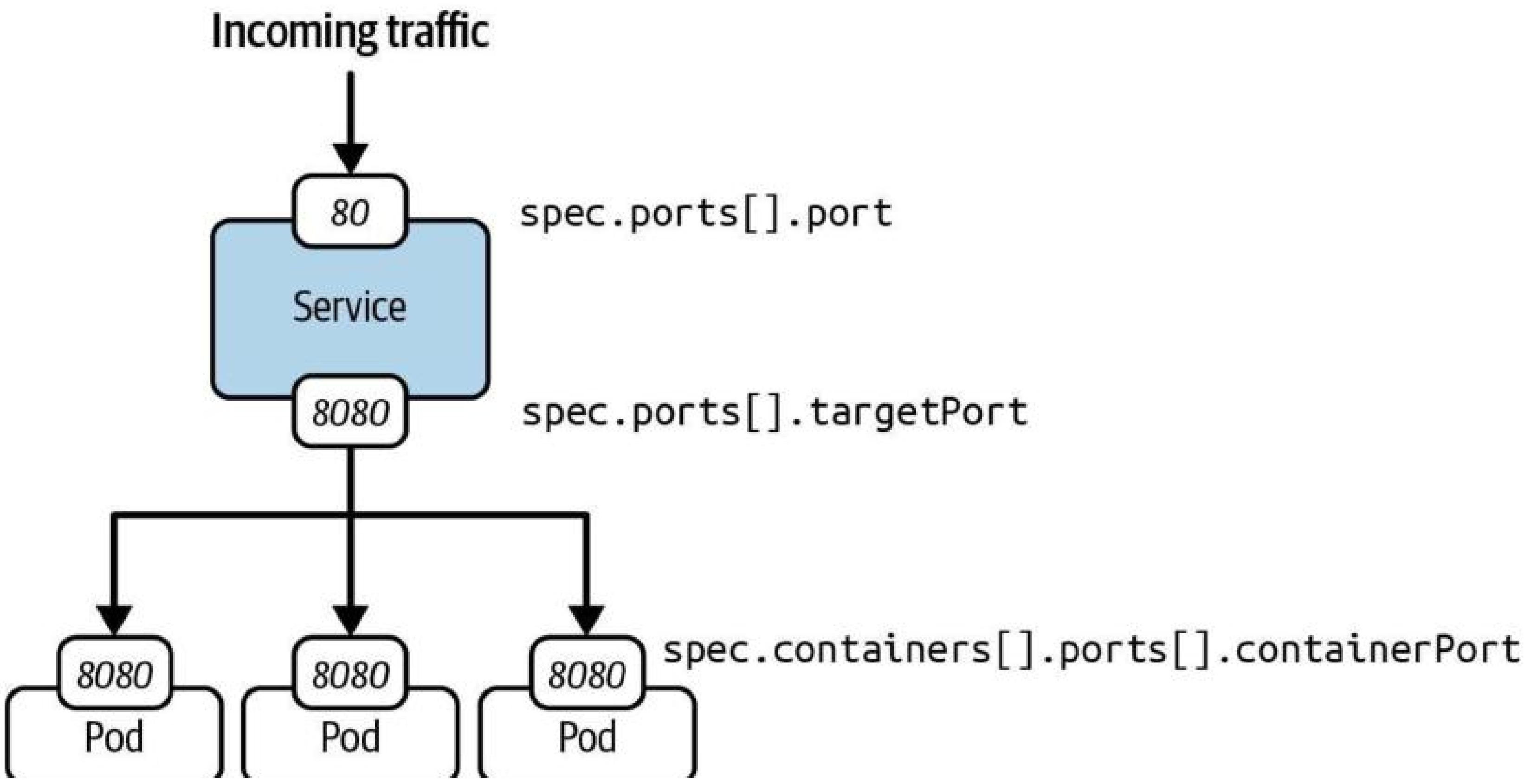
```
kubectl config current-context
bmuschko

kubectl config use-context johndoe
Switched to context "johndoe".

kubectl config current-context
johndoe
```

Services

Providing a stable network endpoint to Pods

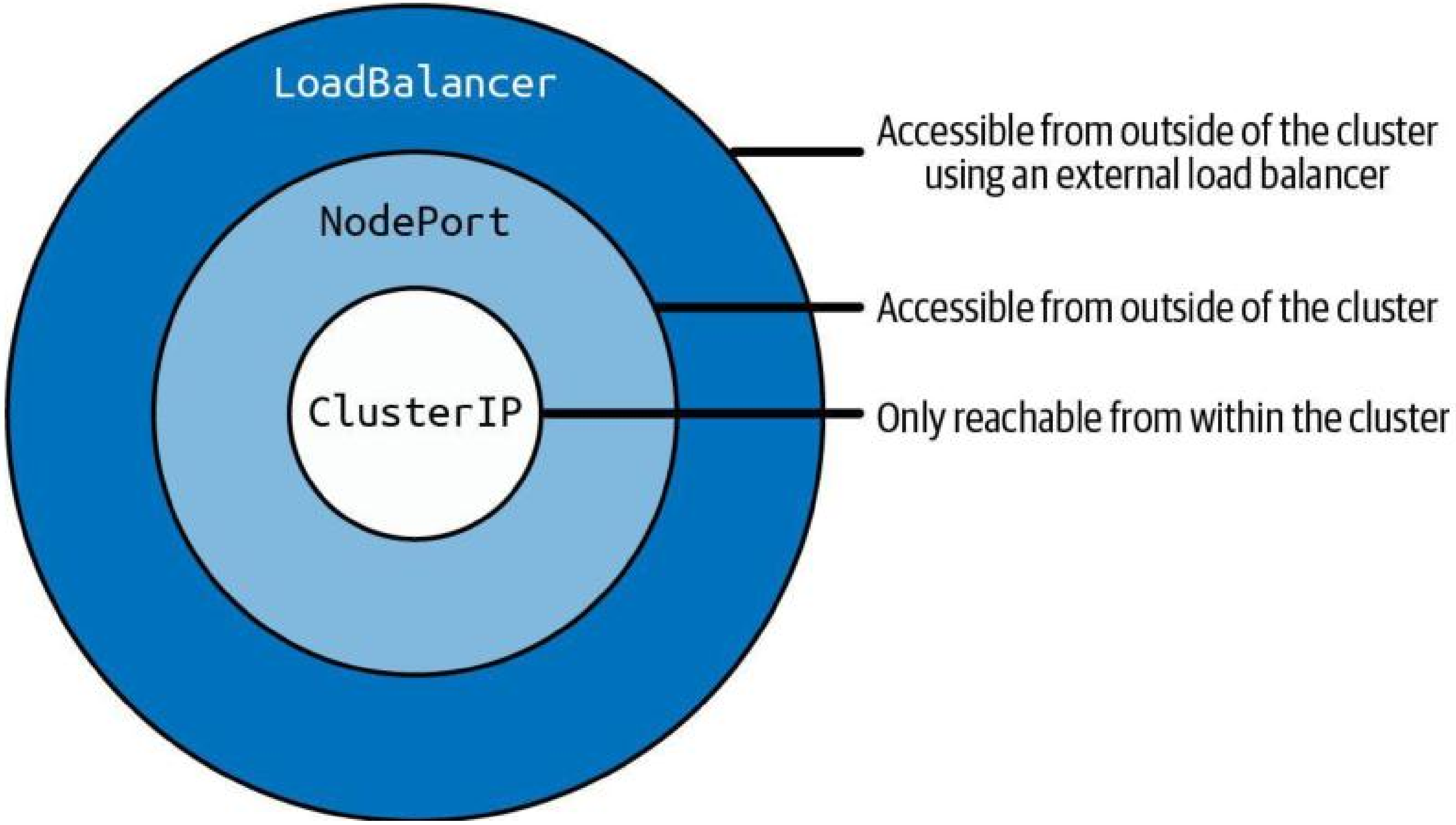
Port Mapping

"How to map the Service port to the container port in a Pod?"



Service Type Inheritance

Services build on top of each other



Troubleshooting Services

Built-in mechanisms for identifying the root cause of a failure

Ingress

Providing external access to Services via HTTP(S)

Network Policies

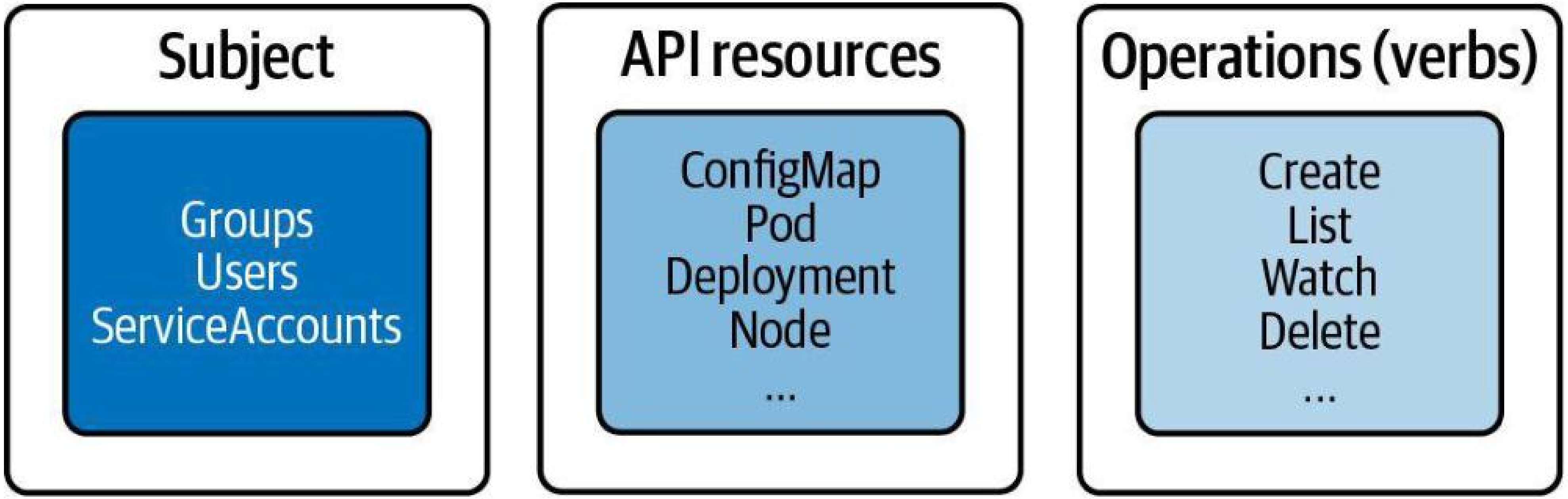
Restricting Pod-to-Pod communication

Authentication and Authorization

Access control for the Kubernetes API

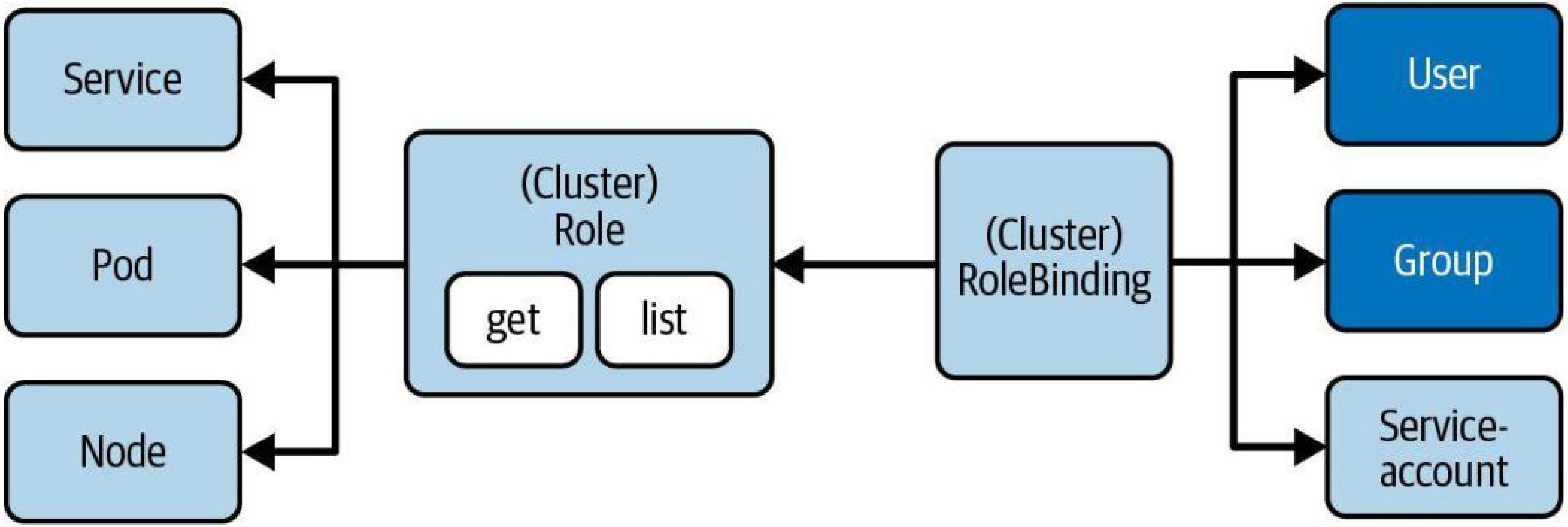
RBAC High-Level Overview

Three key elements for understanding concept



Involved RBAC Primitives

Restrict access to certain operations of API resources for subjects



Namespace and Cluster-Wide Permissions

Defining permission scopes

- The ***Role*** maps API resources to verbs for a **single namespace**.
- The ***RoleBinding*** maps a reference of a Role to one or many subjects for a **single namespace.**
- To apply permissions across all namespaces of a cluster or cluster-wide resources, use the corresponding API primitives ***ClusterRole*** and ***ClusterRoleBinding***. The configuration options of the manifests are the same.

Namespace and Cluster-Wide Permissions

Defining permission scopes

Term	Scope
Role	Namespace specific
Rolebinding	Namespace specific
ClusterRole	Cluster-wide
ClusterRoleBinding	Cluster-wide

Namespace and Cluster-Wide Permissions

Term	Scope	Use Case
Role	Namespace specific	Allow user to list pods in dev namespace Role + Rolebinding
Rolebinding	Namespace specific	Allow user to list pods in dev namespace Role + Rolebinding
ClusterRole	Cluster-wide	Allow a monitoring agent to list pods in any namespace, ClusterRole + RoleBinding
ClusterRoleBinding	Cluster-wide	Allow a user to manage nodes cluster-wide ClusterRole +

Creating a Role with Imperative Approach

Defines verbs and resources in comma-separated list

```
$ kubectl create role read-only --verb=list,get,watch ↵
  --resource=pods,deployments,services
role.rbac.authorization.k8s.io/read-only created
```

Resources: Primitives the operations should apply to

Operations: Only allow listing, getting, watching

Role YAML Manifest

Can define a list of rules in an array

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: read-only
rules:
- apiGroups: [""]
  resources: ["pods", "services"]
  verbs: ["list", "get", "watch"]
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["list", "get", "watch"]
```

Resources with groups need to be spelled out explicitly (in this case `apps/Deployment`)

Getting Role Details

Maps objects of a Kubernetes primitive to verbs

```
$ kubectl describe role read-only
Name:         read-only
Labels:       <none>
Annotations:  <none>
PolicyRule:
  Resources          Non-Resource URLs  Resource Names  Verbs
  ---------          -----------------  --------------  -----
  pods               []                 []              [list get watch]
  services           []                 []              [list get watch]
  deployments.apps   []                 []              [list get watch]
```

Creating a RoleBinding with Imperative Approach

Maps subject to Role

```
$ kubectl create rolebinding read-only-binding --role=read-only
↵
  --user=johndoe
rolebinding.rbac.authorization.k8s.io/read-only-binding created
```

Subject: Binds a user to the Role

Role: Reference the name of the object

RoleBinding YAML Manifest

Roles can be mapped to multiple subjects if needed

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-only-binding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: read-only
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: johndoe
```

Reference to Role

Reference to User

Getting RoleBinding Details

Only shows mapping between Role and subjects

```
$ kubectl describe rolebinding read-only-binding
Name:         read-only-binding
Labels:           <none>
Annotations:    <none>
Role:
  Kind: Role
  Name: read-only
Subjects:
  Kind  Name Namespace
 ------  ------     ------------
  User  johndoe
```

Service Account as Subject

Pod assign the Service Account by name

Service Account YAML Manifest

In a Pod manifest, refer to the Service Account by name

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: sa-api
  namespace: k97
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: list-pods
  namespace: k97
spec:
  serviceAccountName: sa-api
  ...
```

Accessing Service Account Authentication Token

Automounted at specific Volume mount path

```
$ kubectl describe pod list-pods -n k97
...
Containers
  : pods:
    ...
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from  ↵
      kube-api-access-xnkwd (ro)
...

$ kubectl exec -it list-pods -n k97 -- /bin/sh
# cat
```

RoleBinding YAML Manifest

Default RBAC policies don't span beyond `kube-system`

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: serviceaccount-pod-rolebinding
  namespace: k97
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: list-pods-clusterrole
su kind: ServiceAccount
- apiGroup: rbac.authorization.k8s.io
  name: sa-api
```

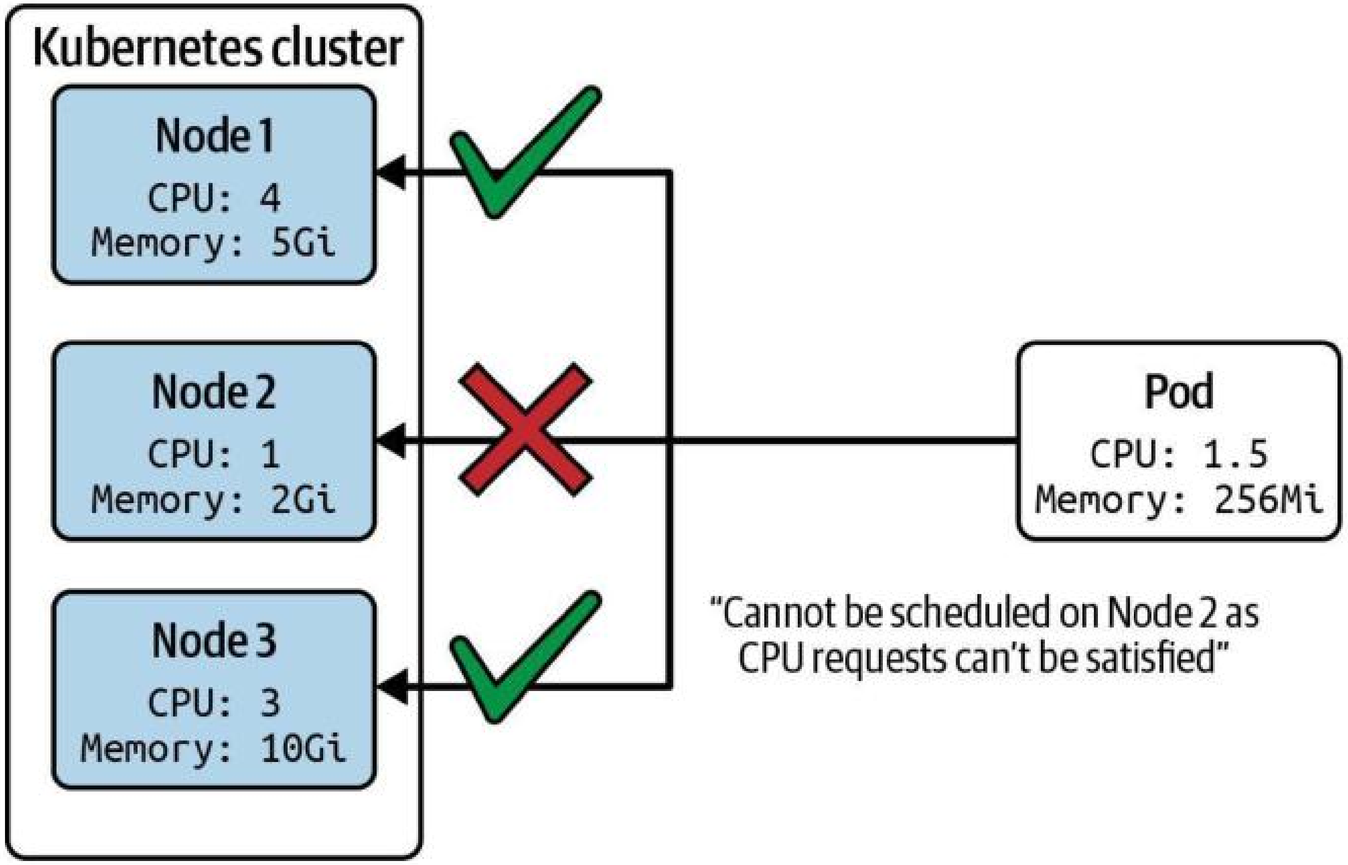
Maps the Service Account as a subject

Resource Management

Resource requests and limits, resource quotas, limit ranges

Example Scheduling Scenario

Node's resource capacity needs to be able to fulfill it

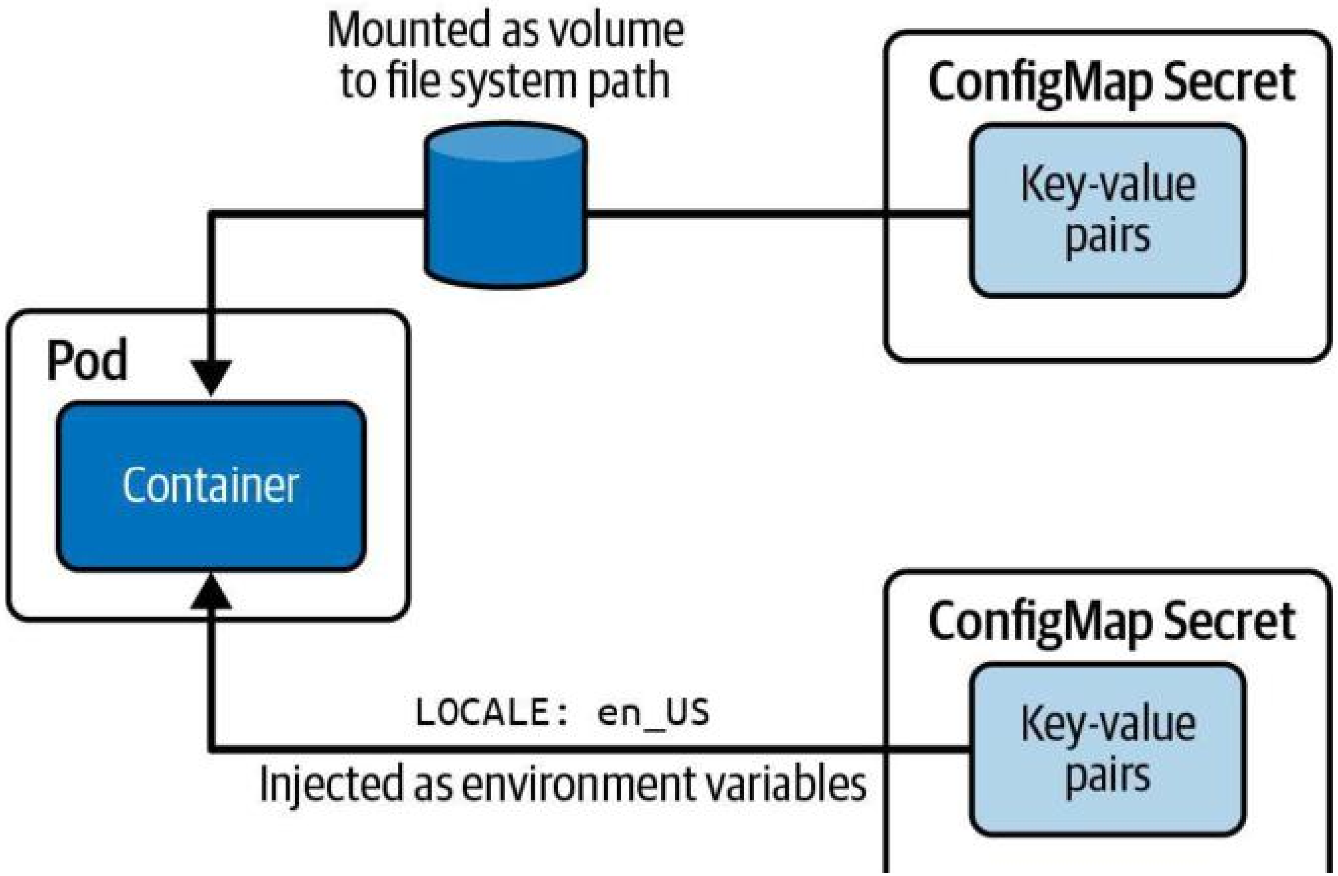


ConfigMaps and Secrets

Defining and consuming configuration data

Consuming Configuration Data

Applicable to ConfigMap and Secret



ENV Variables from Secret

Configuration options and example values

```yaml
envFrom:
 - secretRef:
    name: credentials-secret
```

```
Environment Variables from:
  credentials-secret  Secret
```

ENV Variables from Secret

Configuration options and example values

```
env:
 - name: DB_Host
   valueFrom:
    secretKeyRef:
      name: db-secret
      key: DB_Host
 - name: password
   valueFrom:
     secretKeyRef:
       name: credentials-secrets
       key: password
```

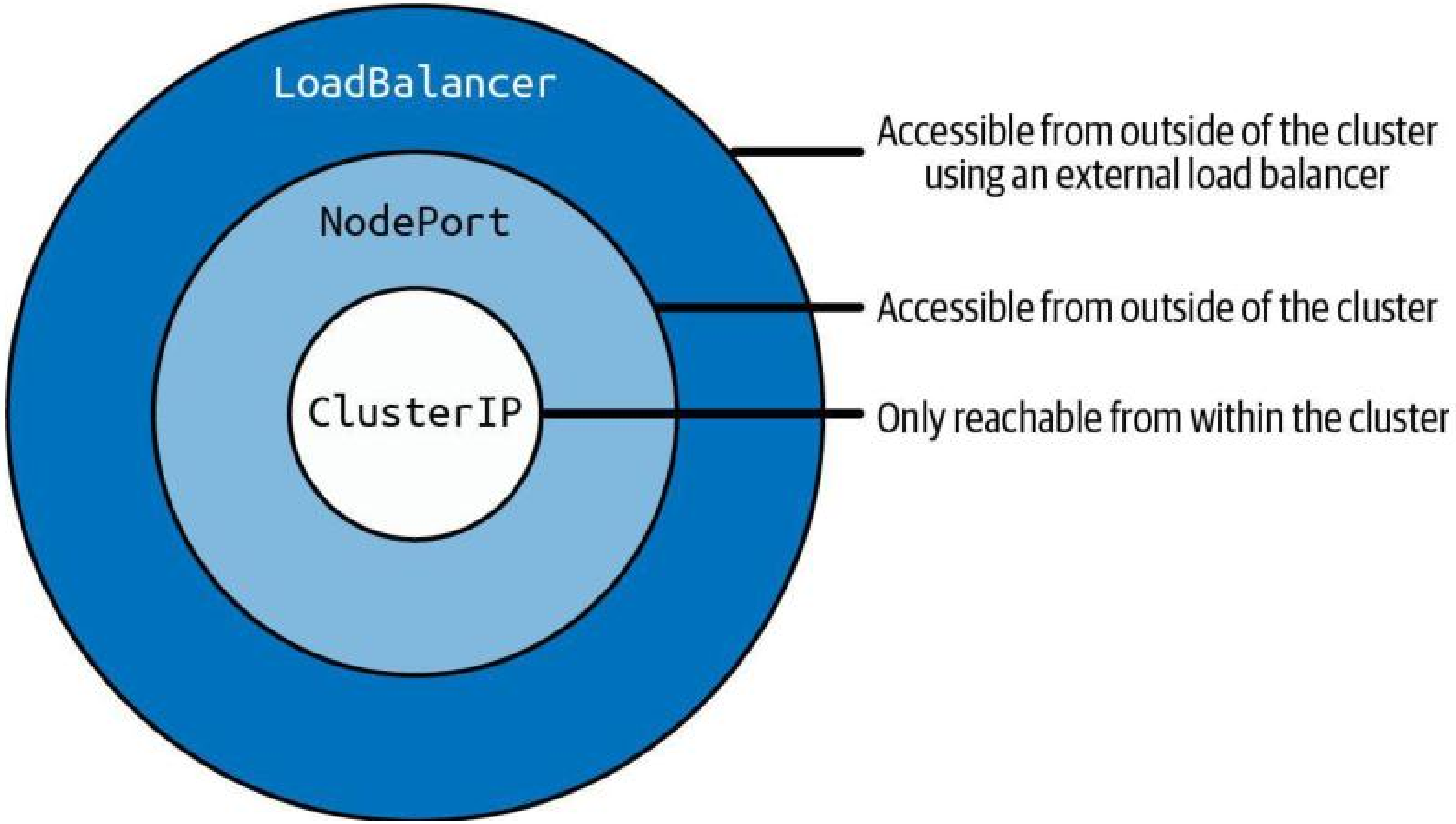
From hello-configmap

From hello-configmap-file

```
Environment:
  DB_Host:   <set to the key 'DB_Host' in secret 'db-secret'>
  password:  <set to the key 'password' of config map 'credentials-secrets'>
```

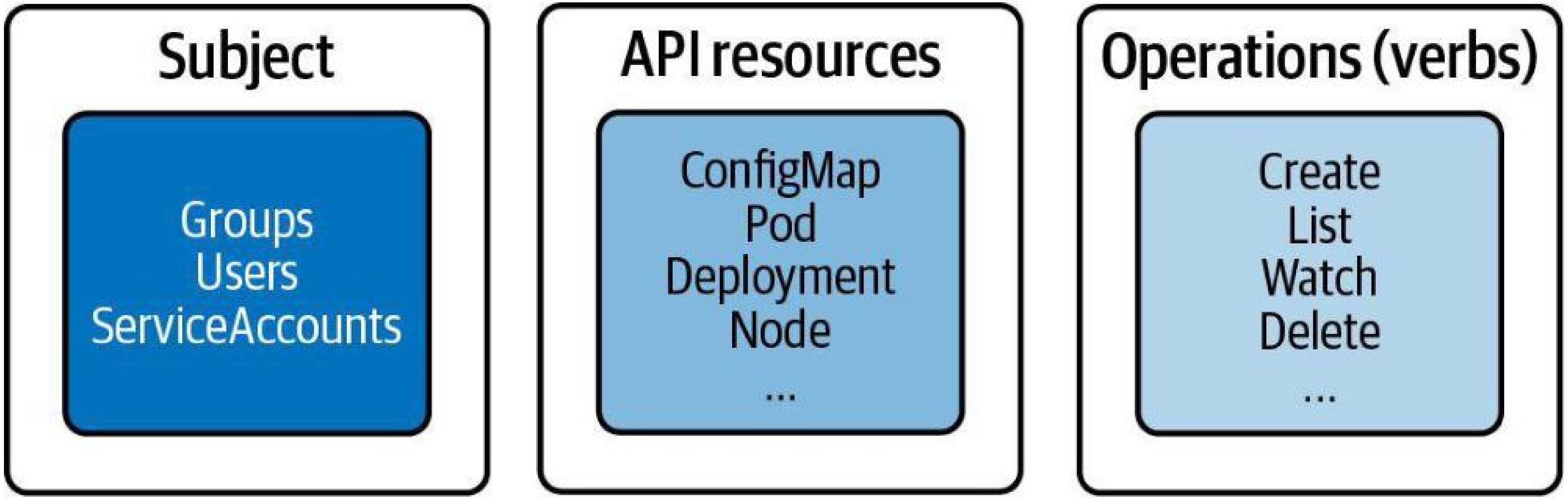
Service Type Inheritance

Services build on top of each other



RBAC High-Level Overview

Three key elements for understanding concept



EXTENDING KUBERNETES

Custom Resource Definitions (CRDs)

Extending the Kubernetes API with custom primitives

What is a Custom Resource Definition (CRD)?

- Some use cases with custom requirements cannot be fulfilled with built-in Kubernetes primitives.
- A CRD allows you to define, create, and persist objects for custom primitives and expose them via the Kubernetes API.
- CRDs are combined with controllers to implement custom logic. The combination is called the Operator pattern.

The Operator Pattern

- An Operator = CRD + Controller
- The CRD extends the Kubernetes API with a new object type (schema, validation, versioning).
- The Controller is a reconciliation loop that watches for CRD instances via the API server and implements the custom business logic.
- Operators are deployed as Pods in the cluster and interact with the API server continuously.
- Example operators: Prometheus Operator, cert-manager, Kafka Operator.

Example CRD — Smoke Testing

- Requirement: A web-based application stack deployed via a Deployment. A Service routes traffic to the Pods.
- Desired functionality: After deployment, run a quick smoke test against the Service's DNS name. Send the result to an external dashboard.
- Goal: Implement the Operator pattern (CRD + controller) for this use case.
- Custom Kind: SmokeTest — attributes: service, path, timeout, retries.

CRD YAML Manifest

Defines the schema for the custom SmokeTest object

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: smoketests.stable.bmuschko.com
spec:
  group: stable.bmuschko.com
  versions:
  - name: v1
    served: true
    storage: true
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            properties:
              service: {type: string}
              path:    {type: string}
              timeout: {type: integer}
              retries: {type: integer}
  scope: Namespaced
  names:
    plural: smoketests
    singular: smoketest
    kind: SmokeTest
    shortNames: [st]
```

CRD · INSTANTIATION

Custom Resource Object YAML Manifest

An instance of the SmokeTest custom kind

```yaml
apiVersion: stable.bmuschko.com/v1
kind: SmokeTest
metadata:
  name: backend-smoke-test
spec:
  service: backend
  path: /health
  timeout: 600
  retries: 3
```

CRD · KUBECTL APPLY

Creating the CRD and Custom Object

Create the CRD first, then create instances

```
kubectl apply -f smoketest-crd.yaml
customresourcedefinition.apiextensions.k8s.io/smoketests.stable.bmuschko.com created

kubectl apply -f backend-smoke-test.yaml
smoketest.stable.bmuschko.com/backend-smoke-test created
```

CRD · INSPECTION

Discovering CRDs

List all installed CRDs

```
kubectl get crds
NAME                            CREATED AT
smoketests.stable.bmuschko.com      2023-05-04T14:49:40Z
```

Discover custom resource types via api-resources

```
kubectl api-resources --api-group=stable.bmuschko.com
NAME        SHORTNAMES  APIVERSION              NAMESPACED  KIND
smoketests  st          stable.bmuschko.com/v1      true        SmokeTest
```

Controller Implementation

- You can CRUD custom resource objects, but no smoke-test logic is initiated without a controller.
- A controller acts as a reconciliation process: it polls for new SmokeTest objects via the API server and executes the smoke test, sending results to the external service.
- Controllers are typically written in Go or Python using a Kubernetes client library (e.g. client-go, kubernetes-client).

EXERCISE

Defining and Interacting with a CRD

EXAM PREP · KEY TAKEAWAYS

Exam Essentials

- You are not expected to implement a CRD schema yourself. Know how to discover and use them.
- Use kubectl get crds to list installed CRDs, and kubectl api-resources --api-group=<group> to see their types.
- Create objects from a CRD schema exactly as you would for built-in resources: kubectl apply -f <manifest>.yaml.
- Controller implementations are out-of-scope for the CKAD exam.

Tea Break

15 minutes

TOPIC 11

Quotas & Limits

Cap namespace usage & per-container resources

GUARDRAILS

Govern Resource Consumption

- ResourceQuota caps the aggregate resources a namespace can consume (pods, requests, limits).
- LimitRange enforces per-container defaults, defaultRequests and maximums.
- Without a LimitRange, Pods with no resource block are rejected in a quota-enforced namespace.
- kubectl describe quota shows Used vs Hard — the primary troubleshooting view.

ResourceQuota YAML Manifest

Limits # of objects and aggregate container resource consumption

```
apiVersion: v1 kind:
ResourceQuota metadata:
  name: awesome-quota
  namespace: team-awesome
spec:



   hard:
     pods: 2
     requests.cpu: "1"
     requests.memory: 1024Mi
     limits.cpu: "4" limits.memory:
     4096Mi
```

What is a LimitRange?

Constraints or defaults the resource allocations for an object type

- Enforces minimum and maximum compute resources usage per Pod or container in a namespace.
- Enforces minimum and maximum storage request per PersistentVolumeClaim in a namespace.
- Enforces a ratio between request and limit for a resource in a namespace.
- Set default requests/limits for compute resources in a namespace and automatically injects them into containers at runtime.

ResourceQuota & LimitRange

LAB 26

Quotas & Limits

ResourceQuota caps the total resources used across a namespace; LimitRange enforces per-container defaults and maximums. CKAD tests both — write the YAML, apply them, and understand the rejection error messages.

You'll build

Apply a ResourceQuota and a LimitRange, watch defaults get injected, then violate the max and the quota.

Key commands: spec: · k run · for i · » ResourceQuota

ResourceQuota & LimitRange

Step 1 — ResourceQuota: namespace-wide ceilings

```
spec:
  hard:
    pods: "5"
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.cpu: "2"
    limits.memory: 2Gi
```

Step 2 — LimitRange: per-container defaults and maximums

```
spec:
  limits:
  - type: Container
    default:        {cpu: 200m, memory: 256Mi}
    defaultRequest: {cpu: 100m, memory: 128Mi}
    max:            {cpu: 500m, memory: 512Mi}
```

ResourceQuota & LimitRange (cont.)

Step 3 — Defaults injected; max enforced

```
k run a --image=nginx:1.25 -n team-a
k get pod a -n team-a -o jsonpath='{.spec.containers[0].resources}'; echo
# a container requesting cpu:1 is rejected: 'maximum cpu per Container is 500m'
```

Step 4 — Exceed the quota

```
for i in 1 2 3 4; do k run quota-$i --image=nginx:1.25 -n team-a; done
k run quota-5 --image=nginx:1.25 -n team-a   # 'exceeded quota: team-a-quota'
k describe quota team-a-quota -n team-a       # Used vs Hard
```

Note ResourceQuota limits aggregate usage; exceeding it rejects new Pods. LimitRange enforces per-container min/max/defaults. Without a LimitRange, Pods with no resource block cannot be created in a quota-enforced namespace.

ResourceQuota & LimitRange

Test it

A Pod with no resource block gets requests/limits injected by the LimitRange, the 6th Pod is rejected with exceeded quota, and describe quota shows Used vs Hard.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Managing Resources for Objects and Namespaces

- Containers should declare a minimum amount of resources needed (requests) and a maximum allowed (limits).
- Application developers should right-size resources with load tests or by monitoring production consumption.
- ResourceQuota enforces resource consumption at the namespace level.
- LimitRange constrains or defaults resource allocations for individual objects (Pod, PVC).

Resource Units in Kubernetes

- CPU is measured in millicores (m). Example: 600m = 0.6 CPU cores.
- Memory is measured in bytes using fixed-point numbers or power-of-two suffixes:
  - Ki = kibibytes • Mi = mebibytes • Gi = gibibytes
  - K = kilobytes • M = megabytes • G = gigabytes
- Example: 256Mi = 268,435,456 bytes. See the official docs for full details.

RESOURCE REQUESTS · DEFINITION

Container Resource Requests

- Containers declare a minimum amount of resources via spec.containers[].resources.requests.
- Resource types: CPU, memory, huge pages, ephemeral storage.
- The scheduler uses requests to decide which node can run the Pod.
- If no node has enough capacity → the Pod stays Pending with status PodExceedsFreeCPU or PodExceedsFreeMemory.

Example Scheduling Scenario

- The scheduler finds a node whose available CPU and memory ≥ the Pod's requests.
- Example: Pod requests 1 CPU + 256 Mi memory. Node must have at least that available.
- Requests do not cap actual usage — they are scheduling guarantees only.
- Once scheduled, actual usage can exceed requests (up to the limit).

Container Resource Requests — Options

- resources.requests.cpu — minimum CPU in cores or millicores (e.g. "1" or "500m")
- resources.requests.memory — minimum memory in bytes with suffix (e.g. "256Mi")
- resources.requests.ephemeral-storage — minimum local ephemeral storage
- resources.requests.hugepages-<size> — huge page allocation
- Values are per container, not per Pod.

Container Resource Requests YAML Manifest

Minimum resource amounts declared for a container

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: rate-limiter
spec:
  containers:
  - name: business-app
    image: bmuschko/nodejs-business-app:1.0.0
    ports:
    - containerPort: 8080
    resources:
      requests:
        memory: "256Mi"
        cpu: "1"
```

RESOURCE LIMITS · DEFINITION

Container Resource Limits

- Containers declare a maximum amount of resources via spec.containers[].resources.limits.
- Resource types: CPU, memory, huge pages, ephemeral storage.
- The container runtime enforces limits at runtime. Behaviour when exceeded:
  - CPU limit exceeded → process is throttled (not killed)
  - Memory limit exceeded → container may be OOMKilled (Out Of Memory)

Container Resource Limits — Options

- resources.limits.cpu — maximum CPU in cores or millicores (e.g. "2" or "1000m")
- resources.limits.memory — maximum memory in bytes with suffix (e.g. "512Mi")
- resources.limits.ephemeral-storage — maximum local ephemeral storage
- resources.limits.hugepages-<size> — maximum huge pages
- Best practice: always set both requests and limits for every container.

RESOURCE LIMITS · DECLARATIVE

Container Resource Limits YAML Manifest

Maximum resource amounts declared for a container

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: rate-limiter
spec:
  containers:
  - name: business-app
    image: bmuschko/nodejs-business-app:1.0.0
    ports:
    - containerPort: 8080
    resources:
      limits:
        memory: "512Mi"
        cpu: "2"
```

Pod Scheduling Details

Find the node a Pod was scheduled on, then inspect its resource usage

```
kubectl get pod rate-limiter -o yaml | grep nodeName:
  nodeName: worker-3

kubectl describe node worker-3
...
Non-terminated Pods:
  Namespace  Name          CPU Requests  CPU Limits  Memory Requests  Memory Limits
  default    rate-limiter  1250m (62%)   0 (0%)      256Mi (6%)       512Mi (13%)
```

Container Resource Requests & Limits — Combined

It is best practice to define both requests and limits for every container

```yaml
spec:
  containers:
  - name: business-app
    image: bmuschko/nodejs-business-app:1.0.0
    resources:
      requests:
        memory: "256Mi"
        cpu: "1"
      limits:
        memory: "512Mi"
        cpu: "2"
```

EXERCISE

Defining Container Resource Requests and Limits

No Requests/Limits Definition

Scheduler rejects the creation of the object

```
$ kubectl create -f nginx-pod.yaml -n team-awesome
Error from server (Forbidden): error when creating "nginx-pod.yaml":
↵ pods "nginx" is forbidden: failed quota: awesome-quota: must
specify ↵limits.cpu,limits.memory,requests.cpu,requests.memory
```

Exceeds Requests/Limits Definition

Scheduler rejects the creation of the object

```
$ kubectl create -f nginx-pod.yaml -n team-awesome
Error from server (Forbidden): error when creating "nginx-pod.yaml":
↵pods "nginx" is forbidden: exceeded quota: awesome-quota,
requested: ↵pods=1,requests.cpu=500m,requests.memory=512Mi, used:
pods=2, ↵requests.cpu=1,requests.memory=1024Mi, limited: pods=2,  ↵
requests.cpu=1,requests.memory=1024Mi
```

LimitRange YAML Manifest

Sets defaults, minimum and maximum CPU allocation for containers

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name:
cpu-resource-constraint
spec:
  limits:
  - default:
      cpu: 500m
    defaultRequest:
      cpu: 500m
    max:
      cpu: "1"
    min:
      cpu: 100m
    type: Container
```

Defines default limits

Defines default requests

Minimum and maximum resource range

Automatically Uses Container Defaults for Pod

Container doesn't provide any resource requests/limits

```
$ kubectl create -f pod.yaml
pod/rate-limiter created

$ kubectl describe pod rate-limiter
...
Containers:
  business-app
   :
     ...
```

Rejected Pod Creation for Exceeded Limits

Requests a higher maximum for CPU resources than defined by LimitRange

```
$ kubectl create -f pod.yaml
Error from server (Forbidden): error when creating "pod.yaml": pods
↵"rate-limiter" is forbidden: maximum cpu usage per Container is 1,
↵but limit is 2
```

WRAP-UP

Day 3 done — you can observe, configure and secure applications.

Tomorrow (half day): Services & Networking, then the final assessment.

COURSE SLIDES

Certified Kubernetes Application Developer (CKAD)

WSQ Course Code: TGS-2025053212

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

04

DAY 4 (½ day)

Services & Networking

Services · Service DNS · Ingress with TLS · NetworkPolicy (CKAD Domain 5 — 20%) + Final Assessment

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your phone camera and submit your attendance.
- A minimum of 75% attendance is required for assessment and funding.

Minimum 75% attendance required

DAY 4 OBJECTIVES

Recap & Today

- So far: design & build (D1), deployment (D2), observability and config & security (D3).
- This morning — Domain 5 Services & Networking (20%): how Pods are reached and isolated.
- Services and DNS for stable in-cluster addressing; Ingress for HTTP/HTTPS; NetworkPolicy for isolation.
- From 1:00 PM — mock-exam practice; the final written assessment starts at 2:00 PM.

TOPIC 1

Services

Stable virtual IPs & DNS for a set of Pods

Why Services?

- Pod IPs are ephemeral — a Service gives a stable virtual IP and DNS name in front of matching Pods.
- ClusterIP (default) is in-cluster only; NodePort opens a port on every node; LoadBalancer asks the cloud for an external IP.
- A Service finds its Pods by label selector; the matching set is tracked as EndpointSlices.
- Empty Endpoints almost always means a selector/label mismatch — the most common Service bug.

THREE WAYS TO EXPOSE

Choosing a Service Type

ClusterIP

- In-cluster only
- Stable virtual IP + DNS
- Default type

NodePort

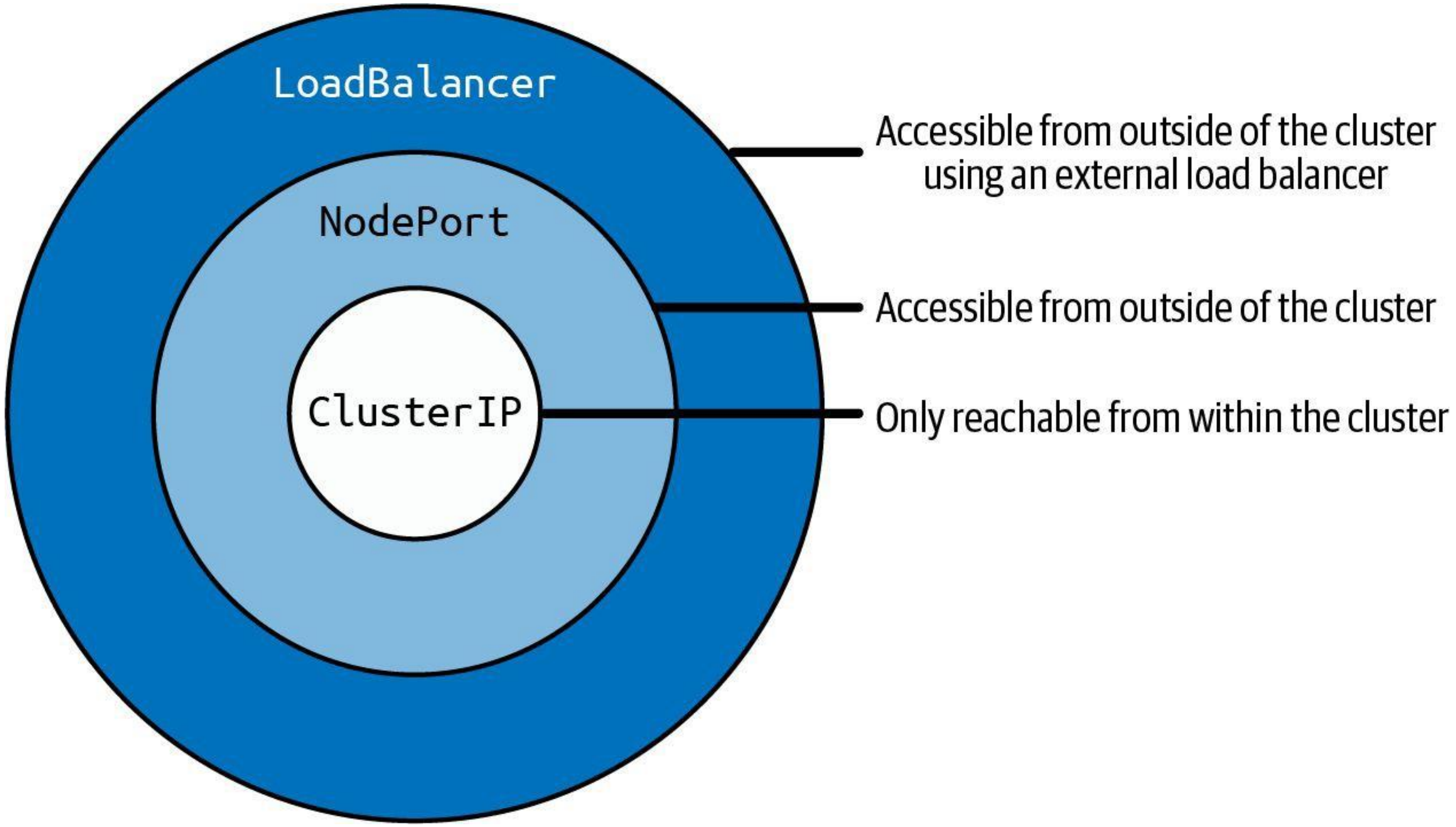
- Port on every node
- 30000–32767
- Reachable from outside

LoadBalancer

- Cloud external IP
- Builds on NodePort
- One Service, public entry

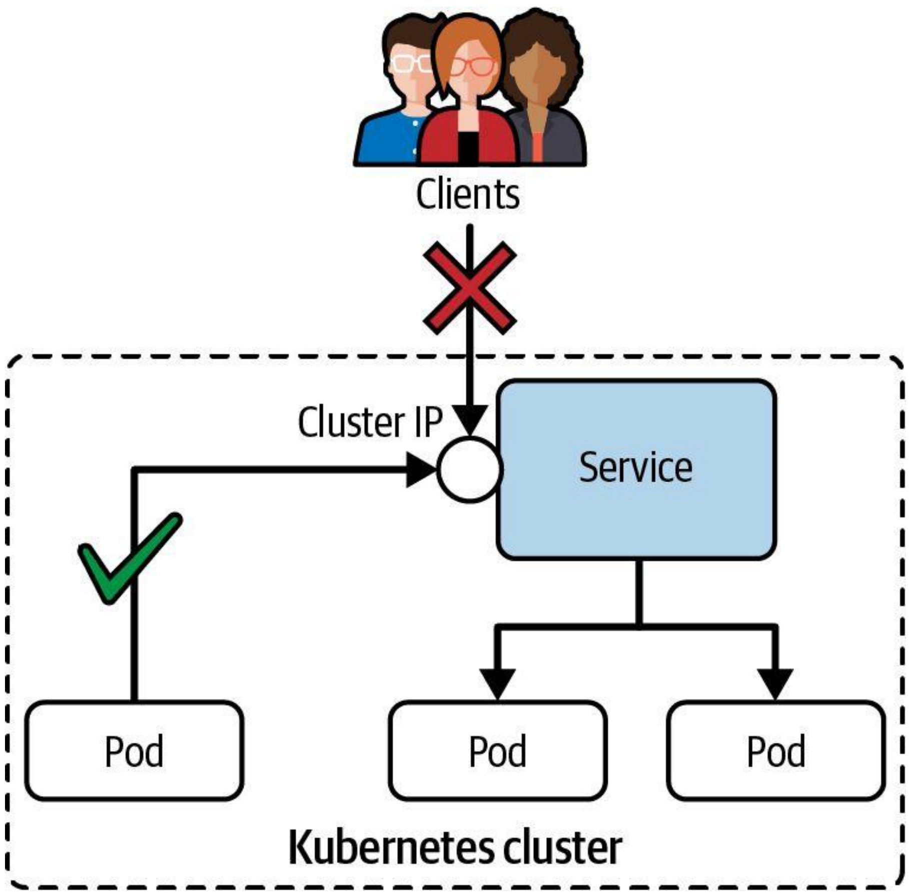
Service Type Inheritance

Services build on top of each other



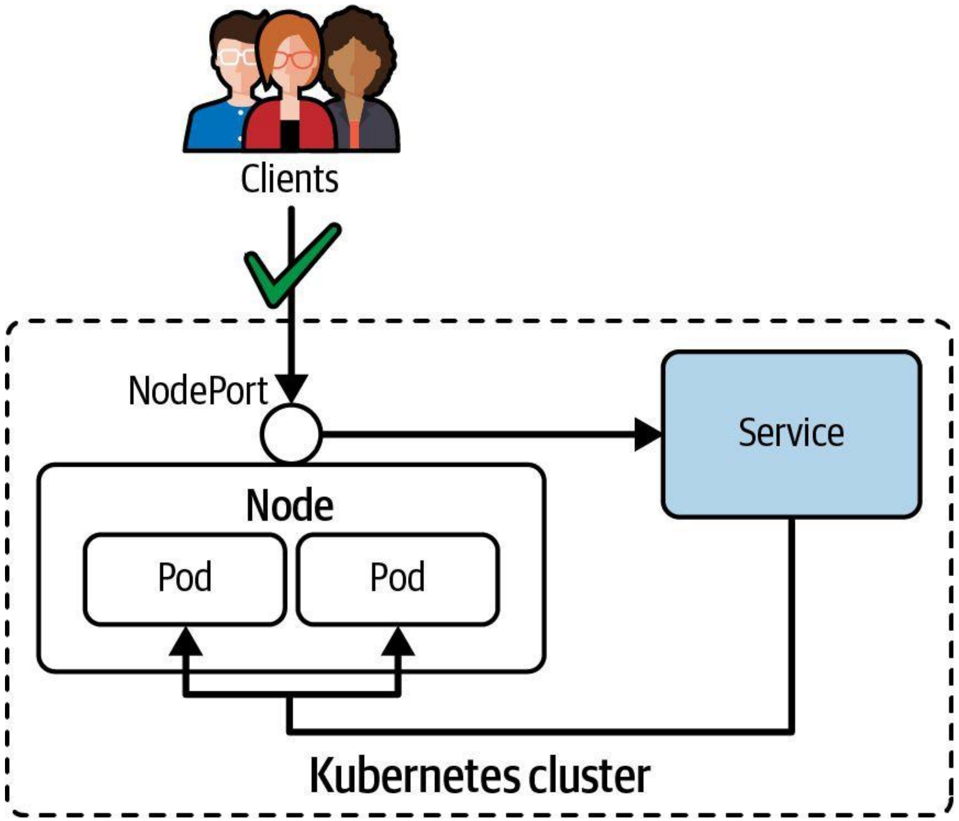
ClusterIP Service Type

Only reachable from within the cluster, e.g. another Pod



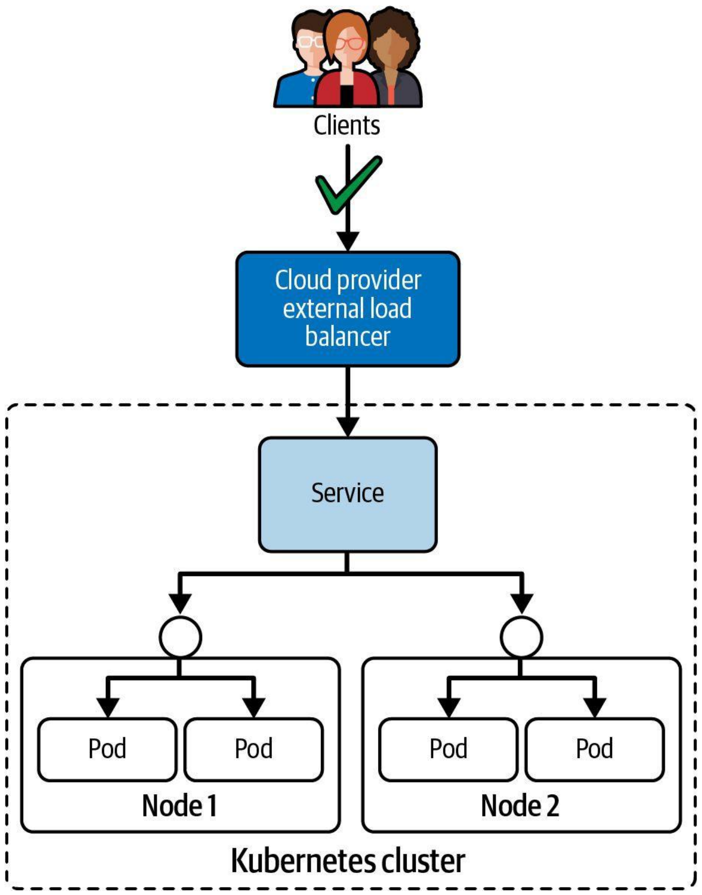
NodePort Service Type

Can be resolved from outside of the Kubernetes cluster



LoadBalancer Service Type

Routes traffic from existing, external load balancer to Service



Service Port Mapping — port, targetPort, nodePort

Three port fields: port (Service), targetPort (Pod container), nodePort (Node)

```
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  type: NodePort
  selector:
    app: web
  ports:
  - port: 80          # port clients use to reach the Service (ClusterIP)
    targetPort: 8080   # port the container listens on
    nodePort: 30080    # port opened on every Node (30000-32767)
                       # omit nodePort for ClusterIP; not needed for LoadBalancer
```

Expose a deployment as a Service in one command

```
kubectl expose deployment web --port=80 --target-port=8080 --type=NodePort
kubectl get svc web
NAME   TYPE       CLUSTER-IP    PORT(S)        AGE
web    NodePort   10.96.1.50    80:30080/TCP   5s
```

Service DNS — How Pods Resolve Services

<service-name>.<namespace>.svc.cluster.local

web-svc	.	production	.	svc	.	cluster.local	.
service name		namespace		resource type		cluster domain	

Short forms (within same namespace)

```
# From a pod in the SAME namespace — just use the service name:
curl http://web-svc/api

# From a pod in a DIFFERENT namespace — include the namespace:
curl http://web-svc.production/api

# Full FQDN always works:
curl http://web-svc.production.svc.cluster.local/api
```

SERVICES · LABEL SELECTOR

Service Selector Matches Pod Labels

Service

```
spec:
  selector:
    app: backend
    tier: api
  ports:
  - port: 80
    targetPort: 8080
```

selects &
routes to

matches

Pod A ✓ Selected

```
labels:
  app: backend
  tier: api
```

Pod B ✗ Not Selected

```
labels:
  app: frontend
```

kubectl get endpoints

Lists the IPs of all matching Pods.
Empty = selector mismatch —
first place to debug!

Services — Quick Reference

Service Types

- ClusterIP (default): internal VIP, cluster-only
  - kubectl create service clusterip my-svc --tcp=80:8080
- NodePort: extends ClusterIP + opens port 30000-32767 on every node
  - kubectl create service nodeport my-svc --tcp=80:8080 --node-port=30080
- LoadBalancer: extends NodePort + provisions cloud LB
- ExternalName: DNS CNAME alias to external hostname

When to Use

- Use ClusterIP when only pods inside the cluster need access
- Use NodePort for simple external access without a cloud provider
- Use LoadBalancer in managed cloud environments (GKE, EKS, AKS)
- Headless service (clusterIP: None): no VIP — direct pod DNS records
  - kubectl create service clusterip my-svc --clusterip=None
- Check: kubectl get svc -A | kubectl describe svc <name>

Services — ClusterIP, NodePort, LoadBalancer

LAB 27 Services

A Service gives a stable virtual IP and DNS name to a set of Pods. CKAD requires fluency with ClusterIP, NodePort and LoadBalancer types, kubectl expose, endpoint debugging and the selector-mismatch pattern.

You'll build

Expose a Deployment as ClusterIP, NodePort and LoadBalancer, inspect EndpointSlices, then debug a selector mismatch.

Key commands: k create · k expose · k patch · » ClusterIP

LAB 27 · COMMANDS

Services — ClusterIP, NodePort, LoadBalancer

Step 1 — Backend Deployment + ClusterIP Service

```
k create deployment web --image=nginx:1.25 --replicas=3
k expose deployment web --port=80 --target-port=80 --name=web-cip
k describe svc web-cip | grep -E "IP:|Endpoints:"
```

Step 2 — NodePort (a port on every node)

```
k expose deployment web --port=80 --target-port=80 --type=NodePort --name=web-np
PORT=$(k get svc web-np -o jsonpath='{.spec.ports[0].nodePort}')
curl -s http://localhost:$PORT | head -3   # 30000-32767
```

Step 3 — LoadBalancer + EndpointSlices

```
k expose deployment web --port=80 --type=LoadBalancer --name=web-lb
k get endpointslices -l kubernetes.io/service-name=web-cip
```

Services — ClusterIP, NodePort, LoadBalancer (cont.)

Step 4 — Debug a selector mismatch (the #1 Service bug)

```
k patch svc web-cip -p '{"spec":{"selector":{"app":"does-not-exist"}}}'
k get endpoints web-cip        # empty = no Pod matches
k patch svc web-cip -p '{"spec":{"selector":{"app":"web"}}}'
```

Note ClusterIP = in-cluster only; NodePort = a port on every node; LoadBalancer = a cloud LB. Empty Endpoints almost always means the selector does not match Pod labels. EndpointSlices (v1.21+) succeed Endpoints.

Services — ClusterIP, NodePort, LoadBalancer

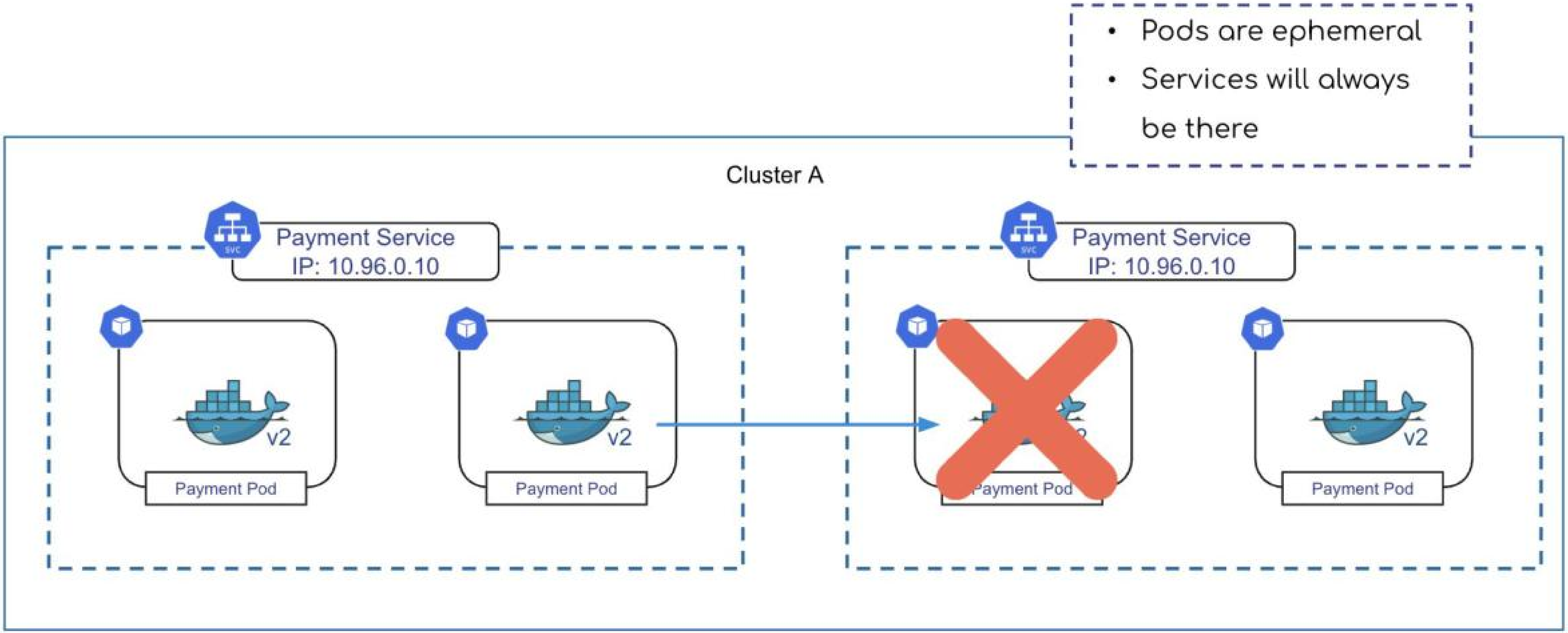
Test it

k describe svc web-cip lists three endpoint IPs, the NodePort serves the page on localhost:$PORT, and a bad selector empties the Endpoints until it is fixed.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

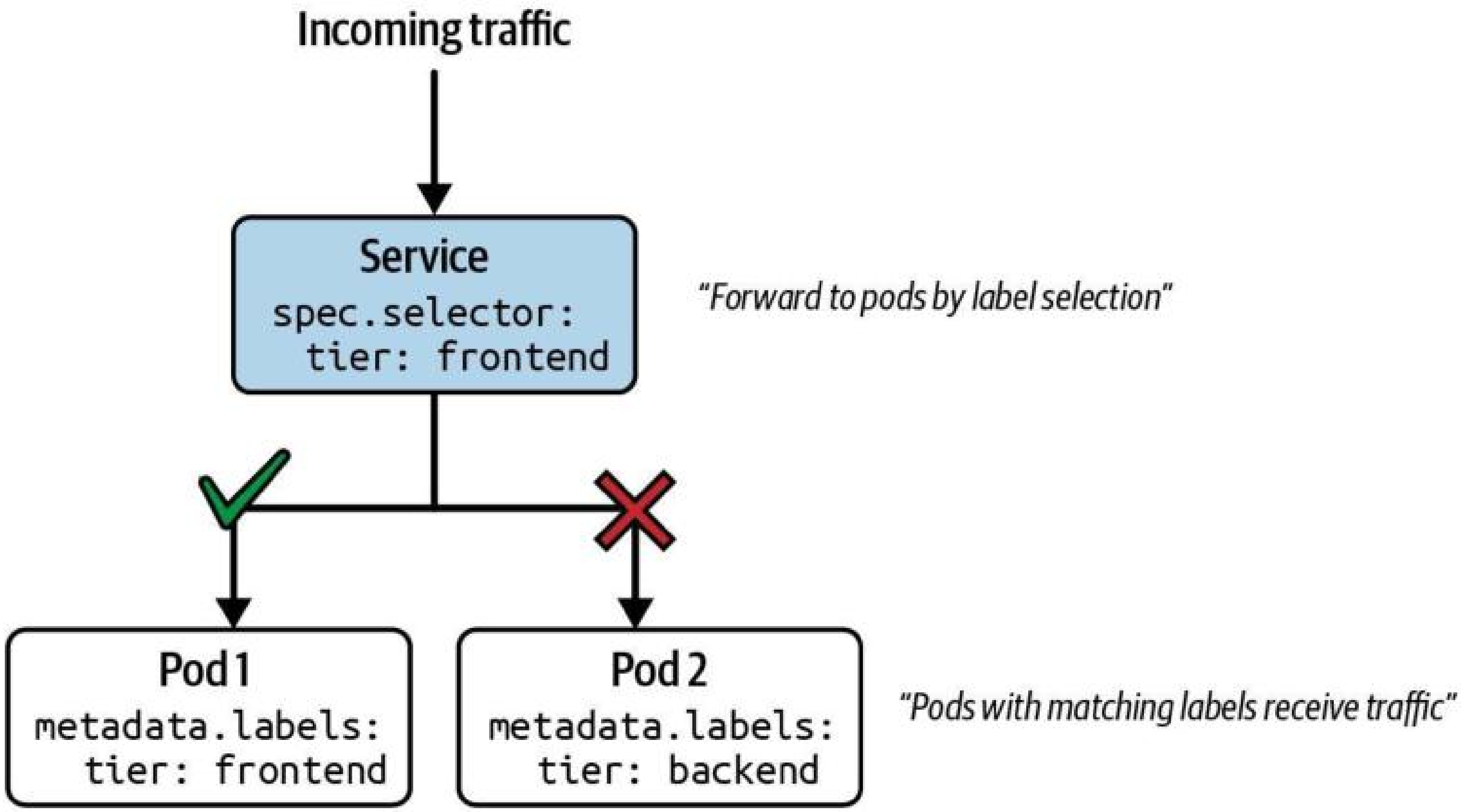
What is a Service?

Stable endpoint for routing traffic to a set of Pods



Request Routing

"How does a Service decide which Pod to forward the request to?"



Creating a Service with Imperative Approach

Needs to provide the service type and ports

```
$ kubectl run echoserver↵
  --image=k8s.gcr.io/echoserver:1.10
  --port=8080
pod/echoserver
created

$ kubectl create↵
  service
  --tcp=80:8080
  echoserver↵
service/echoserver created

$ kubectl run echoserver↵
  --image=k8s.gcr.io/echoserver:1.10
  --expose
  --port=8080↵
service/echoserver
```

Exposed container port

Incoming and outgoing port

Shortcut for creating a Pod + Service

Service YAML Manifest

Needs to provide the service type and ports

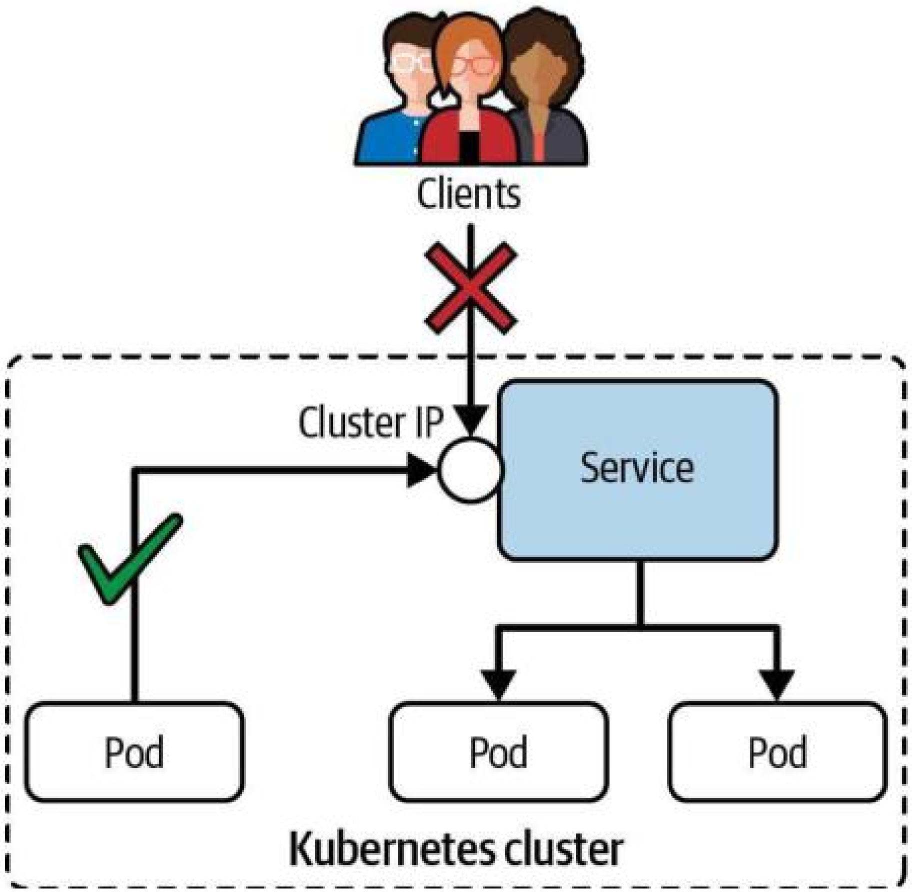
```yaml
apiVersion: v1
kind: Service
metadata:
  name: echoserver
spec:
  selector:
    app: echoserver
  ports:
  - port: 80
    targetPort: 8080
    protocol: TCP
```

Label selection for Pods

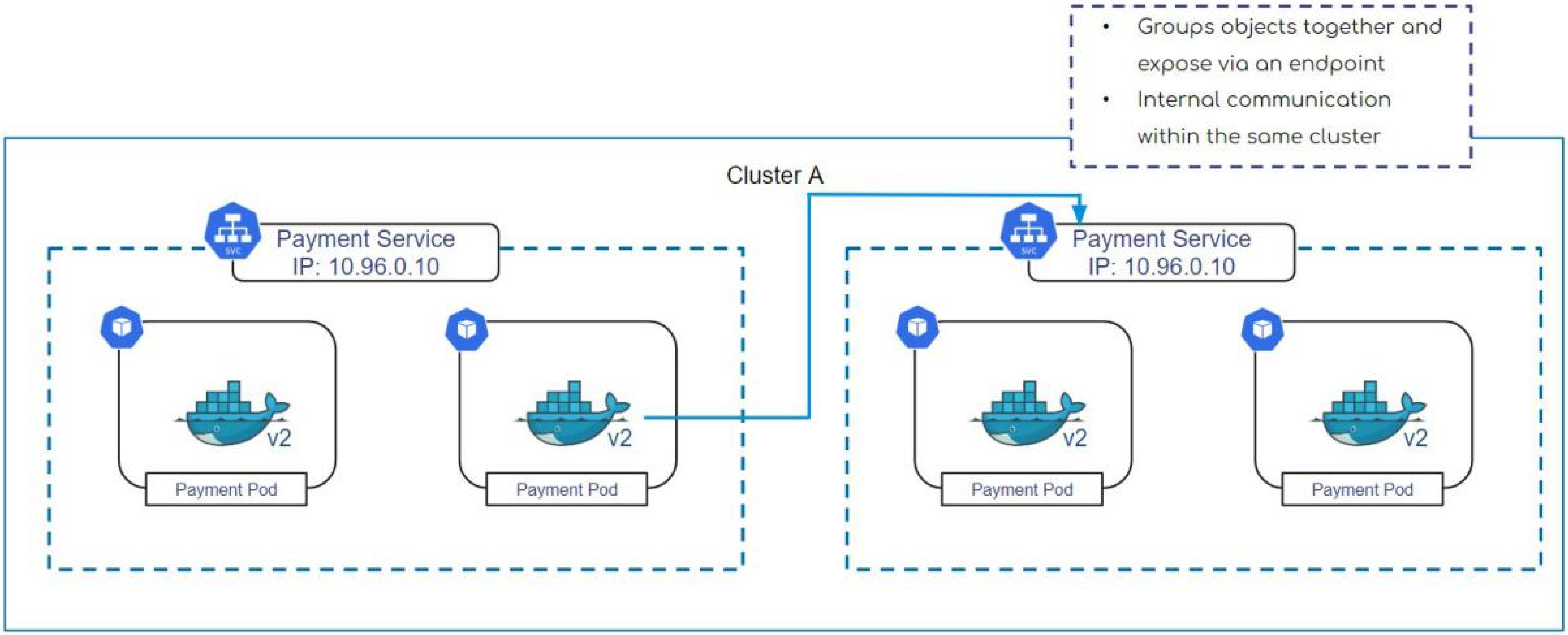
Mapping of incoming to outgoing port

ClusterIP Service Type

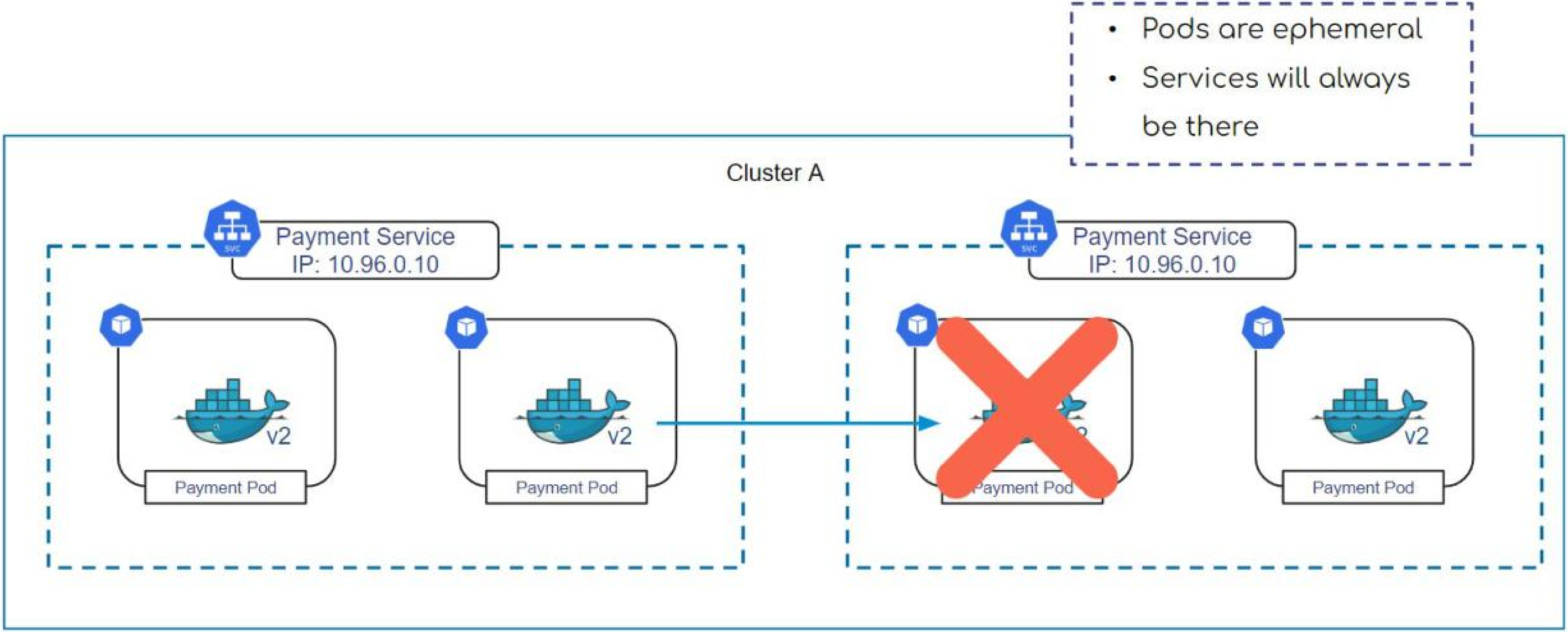
Only reachable from within the cluster, e.g. another Pod



ClusterIP Service



ClusterIP Service



ClusterIP Service YAML Manifest at Runtime

`ClusterIP` service type has been assigned if not already provided

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  ports:
    - port: 80
      targetPort: 80
```

Type: ClusterIP (default)

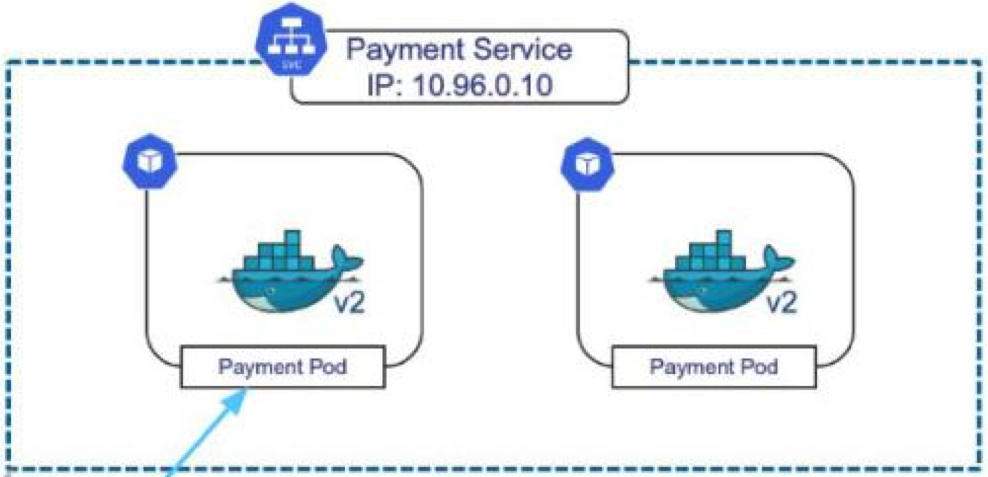
Port of the service

Port that the pod is listening on

ClusterIP Service YAML Manifest at Runtime

`ClusterIP` service type has been assigned if not already provided

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  ports:
    - port: 80
      targetPort: 80
```




```
kubectl create –f my-service.yaml
```

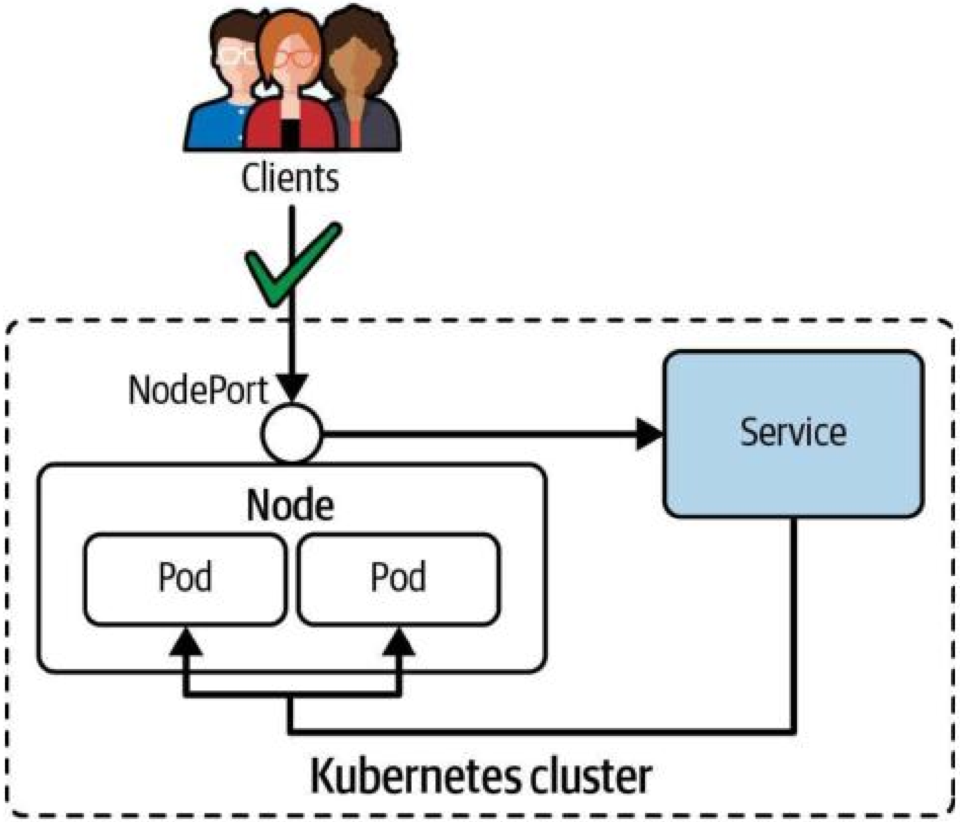
ClusterIP - Binding Pods to Services

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  ports:
    - port: 80
      targetPort: 80
  selector:
    env: dev
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: service-pod-1
  labels:
    env: dev
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
     - containerPort: 80
```

NodePort Service Type

Can be resolved from outside of the Kubernetes cluster



NodePort Service YAML Manifest at Runtime

Service type is `NodePort`, node port has been assigned

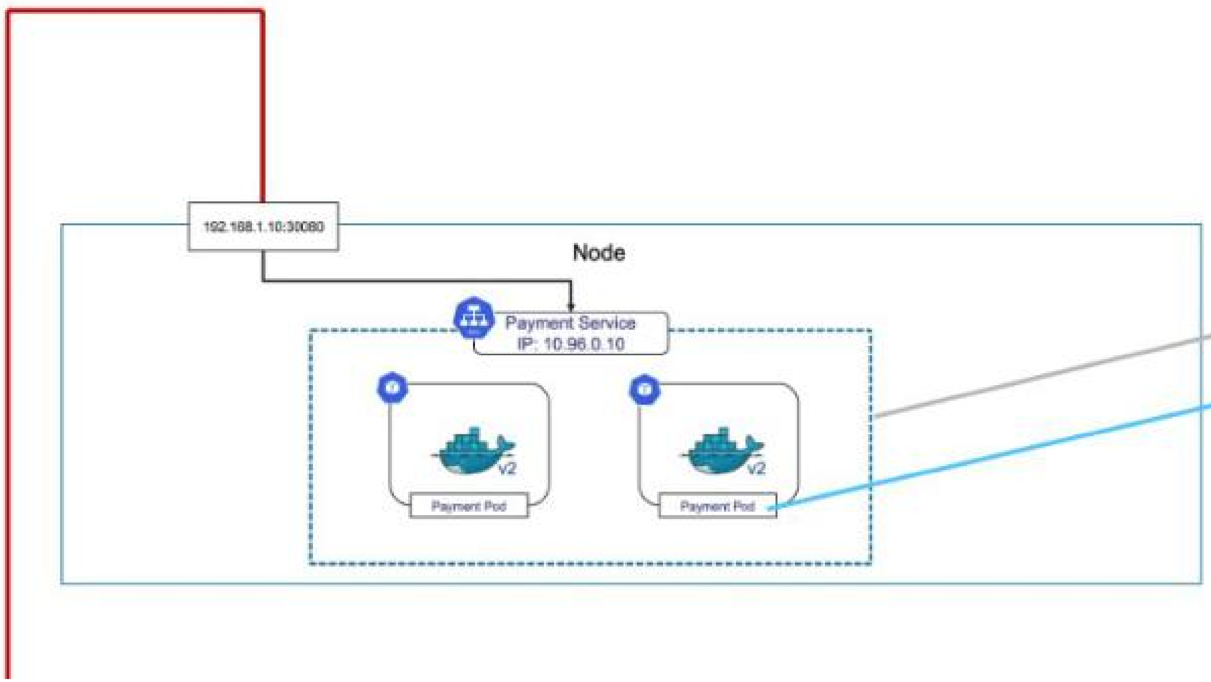
```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: NodePort
  ports:
    - targetPort: 80
      port: 80
      NodePort: 30080
```

Type: *NodePort

NodePort Number

Accessing a NodePort Service

Use the combination of node IP and statically-assigned port




```yaml
apiVersion: v1
kind: Service
metadata:
  name: nodeport-service
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

TROUBLESHOOTING

Troubleshooting Services

Built-in mechanisms for identifying the root cause of a failure

TROUBLESHOOTING SERVICES · STEP 1

Checking Service Connectivity

Identify the Service type and attempt to connect to it

```
kubectl get service echoserver
NAME         TYPE        CLUSTER-IP      EXTERNAL-IP  PORT(S)    AGE
echoserver   ClusterIP   10.96.254.0     <none>       8080/TCP   8s
```

Run a temporary Pod and attempt to reach the Service

```
kubectl run tmp --image=busybox --restart=Never -it --rm \
  -- wget echoserver:8080
Connecting to echoserver (10.96.254.0:8080)
wget: can't connect to remote host (10.96.254.0:8080): Connection refused
pod "tmp" deleted
```

Verifying Endpoints

No endpoints = label selector does not match any Pod

```
kubectl get endpoints echoserver
NAME         ENDPOINTS   AGE
echoserver   <none>      63s
```

Correct configuration exposes Pod IP addresses

```
kubectl get endpoints echoserver
NAME         ENDPOINTS                          AGE
echoserver   10.244.0.3:8080,10.244.0.4:8080   63s
```

Checking Label Selection and Port Assignment

Inspect the Service — verify Selector and TargetPort

```
kubectl describe service echoserver
Name:       echoserver
Selector:   app=echoserver
Type:       ClusterIP
IP:         10.111.148.223
Port:       8080/TCP
TargetPort: 8080/TCP
Endpoints:  10.244.0.3:8080,10.244.0.4:8080
```

Compare with Pod labels — they must match the Selector

```
kubectl describe pod echoserver
Labels:  app=echoserver
Containers:
  echoserver:
    Port:  8080/TCP
```

TROUBLESHOOTING SERVICES · STEP 4

Checking Network Policies

Check for NetworkPolicies that may block ingress traffic

```
kubectl get networkpolicies
No resources found in default namespace.

# If policies exist, inspect them:
kubectl get networkpolicies
NAME                   POD-SELECTOR   AGE
default-deny-ingress   <none>         9s
```

A default-deny-ingress policy blocks ALL ingress — may need an allow rule

TROUBLESHOOTING SERVICES · STEP 5

Ensuring Pod Functionality

Check Pod status — even 'Running' may hide container problems

```
kubectl get pods
NAME         READY   STATUS         RESTARTS   AGE
echoserver   0/1     ErrImagePull   0          2s

# After fix:
kubectl get pods
NAME         READY   STATUS    RESTARTS   AGE
echoserver   1/1     Running   0          5s
```

Exec into the Pod or use logs to verify the application is actually serving

```
kubectl logs echoserver
kubectl exec -it echoserver -- wget -qO- localhost:8080
```

EXERCISE

Troubleshooting a Service

EXAM PREP · KEY TAKEAWAYS

Exam Essentials

- Service misconfiguration is a common failure. Any misconfiguration prevents traffic from reaching the target Pods.
- Common causes: incorrect label selector (Service selector must match Pod labels), wrong port or targetPort assignment.
- Use kubectl get endpoints <service> to verify the Service has resolved Pod IP:port pairs.
- Check for NetworkPolicies that may block ingress traffic to the Pod.
- Verify the Pod itself is healthy: check status, logs, and exec a test request from inside the cluster.

TOPIC 2

Service DNS

How CoreDNS resolves Services and Pods

CLUSTER DNS

Names, Not IPs

- CoreDNS gives every Service a name: <service>.<namespace>.svc.cluster.local.
- Within the same namespace the short name resolves via the Pod's search list.
- Cross-namespace you need at least <service>.<namespace>; the full FQDN always works.
- A headless Service (clusterIP: None) returns every Pod IP instead of a single virtual IP.

Discovering the Service by DNS Lookup

Kubernetes' DNS service maps cluster IP address to Service name

```
$ kubectl run tmp --image=busybox --restart=Never -it --rm ↵
  -- wget echoserver.default.svc.cluster.local:8080
Connecting to echoserver.default.svc.cluster.local:8080 (10.96.254.0:8080)
...
```

Cluster-specific suffix

Service name

Resource type

Namespace

Service DNS

LAB 28 Service DNS

CoreDNS gives every Service a stable DNS name: <service>.<namespace>.svc.cluster.local. CKAD tests cross-namespace resolution, Pod DNS records, headless Services, and reading /etc/resolv.conf to understand ndots:5.

You'll build

Resolve a Service by short name and FQDN, cross namespaces, read a Pod DNS record and /etc/resolv.conf, then a headless Service.

Key commands: k -n · nslookup web · cat /etc/resolv.conf · » FQDN

Service DNS

Step 1 — Resolve from the same namespace (short name)

```
k -n app run client --image=busybox --restart=Never -it --rm -- \
  sh -c 'nslookup web; wget -qO- web | head -3'
```

Step 2 — Resolve from a different namespace

```
nslookup web              # fails cross-namespace
nslookup web.app          # works
nslookup web.app.svc.cluster.local
```

Step 3 — Pod DNS record and resolv.conf

```
# Pod A-record: <dashed-ip>.<namespace>.pod.cluster.local
cat /etc/resolv.conf
# search app.svc.cluster.local svc.cluster.local cluster.local
# ndots:5
```

Service DNS (cont.)

Step 4 — Headless Service returns all Pod IPs

```
k -n app create service clusterip headless --clusterip="None" --tcp=80:80
nslookup headless.app    # returns every Pod IP, not one VIP
```

Note FQDN = <service>.<namespace>.svc.cluster.local. Short names resolve via the search list within the namespace; cross-namespace needs <service>.<namespace>. ndots:5 is why short names check the search list before an absolute lookup. Headless (clusterIP: None) returns per-Pod IPs.

Service DNS

Test it

nslookup web works in-namespace but needs web.app cross-namespace, and the headless Service resolves to all backing Pod IPs.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Headless Services

Headless Service — a Service with **clusterIP: None**. No virtual IP; DNS resolves directly to Pod IPs.

Regular ClusterIP Service

DNS → single virtual IP (kube-proxy load-balances)

Client never sees individual Pod IPs

Best for: stateless services

Headless Service (clusterIP: None)

DNS → returns A record per Pod IP

Client discovers and connects to individual Pods

Best for: StatefulSets, databases, peer discovery

Headless Service — YAML and DNS

Set clusterIP: None to create a headless service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: mysql-headless
spec:
  clusterIP: None          # <-- this makes it headless
  selector:
    app: mysql
  ports:
  - port: 3306
```

DNS — each Pod gets its own A record (used with StatefulSets)

```
# StatefulSet pod DNS pattern:
# <pod-name>.<service-name>.<namespace>.svc.cluster.local
mysql-0.mysql-headless.default.svc.cluster.local
mysql-1.mysql-headless.default.svc.cluster.local
```

Tea Break

15 minutes

TOPIC 3

Ingress & TLS

Layer-7 HTTP/HTTPS routing into the cluster

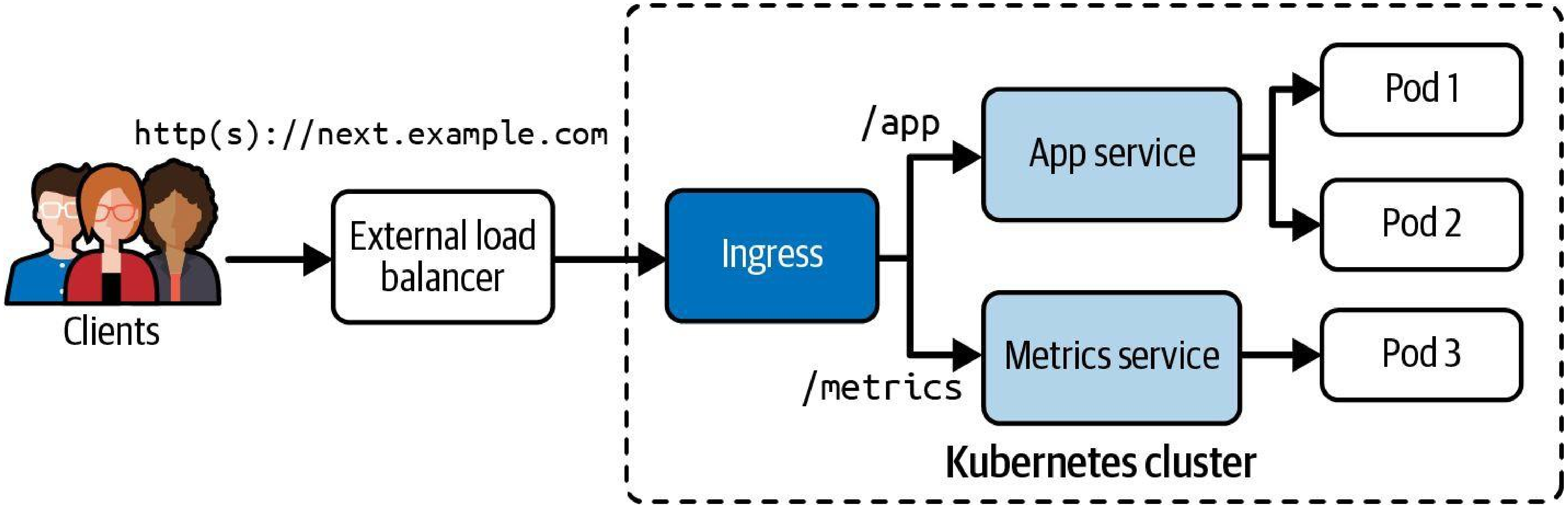
L7 ROUTING

One Entry Point, Many Rules

- An Ingress defines host- and path-based HTTP/HTTPS rules that forward to backend Services.
- An Ingress controller (e.g. ingress-nginx) must be running; ingressClassName selects it.
- TLS terminates at the controller — tls.secretName points at a kubernetes.io/tls Secret.
- pathType: Prefix matches a path and everything below it; Exact requires an exact match.
- apiVersion is networking.k8s.io/v1 (extensions/v1beta1 was removed in 1.22).

Ingress Traffic Routing

The context path is mapped to a backend (Service name and port)



Why Ingress? Services vs Ingress

Without Ingress (Services only)

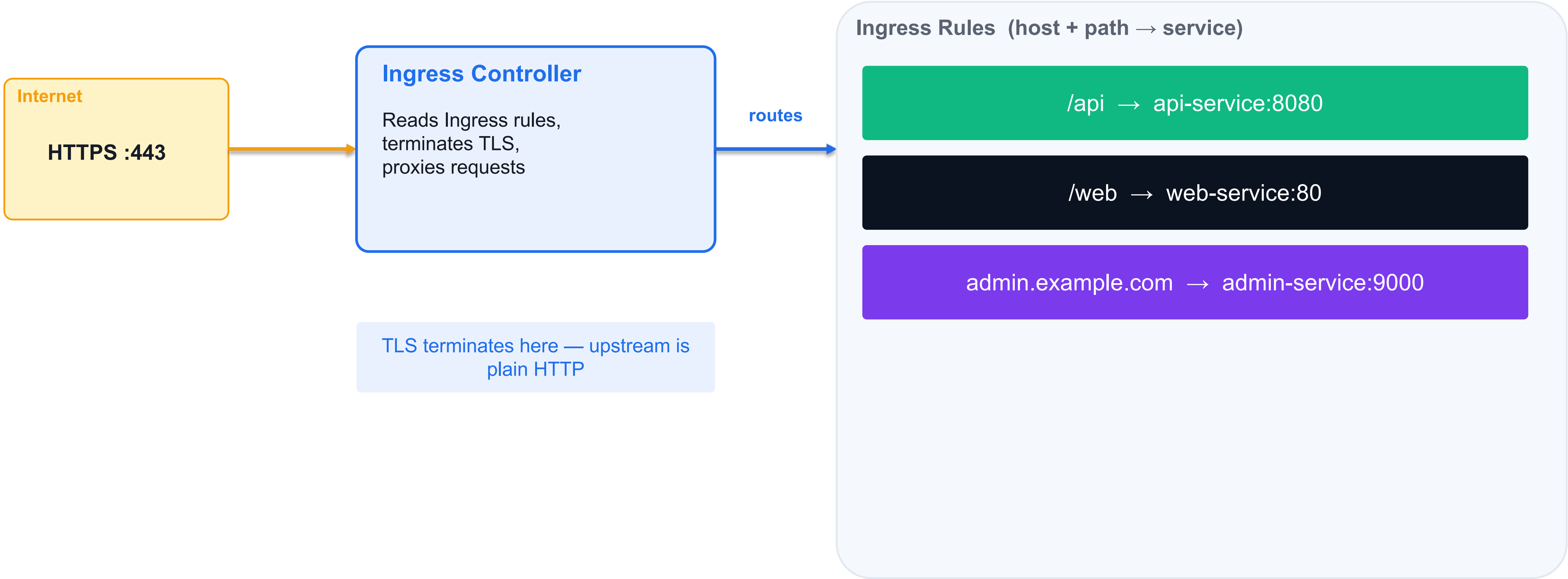
- One Service = one IP/port
- LoadBalancer Service = one cloud LB per service (expensive)
- No SSL/TLS termination at Service level
- No host/path-based routing with Services alone
- Many services = many public IPs to manage

With Ingress

- Single entry point for all HTTP(S) traffic
- L7 routing: route by hostname or URL path
  - example.com/api → api-service
  - example.com/web → web-service
- TLS termination: one cert handles all services
- Requires an IngressController (nginx, traefik, AWS ALB…)

INGRESS · ARCHITECTURE

Ingress — L7 Routing Architecture



Ingress YAML — Anatomy

Path-based routing for two services on the same host

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx          # which IngressController to use
  tls:
  - hosts: [example.com]
    secretName: tls-secret          # kubectl create secret tls
  rules:
  - host: example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service: {name: api-svc, port: {number: 8080}}
      - path: /web
        pathType: Prefix
        backend:
          service: {name: web-svc, port: {number: 80}}
```

IngressClass and IngressController

- An IngressController is a running Pod (e.g. nginx, traefik) that reads Ingress rules and configures its proxy.
- An IngressClass links an Ingress resource to a specific IngressController.
  - spec.ingressClassName: nginx → handled by the nginx IngressController
- If only one IngressClass exists it can be marked as default (isDefaultClass: true).
- Check available classes: kubectl get ingressclass
- CKAD exam: IngressClass is usually pre-configured — just reference it in your Ingress spec.
- Troubleshoot: kubectl describe ingress <name> — check Events for routing errors.

Ingress with TLS

LAB 29 Ingress & TLS

An Ingress provides Layer-7 HTTP/HTTPS routing: hostname and path rules forwarding to backend Services. CKAD tests installing a controller, TLS Secrets, host routing and path-based routing with pathType.

You'll build

Install ingress-nginx, create a TLS Secret, write a host+TLS Ingress, test HTTPS, then add a path-based rule.

Key commands: k create · apiVersion: networking.k8s.io/v1 · curl -k · » Ingress

LAB 29 · COMMANDS

Ingress with TLS

Step 1 — Backend Service + TLS Secret

```
k create deployment web --image=hashicorp/http-echo --replicas=2 -- -text=hello-ingress
k expose deployment web --port=5678 --target-port=5678
k create secret tls demo-tls --cert=tls.crt --key=tls.key
```

Ingress with TLS (cont.)

Step 2 — Ingress with TLS and host routing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: {name: demo}
spec:
  ingressClassName: nginx
  tls:
  - hosts: [demo.local]
    secretName: demo-tls
  rules:
  - host: demo.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend: {service: {name: web, port: {number: 5678}}}
```

Ingress with TLS (cont.)

Step 3 — Test HTTPS routing

```
curl -k --resolve demo.local:$HTTPS_PORT:127.0.0.1 \
  https://demo.local:$HTTPS_PORT/        # hello-ingress
```

Step 4 — Add path-based routing

```
# add a second rule: path /v2 -> service v2:5678 (pathType: Prefix)
curl -k --resolve demo.local:$HTTPS_PORT:127.0.0.1 \
  https://demo.local:$HTTPS_PORT/v2       # hello-v2
```

Note Ingress apiVersion is networking.k8s.io/v1. ingressClassName selects the controller; tls.secretName must point at a kubernetes.io/tls Secret; pathType: Prefix matches a path and everything below it. --resolve + -k let curl test without real DNS or a valid cert.

Ingress with TLS

Test it

curl https://demo.local:$HTTPS_PORT/ returns hello-ingress over TLS, and the /v2 path returns hello-v2 after the second rule is added.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Creating an Ingress with Imperative Approach

The rule definition requires intricate knowledge of syntax

```
$ kubectl create
  ingress↵corellian↵
  --rule="star-alliance.com/corellian/api=corellian:8080"
ingress.networking.k8s.io/corellian created
```

Ingress YAML Manifest

Allows for defining multiple rules that map to backend

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: corellian
spec:
  r host: star-alliance.com
    http:
      paths:
      - backend:
          service:
            name: corellian
            port:
              number: 8080
        path: /corellian/api
        pathType: Exact
```

The host definition is optional

Path Types

Incoming requests match based on assigned type in rule

Path Type	Rule	Incoming Request
Exact	`/corellian/api`	Matches `/corellian/api` but does not match `/corellian/test` or `/corellian/api/`.
Prefix	`/corellian/api`	Matches `/corellian/api` and `/corellian/api/` but does not match `/corellian/test`.

Listing and Describing Ingresses

Renders hosts, IP addresses, and ports

```
$ kubectl get
          ingress
NAME         CLASS    HOSTS                ADDRESS         PORTS   AGE
corellian    <none>   star-alliance.com    192.168.64.15   80      10m
$ kubectl describe ingress corellian
Name:              corellian
Namespace:         default
Address:           192.168.64.15
Default backend:   default-http-backend:80 (<error: \
Rules:             endpoints "default-http-backend" not found>)
  Host                Path  Backends
  ----                ----  --------
  star-alliance.com
                      /corellian/api   corellian:8080 (172.17.0.5:8080)
Annotations:          <none>
Events:               <none>
```

Configuring DNS for an Ingress

Making an Ingress convenient to use by end users

- To resolve the Ingress, you'll need to configure DNS entries to the external address.
- You'll need to either configure an A record, or a CNAME record. The ExternalDNS cluster add-on can help with managing those DNS records for you.
- If you have no domain or are using a local Kubernetes solution, you can add the mapping between IP address and domain name to `/etc/hosts`.

Accessing an Ingress

Use host name, port, and context path

```
$ wget star-alliance.com/corellian/api --timeout=5 --tries=1
--2021-11-30 19:34:57--  http://star-alliance.com/corellian/api
Resolving star-alliance.com (star-alliance.com)... 192.168.64.15
Connecting to star-alliance.com (star-alliance.com)|192.168.64.15|:80...
↵connected.
HTTP request sent, awaiting response... 200 OK
...
$ wget star-alliance.com/corellian/api/ --timeout=5 --tries=1
--2021-11-30 15:36:26--  http://star-alliance.com/corellian/api/
Resolving star-alliance.com (star-alliance.com)... 192.168.64.15
Connecting to star-alliance.com (star-alliance.com)|192.168.64.15|:80...
↵connected.
HTTP request sent, awaiting response... 404 Not Found
2021-11-30 15:36:26 ERROR 404: Not Found.
```

TOPIC 4

NetworkPolicy

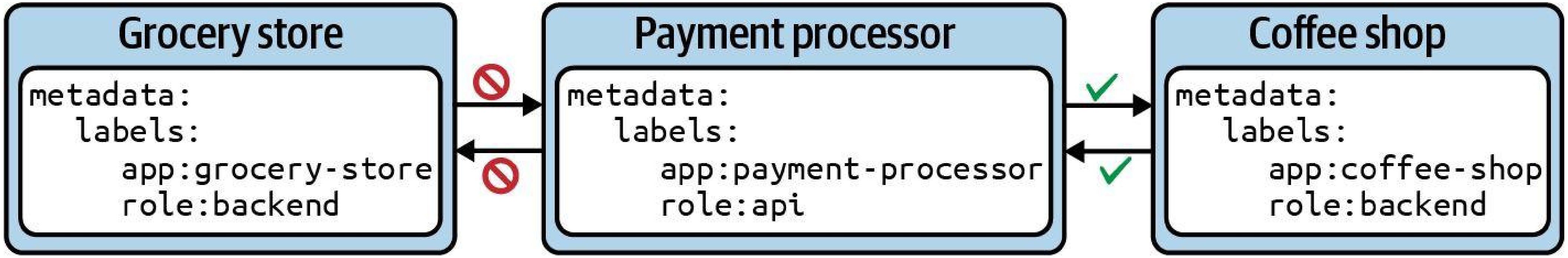
Allow-list Pod-to-Pod traffic

Restrict the Default Open Network

- By default every Pod can talk to every other Pod — NetworkPolicies allow-list specific traffic.
- Default-deny: podSelector: {} with policyTypes:[Ingress] and no ingress rules blocks all inbound.
- Allow by podSelector (Pods by label) or namespaceSelector (whole namespaces) in from/to.
- Egress works the same way — the classic exam pattern is 'deny all egress except DNS (port 53)'.
- Policies are additive: multiple policies combine with logical OR on their allow rules.

Example Network Policy

Restricting access to the payment processor Pod from other Pods



Default Deny Network Policies

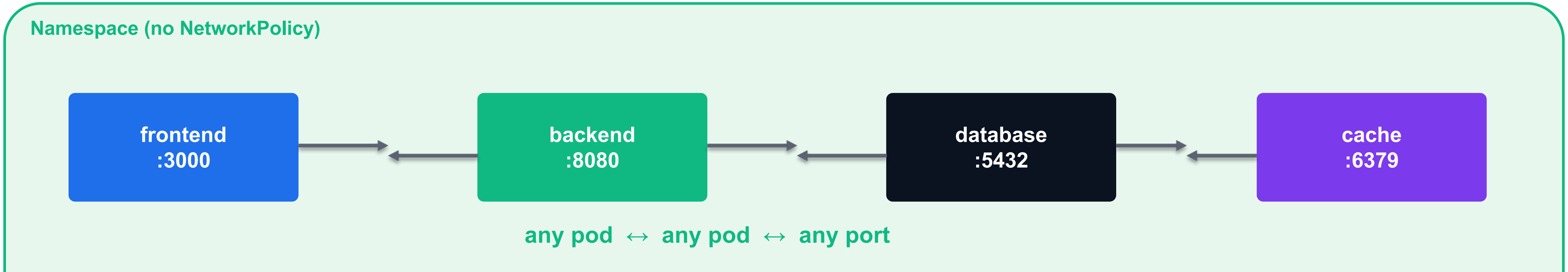
Network policies are additive

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
spec:

  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
```

Selects all Pods in the namespace

Applies in incoming and outgoing traffic

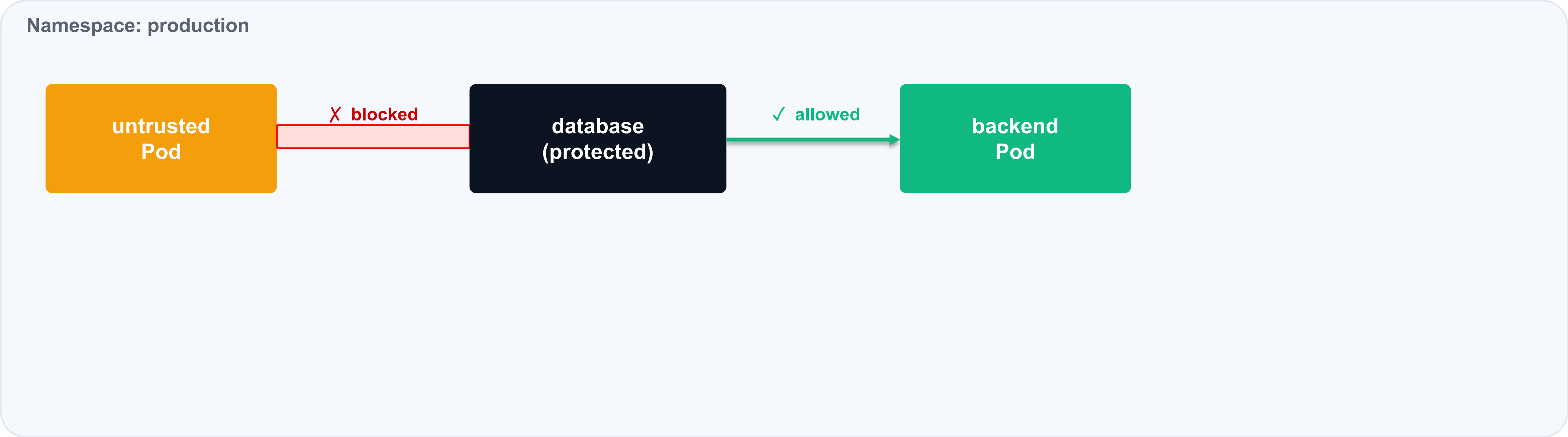
Default: All Pods Can Communicate



Without NetworkPolicy: every Pod can reach every other Pod on any port — no isolation

NETWORKPOLICY · INGRESS RULES

NetworkPolicy — Restrict Ingress Traffic




```yaml
kind: NetworkPolicy
spec:
  podSelector: {matchLabels: {app: database}}
  policyTypes: [Ingress]
  ingress:
  - from:
    - podSelector: {matchLabels: {role: backend}}
    ports:
    - port: 5432
```

C

NetworkPolicy — Restrict Egress Traffic

Allow backend to reach only the database (port 5432) — block everything else

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-egress
  namespace: production
spec:
  podSelector:
    matchLabels:
      role: backend
  policyTypes:
  - Egress
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: database
    ports:
    - protocol: TCP
      port: 5432
  - ports:                     # allow DNS
    - protocol: UDP
      port: 53
```

NetworkPolicy Patterns

- Default-deny ingress: select all pods, no ingress rules → blocks all incoming traffic.
  - kind: NetworkPolicy spec: podSelector: {} policyTypes: [Ingress] ingress: []
- Default-deny egress: select all pods, no egress rules → blocks all outgoing traffic.
  - kind: NetworkPolicy spec: podSelector: {} policyTypes: [Egress] egress: []
- Always allow DNS (port 53 UDP) in egress rules, or pods cannot resolve service names.
- namespaceSelector: target all pods in a specific namespace (e.g. monitoring).
- ipBlock: allow/deny traffic to/from external CIDRs (e.g. 192.168.0.0/16).

NetworkPolicy

LAB 30 NetworkPolicy

By default every Pod can talk to every other Pod. NetworkPolicies allow-list specific sources and destinations. CKAD regularly includes a NetworkPolicy question — you must write a default-deny and a selective-allow policy from scratch.

You'll build

Apply default-deny ingress, a selective podSelector allow, an egress DNS-only lockdown, then a cross-namespace allow.

Key commands: spec: · ingress: · » Default-deny

NetworkPolicy

Step 1 — Default deny: block all ingress

```
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```

Step 2 — Selective allow: only role=allowed reaches the backend

```
spec:
  podSelector:
    matchLabels: {app: backend}
  policyTypes: [Ingress]
  ingress:
  - from:
    - podSelector:
        matchLabels: {role: allowed}
    ports:
    - {protocol: TCP, port: 5678}
```

NetworkPolicy (cont.)

Step 3 — Egress lockdown: allow DNS only

```
spec:
  podSelector: {matchLabels: {role: blocked}}
  policyTypes: [Egress]
  egress:
  - to:
    - namespaceSelector: {}
      podSelector: {matchLabels: {k8s-app: kube-dns}}
    ports: [{protocol: UDP, port: 53}, {protocol: TCP, port: 53}]
```

Step 4 — Cross-namespace allow (namespaceSelector)

```
ingress:
- from:
  - namespaceSelector:
      matchLabels: {purpose: trusted}
```

NetworkPolicy (cont.)

Note Default-deny = podSelector: {} + policyTypes: [Ingress] with no ingress list. podSelector in from matches Pods by label in the same namespace; namespaceSelector matches whole namespaces. Policies are additive — multiple policies combine with logical OR.

NetworkPolicy

Test it

After the allow policy, client-ok (role=allowed) reaches the backend while client-bad stays blocked, proving the NetworkPolicy is enforced.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

What is an Network Policy Controller?

Evaluating Network Policy rules

- A Network Policy cannot work without a Network Policy controller. The Network Policy controller evaluates the collection of rules defined by a Network Policy.
- One option for such a Network Policy controller is Cilium.

Network Policy YAML Manifest

Key-value pairs can be parsed from different sources

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-allow
spec:
  podSelector:
    matchLabels:
      app: payment-processor
      role: api
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: coffeeshop
```

Selects the Pod the policy should apply to by label selection

Allows incoming traffic from the Pod with matching labels within the same namespace

Network Policy Rules

Attributes that form the rules

Attribute	Description
`podSelector`	Selects the Pods in the namespace to apply the network policy to.
`policyTypes`	Defines the type of traffic (i.e., ingress and/or egress) the network policy applies to.
`ingress`	Lists the rules for incoming traffic. Each rule can define `from` and `ports` sections.
`egress`	Lists the rules for outgoing traffic. Each rule can define `to` and `ports` sections.

Behavior of `to` and `from` Selectors

Ingress and egress sections define the same attributes

Attribute	Description
`podSelector`	Selects Pods by label(s) in the same namespace as the Network Policy which should be allowed as ingress sources or egress destinations.
`namespaceSelector`	Selects namespaces by label(s) for which all Pods should be allowed as ingress sources or egress destinations.
`namespaceSelector` and `podSelector`	Selects Pods by label(s) within namespaces by label(s).

Rendering Network Policy Details

Ingress and egress rules can be inspected in details

```
$ kubectl describe networkpolicy api-allow
Name:           api-allow
Namespace:      default
Created on:     2020-09-26 18:02:57 -0600 MDT
Labels:         <none>
Annotations:    <none>
Spec:
  PodSelector:     app=payment-processor,role=api
  Allowing ingress traffic:
    To Port: <any> (traffic allowed to all ports)
    From:
      PodSelector: app=coffeeshop
  Not affecting egress traffic
  Policy Types: Ingress
```

Verifying the Runtime Behavior

Communication from Pods not matching with labels will be blocked

```
$ kubectl run grocery-store --rm -it --image=busybox
↵
  -l app=grocery-store,role=backend -- /bin/sh
/ # wget --spider --timeout=1 10.0.0.51
Connecting to 10.0.0.51 (10.0.0.51:80)
wget: download timed out
/ # exit
pod "grocery-store" deleted

$ kubectl run coffeeshop --rm -it --image=busybox ↵
  -l app=coffeeshop,role=backend -- /bin/sh
/ # wget --spider --timeout=1 10.0.0.51
Connecting to 10.0.0.51 (10.0.0.51:80)
remote file exists
/ # exit
```

Restricting Access to Ports

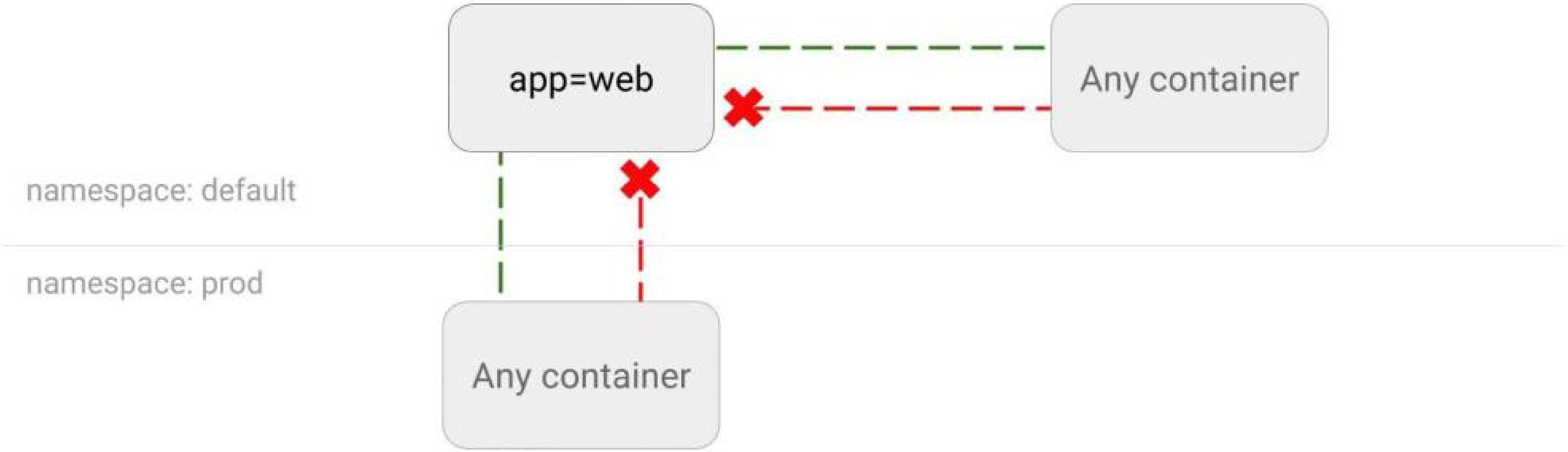
By default, all ports are accessible

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: port-allow
spec:
  podSelector:
    matchLabels:
      app: backend
  ingress:
  - from:
    - podSelector:
    ports:
    - protocol: TCP
      port: 8080
```

Only allow incoming traffic to port 8080

Scenario-Based Learning Resource

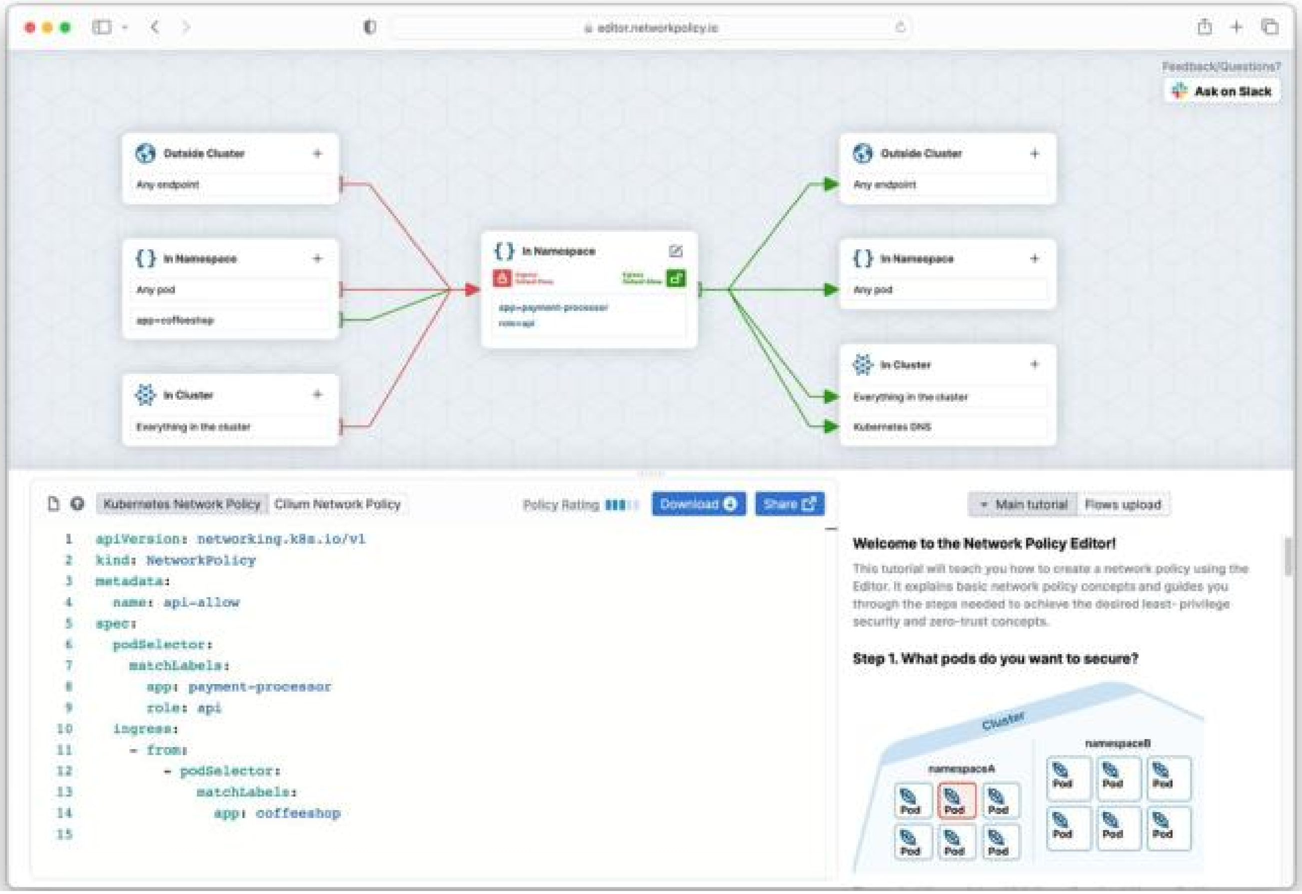
Exercising use cases for Network Policies is helpful with understanding concept



https://github.com/ahmetb/kubernetes-network-policy-recipes

Visualizing and Analysing a Network Policy

The page networkpolicy.io provides policy editor



COURSE COMPLETE

All five CKAD domains complete.

Design & build · Deployment · Observability · Config & Security · Services & Networking.

EXAM TIPS · ENVIRONMENT

CKAD Exam Environment

PSI Secure Browser — Remote Desktop | 2 hours | ~15–20 questions | 66% pass mark

- Single monitor; 1 additional tab allowed (kubernetes.io docs only)
- No notes, no bookmarks — docs must be navigated fresh
- Allowed: kubernetes.io/docs, kubernetes.io/blog, helm.sh/docs
- Remote desktop — use built-in terminal and browser only
- Candidate ID verification required before exam starts

- All questions are practical — live cluster
- Multiple contexts — always switch context per question
- Partial credit possible — attempt every question
- Paste YAML from docs or use --dry-run=client -o yaml
- Flag difficult questions and return — time management critical

kubectl Shortcuts for the Exam

Set alias and enable auto-completion (pre-configured in exam)

```
alias k=kubectl
source <(kubectl completion bash)
complete -o default -F __start_kubectl k

k get po     # pods
k get svc    # services
k get deploy # deployments
k get ns     # namespaces
```

Generate YAML with --dry-run (fastest way to create manifests)

```
k run nginx --image=nginx --dry-run=client -o yaml > pod.yaml
k create deploy myapp --image=myapp:1.0 --dry-run=client -o yaml
k create svc clusterip my-svc --tcp=80:80 --dry-run=client -o yaml
k apply -f pod.yaml
k delete -f pod.yaml --force --grace-period=0
```

Working in the Right Context & Namespace

Each exam question specifies a cluster context — switch before solving

```
kubectl config get-contexts
kubectl config use-context <context-name>
kubectl config current-context
```

Set the default namespace to avoid typing -n every time

```
kubectl config set-context --current --namespace=<namespace>
kubectl config view --minify | grep namespace
kubectl get pods -n kube-system   # one-shot without changing default
```

Exam Strategy & Time Management

- Read the question fully — note context, namespace, and exact requirements
- Switch context FIRST before any kubectl command
- Use --dry-run=client -o yaml to generate YAML skeletons, then edit
  - Faster than writing YAML from scratch every time
- Use 'kubectl explain' for YAML field names (no internet needed)
  - kubectl explain pod.spec.containers.livenessProbe
- Flag and skip — if a question takes >5 min, flag and return
- Verify after each task: kubectl get / describe to confirm result
- Use kubectl api-resources to find short names and API groups
- Practice: KillerCoda.com and killer.sh mock exams are the best prep

Exam Objectives and Overview

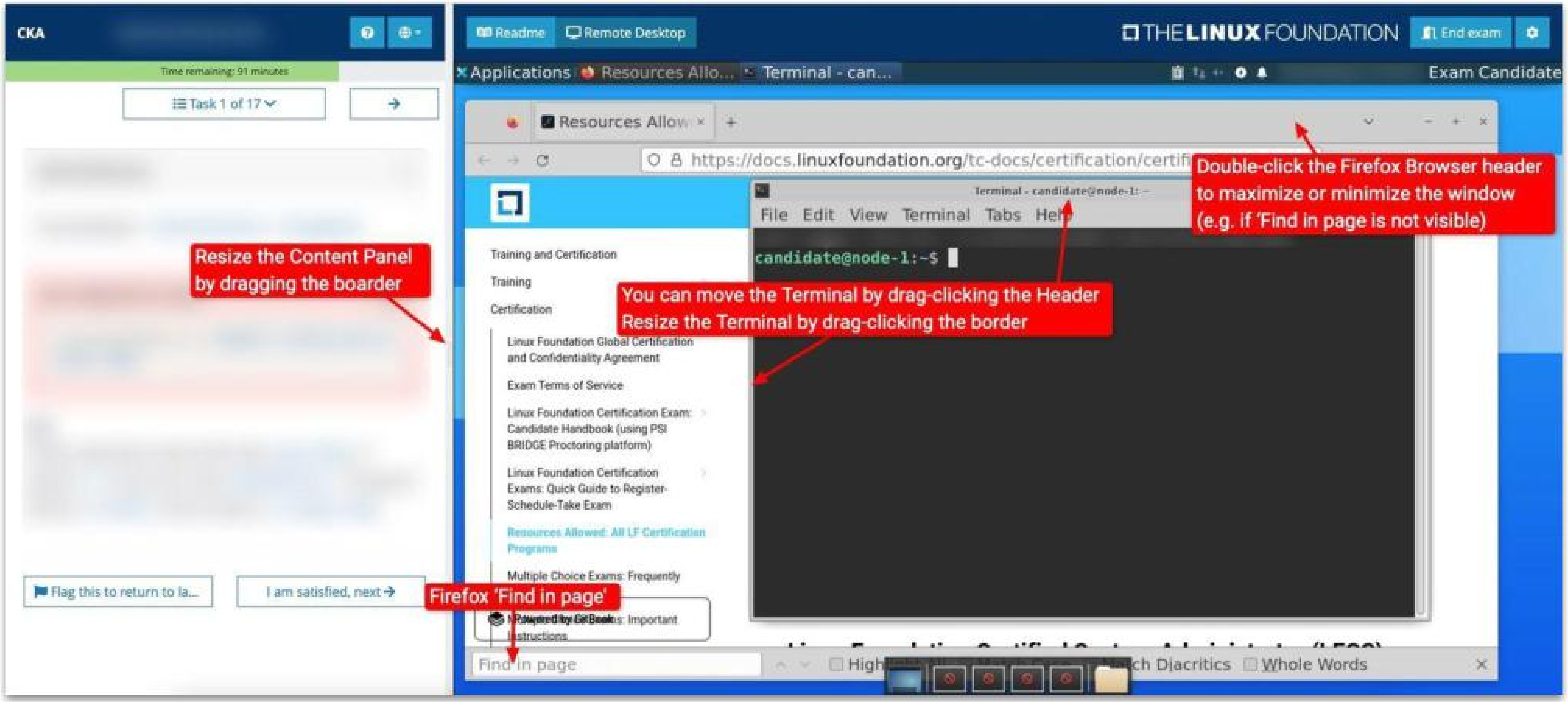
Kubernetes Version Used in Exam

Suffix in curriculum file name indicates version, changes ~ every 3 months

zuzannapn Archive CKAD v.1.26		2216e88 2 weeks ago 152 commits
kcna	add freeCodeCamp KCNA resource (#57)	last year
old-versions	Archive CKAD v.1.26	2 weeks ago
.DS_Store	Archive CKAD v.1.26	2 weeks ago
CKAD_Curriculum_v1.27.pdf	Updating exam curriculum to V1.27	last month
CKA_Curriculum_v1.27.pdf	Updating exam curriculum to V1.27	last month
CKS_Curriculum_ v1.27.pdf	Updating exam curriculum to V1.27	2 weeks ago
KCNA_Curriculum.pdf	KCNA Exam Curriculum	2 years ago
PCA_Curriculum.pdf	PCA Exam Curriculum	10 months ago
README.md	Update README.md	8 months ago

Exam Environment

PSI Remote desktop, single monitor, no bookmarks (announcement)



Using Auto-Completion for kubectl

Preconfigured in exam environment

```
$ kubectl cre<tab>
```

```
$ kubectl create
```

https://kubernetes.io/docs/reference/kubectl/cheatsheet/#kubectl-autocomplete

Lunch Break

1 hour

MOCK EXAM

Mock-Exam Practice — from 1:00 PM

Practise on killer.sh / Killercoda before the graded assessment · ask questions freely

WARM-UP

Mock-Exam Practice (1:00 PM)

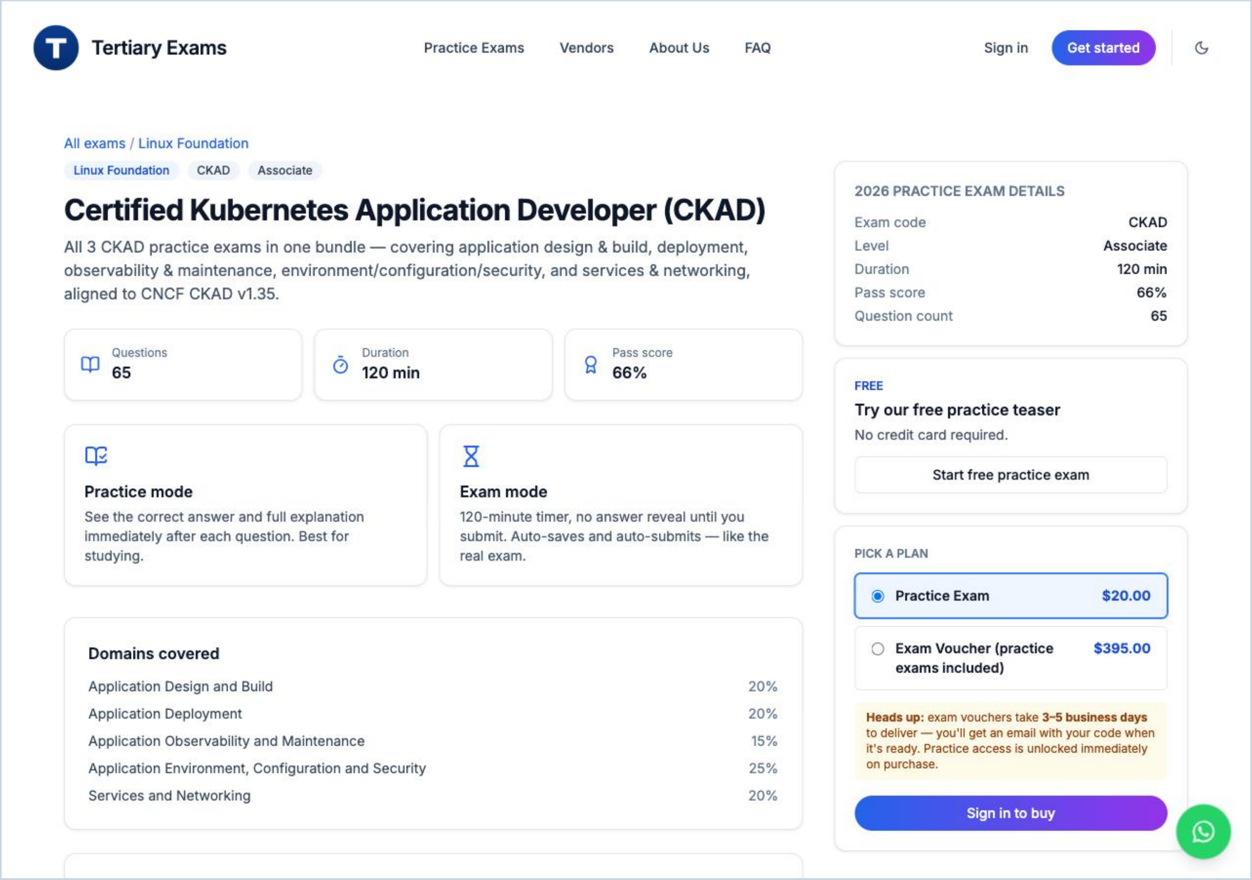
- Warm-up before the graded assessment — work hands-on questions across all five domains.
- Use killer.sh and Killercoda; rehearse the alias k, $do dry-run and kubectl explain workflow.
- Time yourself and flag weak areas — the real CKAD is 2 hours, 100% hands-on.
- Trainer is available for questions; the final assessment begins at 2:00 PM.

PRACTICE EXAM

Take the Online CKAD Practice Exams

- Reinforce all five CKAD domains with realistic, exam-style questions before exam day.
- 3 full practice exams · 65 questions · 120-minute timer · 66% pass mark — aligned to CNCF CKAD v1.35.
- Practice mode reveals the answer and a full explanation after each question; Exam mode mirrors the real timed test.
- Start free with the practice teaser — no credit card required; sign in to unlock the full bundle.

Sign up and practise at:
exams.tertiaryinfotech.com/practice-exams/linuxfoundation/linuxfoundation-ckad



ASSESSMENT

Final Assessment — from 2:00 PM

Written / MCQ on the LMS · open book · covers all five domains and the hands-on labs

FINAL ASSESSMENT

Assessment

Final Assessment

- Written / MCQ assessment on the LMS
- Mock-exam practice from 1:00 PM; final assessment from 2:00 PM
- Covers all five CKAD domains and the hands-on labs
- Open book — slides, Learner Guide & kubernetes.io
- Must be attempted for SSG funding

Funding & Competency

Minimum 75% attendance

Based on SSG digital attendance records.

Assessed as Competent

Pass the written / MCQ assessment.

ASSESSMENT RULES

Briefing for Assessment

Clear your space

Keep only approved materials on the table.

No recording

No photos of the assessment.

Work individually

No discussion during the assessment.

Use the LMS

The assessment is delivered online.

Open book

Slides, Learner Guide & kubernetes.io are allowed.

Time's up

Submit when time is up.

THANK YOU

Thank you — and good luck with the CKAD exam!

Keep practising on killer.sh and Killercoda; the exam is 2 hours, 100% hands-on.